Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

COURSE NOTES · BASIC TO ADVANCED

Tailwind CSS
Utility-First Styling

Complete notes for the Learn Being Forward Tailwind CSS syllabus: every module, topic and subtopic explained step by step, with diagrams, verified examples and a practice checkpoint for each module.

17	3	26	325
modules	levels	minimum days	examples rendered

How to use these notes

These notes follow the **Learn Being Forward Tailwind CSS syllabus** exactly: the same 17 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **3.7.2**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 6	7 days	Set up Tailwind and style pages with spacing, sizing, colour, typography, effects and state variants
2. Intermediate	7 to 12	9 days	Build layouts with Flexbox and Grid, make them responsive with breakpoints and container queries, add dark mode, motion and reusable components
3. Advanced	13 to 17	10 days	Customise the theme with @theme, extend Tailwind with custom utilities, variants and plugins, use it inside frameworks and ship production-quality projects
Total	1 to 17	26 days	About 5 weeks at 6 study days per week

Read
Read the concept for each subtopic and note the key points in your own words.

Type & run
Type out and run every example. Do not copy and paste code. Compare with the OUTPUT box.

Checkpoint
Finish the practice checkpoint at the end of each module before moving on.

Revise
At the end of each level, revise its modules and build one small project.

Every Tailwind example in these notes was compiled with the real Tailwind CSS v4.1.13 compiler (the standalone CLI), and the generated CSS was applied to the HTML in Chromium 141; the picture under the code is a screenshot of what the browser displayed. Where the generated CSS is shown, it is the compiler's actual output (shortened only by removing unrelated rules). Framework code (React, Vue, Angular, shadcn/ui, cva) in Module 15 could not be installed in the offline build environment; it is shown as code with representative output, labelled as such.

HOW TO READ THE BOXES

Dark boxes with a gold top line are **HTML and CSS code**. Dashed cream boxes labelled **OUTPUT** show what the program prints. Gold-bordered boxes are notes and tips, red boxes warn about common mistakes, and navy boxes hold the key idea to remember.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to Tailwind CSS	Beginner	1
02	Setup and installation	Beginner	1
03	Core utilities: spacing, sizing and colours	Beginner	2
04	Typography	Beginner	1
05	Backgrounds, borders and effects	Beginner	1

Module	Topic	Level	Min days
06	State variants	Beginner	1
07	Layout, positioning and Flexbox	Intermediate	2
08	CSS Grid with Tailwind	Intermediate	1
09	Responsive design and container queries	Intermediate	2
10	Dark mode and theming basics	Intermediate	1
11	Transitions, transforms and animations	Intermediate	1
12	Building components and reusing styles	Intermediate	2
13	Customising the theme	Advanced	2
14	Directives, custom utilities, variants and plugins	Advanced	2
15	Tailwind with frameworks and the ecosystem	Advanced	2
16	Performance, accessibility and best practices	Advanced	1
17	Projects and interview preparation	Advanced	3

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 6

Module 01 · Introduction to Tailwind CSS 7

1.1 What is Tailwind CSS? 7
1.2 History of Tailwind CSS 9
1.3 Utility-first vs other approaches 11
1.4 Why do we need Tailwind CSS? 13
1.5 Where is Tailwind CSS used? 15
1.6 Prerequisites recap 16

Module 02 · Setup and installation 18

2.1 Ways to use Tailwind 18
2.2 Setting up a Vite project 21
2.3 How Tailwind works 23
2.4 Editor tooling 27
2.5 Reading the documentation 28

Module 03 · Core utilities: spacing, sizing and colours 30

3.1 The design scale 30
3.2 Spacing 34
3.3 Sizing 36
3.4 Colours 39

Module 04 · Typography 44

4.1 Fonts 44
4.2 Size and weight 47
4.3 Text styling 50
4.4 Lists 55
4.5 The Typography plugin 56

Module 05 · Backgrounds, borders and effects 61

5.1 Backgrounds 61
5.2 Borders 65
5.3 Rings and outlines 68
5.4 Effects 70
5.5 Filters 72
5.6 Images 74

Module 06 · State variants 78

6.1 Variant basics 78
6.2 Interactive states 80
6.3 Structural variants 82
6.4 Form variants 84
6.5 Pseudo-elements 87
6.6 Styling based on other elements 88
6.7 Attribute variants 93

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 7 TO 12

Module 07 · Layout, positioning and Flexbox 99

7.1 Layout utilities 99
7.2 Positioning 103
7.3 Overflow 104
7.4 Flexbox 106
7.5 Common patterns 110

Module 08 · CSS Grid with Tailwind 116

8.1 Grid basics 116
8.2 Placing items 117
8.3 Arbitrary and template grids 119
8.4 Alignment 122
8.5 Common grid layouts 123

Module 09 · Responsive design and container queries 128

9.1 Mobile-first breakpoints 128
9.2 Responsive patterns 132
9.3 Container queries 136
9.4 Other media variants 138
9.5 Testing responsiveness in DevTools 140

Module 10 · Dark mode and theming basics 145

10.1 Dark mode 145
10.2 Designing for dark mode 149
10.3 Theme colours with CSS variables 150

Module 11 · Transitions, transforms and animations 157

11.1 Transitions 157
11.2 Transforms 159
11.3 Animations 161
11.4 Accessibility 165

Module 12 · Building components and reusing styles 168

12.1 Thinking in components 168
12.2 Extracting with CSS 170
12.3 Common UI components 172
12.4 The Forms plugin 183
12.5 Accessibility for components 184

LEVEL 3 · ADVANCED LEVEL · MODULES 13 TO 17

Module 13 · Customising the theme 188

13.1 Theme variables 188
13.2 Extending and overriding 191
13.3 Using theme values elsewhere 193
13.4 Design systems 195
13.5 Legacy JavaScript configuration 196

Module 14 · Directives, custom utilities, variants and plugins 201

14.1 Directives and functions 201
14.2 Content sources 205
14.3 Custom utilities 206
14.4 Custom variants 207
14.5 Plugins 208
14.6 Native CSS with Tailwind 209

Module 15 · Tailwind with frameworks and the ecosystem 214

15.1 Tailwind with frameworks 214
15.2 Dynamic classes 217
15.3 Component libraries 220
15.4 Icons and assets 222

Module 16 · Performance, accessibility and best practices 226

16.1 Performance 226
16.2 Accessibility 228
16.3 Best practices 229
16.4 Upgrading projects 230
16.5 Debugging 231

Module 17 · Projects and interview preparation 234

17.1 Capstone projects 234
17.2 Practice 238
17.3 Interview preparation 238
17.4 Next steps 241

LEVEL 1

Beginner level

Outcome: set up Tailwind and style pages with spacing, sizing, colour, typography, effects and state variants.

6 MODULES · 7 DAYS MINIMUM

Module 01 · Introduction to Tailwind CSS	1 day
Module 02 · Setup and installation	1 day
Module 03 · Core utilities: spacing, sizing and colours	2 days
Module 04 · Typography	1 day
Module 05 · Backgrounds, borders and effects	1 day
Module 06 · State variants	1 day

MODULE 01 · BEGINNER

Introduction to Tailwind CSS

Minimum duration **1 day** | Level **Beginner**

This module explains what Tailwind CSS is, the utility-first idea behind it, and why so many teams use it, before any code is written. It covers the definition and what utility classes are, how Tailwind generates only the CSS a project uses, its history from 2017 to the Tailwind CSS 4 engine and the Tailwind Labs ecosystem, how utility-first compares with traditional CSS, component frameworks such as Bootstrap and inline styles, the common criticisms and the answers to them, why teams adopt it, where it is used, and the HTML and CSS knowledge it assumes.

IN THIS MODULE

1.1 What is Tailwind CSS?
1.2 History of Tailwind CSS
1.3 Utility-first vs other approaches
1.4 Why do we need Tailwind CSS?
1.5 Where is Tailwind CSS used?
1.6 Prerequisites recap

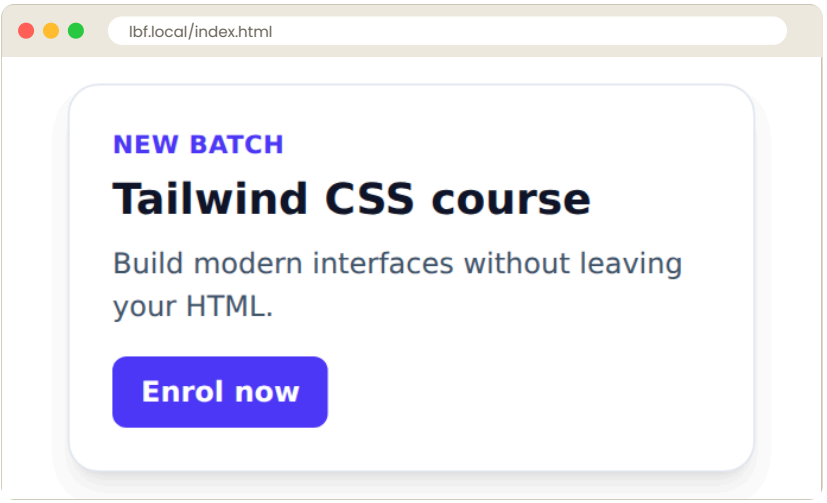
1.1 What is Tailwind CSS?

1.1.1 Definition: a utility-first CSS framework for building designs directly in HTML

Tailwind CSS is a **utility-first CSS framework**. Instead of giving you ready-made components (a "card", a "navbar") like Bootstrap, it gives you thousands of small **utility classes**, each doing one job, that you combine directly in your HTML to build any design. You still write CSS concepts (padding, Flexbox, colours, breakpoints), but you express them with class names instead of writing selectors and declarations in a separate file.

A CARD BUILT ONLY WITH UTILITY CLASSES — HTML

```html
<div class="mx-auto max-w-sm rounded-2xl bg-white p-6 shadow-lg ring-1 ring-slate-200">
  <p class="text-sm font-semibold tracking-wide text-indigo-600 uppercase">New batch</p>
  <h2 class="mt-1 text-2xl font-bold text-slate-900">Tailwind CSS course</h2>
  <p class="mt-2 text-slate-600">Build modern interfaces without leaving your HTML.</p>
  <a href="#" class="mt-4 inline-block rounded-lg bg-indigo-600 px-4 py-2 font-semibold text-white
hover:bg-indigo-700">Enrol now</a>
</div>
```



NO CUSTOM CSS WAS WRITTEN: EVERY STYLE COMES FROM A UTILITY CLASS

Reading the classes left to right is like reading CSS: `mx-auto` (margin-inline: auto), `max-w-sm` (max-width: 24rem), `rounded-2xl` (border-radius: 1rem), `bg-white`, `p-6` (padding: 1.5rem), `shadow-lg`, and so on.

1.1.2 Utility classes: small, single-purpose classes such as p-4, text-center, bg-blue-500

A utility class applies one small, predictable piece of styling, usually one CSS property with a value from a fixed **design scale**. Their names follow a pattern: a **property abbreviation**, optionally a side or axis, and a **value** from the scale. Here is the real CSS Tailwind v4 generated for a few common utilities:

Class	Pattern	Meaning
`p-4`	padding, scale step 4	padding: 1rem (4 × 0.25rem)
`px-6`	padding, x axis, step 6	padding-inline: 1.5rem
`mt-2`	margin, top, step 2	margin-top: 0.5rem
`text-center`	text alignment	text-align: center
`text-xl`	font size from the type scale	font-size: 1.25rem with a matching line height
`bg-blue-500`	background colour, palette blue, shade 500	background-color: var(--color-blue-500)
`rounded-lg`	border radius from the radius scale	border-radius: 0.5rem
`flex`	display	display: flex

COMPILED CSS

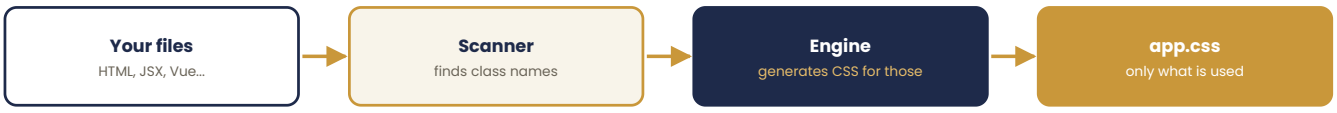
```
.mt-2 {
  margin-top: calc(var(--spacing) * 2);
}
.flex {
  display: flex;
}
.rounded-lg {
  border-radius: var(--radius-lg);
}
.bg-blue-500 {
  background-color: var(--color-blue-500);
}
.p-4 {
  padding: calc(var(--spacing) * 4);
}
.px-6 {
  padding-inline: calc(var(--spacing) * 6);
}
.text-center {
  text-align: center;
}
.text-xl {
  font-size: var(--text-xl);
  line-height: var(--tw-leading, var(--text-xl--line-height));
}
```

1.1.3 Tailwind generates only the CSS your project actually uses

Tailwind knows about tens of thousands of possible classes (every colour × every shade × every property, every spacing value, every variant combination), but it does not ship them all. At build time it **scans your source files** (HTML, JSX, Vue, Angular templates…) for class names and generates CSS **only for the ones it finds**. A typical production site ends up with 10 to 30 KB of CSS (under 10 KB compressed), smaller than most hand-written stylesheets. The proof, compiled here:

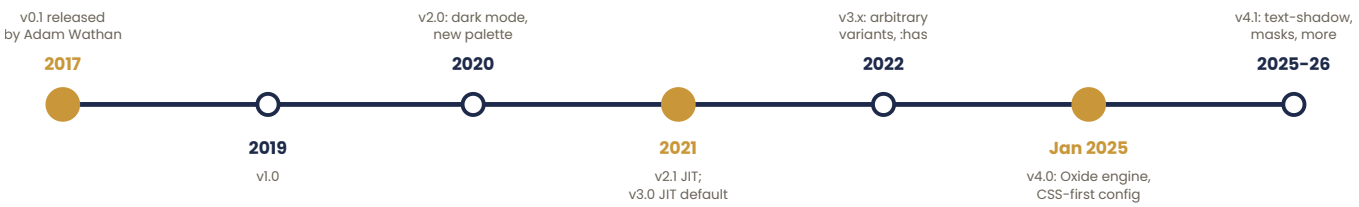
MEASURED WITH THE TAILWIND CLI

```
page using 1 utility class   -> generated CSS: 4.5 KB (of which preflight and theme variables are
most)
page using 20 utility classes -> generated CSS: 8.4 KB
same page, minified           -> 7.2 KB
```



TAILWIND BUILDS CSS FROM THE CLASSES IT FINDS IN YOUR SOURCE

1.2 History of Tailwind CSS



TAILWIND CSS MILESTONES

1.2.1 Created by Adam Wathan and team, first released in 2017

Tailwind was created by **Adam Wathan** with **Jonathan Reinink**, **David Hemphill** and **Steve Schoger**, growing out of the utility classes Wathan wrote while building a side project. His 2017 essay "CSS Utility Classes and Separation of Concerns" argued that semantic class names often just move coupling from CSS to HTML, and that composing small utilities scales better. Version 0.1 was released on 1 November 2017. The company **Tailwind Labs** was founded to develop it full-time, and it became one of the most used CSS frameworks; by the State of CSS surveys of the mid-2020s it was the most-used CSS framework among respondents.

1.2.2 Tailwind CSS 2 and 3 (Just-in-Time engine)

Version 2 (November 2020) added dark mode, an extended colour palette, ring utilities and better form support. In 2021 the **Just-in-Time (JIT) engine** changed everything: instead of generating every possible class up front and deleting unused ones afterwards (which made development builds several megabytes and limited variants), JIT generated CSS on demand as you wrote classes. That enabled **arbitrary values** (`w-[327px]`), every variant everywhere, and instant builds. **Version 3** (December 2021) made JIT the only engine and added arbitrary properties, colored shadows, scroll snap, `print:`, and later `has-*`, container queries via a plugin and more.

1.2.3 Tailwind CSS 4 (2025): new high-performance engine and CSS-first configuration

Tailwind CSS 4.0 (22 January 2025) was a ground-up rewrite with a new engine (codename Oxide, with parts in Rust) that is several times faster for full builds and over 100 times faster for incremental builds. Its biggest change for developers is **CSS-first configuration**: one line, `@import "tailwindcss";`, replaces the old directives, and the design system is defined in CSS with `@theme` variables instead of a `tailwind.config.js` file. It is built on modern CSS: cascade layers, registered custom properties (`@property`), `color-mix()`, the `oklch` colour space for a more vivid palette, native container queries and logical properties, and it needs Safari 16.4+, Chrome 111+ and Firefox 128+. Version 4.1 (April 2025) added text shadows, masks and more. This book uses v4.1.

	Tailwind 3	Tailwind 4
Entry CSS	`@tailwind base; @tailwind components; @tailwind utilities;`	`@import "tailwindcss";`
Configuration	`tailwind.config.js` (JavaScript)	`@theme { --color-brand: ...; }` in CSS
Content paths	listed in `content: [...]`	detected automatically, `@source` to add more
Colours	sRGB hex palette	oklch palette, P3-capable
Container queries	plugin	built in (`@container`, `@md:`)
Engine	JIT in JavaScript	Oxide: faster, Lightning CSS integrated

1.2.4 Tailwind Labs ecosystem: Tailwind Plus, Headless UI, Heroicons

Product	What it is
Tailwind Plus (formerly Tailwind UI)	a paid library of hundreds of professionally designed components, page sections and full templates written with Tailwind, in HTML, React and Vue

Product	What it is
Headless UI	free, unstyled, fully accessible interactive components (menus, dialogs, listboxes, tabs, comboboxes) for React and Vue, designed to be styled with Tailwind
Heroicons	free SVG icon set (outline, solid, mini, micro) by the Tailwind designers
Tailwind Play	an online playground (play.tailwindcss.com)
Tailwind CSS IntelliSense	the official VS Code extension: autocomplete, hover previews, linting
prettier-plugin-tailwindcss	official Prettier plugin that sorts classes in a consistent order

1.3 Utility-first vs other approaches

The same button, styled four ways, makes the differences concrete.

1.3.1 Traditional CSS with custom class names

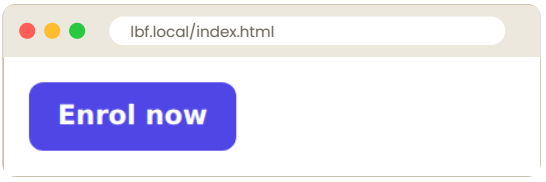
You invent a name, write the HTML with it, switch to a CSS file, and write the rules. Clean HTML, but every new component means new names, and the CSS file only grows, because nobody knows which rules are still used (CSS3 course, Module 18).

CUSTOM CSS — CSS

```css
.btn-enrol {
  display: inline-block; padding: 0.5rem 1rem; border-radius: 0.5rem;
  background-color: #4f46e5; color: #fff; font-weight: 600; text-decoration: none;
}
.btn-enrol:hover { background-color: #4338ca; }
```

INDEX.HTML — HTML

```html
<a class="btn-enrol" href="#">Enrol now</a>
```



HOW THE BROWSER DISPLAYS IT

1.3.2 Component frameworks like Bootstrap

A component framework gives you pre-built classes: `btn btn-primary`. Very fast for standard UI, but every Bootstrap site tends to look like Bootstrap, and anything outside its components means fighting its styles with overrides and higher specificity.

BOOTSTRAP — HTML

```html
<a class="btn btn-primary" href="#">Enrol now</a>   <!-- styles come from bootstrap.css (about 230 KB unminified) -->
```

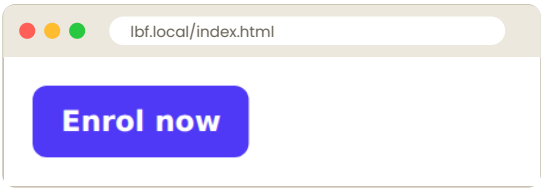
1.3.3 Inline styles vs utility classes (states, responsiveness, design constraints)

At first glance utilities look like inline styles. They are very different:

	Inline `style=""`	Tailwind utilities
Hover, focus, active states	impossible	`hover:bg-indigo-700`, `focus-visible:ring-2`
Responsive design	impossible (no media queries)	`md:px-6`, `lg:text-lg`
Dark mode, print, reduced motion	impossible	`dark:bg-slate-800`, `motion-reduce:transition-none`
Design constraints	any value: `padding: 13px`, 50 different greys	values from a shared scale: `p-3`, `p-4`; one palette
Specificity	highest; hard to override	single class (0,1,0) in a cascade layer
Reuse and caching	repeated in every element, not cacheable	one small cached stylesheet

TAILWIND HTML

```
<a href="#" class="inline-block rounded-lg bg-indigo-600 px-4 py-2 font-semibold text-white hover:bg-indigo-700 focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-indigo-600 md:px-6">Enrol now</a>
```



THE TAILWIND BUTTON, WITH HOVER, KEYBOARD FOCUS AND A LARGER PADDING FROM MD UP, ALL IN THE CLASS LIST

COMPILED CSS

```
.inline-block {
  display: inline-block;
}
.rounded-lg {
  border-radius: var(--radius-lg);
}
.bg-indigo-600 {
  background-color: var(--color-indigo-600);
}
.px-4 {
  padding-inline: calc(var(--spacing) * 4);
}
.hover\:bg-indigo-700 {
  &:hover {
    @media (hover: hover) {
      background-color: var(--color-indigo-700);
    }
  }
}
.md\:px-6 {
  @media (width >= 48rem) {
    padding-inline: calc(var(--spacing) * 6);
  }
}
```

1.3.4 Common criticisms ("ugly HTML") and how Tailwind answers them

Criticism	Answer
"The HTML is ugly and long"	True for raw markup; but in real projects the markup lives in a component (a React/Vue/Angular component, a template partial) written once and reused, so the long class list exists in one place. Prettier sorts classes consistently.
"It breaks separation of concerns"	The separation that matters is between content and presentation logic; a component file containing both its markup and its styles is easier to change and delete than markup in one file coupled to selectors in another.
"It is just inline styles"	No: utilities support states, breakpoints, dark mode and a design scale (1.3.3).
"I have to learn new class names"	Names map closely to CSS properties; IntelliSense autocompletes them and shows the CSS on hover. After a few days they are faster than writing CSS.
"Every site looks the same"	That applies to component frameworks; Tailwind has no default look beyond its scales, and the theme is fully customisable (Module 13).
"You still need to know CSS"	Correct, and important: Tailwind is a faster way to write CSS, not a replacement for understanding it.

1.4 Why do we need Tailwind CSS?

1.4.1 Faster development without switching files

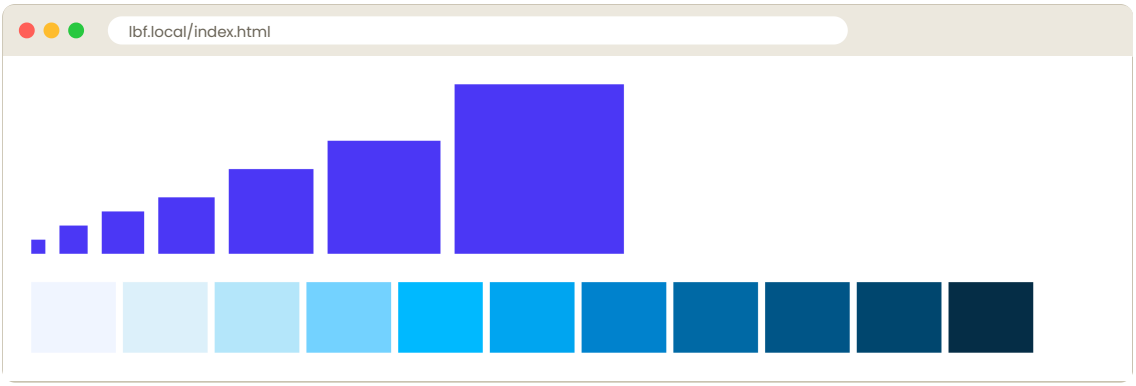
You style an element where you write it. There is no switching between an HTML and a CSS file, no searching for which rule affects which element, and changes are visible immediately with a dev server. Teams consistently report faster prototyping and fewer "where is this style coming from?" moments.

1.4.2 Consistent design through a fixed scale of spacing, colours and sizes

Instead of arbitrary pixel values, every spacing utility comes from one scale (multiples of 0.25rem), every colour from one palette, every font size from one type scale. A whole team naturally produces a consistent design, and changing a token changes it everywhere.

SCALES INSTEAD OF ARBITRARY VALUES HTML

```
<div class="flex flex-wrap items-end gap-2">
  <div class="size-2 bg-indigo-600"></div><div class="size-4 bg-indigo-600"></div><div class="size-6
bg-indigo-600"></div>
  <div class="size-8 bg-indigo-600"></div><div class="size-12 bg-indigo-600"></div><div class="size-16
bg-indigo-600"></div><div class="size-24 bg-indigo-600"></div>
</div>
<div class="mt-4 flex gap-1">
  <div class="h-10 w-12 bg-sky-50"></div><div class="h-10 w-12 bg-sky-100"></div><div class="h-10 w-12
bg-sky-200"></div><div class="h-10 w-12 bg-sky-300"></div>
  <div class="h-10 w-12 bg-sky-400"></div><div class="h-10 w-12 bg-sky-500"></div><div class="h-10 w-
12 bg-sky-600"></div><div class="h-10 w-12 bg-sky-700"></div>
  <div class="h-10 w-12 bg-sky-800"></div><div class="h-10 w-12 bg-sky-900"></div><div class="h-10 w-
12 bg-sky-950"></div>
</div>
```



THE SPACING SCALE (SIZE-2 TO SIZE-24) AND THE SKY PALETTE FROM 50 TO 950

1.4.3 No naming of classes

Naming is famously one of the hardest problems in programming. With utilities you never have to decide whether something is a `.card-wrapper-inner` or a `.card__body`; you describe what it looks like and move on.

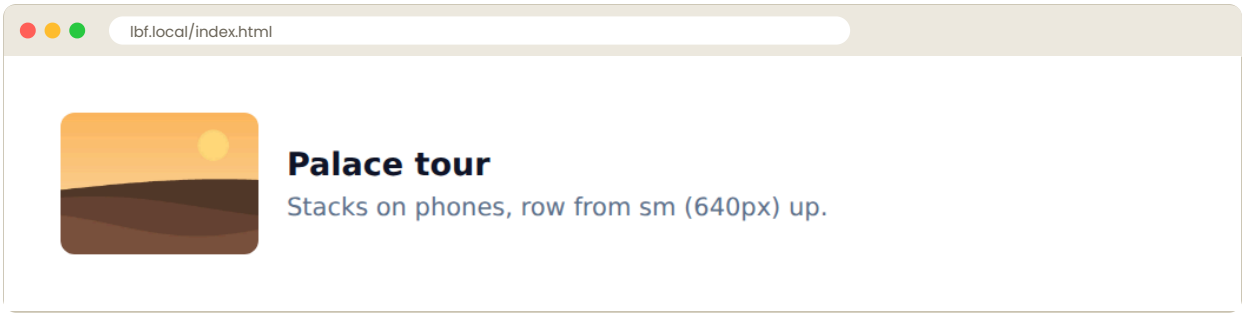
1.4.4 Small production CSS files

Because only used classes are generated and classes are reused across the whole site, the CSS stops growing after the first few pages: the hundredth page mostly reuses `flex`, `p-4`, `text-sm` and friends. Measured in 1.1.3: about 7 KB for a small page, minified, most of it the reset and theme variables.

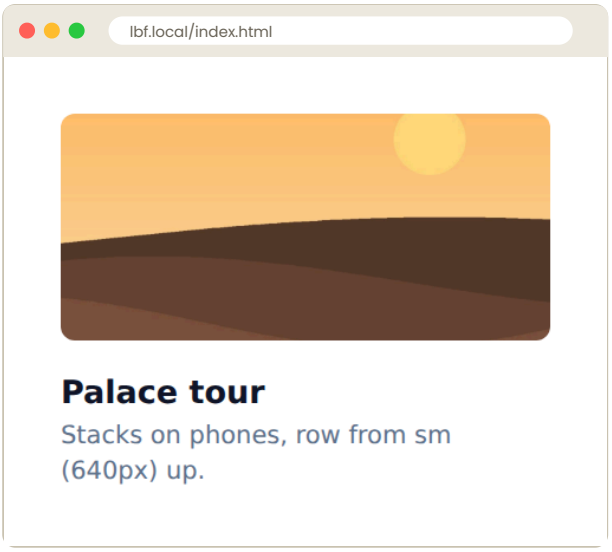
1.4.5 Easy responsive design and dark mode

RESPONSIVE AND DARK-MODE UTILITIES HTML

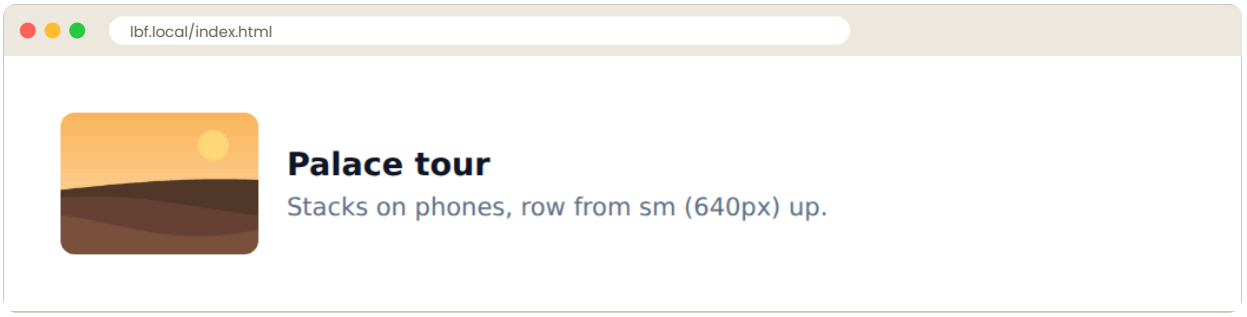
```
<div class="rounded-xl bg-white p-4 text-slate-900 dark:bg-slate-800 dark:text-white">
  <div class="flex flex-col gap-4 sm:flex-row sm:items-center">
    <img src="mysuru-palace.jpg" alt="" class="h-32 w-full rounded-lg object-cover sm:h-20 sm:w-28">
    <div>
      <h3 class="text-lg font-bold">Palace tour</h3>
      <p class="text-sm text-slate-500 dark:text-slate-300">Stacks on phones, row from sm (640px) up.
</p>
    </div>
  </div>
</div>
```



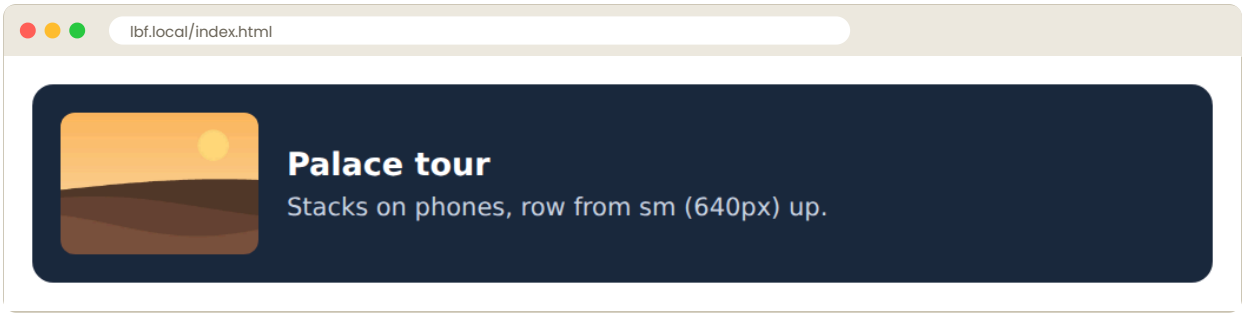
HOW THE BROWSER DISPLAYS IT



340 PX



700 PX. ONE CLASS LIST, TWO LAYOUTS



THE SAME CARD WITH THE SYSTEM SET TO DARK MODE (EMULATED IN CHROMIUM): THE DARK: UTILITIES APPLY

1.4.6 Works with every framework

Tailwind only needs to read your files and output a CSS file, so it works with plain HTML, server templates (PHP, Django, Rails, Thymeleaf), and every JavaScript framework: React, Next.js, Vue, Nuxt, Angular, Svelte, Astro, Solid, Remix, Laravel and more. Official integration guides exist for each (Module 2).

1.5 Where is Tailwind CSS used?

1.5.1 React, Next.js, Vue, Nuxt, Angular and Svelte projects

Tailwind is the default or recommended styling option in many framework starters: `create-next-app` offers it during setup, Laravel and Phoenix include it in their starter kits, and Vite, Nuxt, Astro and SvelteKit have first-class guides. In React and Vue codebases it is now the most common way to style components.

1.5.2 Landing pages, dashboards and SaaS products

Product and marketing sites (the sites of OpenAI, Shopify's developer docs, GitHub Next and many startups have used Tailwind), admin dashboards, SaaS applications and documentation sites benefit most: they need many custom, consistent, responsive screens built quickly.

1.5.3 Component libraries such as shadcn/ui and daisyUI

A large ecosystem builds on Tailwind (Module 15): **shadcn/ui** (copy-into-your-project React components built on Radix and Tailwind, extremely popular), **daisyUI** (a Tailwind plugin adding component classes such as `btn`, `card`), **Flowbite**, **Headless UI**, **Preline**, and the official **Tailwind Plus** templates.

1.6 Prerequisites recap

1.6.1 HTML structure and CSS concepts (box model, Flexbox, Grid, responsive design)

Every Tailwind class is a CSS declaration, so this course assumes the HTML5 and CSS3 courses (or equivalent knowledge). The table maps what you must already understand to where it appears in Tailwind:

CSS concept (CSS3 course module)	Tailwind utilities (this course module)
Box model, box-sizing (5)	`p-*`, `m-*`, `w-*`, `border-*`, `box-border` (3, 5, 7)
Units, rem, colours (4)	the spacing scale, `text-*`, the colour palette (3, 4)
Selectors, pseudo-classes (3, 9)	variants: `hover:`, `first:`, `group-hover:`, `has-*` (6)
Positioning, z-index (7)	`relative`, `absolute`, `inset-0`, `z-10` (7)
Flexbox, Grid (10, 11)	`flex`, `justify-between`, `grid-cols-3`, `col-span-2` (7, 8)
Media and container queries (12, 16)	`sm:`, `md:`, `@container`, `@md:` (9)
Custom properties, cascade layers (15, 16)	`@theme`, `bg-(--brand)`, layers in Tailwind (13, 14)
Transitions, animations (13, 14)	`transition`, `duration-300`, `animate-spin` (11)

PRACTICE CHECKPOINT

Explain in your own words what **"utility-first"** means and compare styling **one button** with custom CSS, Bootstrap and Tailwind.

	Custom CSS	Bootstrap	Tailwind
Markup	`<a class="btn-enrol">`	`<a class="btn btn-primary">`	`<a class="rounded-lg bg-indigo-600 px-4 py-2 font-semibold text-white hover:bg-indigo-700">`
Where styles live	your CSS file	Bootstrap's CSS (+ your overrides)	generated from the classes
Custom look	full control, more work	needs theming/overrides	full control, fast
CSS shipped	whatever you wrote	whole framework unless customised	only used utilities

	Custom CSS	Bootstrap	Tailwind
Hover/responsive	write selectors and media queries	built into components	variants in the class list

Model answer: utility-first means building designs by composing many small, single-purpose classes (each one CSS property with a value from a design scale) directly in the markup, rather than writing custom CSS for each component or using pre-built component classes. Styles stay next to the element they affect, states and breakpoints are expressed as prefixes, and the build generates only the CSS actually used.

MODULE 02 · BEGINNER

Setup and installation

Minimum duration **1 day** | Level **Beginner**

By the end of this module the learner can set up Tailwind CSS in a real project and use editor tooling. It covers every way to use Tailwind (the Play CDN, Tailwind Play, the Vite plugin, the Tailwind CLI, the PostCSS plugin and framework guides), setting up a Vite project step by step, how Tailwind works (automatic content detection and class scanning, why class names must be complete strings, Preflight), the IntelliSense extension and the Prettier plugin for class sorting, and how to read the documentation. The Tailwind CLI commands in this module were really run with Tailwind v4.1.13.

IN THIS MODULE

2.1 Ways to use Tailwind
2.2 Setting up a Vite project
2.3 How Tailwind works
2.4 Editor tooling
2.5 Reading the documentation

2.1 Ways to use Tailwind

Method	Best for	Needs Node.js	Production-ready
Play CDN (script tag)	quick experiments, demos, learning	no	no
Tailwind Play (website)	trying ideas, sharing snippets	no	no
Vite plugin `@tailwindcss/vite`	most new projects (React, Vue, Svelte, plain)	yes	yes (recommended)
Tailwind CLI `@tailwindcss/cli` or the standalone binary	plain HTML sites, any back end, no bundler	npm package: yes; standalone binary: no	yes
PostCSS plugin `@tailwindcss/postcss`	projects already using PostCSS (Next.js, Angular, webpack)	yes	yes
Framework guides	Next.js, Nuxt, Angular, SvelteKit, Laravel, Astro…	yes	yes

2.1.1 Play CDN for quick experiments

The **Play CDN** is a script that compiles Tailwind **in the browser**: add one script tag and start using classes in any HTML file, with no build step. It watches the page and generates CSS on the fly, and supports custom theme variables in a `<style type="text/tailwindcss">` block. It is slow to start, adds a large JavaScript file and depends on a third-party CDN, so it is for **development and learning only**, never for production.

INDEX.HTML WITH THE PLAY CDN

HTML

```html
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
  <style type="text/tailwindcss">
    @theme { --color-brand: #1c2b4a; }
  </style>
</head>
<body>
  <h1 class="p-8 text-3xl font-bold text-brand underline">Hello Tailwind!</h1>
</body>
</html>
```

2.1.2 Tailwind Play (online playground)

Tailwind Play (play.tailwindcss.com) is an official online editor: HTML on the left, live preview on the right, with autocomplete, hover previews of the generated CSS, a CSS tab for `@theme` customisation, device-size previews and shareable links. It is the fastest way to try a utility or send a reproducible example to a colleague or a forum.

2.1.3 Vite plugin (@tailwindcss/vite)

For new projects the recommended integration is the official **Vite plugin**, which is the fastest option and needs no PostCSS configuration. Section 2.2 walks through it.

VITE.CONFIG.JS

JavaScript

```javascript
// vite.config.js
import { defineConfig } from "vite";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [tailwindcss()],
});
```

2.1.4 Tailwind CLI

The **CLI** reads an input CSS file, scans your project for classes and writes an output CSS file; `--watch` rebuilds on every change and `--minify` compresses. It is available as an npm package (`npx @tailwindcss/cli`) and as a **standalone executable** (a single file downloaded from GitHub releases, no Node.js needed), which is what built every example in this book. It really ran on this small project:

SRC/INPUT.CSS

text

```
@import "tailwindcss";
```

INDEX.HTML HTML

```html
<!doctype html>
<html>
<head><link href="./dist/output.css" rel="stylesheet"></head>
<body>
  <h1 class="p-8 text-3xl font-bold underline">Hello Tailwind!</h1>
</body>
</html>
```

BUILDING WITH THE TAILWIND CLI terminal

```
tailwindcss --help | head -1
tailwindcss -i src/input.css -o dist/output.css
wc -c dist/output.css
grep -A2 "\.text-3xl {" dist/output.css
```

TERMINAL

```
$ tailwindcss --help | head -1
≈ tailwindcss v4.1.13
$ tailwindcss -i src/input.css -o dist/output.css
≈ tailwindcss v4.1.13

Done in 39ms
$ wc -c dist/output.css
5239 dist/output.css
$ grep -A2 "\.text-3xl {" dist/output.css
  .text-3xl {
    font-size: var(--text-3xl);
    line-height: var(--tw-leading, var(--text-3xl--line-height));
```

The npm equivalents: `npm install tailwindcss @tailwindcss/cli`, then `npx @tailwindcss/cli -i ./src/input.css -o ./dist/output.css --watch` during development and `--minify` for production.

2.1.5 PostCSS plugin

If a project already runs PostCSS (Next.js, Angular, older webpack set-ups), install `@tailwindcss/postcss` and add it to the PostCSS configuration. In Tailwind 4 you no longer need `autoprefixer` or `postcss-import`: Tailwind handles imports and vendor prefixes itself (it integrates Lightning CSS).

POSTCSS.CONFIG.MJS JavaScript

```javascript
// postcss.config.mjs
export default {
  plugins: {
    "@tailwindcss/postcss": {},
  },
};
```

2.1.6 Framework guides (Next.js, Vite + React, Angular, Nuxt)

Framework	Integration	CSS entry file
Next.js	`create-next-app` offers Tailwind; otherwise `@tailwindcss/postcss`	`app/globals.css`
Vite + React / Vue / Svelte	`@tailwindcss/vite`	`src/index.css` or `src/style.css`

Framework	Integration	CSS entry file
Angular	`@tailwindcss/postcss` with a `.postcssrc.json`	`src/styles.css`
Nuxt	`@tailwindcss/vite` in `nuxt.config.ts` (vite.plugins)	`assets/css/main.css`
Laravel	starter kits include it; `@tailwindcss/vite`	`resources/css/app.css`
SvelteKit, Astro, Remix	official guides at tailwindcss.com/docs/installation/framework-guides	varies

In every case the CSS entry file contains `@import "tailwindcss";`, and the framework's root layout or entry imports that CSS file.

2.2 Setting up a Vite project

2.2.1 Creating the project and installing Tailwind

CREATE THE PROJECT AND INSTALL TAILWIND terminal

```
npm create vite@latest lbf-tailwind -- --template vanilla
cd lbf-tailwind
npm install tailwindcss @tailwindcss/vite
```

TERMINAL (REPRESENTATIVE)

```
Scaffolding project in /home/asha/lbf-tailwind...

Done. Now run:
  cd lbf-tailwind
  npm install
  npm run dev

added 31 packages, and audited 32 packages in 4s
found 0 vulnerabilities
```

Then register the plugin in `vite.config.js` (shown in 2.1.3). The project structure:

PROJECT STRUCTURE text

```
lbf-tailwind/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.js        // imports "./style.css"
    └── style.css      // @import "tailwindcss";
```

2.2.2 Adding @import "tailwindcss"; to the CSS file

Replace the contents of `src/style.css` with a single line. That import brings in the theme variables, Preflight (the base reset) and all utilities, each in its own cascade layer, and is where you will later add `@theme` customisations.

SRC/STYLE.CSS CSS

```
@import "tailwindcss";
```

What that one line expands to, from the real compiled output (opening lines only):

THE START OF THE GENERATED FILE style.css

```
@import "tailwindcss";
```

COMPILED CSS (TAILWIND v4.1.13)

```
/*! tailwindcss v4.1.13 | MIT License | https://tailwindcss.com */
@layer properties;
@layer theme, base, components, utilities;
@layer theme {
  :root, :host {
    --font-sans: ui-sans-serif, system-ui, sans-serif, 'Apple Color Emoji', 'Segoe UI Emoji', 'Segoe
UI Symbol',
    'Noto Color Emoji';
    --font-mono: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, 'Liberation Mono', 'Courier
New',
    monospace;
    --text-3xl: 1.875rem;
    --text-3xl--line-height: calc(2.25 / 1.875);
    --font-weight-bold: 700;
    --default-font-family: var(--font-sans);
    --default-mono-font-family: var(--font-mono);
  ... (about 250 theme variables) ...
}
@layer base {
  *, ::after, ::before, ::backdrop, ::file-selector-button {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
    border: 0 solid;
  }
  ...
```

Note the first line, `@layer theme, base, components, utilities;`: Tailwind declares cascade layers (CSS3 course, Module 16) so utilities always override base styles and component classes, whatever their order or specificity.

2.2.3 Running the dev server and production build

INDEX.HTML HTML

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>LBF Tailwind</title>
  <script type="module" src="/src/main.js"></script>
</head>
<body class="bg-slate-50">
  <h1 class="p-8 text-3xl font-bold text-indigo-700 underline">Hello Tailwind!</h1>
</body>
</html>
```

DEVELOPMENT SERVER terminal

```
npm run dev
```

TERMINAL (REPRESENTATIVE)

```
  VITE v7.1.5  ready in 286 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

Edit a class in index.html and save: Vite's hot module replacement updates the page instantly. For deployment, `npm run build` writes optimised, minified, hashed files to `dist/`:

PRODUCTION BUILD terminal

```
npm run build
```

TERMINAL (REPRESENTATIVE)

```
vite v7.1.5 building for production...
✓ 4 modules transformed.
dist/index.html                  0.46 kB │ gzip: 0.30 kB
dist/assets/index-C9x2pQ1a.css   5.62 kB │ gzip: 1.71 kB
dist/assets/index-BqT4nM8s.js    0.71 kB │ gzip: 0.40 kB
✓ built in 198ms
```

INDEX.HTML HTML

```
<h1 class="p-8 text-3xl font-bold text-indigo-700 underline">Hello Tailwind!</h1>
```

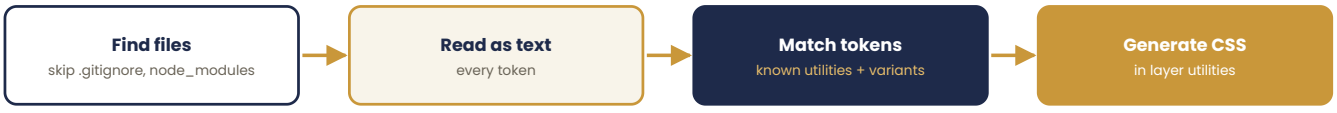


THE PAGE IN THE BROWSER

2.3 How Tailwind works

2.3.1 Automatic content detection and class scanning

Tailwind 4 finds your source files automatically: it scans the project from the working directory, **ignoring** files listed in `.gitignore`, `node_modules`, binary files, CSS files and lock files. It reads each file as **plain text**, without understanding the language, and picks out every token that **could** be a class name (sequences of letters, numbers, hyphens, colons, brackets…). For each token that matches a utility or variant it knows, it generates the CSS. If your templates live somewhere unusual (a package in node_modules, a separate folder), add them with `@source` (Module 14).



TAILWIND'S SCANNING PROCESS

Because scanning is text-based, classes are found in any file type, even in strings and comments. This compile used a JavaScript file and a Markdown file as the only sources:

INPUT.CSS text

```
@import "tailwindcss";
```

APP.JS JavaScript

```
const badge = `<span class="bg-emerald-500">OK</span>`;
```

NOTES.MD text

```
Remember to use text-rose-600 for errors and p-7 for cards.
```

CLASSES FOUND IN A .JS FILE AND A .MD FILE terminal

```
tailwindcss -i input.css -o out.css 2>/dev/null
grep -o "\.[a-z0-9-]*\(bg-emerald-500\|text-rose-600\|p-7\)" out.css | sort -u
```

TERMINAL

```
$ tailwindcss -i input.css -o out.css 2>/dev/null
$ grep -o "\.[a-z0-9-]*\(bg-emerald-500\|text-rose-600\|p-7\)" out.css | sort -u
.bg-emerald-500
.p-7
.text-rose-600
```

2.3.2 Why class names must be complete strings (no dynamic string building)

Because Tailwind only sees the **source text**, it cannot run your code. A class built by concatenation at run time, such as `` `text-${color}-600` ``, never appears in full anywhere in the source, so its CSS is **never generated**, even though the browser later receives a correct-looking class name. Always write complete class names, and map values to them:

DYNAMIC VS COMPLETE CLASS NAMES JavaScript

```
// ✗ Broken: Tailwind never sees "text-red-600" or "text-green-600"
const cls = `text-${status === "error" ? "red" : "green"}-600`;

// ✓ Works: the full class names appear in the source
const colours = { error: "text-red-600", success: "text-green-600" };
const cls2 = colours[status];
```

INPUT.CSS text

```
@import "tailwindcss";
```

BROKEN.JS JavaScript

```
const cls = `text-${status === "error" ? "red" : "green"}-600`;
```

FIXED.JS JavaScript

```
const colours = { info: "text-sky-600" };
```

PROOF: WHICH CLASSES WERE GENERATED

terminal

```
tailwindcss -i input.css -o out.css 2>/dev/null
echo "text-red-600   generated: $(grep -c '\.text-red-600 ' out.css)"
echo "text-green-600 generated: $(grep -c '\.text-green-600 ' out.css)"
echo "text-sky-600   generated: $(grep -c '\.text-sky-600 ' out.css)"
```

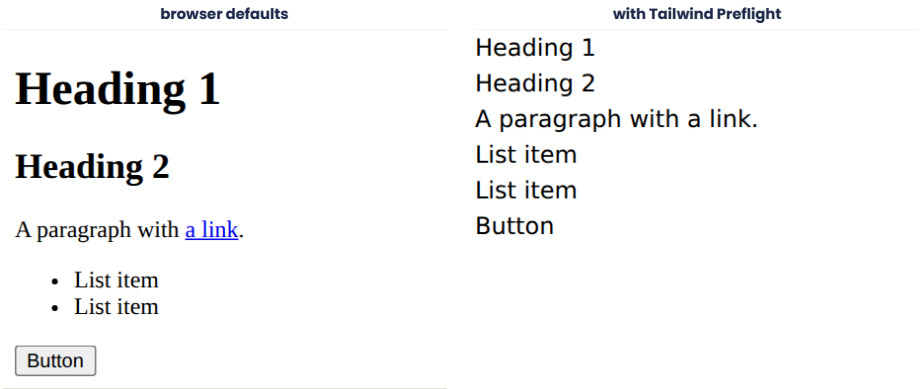
TERMINAL

```
$ tailwindcss -i input.css -o out.css 2>/dev/null
$ echo "text-red-600   generated: $(grep -c '\.text-red-600 ' out.css)"
text-red-600   generated: 0
$ echo "text-green-600 generated: $(grep -c '\.text-green-600 ' out.css)"
text-green-600 generated: 0
$ echo "text-sky-600   generated: $(grep -c '\.text-sky-600 ' out.css)"
text-sky-600   generated: 1
```

Only `text-sky-600`, written in full, was generated; the dynamically built red and green classes produced no CSS at all. Module 15 shows the patterns frameworks use (lookup objects, `clsx`, `cva`).

2.3.3 Preflight: Tailwind's base reset

Preflight is Tailwind's opinionated reset, in the `base` layer (built on modern-normalize). It removes all default margins, makes headings and text the same size and weight as normal text, removes list bullets, makes images block-level and responsive, sets `box-sizing: border-box` everywhere, resets borders to `0 solid` so that just adding `border` works, and makes form controls inherit fonts. The result: an unstyled page looks plain until you add utilities, and nothing is styled by surprise.



PREFLIGHT REMOVES DEFAULT MARGINS, SIZES, BULLETS AND BUTTON STYLES

COMPILED CSS: PREFLIGHT

```
@layer base {
  *, ::after, ::before, ::backdrop, ::file-selector-button {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
    border: 0 solid;
  }
  html, :host {
    line-height: 1.5;
    -webkit-text-size-adjust: 100%;
    tab-size: 4;
    font-family: var(--default-font-family, ui-sans-serif, system-ui, sans-serif, 'Apple Color Emoji',
'Segoe UI Emoji', 'Segoe UI Symbol', 'Noto Color Emoji');
    font-feature-settings: var(--default-font-feature-settings, normal);
    font-variation-settings: var(--default-font-variation-settings, normal);
    -webkit-tap-highlight-color: transparent;
  }
  hr {
    height: 0;
    color: inherit;
    border-top-width: 1px;
  }
  abbr:where([title]) {
    -webkit-text-decoration: underline dotted;
    text-decoration: underline dotted;
  }
  h1, h2, h3, h4, h5, h6 {
    font-size: inherit;
    font-weight: inherit;
  }
  a {
    color: inherit;
    -webkit-text-decoration: inherit;
    text-decoration: inherit;
  }
  b, strong {
    font-weight: bolder;
  }
  code, kbd, samp, pre {
    font-family: var(--default-mono-font-family, ui-monospace, SFMono-Regular, Menlo, Monaco,
Consolas, 'Liberation Mono', 'Courier New', monospace);
    font-feature-settings: var(--default-mono-font-feature-settings, normal);
    font-variation-settings: var(--default-mono-font-variation-settings, normal);
  ...
}
```

The first 40 lines of the real base layer.

WATCH OUT

Because Preflight removes list styles and heading sizes, content written by users or a CMS (a blog post body) looks unstyled. Style it with the Typography plugin's `prose` class (Module 4) or explicit utilities; `list-disc` restores bullets. If you are adding Tailwind to an existing site, you can import only the theme and utilities layers without Preflight.

SKIPPING PREFLIGHT CSS

```
/* Tailwind without Preflight (for adding to an existing site) */
@layer theme, base, components, utilities;
@import "tailwindcss/theme.css" layer(theme);
@import "tailwindcss/utilities.css" layer(utilities);
```

2.4 Editor tooling

2.4.1 Tailwind CSS IntelliSense extension for VS Code

Install **Tailwind CSS IntelliSense** (by Tailwind Labs) from the VS Code Marketplace. It reads your project's CSS entry file and theme, then provides: **autocomplete** of every class and variant as you type inside `class=""`, **hover previews** showing the exact generated CSS, **colour swatches** next to colour classes, and **linting** that warns about conflicting classes (`p-4 p-6`) or invalid `@apply` usage. Useful settings: enable `editor.quickSuggestions` for strings, and add `tailwindCSS.classFunctions` for helper functions such as `cva` and `clsx`.



INTELLISENSE: COMPLETIONS WITH COLOUR SWATCHES AND A CSS PREVIEW (SCHEMATIC)

2.4.2 Prettier plugin for automatic class sorting

Long class lists are easier to read when every file orders them the same way. The official **prettier-plugin-tailwindcss** sorts classes in the same order Tailwind outputs them in CSS: base and layout classes first (positioning, display, box model), then decoration, then variants such as `hover:` and `md:` at the end. It removes the "how should I order these?" debate in code review.

INSTALL terminal

```
npm install -D prettier prettier-plugin-tailwindcss
```

TERMINAL (REPRESENTATIVE)

```
added 2 packages in 2s
```

.PRETTIERRC text

```
{
  "plugins": ["prettier-plugin-tailwindcss"],
  "tailwindStylesheet": "./src/style.css"
}
```

BEFORE AND AFTER "NPX PRETTIER --WRITE" index.html

```
<button class="text-white hover:bg-indigo-700 px-4 md:px-6 rounded-lg py-2 bg-indigo-600 font-semibold">Enrol</button>
```

AFTER SORTING (REPRESENTATIVE)

```
<button class="rounded-lg bg-indigo-600 px-4 py-2 font-semibold text-white hover:bg-indigo-700 md:px-6">Enrol</button>
```

2.5 Reading the documentation

2.5.1 Searching utilities in the official docs

The documentation at **tailwindcss.com/docs** is excellent and organised like CSS itself: Layout, Flexbox and Grid, Spacing, Sizing, Typography, Backgrounds, Borders, Effects, Filters, Tables, Transitions and Animation, Transforms, Interactivity, SVG and Accessibility. Press **Ctrl+K** (Cmd+K) and type either the **CSS property** you want (`letter-spacing`) or a class you half remember (`tracking`). Each utility page shows a table of every class and the CSS it generates, interactive examples, and sections on responsive and state variants and customising the theme. The "Core concepts" pages (utility classes, hover and focus, responsive design, dark mode, theme variables) are worth reading once from start to finish.

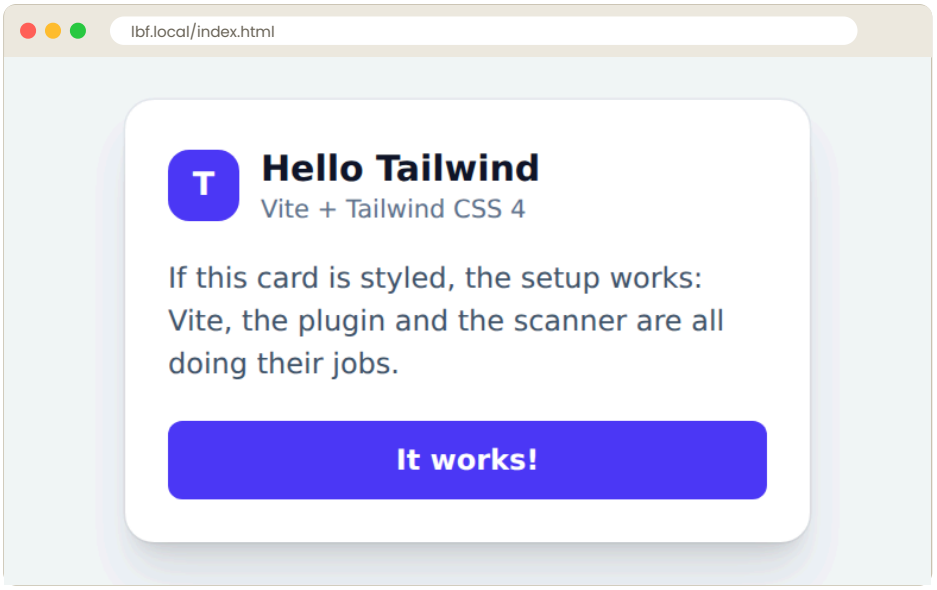
I want...	Search for	Finds
space between letters	letter-spacing	`tracking-*`
line height	line-height	`leading-*`, or `text-lg/7`
to stop an item shrinking	flex-shrink	`shrink-0`
a box that keeps its shape	aspect-ratio	`aspect-video`, `aspect-square`
to hide visually but keep for screen readers	screen readers	`sr-only`
to style the parent when a child is checked	has	`has-checked:`

PRACTICE CHECKPOINT

Create a **Vite project with Tailwind**, install **IntelliSense** and the **Prettier plugin**, and build a **"Hello Tailwind" card**.

INDEX.HTML (BODY): SAMPLE SOLUTION HTML

```
<div class="flex min-h-[280px] items-center justify-center bg-slate-100 p-6">
  <div class="w-full max-w-sm rounded-2xl bg-white p-6 shadow-xl ring-1 ring-slate-900/5">
    <div class="flex items-center gap-3">
      <span class="grid size-10 place-items-center rounded-xl bg-indigo-600 text-lg font-bold text-
white">T</span>
      <div>
        <h1 class="text-xl font-bold text-slate-900">Hello Tailwind</h1>
        <p class="text-sm text-slate-500">Vite + Tailwind CSS 4</p>
      </div>
    </div>
    <p class="mt-4 text-slate-600">If this card is styled, the setup works: Vite, the plugin and the
scanner are all doing their jobs.</p>
    <button class="mt-5 w-full rounded-lg bg-indigo-600 py-2.5 font-semibold text-white hover:bg-
indigo-700">It works!</button>
  </div>
</div>
```



THE "HELLO TAILWIND" CARD

MODULE 03 · BEGINNER

Core utilities: spacing, sizing and colours

Minimum duration **2 days** | Level **Beginner**

This module covers the building blocks used on nearly every element: spacing, sizing and colour. It explains the spacing scale and how numbers map to rem values, arbitrary values, arbitrary properties and the CSS-variable shorthand; padding, margin (including auto and negative margins) and space between children; width, height, size, minimum and maximum sizes and aspect ratios; and the default colour palette with shades 50 to 950, text, background and border colours, the opacity modifier, opacity utilities and how to choose accessible colour combinations. Every class shown was compiled with Tailwind v4.1.13 and its generated CSS is quoted from the compiler.

IN THIS MODULE

3.1 The design scale
3.2 Spacing
3.3 Sizing
3.4 Colours

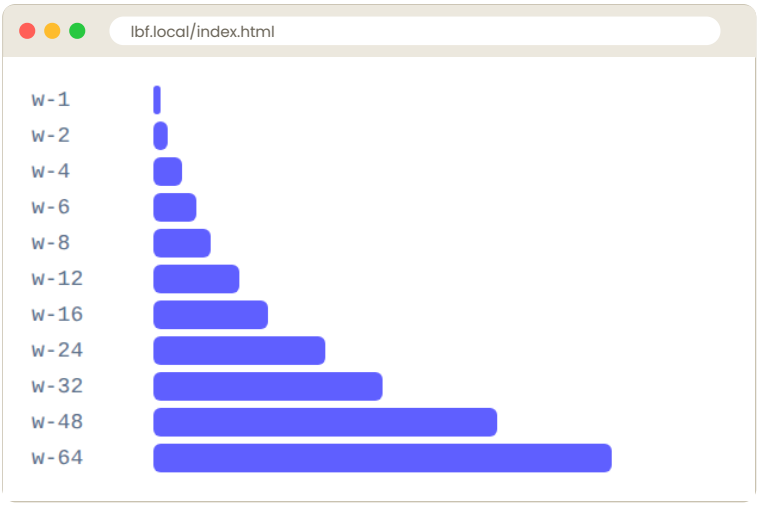
3.1 The design scale

3.1.1 Spacing scale and how numbers map to rem values

In Tailwind 4 every spacing and sizing utility (padding, margin, gap, width, height, inset, translate…) is calculated from **one theme variable**, `--spacing: 0.25rem`. The number in the class is a multiplier: `p-4` = `calc(var(--spacing) * 4)` = 1rem = 16px at the default root size. Any number works, including decimals in steps of 0.5 (`p-2.5`, `mt-13`, `w-97`), which is a change from v3, where only predefined steps existed. `px` means 1px and `0` means zero.

Class number	Calculation	rem	px (16px root)
0	0 × 0.25rem	0rem	0px
0.5	0.5 × 0.25rem	0.125rem	2px
1	1 × 0.25rem	0.25rem	4px
1.5	1.5 × 0.25rem	0.375rem	6px
2	2 × 0.25rem	0.5rem	8px
3	3 × 0.25rem	0.75rem	12px
4	4 × 0.25rem	1rem	16px
5	5 × 0.25rem	1.25rem	20px
6	6 × 0.25rem	1.5rem	24px
8	8 × 0.25rem	2rem	32px

Class number	Calculation	rem	px (16px root)
10	10 × 0.25rem	2.5rem	40px
12	12 × 0.25rem	3rem	48px
16	16 × 0.25rem	4rem	64px
20	20 × 0.25rem	5rem	80px
24	24 × 0.25rem	6rem	96px
32	32 × 0.25rem	8rem	128px



WIDTHS FROM THE SAME SCALE, RENDERED

COMPILED CSS

```
.mt-2\.5 {
  margin-top: calc(var(--spacing) * 2.5);
}
.h-px {
  height: 1px;
}
.w-13 {
  width: calc(var(--spacing) * 13);
}
.gap-0 {
  gap: calc(var(--spacing) * 0);
}
.p-4 {
  padding: calc(var(--spacing) * 4);
}
```

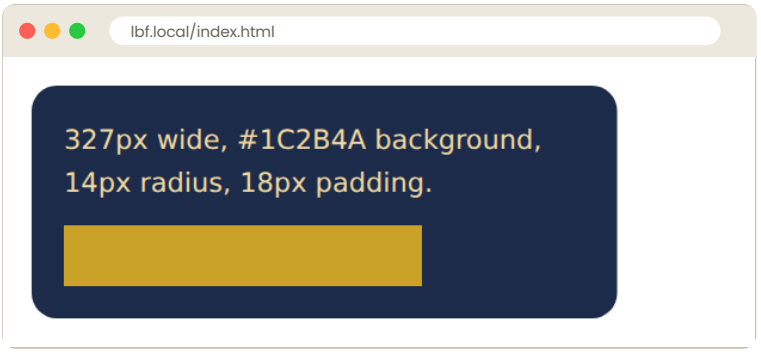
3.1.2 Arbitrary values w-[327px], bg-[#1C2B4A]

When a design needs a value that is not on the scale (a pixel-exact width from a mock-up, a brand colour), put it in **square brackets**. Any valid CSS value works, including `calc()`, `clamp()`, `var()` and URLs. Spaces are written as **underscores** (`grid-cols-[1fr_2fr]`); a real underscore is escaped as `\_`. When a value could be ambiguous (is `[var(--x)]` a colour or a length?), add a type hint: `text-(length:--x)` or `bg-[color:var(--x)]`.

ARBITRARY VALUES

HTML

```html
<div class="w-[327px] rounded-[14px] bg-[#1C2B4A] p-[18px] text-[#F0D9A0]">
  <p class="text-[15px] leading-[1.6]">327px wide, #1C2B4A background, 14px radius, 18px padding.</p>
  <div class="mt-3 h-[calc(2rem+2px)] w-[min(100%,200px)] bg-[#C9A227]"></div>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```css
.h-\[calc\(2rem\+2px\)\] {
  height: calc(2rem + 2px);
}
.w-\[327px\] {
  width: 327px;
}
.w-\[min\(100\%\,200px\)\] {
  width: min(100%, 200px);
}
.grid-cols-\[1fr_2fr\] {
  grid-template-columns: 1fr 2fr;
}
.rounded-\[14px\] {
  border-radius: 14px;
}
.bg-\[\#1C2B4A\] {
  background-color: #1C2B4A;
}
.p-\[18px\] {
  padding: 18px;
}
.text-\[\#F0D9A0\] {
  color: #F0D9A0;
}
```

TIP

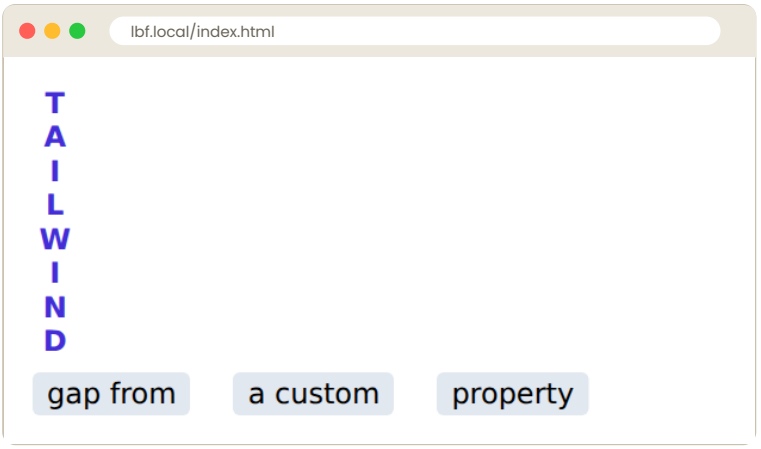
Use arbitrary values sparingly. If the same arbitrary value appears more than two or three times, it belongs in the theme as a token (Module 13): `bg-[#1C2B4A]` repeated everywhere should become `bg-navy`.

3.1.3 Arbitrary properties [mask-type:luminance]

For a CSS property Tailwind has no utility for, write the whole declaration in brackets: `[property:value]`. It still works with variants (`hover:[mask-type:alpha]`, `md:[writing-mode:vertical-rl]`), so you never need to leave the class list for a one-off property. It is also handy for setting custom properties: `[--card-gap:12px]`.

ARBITRARY PROPERTIES HTML

```
<p class="[writing-mode:vertical-rl] [text-orientation:upright] font-bold text-indigo-700">TAILWIND</p>
<div class="mt-2 [--gap:1.5rem] flex gap-(--gap)"><span class="rounded bg-slate-200 px-2">gap from</span><span class="rounded bg-slate-200 px-2">a custom</span><span class="rounded bg-slate-200 px-2">property</span></div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.gap-\(--gap\) {
  gap: var(--gap);
}
.\[--gap\:1\.5rem\] {
  --gap: 1.5rem;
}
.\[text-orientation\:upright\] {
  text-orientation: upright;
}
.\[writing-mode\:vertical-rl\] {
  writing-mode: vertical-rl;
}
```

3.1.4 CSS variable shorthand bg-(--brand)

Tailwind 4 has a shorthand for using a CSS custom property as a value: **parentheses** with the variable name, `bg-(--brand)`, which generates `background-color: var(--brand)`. (In v3 the equivalent was `bg-[--brand]`.) It connects utilities to values set by JavaScript, a CMS or a parent element, for example a per-course colour coming from the database.

ONE CLASS, THREE COLOURS FROM INLINE VARIABLES HTML

```
<div class="flex gap-3">
  <div style="--brand: #1c2b4a" class="rounded-lg bg-(--brand) px-4 py-3 text-white">HTML5</div>
  <div style="--brand: #c9a227" class="rounded-lg bg-(--brand) px-4 py-3 text-white">CSS3</div>
  <div style="--brand: #0e7490" class="rounded-lg bg-(--brand) px-4 py-3 text-white">Tailwind</div>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.bg-\(--brand\) {
  background-color: var(--brand);
}
```

3.2 Spacing

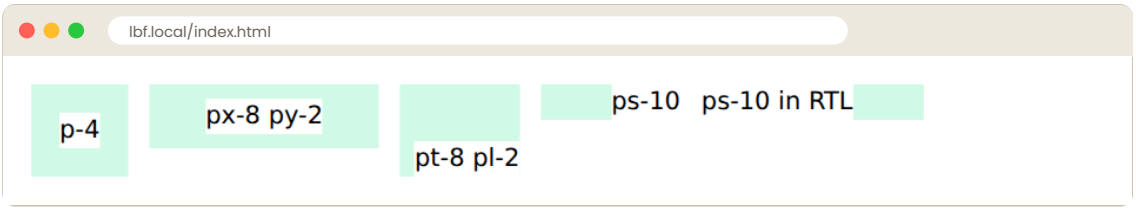
3.2.1 Padding: p-*, px-*, py-*, pt-*, ps-*, pe-*

Class	CSS property	Class	CSS property
`p-4`	padding	`pt-4`	padding-top
`px-4`	padding-inline (left + right)	`pr-4`	padding-right
`py-4`	padding-block (top + bottom)	`pb-4`	padding-bottom
`ps-4`	padding-inline-start (logical)	`pl-4`	padding-left
`pe-4`	padding-inline-end (logical)	`p-[5px]`	arbitrary

PADDING UTILITIES (GREEN = PADDING)

HTML

```
<div class="flex flex-wrap items-start gap-3 text-sm">
  <div class="bg-emerald-100"><div class="p-4 bg-emerald-300/0"><div class="bg-white">p-4</div></div>
</div>
  <div class="bg-emerald-100"><div class="px-8 py-2"><div class="bg-white">px-8 py-2</div></div></div>
  <div class="bg-emerald-100"><div class="pt-8 pl-2"><div class="bg-white">pt-8 pl-2</div></div></div>
  <div class="bg-emerald-100"><div class="ps-10"><div class="bg-white">ps-10</div></div></div>
  <div dir="rtl" class="bg-emerald-100"><div class="ps-10"><div class="bg-white">ps-10 in RTL</div>
</div></div>
</div>
```



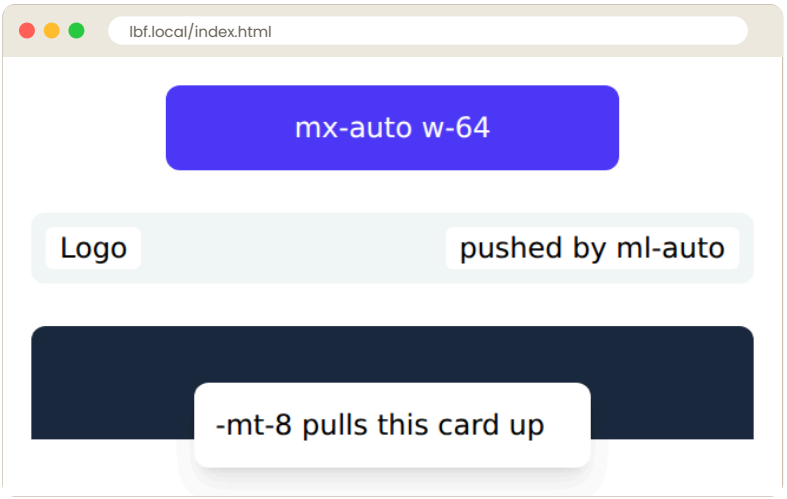
PS-* FOLLOWS THE WRITING DIRECTION: IN RIGHT-TO-LEFT TEXT THE PADDING MOVES TO THE RIGHT

3.2.2 Margin: m-*, mx-auto, negative margins -mt-4

Margin utilities mirror padding (`m-`, `mx-`, `my-`, `mt-`, `ms-`…), plus `auto` (`mx-auto` centres a block with a width, `ml-auto` pushes a flex item to the right) and **negative** values with a leading minus (`-mt-4`, `-mx-2`).

AUTO AND NEGATIVE MARGINS HTML

```
<div class="mx-auto w-64 rounded-lg bg-indigo-600 p-3 text-center text-white">mx-auto w-64</div>
<div class="mt-6 flex rounded-lg bg-slate-100 p-2"><span class="rounded bg-white px-2">Logo</span>
<span class="ml-auto rounded bg-white px-2">pushed by ml-auto</span></div>
<div class="mt-6 h-16 rounded-t-lg bg-slate-800"></div>
<div class="relative mx-auto -mt-8 w-56 rounded-lg bg-white p-3 shadow-lg">-mt-8 pulls this card
up</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

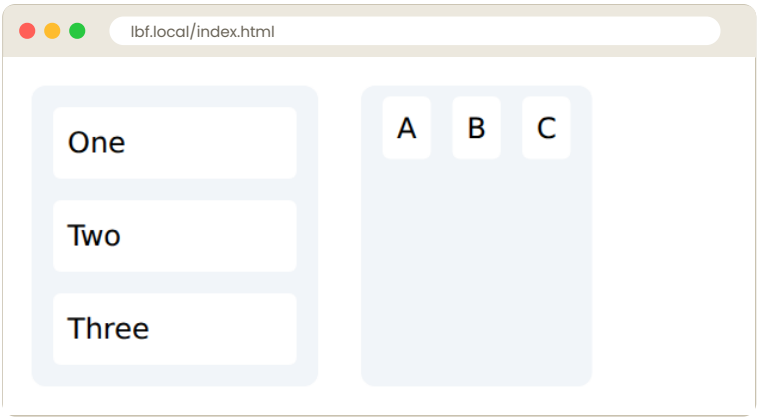
```
.-mx-2 {
  margin-inline: calc(var(--spacing) * -2);
}
.mx-auto {
  margin-inline: auto;
}
.my-4 {
  margin-block: calc(var(--spacing) * 4);
}
.-mt-8 {
  margin-top: calc(var(--spacing) * -8);
}
.ml-auto {
  margin-left: auto;
}
```

3.2.3 Space between children: space-x-*, space-y-*

`space-y-4` adds a top margin to every child except the first, putting equal space **between** stacked items; `space-x-*` does the same horizontally. In Tailwind 4 it uses `:where(& > :not(:last-child))` with a bottom/right margin. For flex and grid containers, prefer `gap-*` (Module 7), which handles wrapping and does not depend on margins.

SPACE-Y-3 AND SPACE-X-3 HTML

```
<div class="flex gap-6">
  <div class="w-40 space-y-3 rounded-lg bg-slate-100 p-3"><div class="rounded bg-white p-2">One</div>
<div class="rounded bg-white p-2">Two</div><div class="rounded bg-white p-2">Three</div></div>
  <div class="space-x-3 rounded-lg bg-slate-100 p-3"><span class="rounded bg-white p-2">A</span><span
class="rounded bg-white p-2">B</span><span class="rounded bg-white p-2">C</span></div>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.space-y-3 {
  :where(& > :not(:last-child)) {
    --tw-space-y-reverse: 0;
    margin-block-start: calc(calc(var(--spacing) * 3) * var(--tw-space-y-reverse));
    margin-block-end: calc(calc(var(--spacing) * 3) * calc(1 - var(--tw-space-y-reverse)));
  }
}
```

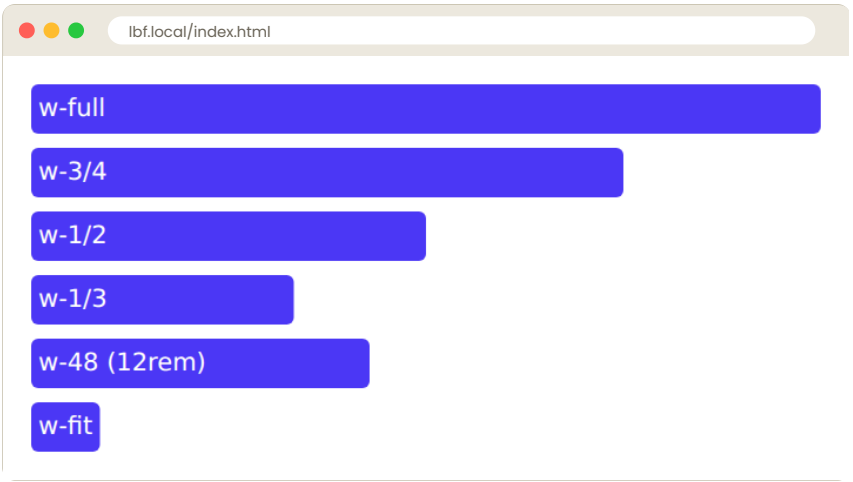
3.3 Sizing

3.3.1 Width: w-*, w-full, w-screen, w-1/2, w-fit

Class	CSS	Use
`w-64`	width: 16rem (scale × 0.25rem)	fixed widths
`w-1/2`, `w-2/3`, `w-3/4`	width: calc(1/2 * 100%)	fractions of the parent
`w-full`	width: 100%	fill the parent
`w-screen`	width: 100vw	full viewport width
`w-auto`	width: auto	reset
`w-fit`, `w-min`, `w-max`	fit-content, min-content, max-content	intrinsic sizing (CSS3 Module 5)
`w-sm`, `w-md` … `w-7xl`	width: var(--container-sm) = 24rem …	container sizes
`w-dvw`, `w-svw`, `w-lvw`	dynamic/small/large viewport width	mobile viewports

WIDTH UTILITIES — HTML

```
<div class="space-y-2 text-sm text-white">
  <div class="w-full rounded bg-indigo-600 p-1">w-full</div>
  <div class="w-3/4 rounded bg-indigo-600 p-1">w-3/4</div>
  <div class="w-1/2 rounded bg-indigo-600 p-1">w-1/2</div>
  <div class="w-1/3 rounded bg-indigo-600 p-1">w-1/3</div>
  <div class="w-48 rounded bg-indigo-600 p-1">w-48 (12rem)</div>
  <div class="w-fit rounded bg-indigo-600 p-1">w-fit</div>
</div>
```



HOW THE BROWSER DISPLAYS IT

3.3.2 Height: h-*, h-screen, h-dvh

Height utilities mirror width: `h-12`, `h-full` (100% of a parent with a definite height), `h-screen` (100vh), `h-dvh` / `h-svh` / `h-lvh` (the dynamic, small and large viewport heights that fix mobile toolbar problems, CSS3 Module 4), `h-fit`, `h-auto`. For full-screen hero sections prefer `min-h-dvh` so content can still grow.

COMPILED CSS

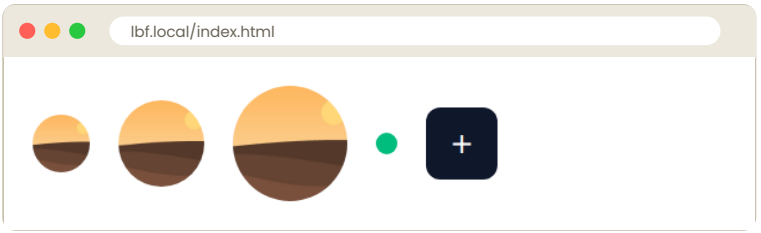
```
.h-12 {
  height: calc(var(--spacing) * 12);
}
.h-dvh {
  height: 100dvh;
}
.h-full {
  height: 100%;
}
.h-screen {
  height: 100vh;
}
.h-svh {
  height: 100svh;
}
.min-h-dvh {
  min-height: 100dvh;
}
```

3.3.3 size-* for width and height together

`size-*` sets width and height to the same value in one class: icons (`size-5`), avatars (`size-12`), square buttons (`size-10`), dots (`size-2`).

SIZE-* FOR AVATARS, A STATUS DOT AND A SQUARE BUTTON — HTML

```
<div class="flex items-center gap-4">
  <img src="mysuru-palace.jpg" alt="" class="size-8 rounded-full object-cover">
  <img src="mysuru-palace.jpg" alt="" class="size-12 rounded-full object-cover">
  <img src="mysuru-palace.jpg" alt="" class="size-16 rounded-full object-cover">
  <span class="size-3 rounded-full bg-emerald-500"></span>
  <button class="grid size-10 place-items-center rounded-lg bg-slate-900 text-white">+</button>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.size-12 {
  width: calc(var(--spacing) * 12);
  height: calc(var(--spacing) * 12);
}
```

3.3.4 Min and max: min-w-*, max-w-* (max-w-md, max-w-7xl), min-h-screen

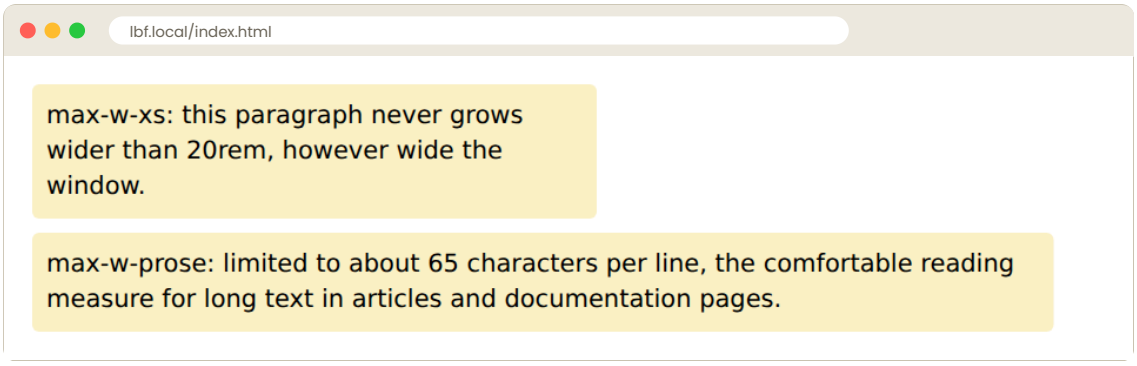
`max-w-*` uses **container sizes** from `max-w-3xs` (16rem) to `max-w-7xl` (80rem), plus `max-w-prose` (65ch, ideal line length) and `max-w-full`. The classic centred page container is `mx-auto max-w-7xl px-4`. `min-h-screen` or `min-h-dvh` make a section at least the height of the viewport; `min-w-0` lets flex items shrink below their content (CSS3 Module 10).

max-w-*	Value	max-w-*	Value
3xs	16rem (256px)	xl	36rem (576px)
2xs	18rem (288px)	2xl	42rem (672px)
xs	20rem (320px)	3xl	48rem (768px)
sm	24rem (384px)	4xl	56rem (896px)
md	28rem (448px)	5xl	64rem (1024px)
lg	32rem (512px)	7xl	80rem (1280px)

MAX-W-XS AND MAX-W-PROSE

HTML

```
<div class="space-y-2 text-sm">
  <p class="max-w-xs rounded bg-amber-100 p-2">max-w-xs: this paragraph never grows wider than 20rem, however wide the window.</p>
  <p class="max-w-prose rounded bg-amber-100 p-2">max-w-prose: limited to about 65 characters per line, the comfortable reading measure for long text in articles and documentation pages.</p>
</div>
```



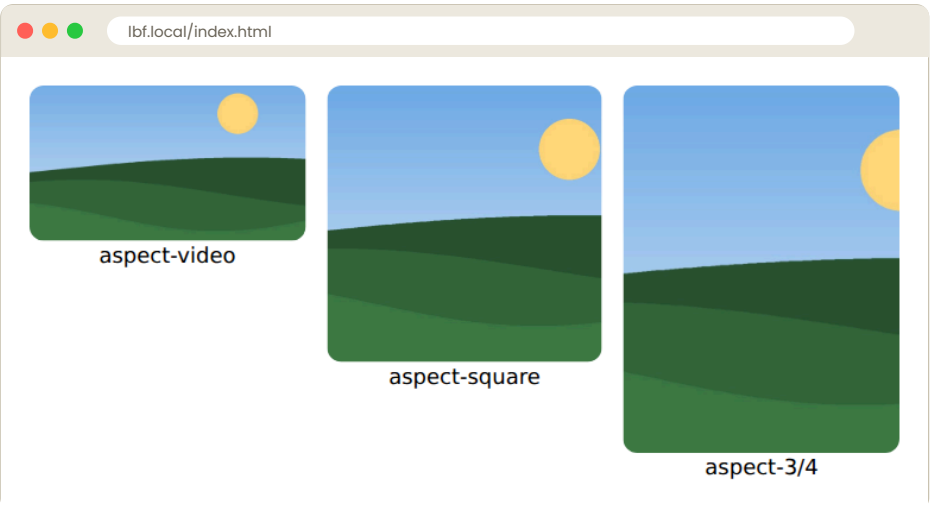
HOW THE BROWSER DISPLAYS IT

3.3.5 aspect-video, aspect-square

`aspect-video` (16 / 9), `aspect-square` (1 / 1), `aspect-auto`, fractions such as `aspect-3/2` and `aspect-4/3`, and arbitrary ratios `aspect-[21/9]`. Combine with `object-cover` so images fill the shape without distortion.

ASPECT RATIOS HTML

```
<div class="grid grid-cols-3 gap-3 text-center text-xs">
  <figure><img src="hills.jpg" alt="" class="aspect-video w-full rounded-lg object-cover">
<figcaption>aspect-video</figcaption></figure>
  <figure><img src="hills.jpg" alt="" class="aspect-square w-full rounded-lg object-cover">
<figcaption>aspect-square</figcaption></figure>
  <figure><img src="hills.jpg" alt="" class="aspect-3/4 w-full rounded-lg object-cover">
<figcaption>aspect-3/4</figcaption></figure>
</div>
```

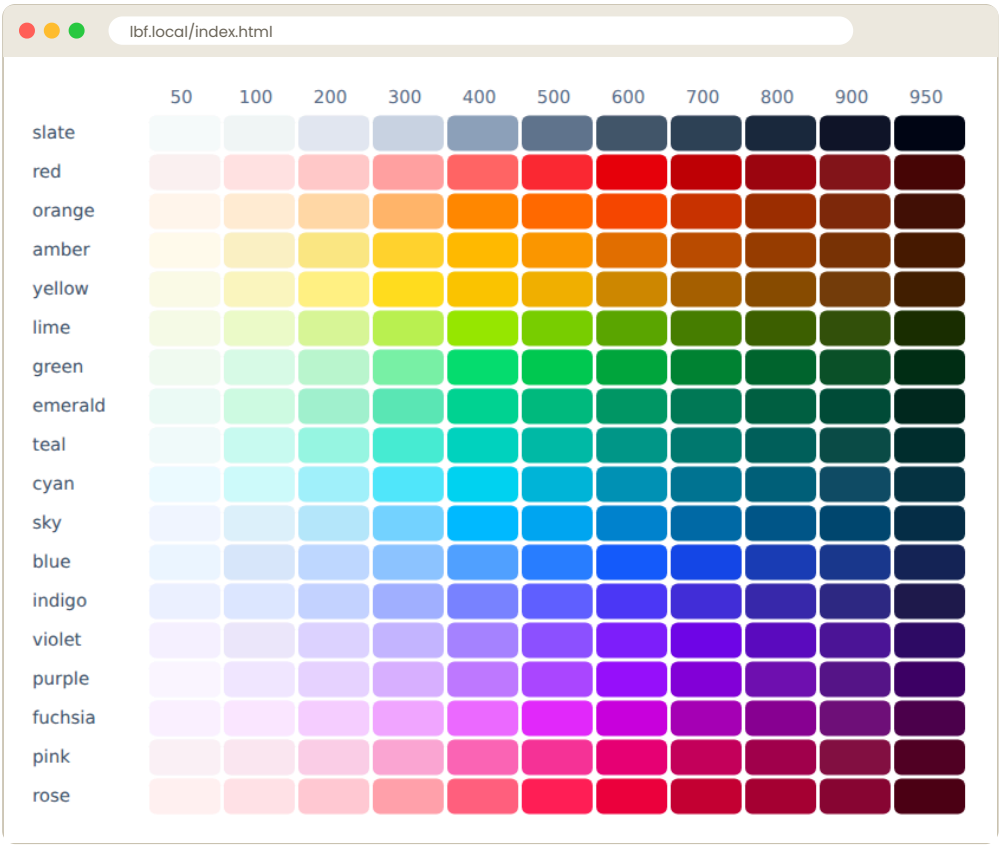


HOW THE BROWSER DISPLAYS IT

3.4 Colours

3.4.1 The default colour palette and shades 50 to 950

Tailwind ships a carefully designed palette of **22 colours** (slate, gray, zinc, neutral, stone, red, orange, amber, yellow, lime, green, emerald, teal, cyan, sky, blue, indigo, violet, purple, fuchsia, pink, rose), each with **11 shades**: 50 (lightest) to 950 (darkest), plus `black`, `white`, `transparent`, `current` (currentColor) and `inherit`. In v4 the palette is defined in **oklch**, which makes it more vivid on modern screens and perceptually even. Shade 500 is the "base" tone; 600–700 suit text and buttons on white; 50–100 suit tinted backgrounds.



THE DEFAULT PALETTE (18 OF 22 COLOURS), COMPILED AND RENDERED

COMPILED CSS

```
.bg-indigo-600 {
  background-color: var(--color-indigo-600);
}
```

THEME VARIABLES IN THE COMPILED CSS

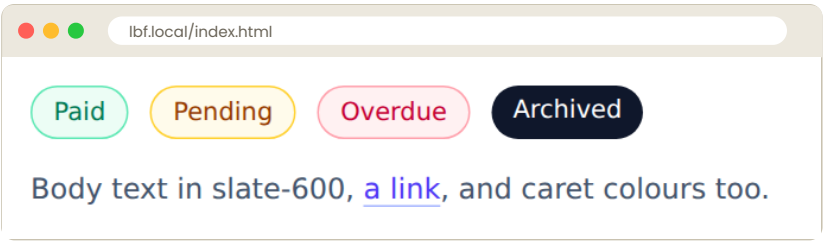
```
--color-indigo-50: oklch(96.2% 0.018 272.314);
--color-indigo-500: oklch(58.5% 0.233 277.117);
--color-indigo-950: oklch(25.7% 0.09 281.288);
```

3.4.2 Text, background and border colours

The same colour names work with every colour utility: `text-*`, `bg-*`, `border-*`, `outline-*`, `ring-*`, `divide-*`, `decoration-*`, `accent-*`, `caret-*`, `fill-*`, `stroke-*`, `shadow-*`, `from-/via-/to-*` (gradients).

STATUS BADGES FROM TEXT, BACKGROUND AND BORDER COLOURS HTML

```
<div class="flex flex-wrap gap-3">
  <span class="rounded-full border border-emerald-300 bg-emerald-50 px-3 py-1 text-sm font-medium text-emerald-700">Paid</span>
  <span class="rounded-full border border-amber-300 bg-amber-50 px-3 py-1 text-sm font-medium text-amber-800">Pending</span>
  <span class="rounded-full border border-rose-300 bg-rose-50 px-3 py-1 text-sm font-medium text-rose-700">Overdue</span>
  <span class="rounded-full bg-slate-900 px-3 py-1 text-sm font-medium text-white">Archived</span>
</div>
<p class="mt-4 text-slate-600">Body text in slate-600, <a href="#" class="text-indigo-600 underline decoration-indigo-300 underline-offset-4">a link</a>, and <span class="caret-pink-500">caret colours</span> too.</p>
```



HOW THE BROWSER DISPLAYS IT

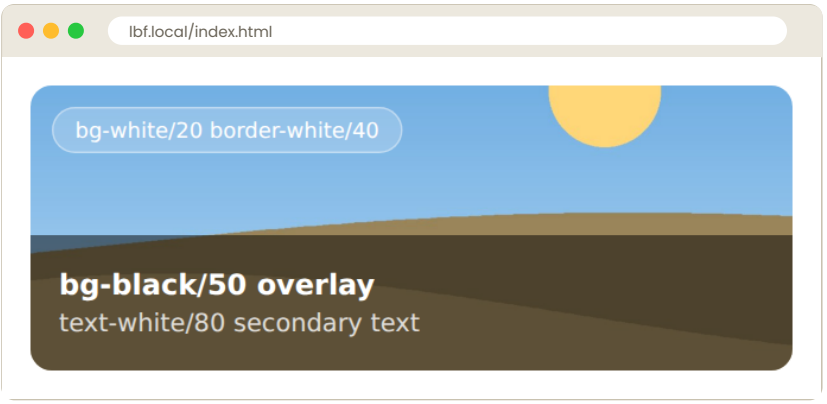
3.4.3 Opacity modifier bg-black/50

Add `/` and an opacity to any colour utility: `bg-black/50`, `text-white/80`, `border-indigo-500/30`, `bg-sky-500/[.07]`. Tailwind 4 implements it with `color-mix()` (with an oklab fallback), so only that colour becomes translucent, unlike the opacity property (CSS3 Module 4).

THE OPACITY MODIFIER

HTML

```
<div class="relative h-40 overflow-hidden rounded-xl bg-[url(beach.jpg)] bg-cover bg-center">
  <div class="absolute inset-x-0 bottom-0 bg-black/50 p-4 text-white">
    <p class="font-bold">bg-black/50 overlay</p><p class="text-sm text-white/80">text-white/80
secondary text</p>
  </div>
  <span class="absolute top-3 left-3 rounded-full border border-white/40 bg-white/20 px-3 py-1 text-xs
text-white">bg-white/20 border-white/40</span>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.bg-black\/50 {
  background-color: color-mix(in srgb, #000 50%, transparent);
  @supports (color: color-mix(in lab, red, red)) {
    background-color: color-mix(in oklab, var(--color-black) 50%, transparent);
  }
}
```

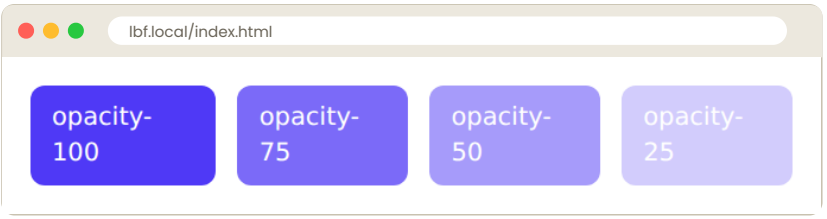
3.4.4 opacity-*

`opacity-0` to `opacity-100` (any number, as a percentage) sets the CSS `opacity` of the whole element and its children: use it for disabled states, fades and hover reveals, not for translucent backgrounds behind text.

OPACITY UTILITIES

HTML

```
<div class="flex gap-3"><div class="rounded-lg bg-indigo-600 px-3 py-2 text-sm text-white opacity-100">opacity-100</div><div class="rounded-lg bg-indigo-600 px-3 py-2 text-sm text-white opacity-75">opacity-75</div><div class="rounded-lg bg-indigo-600 px-3 py-2 text-sm text-white opacity-50">opacity-50</div><div class="rounded-lg bg-indigo-600 px-3 py-2 text-sm text-white opacity-25">opacity-25</div></div>
```



HOW THE BROWSER DISPLAYS IT

3.4.5 Choosing accessible colour combinations

A reliable rule of thumb with the default palette on a white background: shades **600 and darker** pass WCAG AA (4.5:1) for normal text for most colours; **500** often passes only for large text; 400 and lighter are for decoration, borders and backgrounds. On dark backgrounds, mirror it: 300–400 text on 900–950. Yellow, amber, lime and the grey family behave differently, so check. This script computed the real contrast of each shade of four colours against white, using the colours exactly as Chromium renders Tailwind's oklch values:

CONTRAST ON WHITE, COMPUTED IN CHROMIUM

```
text-slate-400    2.63:1  fails
text-slate-500    4.76:1  AA
text-slate-600    7.58:1  AA
text-slate-700    10.36:1  AA
text-indigo-400   3.13:1  large text only
text-indigo-500   4.58:1  AA
text-indigo-600   6.46:1  AA
text-indigo-700   8.09:1  AA
text-emerald-400  1.94:1  fails
text-emerald-500  2.47:1  fails
text-emerald-600  3.65:1  large text only
text-emerald-700  5.36:1  AA
text-amber-400    1.72:1  fails
text-amber-500    2.13:1  fails
text-amber-600    3.20:1  large text only
text-amber-700    5.03:1  AA
```

NOTE

Colour alone must never carry meaning: the badges in 3.4.2 also say "Paid", "Overdue" in words (CSS3 Module 20).

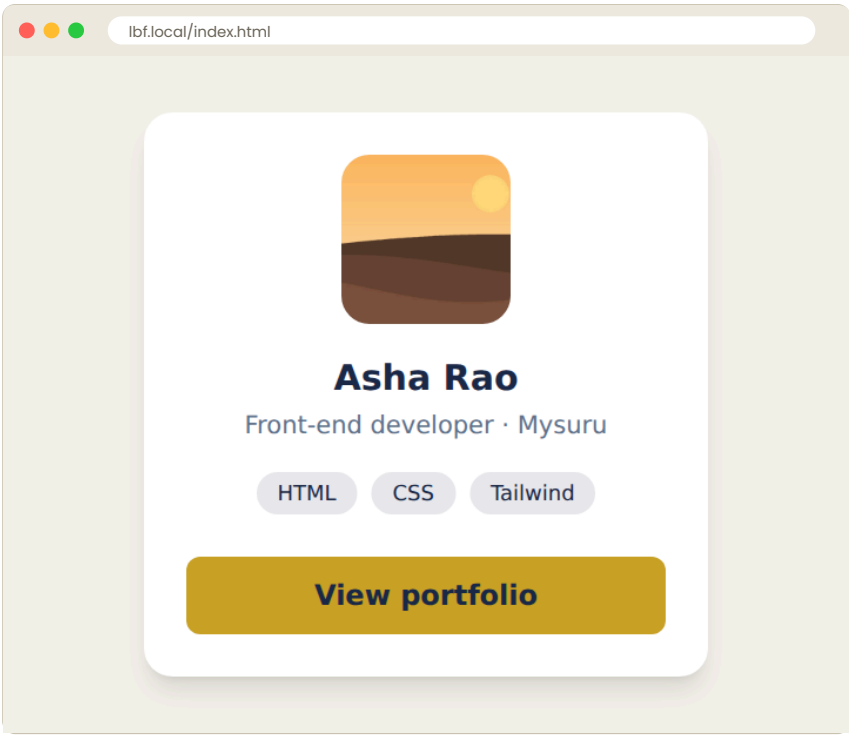
PRACTICE CHECKPOINT

A **centred profile card** with consistent spacing from the scale, **fixed width**, **square avatar** and **brand colours using arbitrary values**.

PROFILE-CARD.HTML: SAMPLE SOLUTION

HTML

```
<div class="min-h-[340px] bg-[#F4F1EA] p-8">
  <article class="mx-auto w-80 rounded-2xl bg-white p-6 text-center shadow-lg">
    <img src="mysuru-palace.jpg" alt="Asha Rao" class="mx-auto size-24 rounded-2xl object-cover">
    <h2 class="mt-4 text-xl font-bold text-[#1C2B4A]">Asha Rao</h2>
    <p class="mt-1 text-sm text-slate-500">Front-end developer · Mysuru</p>
    <div class="mt-4 flex justify-center gap-2">
      <span class="rounded-full bg-[#1C2B4A]/10 px-3 py-1 text-xs font-medium text-
[#1C2B4A]">HTML</span>
      <span class="rounded-full bg-[#1C2B4A]/10 px-3 py-1 text-xs font-medium text-
[#1C2B4A]">CSS</span>
      <span class="rounded-full bg-[#1C2B4A]/10 px-3 py-1 text-xs font-medium text-
[#1C2B4A]">Tailwind</span>
    </div>
    <a href="#" class="mt-6 block rounded-lg bg-[#C9A227] px-4 py-2.5 font-semibold text-[#1C2B4A]
hover:bg-[#b8921d]">View portfolio</a>
  </article>
</div>
```



THE PROFILE CARD

MODULE 04 · BEGINNER

Typography

Minimum duration **1 day** | Level **Beginner**

Typography utilities control fonts, sizes, weights and how text is laid out. This module covers the three default font families and how to add a custom font such as Poppins through the theme; the type scale from text-xs to text-9xl, line-height modifiers and font weights; alignment, decoration, transforms, letter spacing and line height; truncation, line clamping and modern wrapping; text shadows (new in v4.1); list styles; and the official Typography plugin, whose prose classes style whole articles you do not control. Every example was compiled with Tailwind v4.1.13 and rendered in Chromium.

IN THIS MODULE

4.1 Fonts
4.2 Size and weight
4.3 Text styling
4.4 Lists
4.5 The Typography plugin

4.1 Fonts

4.1.1 font-sans, font-serif, font-mono

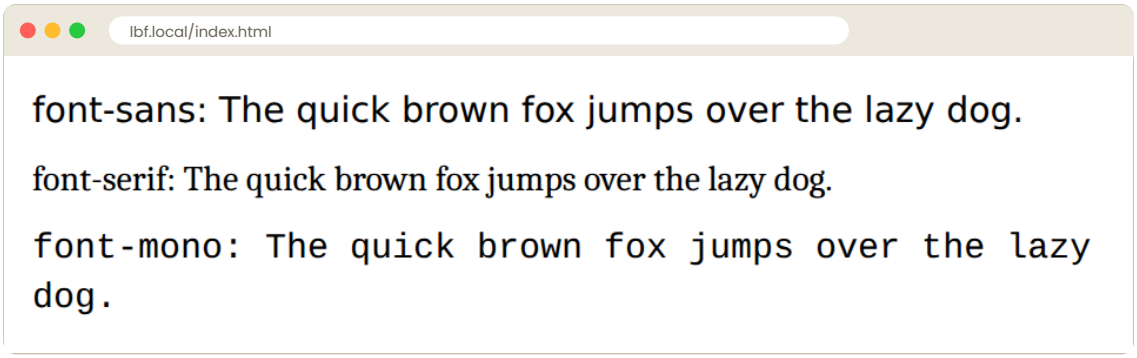
Tailwind ships three font-family utilities. Each points at a theme variable holding a **system font stack**, so text uses fonts already on the device and nothing has to download. `font-sans` is also the default for the whole page, because Preflight sets `html { font-family: var(--default-font-family) }`, which resolves to `--font-sans`.

Utility	Theme variable	Default stack (start)
`font-sans`	`--font-sans`	ui-sans-serif, system-ui, sans-serif, emoji fonts
`font-serif`	`--font-serif`	ui-serif, Georgia, Cambria, "Times New Roman", Times, serif
`font-mono`	`--font-mono`	ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, monospace

THE THREE DEFAULT FAMILIES

HTML

```
<p class="font-sans text-xl">font-sans: The quick brown fox jumps over the lazy dog.</p>
<p class="font-serif text-xl mt-3">font-serif: The quick brown fox jumps over the lazy dog.</p>
<p class="font-mono text-xl mt-3">font-mono: The quick brown fox jumps over the lazy dog.</p>
```



ON A MAC THESE BECOME SAN FRANCISCO, NEW YORK AND SF MONO; ON WINDOWS SEGOE UI, CAMBRIA AND CONSOLAS

COMPILED CSS

```
.font-mono {
  font-family: var(--font-mono);
}
.font-sans {
  font-family: var(--font-sans);
}
.font-serif {
  font-family: var(--font-serif);
}
```

The generated rule only references the variable; the stack itself lives once in the theme layer as `--font-sans: ui-sans-serif, system-ui, sans-serif, "Apple Color Emoji", ...`. That is why changing a font for the whole project is a one-line theme change, as the next section shows.

4.1.2 Adding a custom font (for example Poppins)

A custom font takes two steps. First load the font file, either with a normal CSS `@font-face` rule or with a `<link>` to a font service such as Google Fonts. Then register it in `@theme` as a `--font-*` variable. The name after `--font-` becomes the utility: `--font-display` creates `font-display`. Redefining `--font-sans` changes the default font for the entire site.

1. **Load the files** (self-hosted `@font-face` is fastest and avoids a third-party request; Google Fonts is quickest to set up).
2. **Register a theme variable** `--font-poppins: "Poppins", sans-serif;` in `@theme`, always ending with a generic fallback.
3. **Use the utility** `font-poppins`, or override `--font-sans` to make it the default everywhere.

APP.CSS CSS

```css
@import "tailwindcss";

@font-face {
  font-family: "Poppins";
  src: url("fonts/Poppins-Regular.ttf") format("truetype");
  font-weight: 400;
  font-display: swap;
}
@font-face {
  font-family: "Poppins";
  src: url("fonts/Poppins-Bold.ttf") format("truetype");
  font-weight: 700;
  font-display: swap;
}
@font-face {
  font-family: "Lora";
  src: url("fonts/Lora-Variable.ttf") format("truetype");
  font-weight: 400 700;
}

@theme {
  --font-poppins: "Poppins", ui-sans-serif, sans-serif;
  --font-lora: "Lora", ui-serif, Georgia, serif;
}
```

INDEX.HTML HTML

```html
<h1 class="font-poppins text-3xl font-bold text-indigo-900">Learn Being Forward</h1>
<p class="font-poppins mt-1 text-slate-600">Poppins loaded with @font-face and registered in @theme.
</p>
<h2 class="font-lora mt-4 text-2xl font-bold">Lora for headings</h2>
<p class="mt-1">This paragraph still uses the default font-sans stack.</p>
```



TWO SELF-HOSTED FONTS REGISTERED AS FONT-POPPINS AND FONT-LORA

With the Google Fonts service the `@font-face` rules come from their stylesheet instead, and only the theme registration remains in your CSS:

LOADING POPPINS FROM GOOGLE FONTS HTML

```html
<!-- index.html, inside <head> -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swap"
rel="stylesheet">
```

APP.CSS: MAKING POPPINS THE SITE DEFAULT CSS

```css
@import "tailwindcss";

@theme {
  /* replace the default sans stack: every element now uses Poppins */
  --font-sans: "Poppins", ui-sans-serif, system-ui, sans-serif;
}
```

COMPILED CSS

```css
.font-poppins {
  font-family: var(--font-poppins);
}
```

NOTE

Put `@font-face` rules **outside** `@theme`. The theme block only accepts custom properties (and `@keyframes`). Using `font-display: swap` shows fallback text immediately while the font downloads (HTML5 and CSS3 books, web-font modules).

TIP

Only load the weights you use. Each weight of a static font is a separate file; a variable font such as Lora covers a whole range (here 400 to 700) in one file.

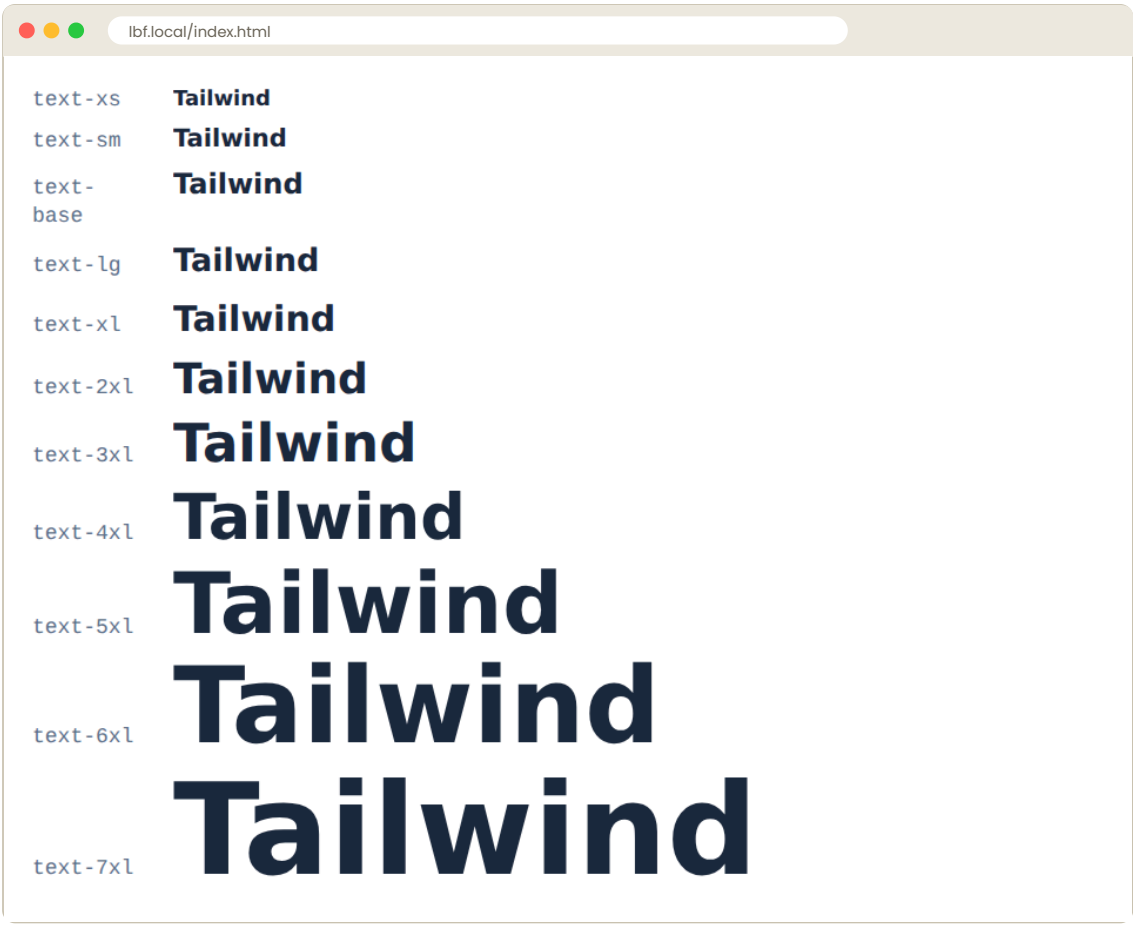
4.2 Size and weight

4.2.1 text-xs to text-9xl

The type scale has 13 steps. Each `text-*` size utility sets **two** properties: `font-size` and a matching `line-height` chosen for that size (tight for large headings, roomier for body text). Both come from theme variables `--text-lg` and `--text-lg--line-height`.

Class	font-size	px	line-height
`text-xs`	0.75rem	12px	calc(1 / 0.75)
`text-sm`	0.875rem	14px	calc(1.25 / 0.875)
`text-base`	1rem	16px	calc(1.5 / 1)
`text-lg`	1.125rem	18px	calc(1.75 / 1.125)
`text-xl`	1.25rem	20px	calc(1.75 / 1.25)
`text-2xl`	1.5rem	24px	calc(2 / 1.5)
`text-3xl`	1.875rem	30px	calc(2.25 / 1.875)
`text-4xl`	2.25rem	36px	calc(2.5 / 2.25)
`text-5xl`	3rem	48px	1
`text-6xl`	3.75rem	60px	1
`text-7xl`	4.5rem	72px	1
`text-8xl`	6rem	96px	1

Class	font-size	px	line-height
`text-9xl`	8rem	128px	1



TEXT-XS TO TEXT-7XL RENDERED AT REAL SIZE

COMPILED CSS

```
.text-5xl {
  font-size: var(--text-5xl);
  line-height: var(--tw-leading, var(--text-5xl--line-height));
}
.text-lg {
  font-size: var(--text-lg);
  line-height: var(--tw-leading, var(--text-lg--line-height));
}
.text-sm {
  font-size: var(--text-sm);
  line-height: var(--tw-leading, var(--text-sm--line-height));
}
```

Notice that `text-5xl` and above use `line-height: 1` (headings are usually one line and need no extra space), while small sizes get ratios around 1.4 to 1.5 for readability. A fluid or one-off size can use an arbitrary value: `text-[clamp(1.5rem,4vw,3rem)]`.

4.2.2 Line height with size modifiers text-lg/7

To change the line height together with the size, add a slash modifier. `text-lg/7` means font size `lg` with line height `leading-7` (7 × 0.25rem = 1.75rem). Named values work too: `text-sm/relaxed`, and so do arbitrary ones: `text-base/[1.9]`. This replaces the pair `text-lg leading-7` with one class.

text-lg/5

Tight: text-lg/5 makes the lines of this paragraph sit very close together, which is hard to read.

text-lg/7

Default rhythm: text-lg/7 gives a comfortable line height for body copy of this size.

text-lg/10

Loose: text-lg/10 spaces the lines so far apart that the text starts to fall apart.

SAME SIZE, THREE LINE HEIGHTS

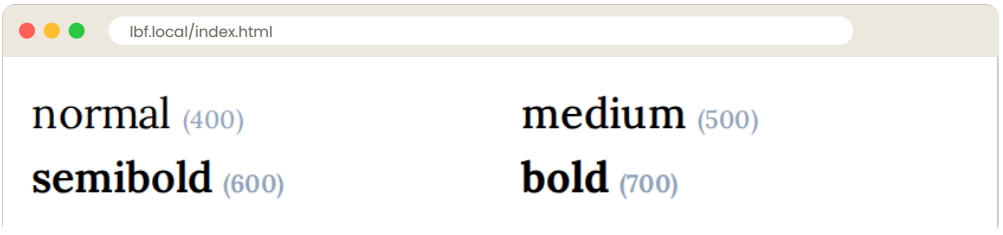
COMPILED CSS

```
.text-base\/\[1\.9\] {
  font-size: var(--text-base);
  line-height: 1.9;
}
.text-lg\/7 {
  font-size: var(--text-lg);
  line-height: calc(var(--spacing) * 7);
}
.text-sm\/relaxed {
  font-size: var(--text-sm);
  line-height: var(--leading-relaxed);
}
```

4.2.3 font-light to font-black

Weight utilities map to the numeric `font-weight` values from 100 to 900. A weight only looks different if the font provides it: a font with just Regular and Bold files will render `font-medium` like Regular and `font-black` like Bold (or a synthesised bold).

Class	Weight	Class	Weight
`font-thin`	100	`font-semibold`	600
`font-extralight`	200	`font-bold`	700
`font-light`	300	`font-extrabold`	800
`font-normal`	400	`font-black`	900
`font-medium`	500	`font-[650]`	arbitrary



LORA IS A VARIABLE FONT (400 TO 700), SO EVERY STEP BETWEEN RENDERS DISTINCTLY

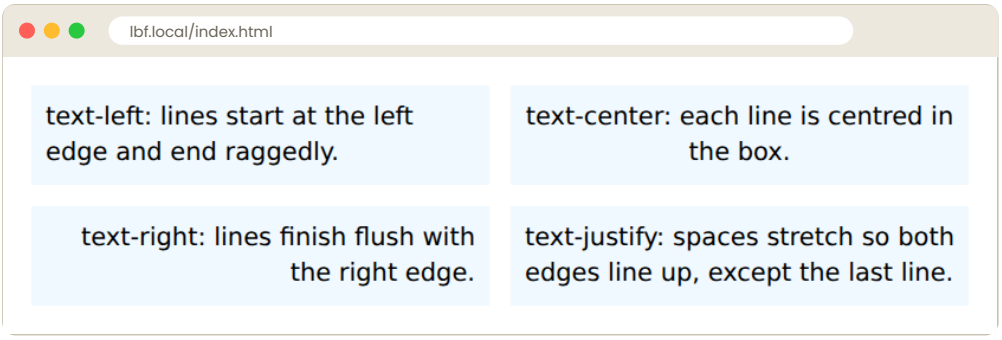
4.3 Text styling

4.3.1 Alignment: text-left, text-center, text-justify

`text-left`, `text-center`, `text-right` and `text-justify` set `text-align`. For right-to-left support prefer the logical `text-start` and `text-end`, which flip automatically. Alignment is often responsive: `text-center md:text-left` centres a hero on phones and left-aligns it on wider screens.

TEXT ALIGNMENT HTML

```
<div class="grid grid-cols-2 gap-3 text-sm">
  <p class="text-left bg-sky-50 p-2">text-left: lines start at the left edge and end raggedly.</p>
  <p class="text-center bg-sky-50 p-2">text-center: each line is centred in the box.</p>
  <p class="text-right bg-sky-50 p-2">text-right: lines finish flush with the right edge.</p>
  <p class="text-justify bg-sky-50 p-2">text-justify: spaces stretch so both edges line up, except the last line.</p>
</div>
```



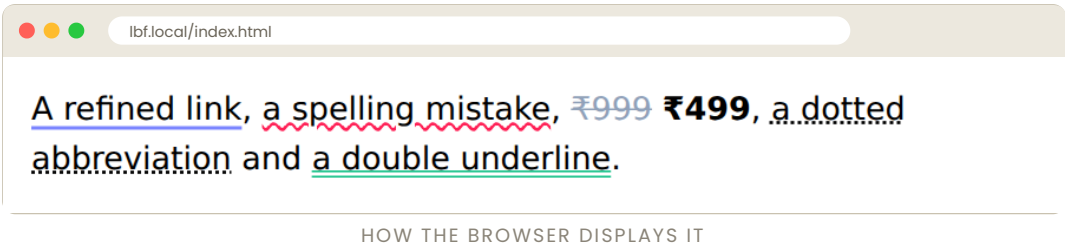
HOW THE BROWSER DISPLAYS IT

4.3.2 Decoration: underline, line-through, decoration-*, underline-offset-*

`underline`, `overline`, `line-through` and `no-underline` choose the line. Then style it: colour with `decoration-pink-500`, thickness with `decoration-2` or `decoration-from-font`, style with `decoration-wavy`, `decoration-dotted`, `decoration-double`, and distance from the text with `underline-offset-4`. These make link styles far more refined than the browser default.

TEXT DECORATION UTILITIES HTML

```
<p class="text-lg">
  <a href="#" class="underline decoration-indigo-400 decoration-2 underline-offset-4 hover:decoration-indigo-700">A refined link</a>,
  <span class="underline decoration-wavy decoration-rose-500">a spelling mistake</span>,
  <span class="line-through text-slate-400">&#8377;999</span> <b>&#8377;499</b>,
  <span class="underline decoration-dotted underline-offset-2">a dotted abbreviation</span>
  and <span class="underline decoration-double decoration-emerald-500">a double underline</span>.
</p>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.line-through {
  text-decoration-line: line-through;
}
.underline {
  text-decoration-line: underline;
}
.decoration-indigo-400 {
  text-decoration-color: var(--color-indigo-400);
}
.decoration-wavy {
  text-decoration-style: wavy;
}
.decoration-2 {
  text-decoration-thickness: 2px;
}
.underline-offset-4 {
  text-underline-offset: 4px;
}
```

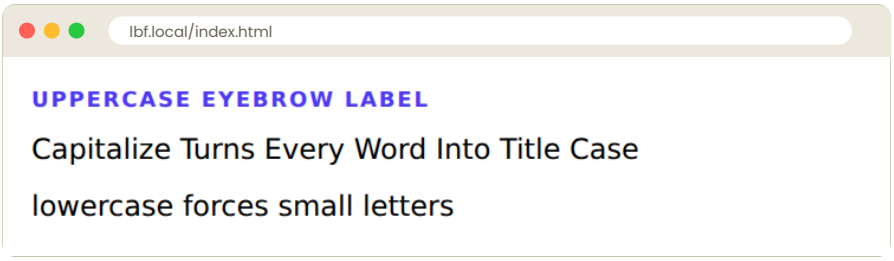
4.3.3 Transform: uppercase, lowercase, capitalize

`uppercase`, `lowercase`, `capitalize` and `normal-case` set `text-transform`. The HTML keeps its original text, which matters: screen readers and copy-paste use the source, so write content in normal sentence case and apply `uppercase` for visual style (a small label, a button). Uppercase text usually needs extra letter spacing to stay readable.

TEXT TRANSFORMS

HTML

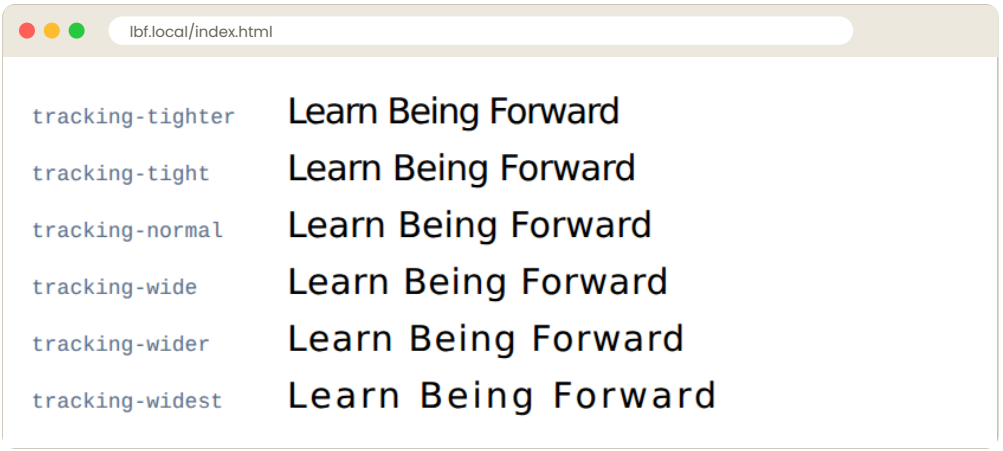
```
<p class="uppercase text-xs font-semibold tracking-widest text-indigo-600">uppercase eyebrow label</p>
<p class="capitalize mt-2">capitalize turns every word into title case</p>
<p class="lowercase mt-2">LOWERCASE FORCES SMALL LETTERS</p>
```



HOW THE BROWSER DISPLAYS IT

4.3.4 Spacing: tracking-*, leading-*

Letter spacing uses `tracking-tighter` (-0.05em) through `tracking-tight`, `tracking-normal`, `tracking-wide` to `tracking-widest` (0.1em). Large headings often look better slightly tighter; small uppercase labels need wider tracking. **Line height** on its own uses `leading-*`: a spacing number (`leading-6` = 1.5rem) or a ratio name (`leading-none` 1, `leading-tight` 1.25, `leading-snug` 1.375, `leading-normal` 1.5, `leading-relaxed` 1.625, `leading-loose` 2).



LETTER SPACING FROM TIGHTER TO WIDEST

COMPILED CSS

```
.leading-6 {
  --tw-leading: calc(var(--spacing) * 6);
  line-height: calc(var(--spacing) * 6);
}
.leading-relaxed {
  --tw-leading: var(--leading-relaxed);
  line-height: var(--leading-relaxed);
}
.tracking-tight {
  --tw-tracking: var(--tracking-tight);
  letter-spacing: var(--tracking-tight);
}
```

NOTE

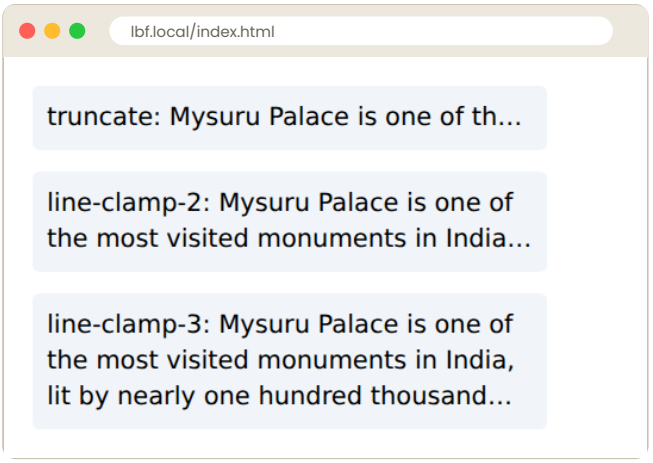
In v4, `leading-*` also sets the `--tw-leading` variable, and a size utility such as `text-lg` reads it with a fallback: `line-height: var(--tw-leading, var(--text-lg--line-height))`. That is why `leading-*` wins over the size's built-in line height no matter which class comes first.

4.3.5 Overflow: truncate, line-clamp-*, text-ellipsis

`truncate` is a three-property shortcut for single-line ellipsis: `overflow: hidden; text-overflow: ellipsis; white-space: nowrap`. For several lines use `line-clamp-2`, `line-clamp-3` and so on, which cut the text after that many lines and add an ellipsis. `text-ellipsis` and `text-clip` set only `text-overflow`, for when you build the other properties yourself. `line-clamp-none` removes the clamp, for example at a larger breakpoint.

SINGLE-LINE TRUNCATION AND MULTI-LINE CLAMPING HTML

```
<div class="w-72 space-y-3 text-sm">
  <p class="truncate rounded bg-slate-100 p-2">truncate: Mysuru Palace is one of the most visited monuments in India, lit by thousands of bulbs.</p>
  <div class="rounded bg-slate-100 p-2"><p class="line-clamp-2">line-clamp-2: Mysuru Palace is one of the most visited monuments in India, lit by nearly one hundred thousand bulbs on Sunday evenings and during Dasara.</p></div>
  <div class="rounded bg-slate-100 p-2"><p class="line-clamp-3">line-clamp-3: Mysuru Palace is one of the most visited monuments in India, lit by nearly one hundred thousand bulbs on Sunday evenings and during Dasara. The Indo-Saracenic building was completed in 1912.</p></div>
</div>
```



PADDING SITS ON A WRAPPER: PADDING ON THE CLAMPED ELEMENT ITSELF WOULD REVEAL PART OF THE HIDDEN NEXT LINE

COMPILED CSS

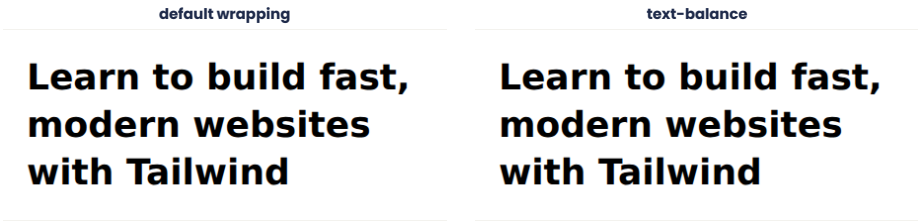
```
.line-clamp-2 {
  overflow: hidden;
  display: -webkit-box;
  -webkit-box-orient: vertical;
  -webkit-line-clamp: 2;
}
.truncate {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
```

WATCH OUT

`truncate` needs a constrained width to have any effect. Inside a flex row, a child will not shrink below its content width unless it has `min-w-0`, so the classic fix for "truncate does nothing in my flex item" is `min-w-0` on that item (Module 7).

4.3.6 Wrapping: text-balance, text-pretty, break-words

`text-balance` makes the browser distribute words so that all lines of a short block (a heading) have similar length, instead of one long line and one orphan word. `text-pretty` is the gentler version for paragraphs: it mainly avoids a single word alone on the last line. For long unbroken strings (URLs, IDs) `break-words` lets them wrap at any point if they would otherwise overflow, and `break-all` breaks anywhere. `whitespace-nowrap` and `whitespace-pre-line` control how spaces and newlines are treated.



TEXT-BALANCE EVENS OUT THE LINE LENGTHS OF A HEADING



LONG UNBROKEN STRINGS

COMPILED CSS

```
.text-balance {
  text-wrap: balance;
}
.text-pretty {
  text-wrap: pretty;
}
.break-words {
  overflow-wrap: break-word;
}
.break-all {
  word-break: break-all;
}
.whitespace-nowrap {
  white-space: nowrap;
}
```

4.3.7 Text shadows

Tailwind 4.1 added text-shadow utilities: `text-shadow-2xs`, `text-shadow-xs`, `text-shadow-sm`, `text-shadow-md` and `text-shadow-lg`, plus `text-shadow-none`. Colour them with `text-shadow-indigo-500` and adjust opacity with a modifier (`text-shadow-lg/30` or `text-shadow-black/50`). The main practical use is keeping white text readable over a photo.

TEXT SHADOW ON A PHOTO AND AS A COLOURED OUTLINE EFFECT

HTML

```
<div class="relative h-40 w-96 overflow-hidden rounded-xl">
  <img src="hills.jpg" alt="" class="absolute inset-0 h-full w-full object-cover">
  <p class="absolute left-4 top-4 text-2xl font-bold text-white">No shadow</p>
  <p class="absolute bottom-4 left-4 text-2xl font-bold text-white text-shadow-lg text-shadow-
black/60">text-shadow-lg</p>
</div>
<p class="mt-4 text-3xl font-black text-white text-shadow-md text-shadow-indigo-700">Coloured
shadow</p>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.text-shadow-black\/60 {
  --tw-text-shadow-color: color-mix(in srgb, #000 60%, transparent);
  @supports (color: color-mix(in lab, red, red)) {
    --tw-text-shadow-color: color-mix(in oklab, color-mix(in oklab, var(--color-black) 60%,
transparent) var(--tw-text-shadow-alpha), transparent);
  }
}
.text-shadow-lg {
  text-shadow: 0px 1px 2px var(--tw-text-shadow-color, rgb(0 0 0 / 0.1)), 0px 3px 2px var(--tw-text-
shadow-color, rgb(0 0 0 / 0.1)), 0px 4px 8px var(--tw-text-shadow-color, rgb(0 0 0 / 0.1));
}
```

4.4 Lists

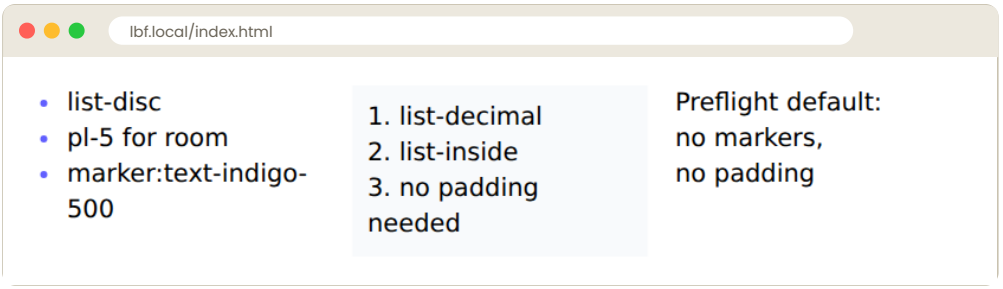
4.4.1 list-disc, list-decimal, list-inside

Preflight removes list bullets and padding (`list-style: none`), because most lists in an interface are menus and card grids, not text lists. To get markers back, add `list-disc` or `list-decimal` (or `list-[upper-roman]`), and some left padding (`pl-5`) because the markers sit **outside** the content box by default. `list-inside` puts the marker inside the box instead, so no padding is needed. The `marker:` variant styles the bullets themselves.

LIST MARKERS

HTML

```
<div class="grid grid-cols-3 gap-4 text-sm">
  <ul class="list-disc pl-5 marker:text-indigo-500">
    <li>list-disc</li><li>pl-5 for room</li><li>marker:text-indigo-500</li>
  </ul>
  <ol class="list-decimal list-inside bg-slate-50 p-2">
    <li>list-decimal</li><li>list-inside</li><li>no padding needed</li>
  </ol>
  <ul>
    <li>Preflight default:</li><li>no markers,</li><li>no padding</li>
  </ul>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.list-inside {
  list-style-position: inside;
}
.list-decimal {
  list-style-type: decimal;
}
.list-disc {
  list-style-type: disc;
}
.marker\:text-indigo-500 {
  & *::marker {
    color: var(--color-indigo-500);
  }
  &::marker {
    color: var(--color-indigo-500);
  }
  & *::-webkit-details-marker {
    color: var(--color-indigo-500);
  }
  &::-webkit-details-marker {
    color: var(--color-indigo-500);
  }
}
```

WATCH OUT

Safari (WebKit) stops announcing a `<ul>` as a list to VoiceOver when `list-style: none` is applied. For a navigation menu built from a list, add `role="list"` to the `<ul>` to keep the list semantics.

4.5 The Typography plugin

4.5.1 prose classes for styling article content

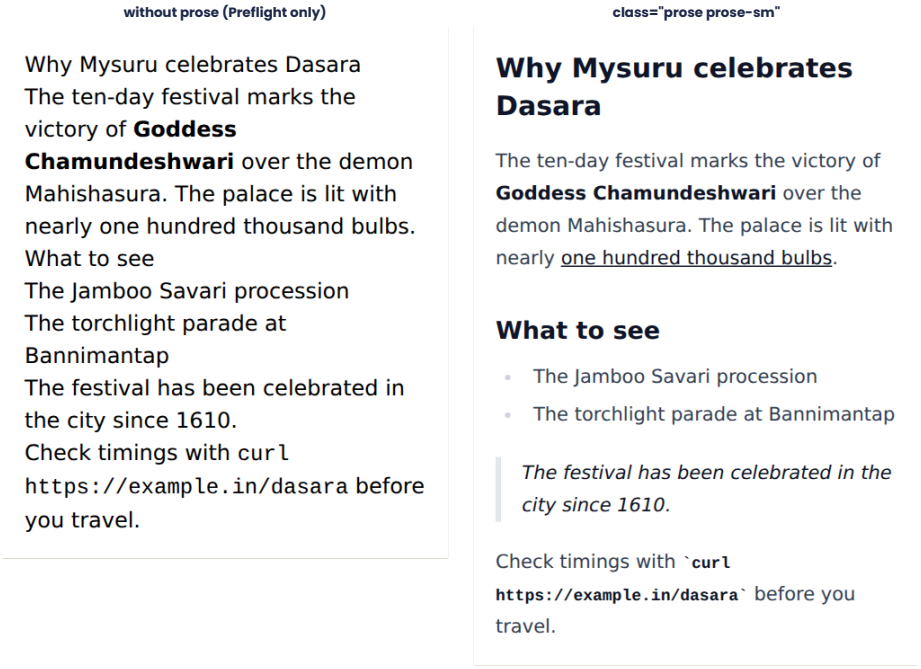
Utility classes are perfect for components you write, but not for HTML you do not control: a blog post from Markdown, CMS content, or rich-text editor output. Because Preflight removes all default styles, such content renders as a flat wall of same-sized text. The official **@tailwindcss/typography** plugin solves this with one class, `prose`, which adds carefully tuned typographic styles to every descendant element (headings, paragraphs, lists, quotes, code, tables, images).

INSTALLING THE PLUGIN (THE STANDALONE CLI ALREADY BUNDLES IT) shell

```
npm install -D @tailwindcss/typography
```

APP.CSS: REGISTERING THE PLUGIN CSS

```css
@import "tailwindcss";
@plugin "@tailwindcss/typography";
```



THE SAME PLAIN HTML WITH AND WITHOUT THE TYPOGRAPHY PLUGIN

THE ARTICLE MARKUP (ONLY THE ARTICLE ELEMENT HAS A CLASS) HTML

```html
<article class="prose prose-sm lg:prose-lg prose-indigo dark:prose-invert">
    <h2>Why Mysuru celebrates Dasara</h2>
    <p>The ten-day festival marks the victory of <strong>Goddess Chamundeshwari</strong> over the
demon Mahishasura. The palace is lit with nearly <a href="#">one hundred thousand bulbs</a>.</p>
    <h3>What to see</h3>
    <ul>
      <li>The Jamboo Savari procession</li>
      <li>The torchlight parade at Bannimantap</li>
    </ul>
    <blockquote>The festival has been celebrated in the city since 1610.</blockquote>
    <p>Check timings with <code>curl https://example.in/dasara</code> before you travel.</p>
</article>
```

The plugin's classes are modifiers you combine:

Class	Effect
`prose`	Base typographic styles, max width about 65 characters
`prose-sm`, `prose-lg`, `prose-xl`, `prose-2xl`	Scale the whole article; responsive: `md:prose-lg`
`prose-slate`, `prose-zinc`, `prose-stone` …	Grey palette for text, borders and quotes
`prose-invert`	Light text for a dark background; usually `dark:prose-invert`
`prose-a:text-indigo-600`, `prose-headings:font-poppins`, `prose-img:rounded-xl`	Element modifiers: style one element type inside the article with any utility

Class	Effect
`max-w-none`	Remove the default reading width
`not-prose`	Opt a block (a custom component inside the article) out of prose styles

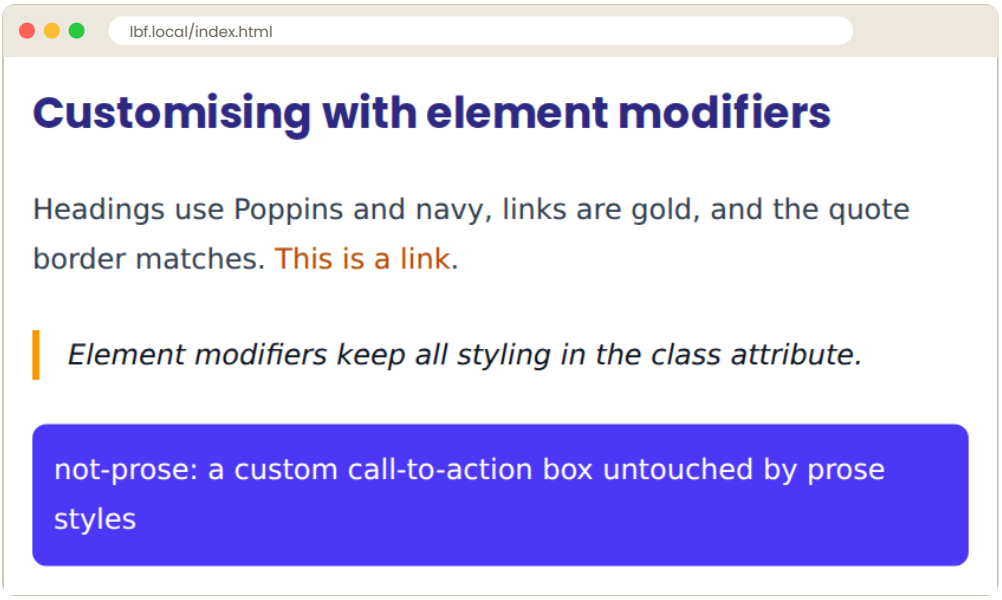
APP.CSS CSS

```css
@import "tailwindcss";

@plugin "@tailwindcss/typography";
@font-face { font-family: "Poppins"; src: url("fonts/Poppins-Bold.ttf"); font-weight: 700; }
@theme { --font-poppins: "Poppins", sans-serif; }
```

INDEX.HTML HTML

```html
<article class="prose prose-headings:font-poppins prose-headings:text-indigo-900 prose-a:text-amber-700 prose-a:no-underline hover:prose-a:underline prose-blockquote:border-amber-500 max-w-none">
  <h2>Customising with element modifiers</h2>
  <p>Headings use Poppins and navy, links are gold, and the quote border matches. <a href="#">This is a link</a>.</p>
  <blockquote>Element modifiers keep all styling in the class attribute.</blockquote>
  <div class="not-prose mt-4 rounded-lg bg-indigo-600 p-3 text-white">not-prose: a custom call-to-action box untouched by prose styles</div>
</article>
```



PROSE WITH ELEMENT MODIFIERS AND A NOT-PROSE BLOCK

PRACTICE CHECKPOINT

A **blog article page** with a custom font, a clear type scale, clamped preview text on cards and prose styling for the article body.

The solution registers Lora for headings and Poppins as the default sans font, shows two article cards whose excerpts are clamped to three lines and whose titles are balanced, and renders the article body with `prose` customised through element modifiers.

APP.CSS CSS

```css
@import "tailwindcss";

@plugin "@tailwindcss/typography";

@font-face { font-family: "Poppins"; src: url("fonts/Poppins-Regular.ttf"); font-weight: 400; }
@font-face { font-family: "Poppins"; src: url("fonts/Poppins-Bold.ttf"); font-weight: 700; }
@font-face { font-family: "Lora"; src: url("fonts/Lora-Variable.ttf"); font-weight: 400 700; }

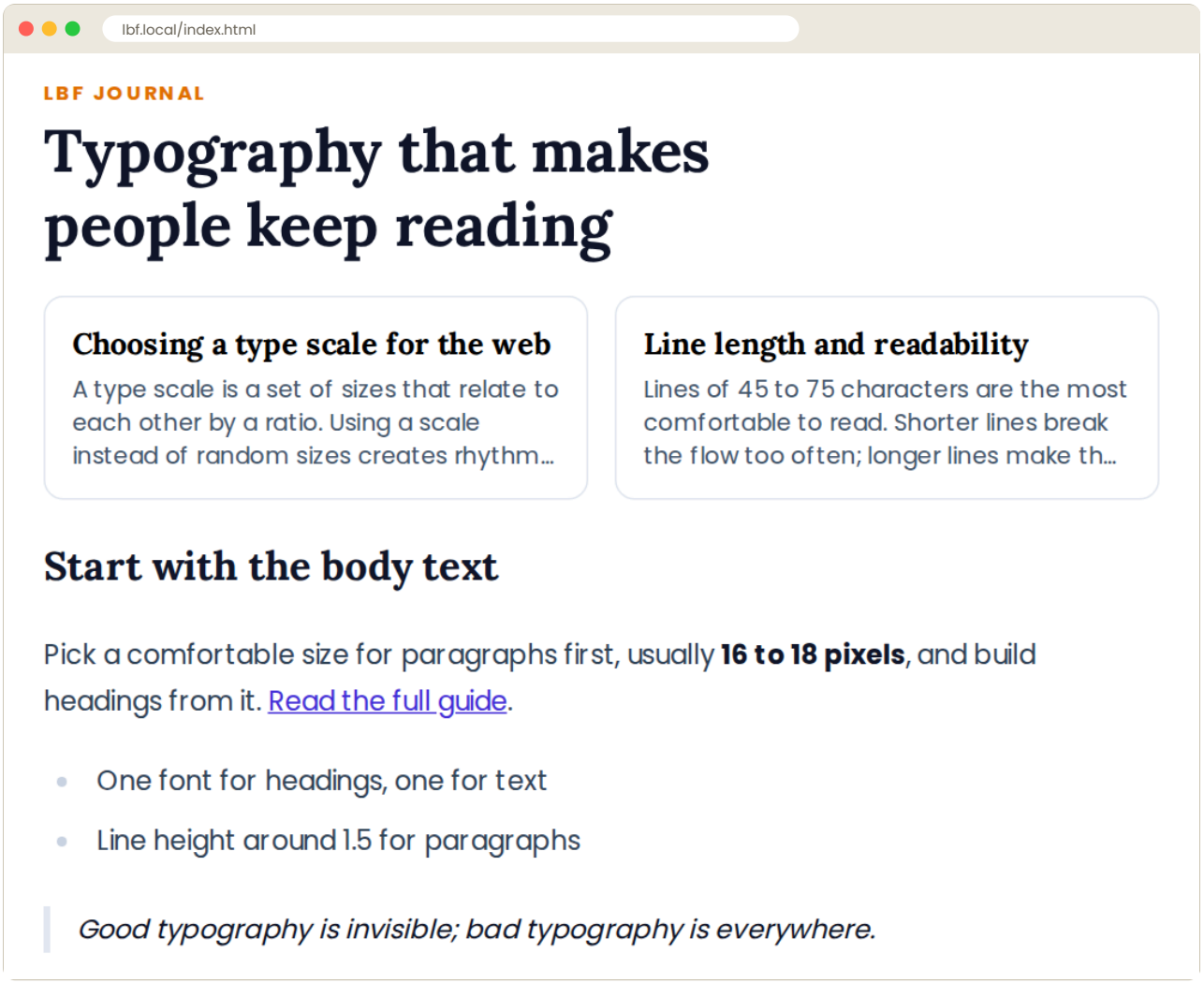
@theme {
  --font-sans: "Poppins", ui-sans-serif, system-ui, sans-serif;
  --font-display: "Lora", ui-serif, Georgia, serif;
}
```

ARTICLE.HTML: SAMPLE SOLUTION HTML

```html
<div class="mx-auto max-w-2xl">
  <p class="text-xs font-bold uppercase tracking-widest text-amber-600">LBF Journal</p>
  <h1 class="font-display mt-1 text-4xl/tight font-bold text-balance text-slate-900">Typography that
makes people keep reading</h1>

  <div class="mt-5 grid grid-cols-2 gap-4">
    <a href="#" class="rounded-xl border border-slate-200 p-4 hover:border-indigo-300">
      <h2 class="font-display text-lg/snug font-bold text-balance">Choosing a type scale for the
web</h2>
      <p class="mt-1 line-clamp-3 text-sm text-slate-600">A type scale is a set of sizes that relate
to each other by a ratio. Using a scale instead of random sizes creates rhythm and makes the hierarchy
of a page obvious at a glance, even before anyone reads a word.</p>
    </a>
    <a href="#" class="rounded-xl border border-slate-200 p-4 hover:border-indigo-300">
      <h2 class="font-display text-lg/snug font-bold text-balance">Line length and readability</h2>
      <p class="mt-1 line-clamp-3 text-sm text-slate-600">Lines of 45 to 75 characters are the most
comfortable to read. Shorter lines break the flow too often; longer lines make the eye lose its place
when it returns to the start of the next line.</p>
    </a>
  </div>

  <article class="prose prose-slate mt-6 prose-headings:font-display prose-a:text-indigo-700">
    <h2>Start with the body text</h2>
    <p>Pick a comfortable size for paragraphs first, usually <strong>16 to 18 pixels</strong>, and
build headings from it. <a href="#">Read the full guide</a>.</p>
    <ul>
      <li>One font for headings, one for text</li>
      <li>Line height around 1.5 for paragraphs</li>
    </ul>
    <blockquote>Good typography is invisible; bad typography is everywhere.</blockquote>
  </article>
</div>
```



THE BLOG ARTICLE PAGE

Load fonts with @font-face, register them as `--font-*` in `@theme`, size text with `text-*` (plus `/leading` modifiers), and let `prose` style any HTML you do not write yourself.

MODULE 05 · BEGINNER

Backgrounds, borders and effects

Minimum duration **1 day** | Level **Beginner**

These utilities add visual depth. The module covers background colours and images, sizing and positioning of background images, linear, radial and conic gradients with colour stops; borders on every side with colours and styles, rounded corners and dividers between children; rings and outlines; shadows (including coloured and inset shadows), opacity and blend modes; filters and backdrop filters for glassmorphism; and object-fit for images. All examples were compiled with Tailwind v4.1.13 and rendered in Chromium.

IN THIS MODULE

5.1 Backgrounds
5.2 Borders
5.3 Rings and outlines
5.4 Effects
5.5 Filters
5.6 Images

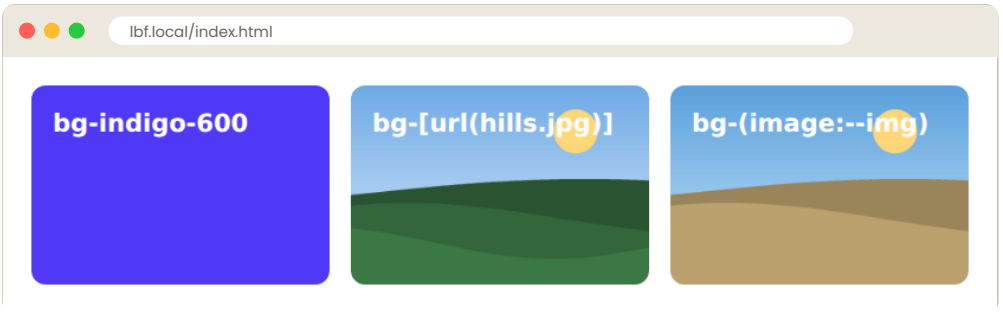
5.1 Backgrounds

5.1.1 Background colours and images bg-[url(...)]

`bg-{colour}-{shade}` sets `background-color` from the palette (Module 3), with the opacity modifier for transparency: `bg-indigo-600/20`. For an image use an arbitrary value with `url()`: `bg-[url(/img/hero.jpg)]`. For an image path that is decided at runtime (a CMS photo), set a custom property inline and use the variable shorthand: `style="--img: url(...)"` with `bg-(image:--img)`. A project-wide image can also be registered as a theme value or written as a custom utility (Module 14).

A COLOUR, AN ARBITRARY IMAGE AND AN IMAGE FROM A CSS VARIABLE — HTML

```
<div class="grid grid-cols-3 gap-3 text-sm font-semibold text-white">
  <div class="h-28 rounded-lg bg-indigo-600 p-3">bg-indigo-600</div>
  <div class="h-28 rounded-lg bg-[url(hills.jpg)] bg-cover p-3">bg-[url(hills.jpg)]</div>
  <div style="--img: url(beach.jpg)" class="h-28 rounded-lg bg-(image:--img) bg-cover p-3">bg-(image:-
-img)</div>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.bg-indigo-600 {
  background-color: var(--color-indigo-600);
}
.bg-indigo-600\/20 {
  background-color: color-mix(in srgb, oklch(51.1% 0.262 276.966) 20%, transparent);
  @supports (color: color-mix(in lab, red, red)) {
    background-color: color-mix(in oklab, var(--color-indigo-600) 20%, transparent);
  }
}
.bg-\(image\:--img\) {
  background-image: var(--img);
}
.bg-\[url\(hills\.jpg\)\] {
  background-image: url(hills.jpg);
}
```

NOTE

The type hint `image:` in `bg-(image:--img)` tells Tailwind the variable holds an image. Without it, `bg-(--x)` is treated as a colour, because a variable could hold anything.

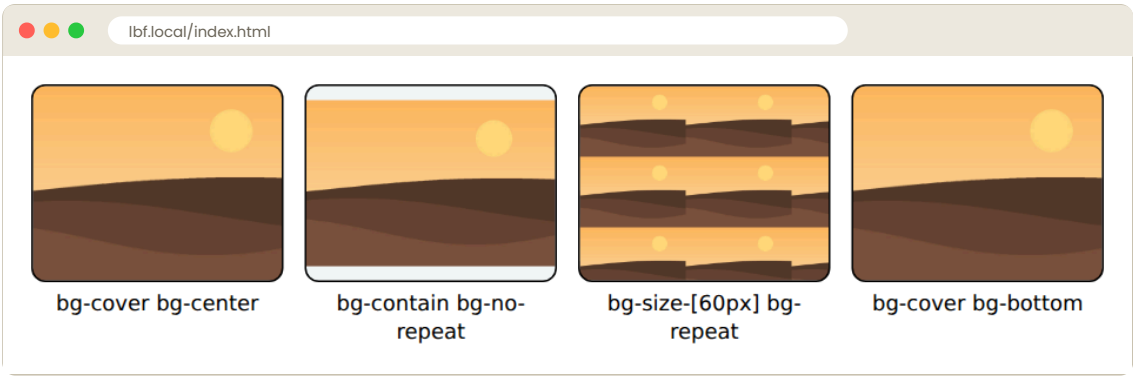
5.1.2 bg-cover, bg-center, bg-no-repeat, bg-fixed

Group	Utilities	CSS
Size	`bg-cover`, `bg-contain`, `bg-auto`, `bg-size-[200px]`	background-size
Position	`bg-center`, `bg-top`, `bg-bottom-right` …, `bg-position-[30%_20%]`	background-position
Repeat	`bg-repeat`, `bg-no-repeat`, `bg-repeat-x`, `bg-repeat-space`	background-repeat
Attachment	`bg-fixed`, `bg-local`, `bg-scroll`	background-attachment
Clip	`bg-clip-text`, `bg-clip-padding`, `bg-clip-content`	background-clip
Origin	`bg-origin-border`, `bg-origin-padding`, `bg-origin-content`	background-origin

ONE IMAGE, FOUR SIZING AND POSITIONING COMBINATIONS

HTML

```
<div class="grid grid-cols-4 gap-3 text-center text-xs">
  <div><div class="h-28 rounded-lg border bg-slate-100 bg-[url(mysuru-palace.jpg)] bg-cover bg-
center"></div><p class="mt-1">bg-cover bg-center</p></div>
  <div><div class="h-28 rounded-lg border bg-slate-100 bg-[url(mysuru-palace.jpg)] bg-contain bg-
center bg-no-repeat"></div><p class="mt-1">bg-contain bg-no-repeat</p></div>
  <div><div class="h-28 rounded-lg border bg-slate-100 bg-[url(mysuru-palace.jpg)] bg-size-[60px] bg-
repeat"></div><p class="mt-1">bg-size-[60px] bg-repeat</p></div>
  <div><div class="h-28 rounded-lg border bg-slate-100 bg-[url(mysuru-palace.jpg)] bg-cover bg-
bottom"></div><p class="mt-1">bg-cover bg-bottom</p></div>
</div>
```

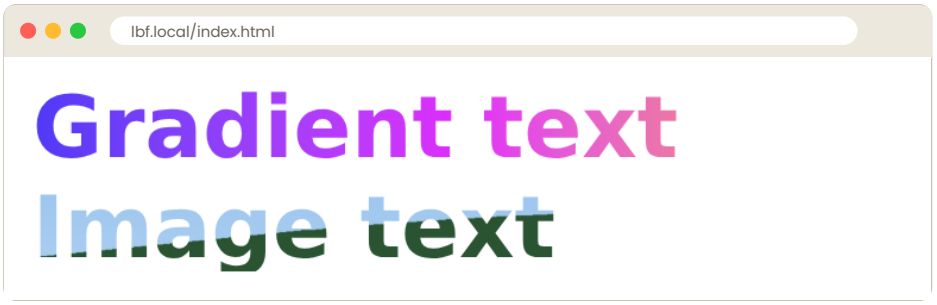


HOW THE BROWSER DISPLAYS IT

`bg-fixed` keeps the image still while the page scrolls (a parallax-like effect). It is ignored by iOS Safari and costs repainting on scroll, so treat it as an enhancement. `bg-clip-text` combined with `text-transparent` paints a gradient or image inside the letters:

INDEX.HTML HTML

```
<h2 class="bg-linear-to-r from-indigo-600 via-fuchsia-500 to-amber-500 bg-clip-text text-5xl font-black text-transparent">Gradient text</h2>
<h2 class="mt-2 bg-[url(hills.jpg)] bg-cover bg-center bg-clip-text text-5xl font-black text-transparent">Image text</h2>
```



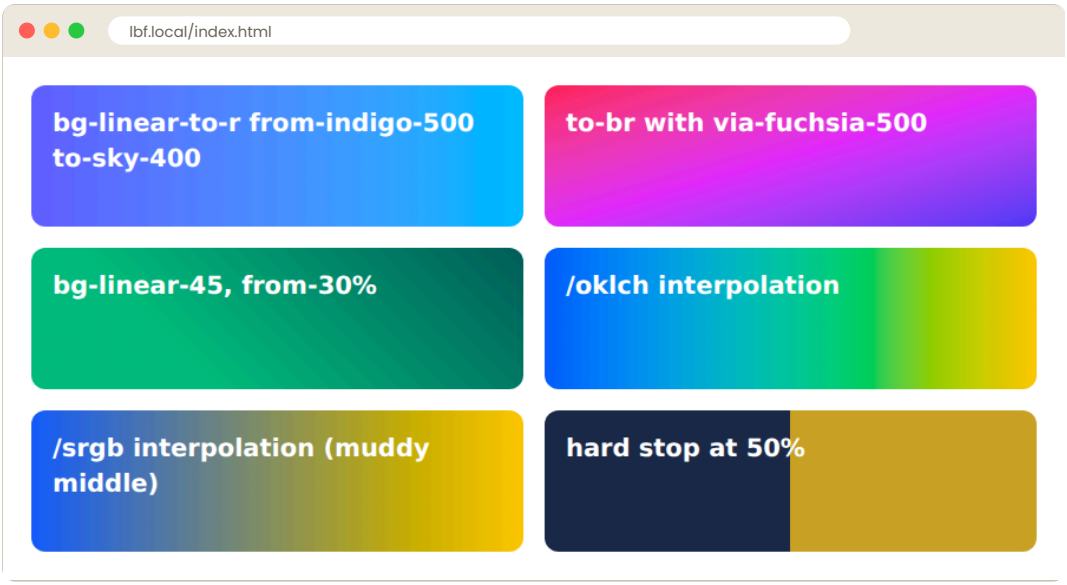
BG-CLIP-TEXT WITH TEXT-TRANSPARENT

5.1.3 Gradients: bg-linear-to-r, from-*, via-*, to-*

A gradient is built from a direction utility and colour stops. In v4 linear gradients are `bg-linear-to-{t, tr, r, br, b, bl, l, tl}` (v3 called them `bg-gradient-to-*`, which still works as an alias) or any angle: `bg-linear-45`, `-bg-linear-12`. The stops are `from-*`, optional `via-*` and `to-*`, each with optional positions: `from-10%`, `via-30%`, `to-90%`.

LINEAR GRADIENTS WITH DIRECTIONS, ANGLES, POSITIONS AND INTERPOLATION HTML

```
<div class="grid grid-cols-2 gap-3 text-sm font-semibold text-white">
  <div class="h-20 rounded-lg bg-linear-to-r from-indigo-500 to-sky-400 p-3">bg-linear-to-r from-indigo-500 to-sky-400</div>
  <div class="h-20 rounded-lg bg-linear-to-br from-rose-500 via-fuchsia-500 to-indigo-600 p-3">to-br with via-fuchsia-500</div>
  <div class="h-20 rounded-lg bg-linear-45 from-emerald-500 from-30% to-teal-800 p-3">bg-linear-45, from-30%</div>
  <div class="h-20 rounded-lg bg-linear-to-r/oklch from-blue-600 to-yellow-400 p-3">/oklch interpolation</div>
  <div class="h-20 rounded-lg bg-linear-to-r/srgb from-blue-600 to-yellow-400 p-3">/srgb interpolation (muddy middle)</div>
  <div class="h-20 rounded-lg bg-linear-to-r from-[#1C2B4A] from-50% to-[#C9A227] to-50% p-3">hard stop at 50%</div>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.bg-linear-45 {
  --tw-gradient-position: 45deg;
  @supports (background-image: linear-gradient(in lab, red, red)) {
    --tw-gradient-position: 45deg in oklab;
  }
  background-image: linear-gradient(var(--tw-gradient-stops));
}
.bg-linear-to-r {
  --tw-gradient-position: to right;
  @supports (background-image: linear-gradient(in lab, red, red)) {
    --tw-gradient-position: to right in oklab;
  }
  background-image: linear-gradient(var(--tw-gradient-stops));
}
.from-indigo-500 {
  --tw-gradient-from: var(--color-indigo-500);
  --tw-gradient-stops: var(--tw-gradient-via-stops, var(--tw-gradient-position), var(--tw-gradient-
from) var(--tw-gradient-from-position), var(--tw-gradient-to) var(--tw-gradient-to-position));
}
```

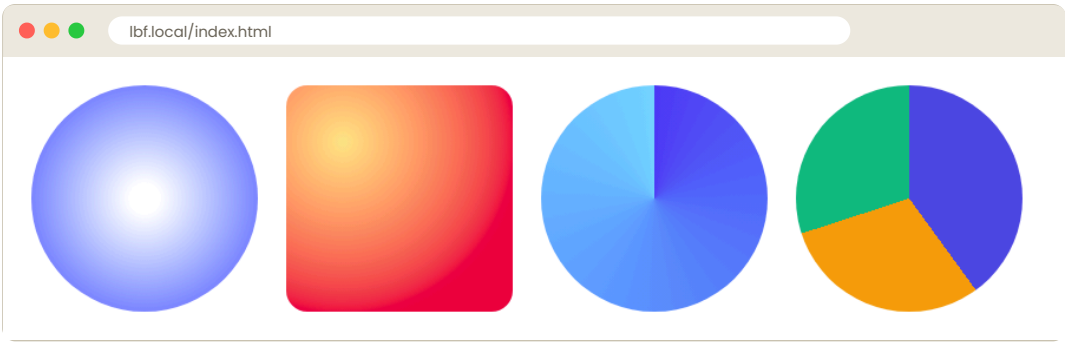
The interpolation modifier (`/oklch`, `/srgb`, `/hsl`, `/longer`) chooses the colour space the gradient travels through. Tailwind 4 defaults to **oklab**, which avoids the grey, muddy middle you get from blue to yellow in sRGB, as the figure shows.

5.1.4 Radial and conic gradients

v4 adds `bg-radial` (with an optional position: `bg-radial-[at_25%_25%]`) and `bg-conic` (with a starting angle: `bg-conic-180`). They use the same `from-`, `via-`, `to-` stops. Conic gradients make pie charts and colour wheels; radial gradients make spotlights and soft glows.

RADIAL (CENTRE AND OFFSET) AND CONIC (SMOOTH AND A PIE CHART) — HTML

```
<div class="flex gap-4">
  <div class="size-32 rounded-full bg-radial from-white from-10% to-indigo-600"></div>
  <div class="size-32 rounded-xl bg-radial-[at_25%_25%] from-amber-200 to-rose-600 to-75%"></div>
  <div class="size-32 rounded-full bg-conic from-indigo-600 to-sky-300"></div>
  <div class="size-32 rounded-full bg-conic-[from_0deg,_#4f46e5_0_40%,_#f59e0b_40%_70%,_#10b981_70%]">
</div>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.bg-conic {
  --tw-gradient-position: in oklab;
  background-image: conic-gradient(var(--tw-gradient-stops));
}
.bg-radial {
  --tw-gradient-position: in oklab;
  background-image: radial-gradient(var(--tw-gradient-stops));
}
.bg-radial-\[at_25\%_25\%\] {
  --tw-gradient-position: at 25% 25%;
  background-image: radial-gradient(var(--tw-gradient-stops,at 25% 25%));
}
```

5.2 Borders

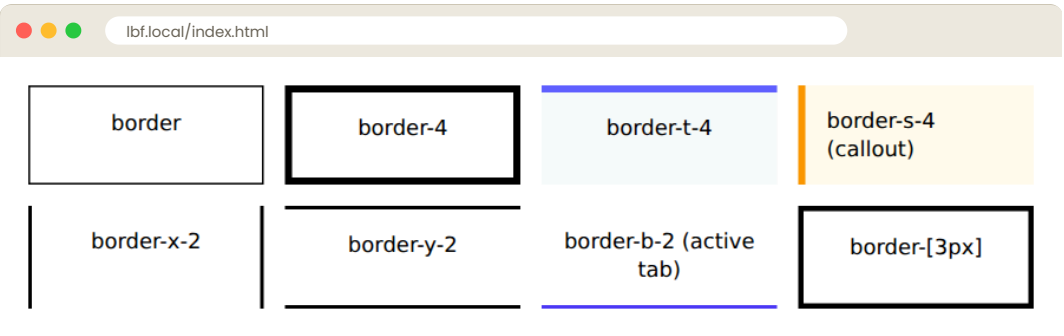
5.2.1 border, border-2, border-t, side-specific borders

`border` adds a 1px border on all sides; `border-0`, `border-2`, `border-4`, `border-8` and `border-[3px]` set the width. Side utilities follow the same pattern as padding: `border-t`, `border-r-4`, `border-x`, `border-y-2`, and logical `border-s` / `border-e`. Because Preflight sets `border: 0 solid` on every element, a width alone is enough to draw a visible line.

BORDER WIDTHS AND SIDES

HTML

```
<div class="grid grid-cols-4 gap-3 text-center text-xs">
  <div class="border p-3">border</div>
  <div class="border-4 p-3">border-4</div>
  <div class="border-t-4 border-indigo-500 bg-slate-50 p-3">border-t-4</div>
  <div class="border-s-4 border-amber-500 bg-amber-50 p-3 text-left">border-s-4 (callout)</div>
  <div class="border-x-2 p-3">border-x-2</div>
  <div class="border-y-2 p-3">border-y-2</div>
  <div class="border-b-2 border-indigo-600 p-3">border-b-2 (active tab)</div>
  <div class="border-[3px] p-3">border-[3px]</div>
</div>
```



HOW THE BROWSER DISPLAYS IT

NOTE

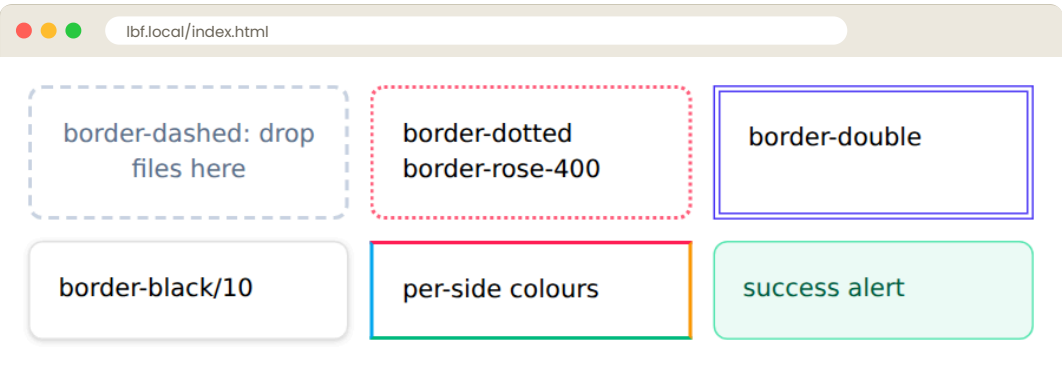
In v3 a bare `border` was grey (gray-200). In v4 the default border colour is `currentColor`, the text colour, so always pair a border with a colour utility such as `border-slate-200`. This is one of the most visible changes when upgrading (Module 16).

5.2.2 Border colours and styles

Colour with `border-{colour}` or per side (`border-t-rose-500`), with opacity modifiers (`border-black/10`, a subtle line that works on any background). Styles are `border-solid` (default), `border-dashed`, `border-dotted`, `border-double` and `border-none`.

BORDER COLOURS AND STYLES HTML

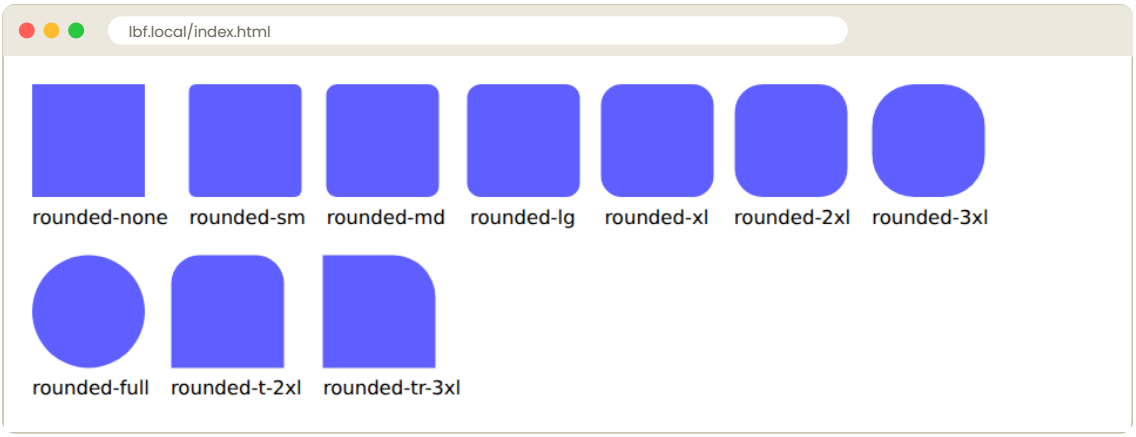
```
<div class="grid grid-cols-3 gap-3 text-sm">
  <div class="rounded-lg border-2 border-dashed border-slate-300 p-4 text-center text-slate-500">border-dashed: drop files here</div>
  <div class="rounded-lg border-2 border-dotted border-rose-400 p-4">border-dotted border-rose-400</div>
  <div class="border-4 border-double border-indigo-600 p-4">border-double</div>
  <div class="rounded-lg border border-black/10 p-4 shadow-sm">border-black/10</div>
  <div class="border-2 border-t-rose-500 border-r-amber-500 border-b-emerald-500 border-l-sky-500 p-4">per-side colours</div>
  <div class="rounded-lg border border-emerald-300 bg-emerald-50 p-4 text-emerald-800">success alert</div>
</div>
```



HOW THE BROWSER DISPLAYS IT

5.2.3 Rounded corners: rounded, rounded-lg, rounded-full

Radius utilities read `--radius-*` theme variables: `rounded-xs` (2px), `rounded-sm` (4px), `rounded-md` (6px), `rounded-lg` (8px), `rounded-xl` (12px), `rounded-2xl` (16px), `rounded-3xl` (24px), `rounded-4xl` (32px), plus `rounded-none` and `rounded-full` (a pill or circle). Individual sides and corners: `rounded-t-lg`, `rounded-tr-xl`, logical `rounded-s-lg`, `rounded-ee-xl`.



THE RADIUS SCALE

COMPILED CSS

```
.rounded {
  border-radius: 0.25rem;
}
.rounded-full {
  border-radius: calc(infinity * 1px);
}
.rounded-lg {
  border-radius: var(--radius-lg);
}
.rounded-t-2xl {
  border-top-left-radius: var(--radius-2xl);
  border-top-right-radius: var(--radius-2xl);
}
```

NOTE

`rounded` with no suffix was renamed in v4: it now means 4px (the old `rounded` value), but `rounded-sm` became 4px and the new `rounded-xs` is 2px. The same shift happened to shadows and blur. The upgrade tool renames these automatically.

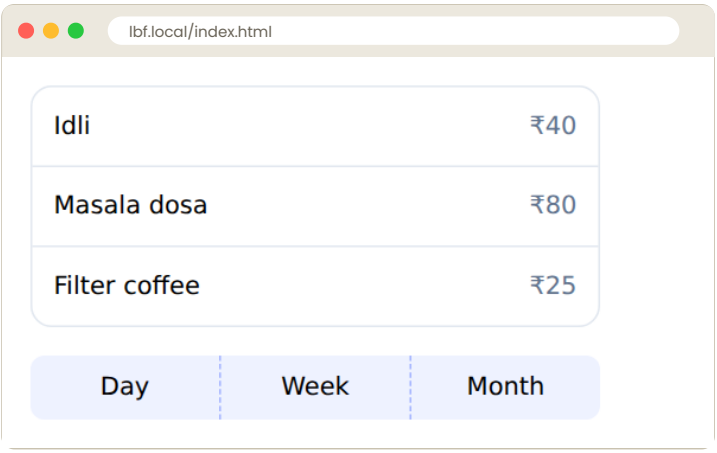
5.2.4 Dividers between children: divide-y, divide-*

`divide-y` draws a border **between** stacked children (a top border on every child except the first) and `divide-x` does it horizontally. Add a colour and style: `divide-slate-200`, `divide-dashed`. It is ideal for lists, menus and table-like rows, because you never have to remove the line from the last item.

DIVIDE-Y FOR A MENU, DIVIDE-X FOR A SEGMENTED CONTROL

HTML

```
<ul class="w-80 divide-y divide-slate-200 rounded-xl border border-slate-200 text-sm">
  <li class="flex justify-between p-3"><span>Idli</span><span class="text-slate-500">&#8377;40</span>
</li>
  <li class="flex justify-between p-3"><span>Masala dosa</span><span class="text-slate-
500">&#8377;80</span></li>
  <li class="flex justify-between p-3"><span>Filter coffee</span><span class="text-slate-
500">&#8377;25</span></li>
</ul>
<div class="mt-4 flex w-80 divide-x divide-dashed divide-indigo-300 rounded-lg bg-indigo-50 text-
center text-sm">
  <div class="flex-1 p-2">Day</div><div class="flex-1 p-2">Week</div><div class="flex-1 p-
2">Month</div>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.divide-y {
  :where(& > :not(:last-child)) {
    --tw-divide-y-reverse: 0;
    border-bottom-style: var(--tw-border-style);
    border-top-style: var(--tw-border-style);
    border-top-width: calc(1px * var(--tw-divide-y-reverse));
    border-bottom-width: calc(1px * calc(1 - var(--tw-divide-y-reverse)));
  }
}
.divide-slate-200 {
  :where(& > :not(:last-child)) {
    border-color: var(--color-slate-200);
  }
}
```

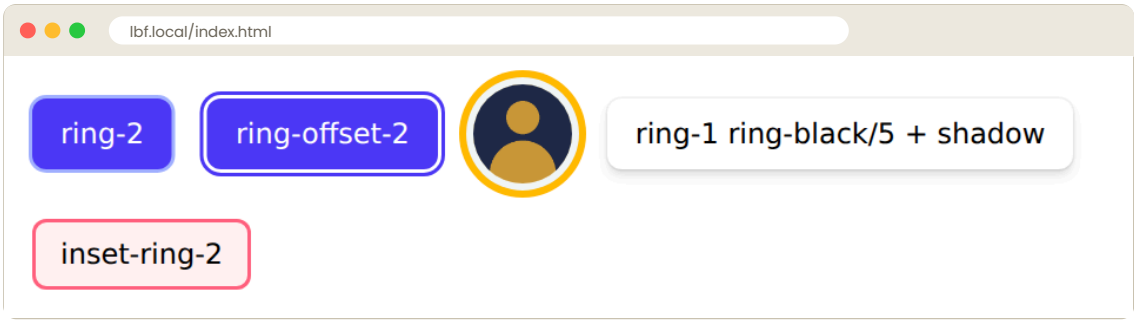
5.3 Rings and outlines

5.3.1 ring-* and ring-offset-*

A **ring** is a box-shadow that looks like an extra border, but it takes up no space and does not shift the layout. `ring` is 1px in v4 (3px in v3), `ring-2`, `ring-4` and so on are thicker; colour with `ring-indigo-500`. `ring-offset-2` leaves a gap between element and ring, filled with `ring-offset-white` (or any colour). `inset-ring` draws it on the inside. Rings are combined with shadows, because Tailwind composes them through separate variables.

RINGS WITH OFFSETS AND INSET RINGS — HTML

```
<div class="flex flex-wrap items-center gap-5">
  <button class="rounded-lg bg-indigo-600 px-4 py-2 text-white ring-2 ring-indigo-300">ring-2</button>
  <button class="rounded-lg bg-indigo-600 px-4 py-2 text-white ring-2 ring-indigo-600 ring-offset-
2">ring-offset-2</button>
  <img src="avatar.png" alt="" class="size-14 rounded-full ring-4 ring-amber-400 ring-offset-4 ring-
offset-slate-100">
  <div class="rounded-lg bg-white px-4 py-2 shadow-md ring-1 ring-black/5">ring-1 ring-black/5 +
shadow</div>
  <div class="rounded-lg bg-rose-50 px-4 py-2 inset-ring-2 inset-ring-rose-400">inset-ring-2</div>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.ring-2 {
  --tw-ring-shadow: var(--tw-ring-inset,) 0 0 0 calc(2px + var(--tw-ring-offset-width)) var(--tw-ring-color, currentcolor);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow), var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}
.ring-offset-2 {
  --tw-ring-offset-width: 2px;
  --tw-ring-offset-shadow: var(--tw-ring-inset,) 0 0 0 var(--tw-ring-offset-width) var(--tw-ring-offset-color);
}
```

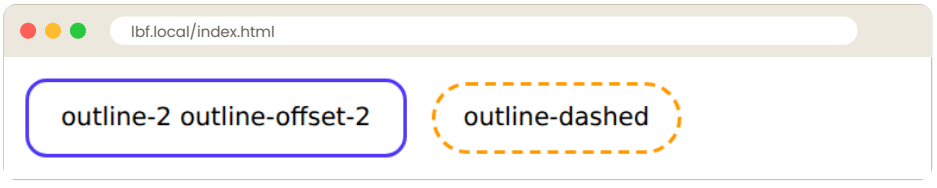
5.3.2 outline-*

Outlines are the browser's native focus indicator and, unlike rings, they follow the border radius in modern browsers and show up in Windows High Contrast mode. Utilities: `outline` (1px solid), `outline-2`, `outline-offset-2`, `outline-indigo-500`, `outline-dashed`, and `outline-hidden`. In v4, `outline-hidden` hides the outline visually but keeps a transparent one for forced-colours mode, while `outline-none` really sets `outline-style: none`.

OUTLINES FOLLOW THE BORDER RADIUS

HTML

```
<div class="flex gap-5 text-sm">
  <button class="rounded-lg bg-white px-4 py-2 outline-2 outline-offset-2 outline-indigo-600">outline-2 outline-offset-2</button>
  <button class="rounded-full bg-white px-4 py-2 outline-2 outline-dashed outline-amber-500">outline-dashed</button>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.outline-hidden {
  --tw-outline-style: none;
  outline-style: none;
  @media (forced-colors: active) {
    outline: 2px solid transparent;
    outline-offset: 2px;
  }
}
.outline-2 {
  outline-style: var(--tw-outline-style);
  outline-width: 2px;
}
.outline-offset-2 {
  outline-offset: 2px;
}
.outline-indigo-600 {
  outline-color: var(--color-indigo-600);
}
```

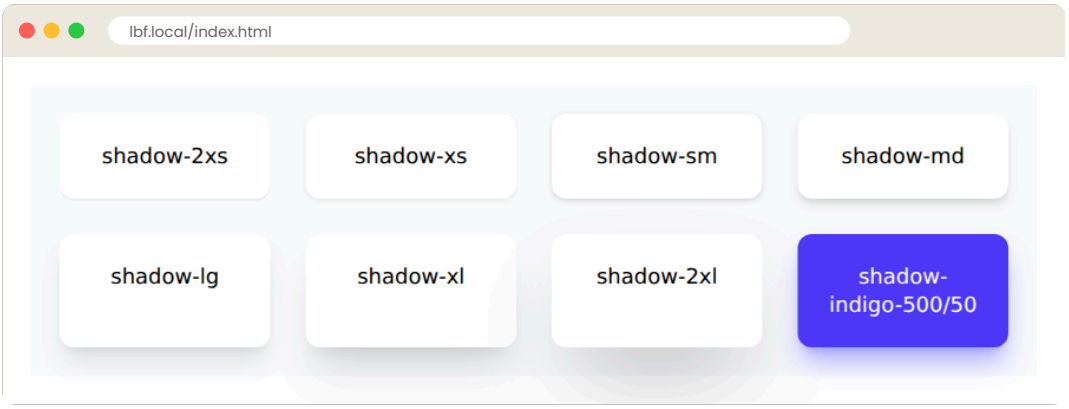
WATCH OUT

Never remove focus indication without replacing it. The common pattern is `focus:outline-hidden focus-visible:ring-2 focus-visible:ring-indigo-500`, which hides the mouse-click outline but shows a ring for keyboard users (Module 6).

5.4 Effects

5.4.1 Shadows: shadow-sm to shadow-2xl, coloured shadows

The shadow scale is `shadow-2xs`, `shadow-xs`, `shadow-sm`, `shadow-md`, `shadow-lg`, `shadow-xl`, `shadow-2xl` and `shadow-none`. Coloured shadows use a colour utility, usually with opacity: `shadow-lg shadow-indigo-500/40`. A coloured glow under a coloured button looks more natural than a grey shadow.



THE SHADOW SCALE AND A COLOURED SHADOW

COMPILED CSS

```
.shadow-lg {
  --tw-shadow: 0 10px 15px -3px var(--tw-shadow-color, rgb(0 0 0 / 0.1)), 0 4px 6px -4px var(--tw-shadow-color, rgb(0 0 0 / 0.1));
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow), var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}
.shadow-indigo-500\/50 {
  --tw-shadow-color: color-mix(in srgb, oklch(58.5% 0.233 277.117) 50%, transparent);
  @supports (color: color-mix(in lab, red, red)) {
    --tw-shadow-color: color-mix(in oklab, color-mix(in oklab, var(--color-indigo-500) 50%, transparent) var(--tw-shadow-alpha), transparent);
  }
}
```

The generated CSS shows how Tailwind composes effects. Each effect writes its own variable (`--tw-shadow`, `--tw-ring-shadow`, `--tw-inset-shadow`…) and every one of them sets the same full `box-shadow` list, so a ring and a shadow on one element do not overwrite each other.

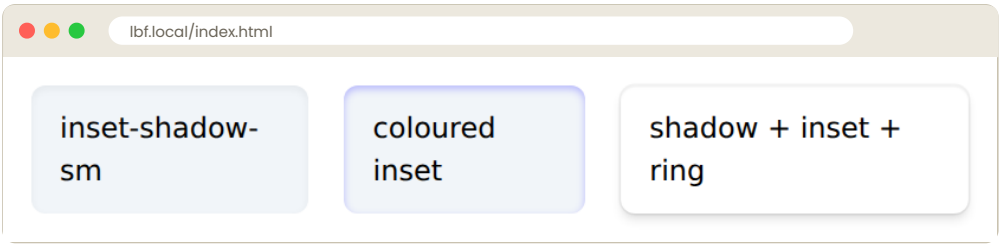
5.4.2 Inset shadows

v4 adds `inset-shadow-2xs`, `inset-shadow-xs` and `inset-shadow-sm` for pressed or recessed surfaces such as input fields, with colours like `inset-shadow-black/20`. They can be stacked with a normal shadow and a ring on the same element, up to four layers.

INSET SHADOWS

HTML

```
<div class="flex gap-5">
  <div class="rounded-lg bg-slate-100 px-4 py-3 inset-shadow-sm">inset-shadow-sm</div>
  <div class="rounded-lg bg-slate-100 px-4 py-3 inset-shadow-sm inset-shadow-indigo-500/40">coloured inset</div>
  <div class="rounded-lg bg-white px-4 py-3 shadow-md inset-shadow-xs ring-1 ring-black/5">shadow + inset + ring</div>
</div>
```



HOW THE BROWSER DISPLAYS IT

5.4.3 opacity-*

`opacity-0` to `opacity-100` (any number, as a percentage) fades the whole element including its children. Remember the difference from the colour modifier: `bg-black/50` makes only the background translucent, `opacity-50` fades everything. `opacity-0` elements still take up space and receive clicks unless you also add `pointer-events-none` or `invisible`.

5.4.4 Blend modes

`mix-blend-multiply`, `mix-blend-screen`, `mix-blend-overlay`, `mix-blend-difference` and the other CSS blend modes control how an element blends with what is behind it. `bg-blend-*` blends an element's own background layers (a colour with its background image), which is a quick way to tint photos.

BACKGROUND AND MIX BLEND MODES HTML

```
<div class="grid grid-cols-3 gap-3 text-center text-xs">
  <div><div class="h-24 rounded-lg bg-[url(mysuru-palace.jpg)] bg-cover"></div><p class="mt-
1">original</p></div>
  <div><div class="h-24 rounded-lg bg-indigo-500 bg-[url(mysuru-palace.jpg)] bg-cover bg-blend-
multiply"></div><p class="mt-1">bg-indigo-500 bg-blend-multiply</p></div>
  <div><div class="h-24 rounded-lg bg-amber-400 bg-[url(mysuru-palace.jpg)] bg-cover bg-blend-
luminosity"></div><p class="mt-1">bg-amber-400 bg-blend-luminosity</p></div>
</div>
<div class="relative mt-4 h-20 overflow-hidden rounded-lg bg-[url(hills.jpg)] bg-cover">
  <p class="absolute inset-0 grid place-items-center text-4xl font-black text-white mix-blend-
difference">mix-blend-difference</p>
</div>
```

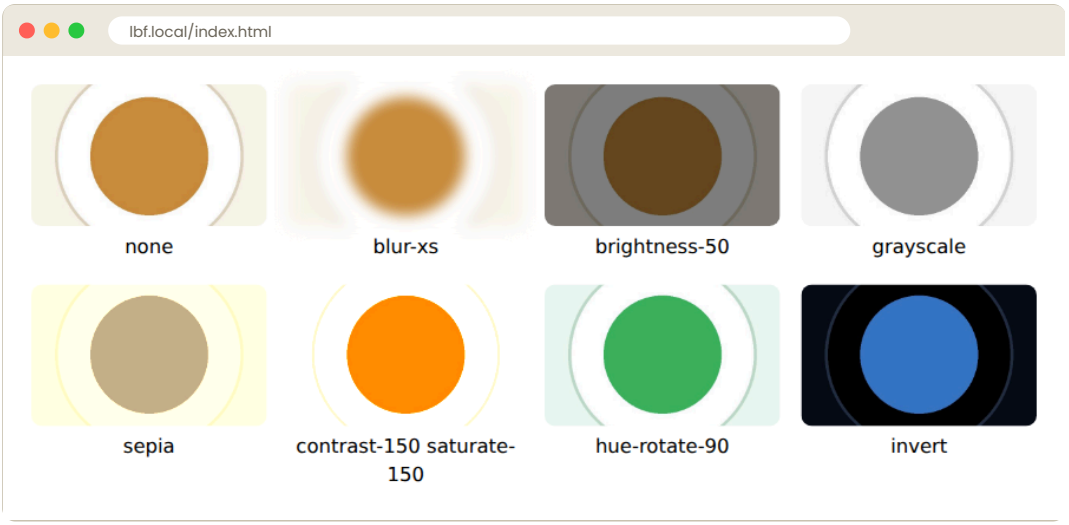


HOW THE BROWSER DISPLAYS IT

5.5 Filters

5.5.1 blur-*, brightness-*, grayscale, drop-shadow-*

Filter utilities apply CSS `filter` functions and can be combined freely (Tailwind joins them through variables): `blur-sm` to `blur-3xl`, `brightness-50` to `brightness-150`, `contrast-*`, `grayscale`, `sepia`, `invert`, `saturate-*`, `hue-rotate-90` and `drop-shadow-*`. Unlike `box-shadow`, `drop-shadow` follows the shape of the visible pixels, so it works on transparent PNG and SVG icons.



FILTERS APPLIED TO THE SAME PHOTO

INDEX.HTML HTML

```html
<div class="flex items-center gap-8">
  <img src="logo.svg" alt="" class="h-16 shadow-lg">
  <img src="logo.svg" alt="" class="h-16 drop-shadow-lg drop-shadow-indigo-500/50">
</div>
```



SHADOW-LG DRAWS A RECTANGLE; DROP-SHADOW-LG FOLLOWS THE LOGO'S SHAPE

COMPILED CSS

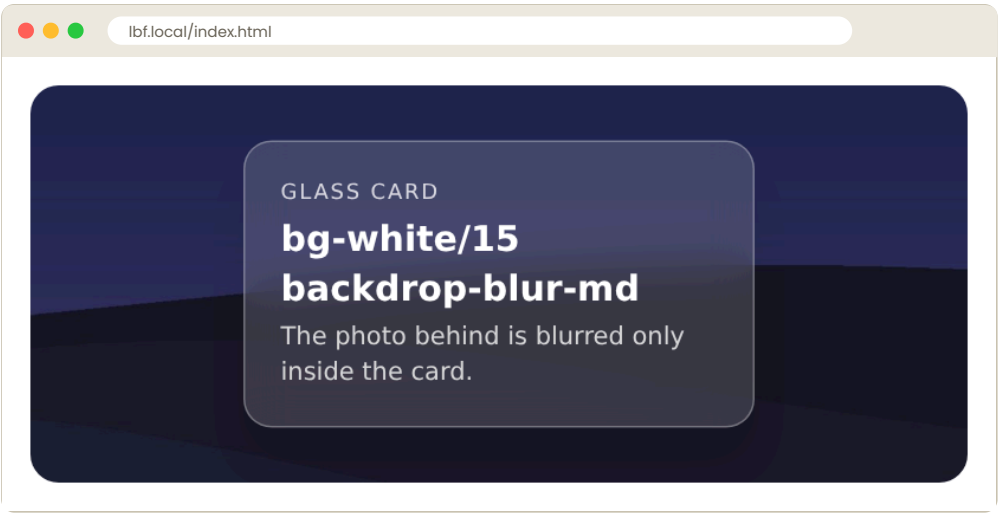
```css
.brightness-50 {
  --tw-brightness: brightness(50%);
  filter: var(--tw-blur,) var(--tw-brightness,) var(--tw-contrast,) var(--tw-grayscale,) var(--tw-hue-rotate,) var(--tw-invert,) var(--tw-saturate,) var(--tw-sepia,) var(--tw-drop-shadow,);
}
.drop-shadow-lg {
  --tw-drop-shadow-size: drop-shadow(0 4px 4px var(--tw-drop-shadow-color, rgb(0 0 0 / 0.15)));
  --tw-drop-shadow: drop-shadow(var(--drop-shadow-lg));
  filter: var(--tw-blur,) var(--tw-brightness,) var(--tw-contrast,) var(--tw-grayscale,) var(--tw-hue-rotate,) var(--tw-invert,) var(--tw-saturate,) var(--tw-sepia,) var(--tw-drop-shadow,);
}
.grayscale {
  --tw-grayscale: grayscale(100%);
  filter: var(--tw-blur,) var(--tw-brightness,) var(--tw-contrast,) var(--tw-grayscale,) var(--tw-hue-rotate,) var(--tw-invert,) var(--tw-saturate,) var(--tw-sepia,) var(--tw-drop-shadow,);
}
```

5.5.2 Backdrop filters: backdrop-blur-* (glassmorphism)

Backdrop filters apply to whatever is **behind** the element, seen through its translucent background. The "frosted glass" or glassmorphism look is a semi-transparent background, a backdrop blur, a light border and a shadow: `bg-white/20 backdrop-blur-md border border-white/30 shadow-lg`. Other backdrop utilities mirror the filters: `backdrop-brightness-*`, `backdrop-saturate-*`, `backdrop-grayscale`. Sticky headers often use `bg-white/70 backdrop-blur` so content scrolling beneath stays faintly visible.

GLASSMORPHISM WITH BACKDROP-BLUR HTML

```
<div class="relative grid h-56 place-items-center overflow-hidden rounded-2xl bg-[url(night.jpg)] bg-cover bg-center">
  <div class="w-72 rounded-2xl border border-white/30 bg-white/15 p-5 text-white shadow-xl backdrop-blur-md">
    <p class="text-xs uppercase tracking-widest text-white/80">Glass card</p>
    <p class="mt-1 text-xl font-semibold">bg-white/15 backdrop-blur-md</p>
    <p class="mt-1 text-sm text-white/80">The photo behind is blurred only inside the card.</p>
  </div>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.backdrop-blur-md {
  --tw-backdrop-blur: blur(var(--blur-md));
  -webkit-backdrop-filter: var(--tw-backdrop-blur,) var(--tw-backdrop-brightness,) var(--tw-backdrop-contrast,) var(--tw-backdrop-grayscale,) var(--tw-backdrop-hue-rotate,) var(--tw-backdrop-invert,) var(--tw-backdrop-opacity,) var(--tw-backdrop-saturate,) var(--tw-backdrop-sepia,);
  backdrop-filter: var(--tw-backdrop-blur,) var(--tw-backdrop-brightness,) var(--tw-backdrop-contrast,) var(--tw-backdrop-grayscale,) var(--tw-backdrop-hue-rotate,) var(--tw-backdrop-invert,) var(--tw-backdrop-opacity,) var(--tw-backdrop-saturate,) var(--tw-backdrop-sepia,);
}
```

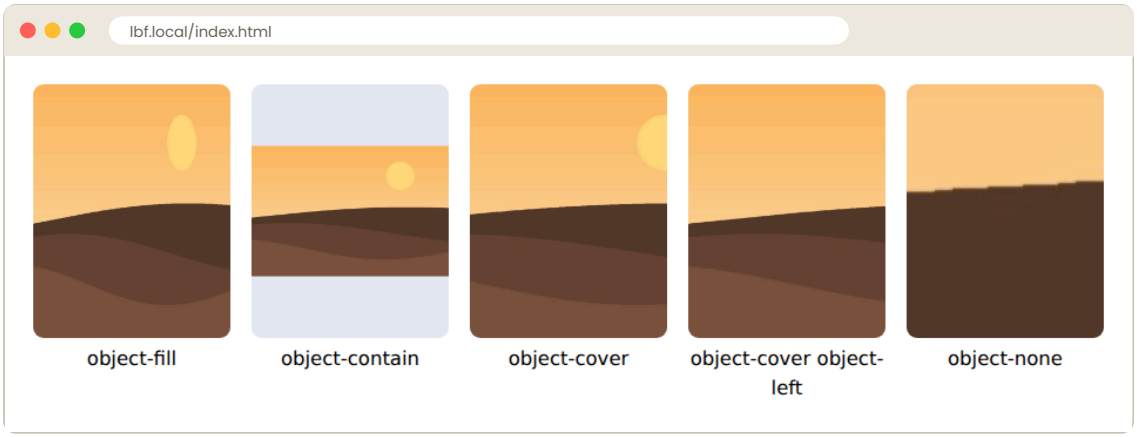
TIP

Glass effects need enough contrast for the text on top. Check the card text against the brightest part of the background, and consider a slightly more opaque background (`bg-black/30`) for important content.

5.6 Images

5.6.1 object-cover, object-contain, object-center

When an `<img>` or `<video>` is given a fixed width and height, `object-fit` decides how the picture fills the box. `object-cover` fills and crops (thumbnails, avatars, cards), `object-contain` shows the whole picture with empty bands, `object-fill` stretches (usually wrong), `object-none` keeps the original size and `object-scale-down` uses the smaller of none and contain. `object-top`, `object-center`, `object-bottom` and `object-[25%_75%]` choose which part stays visible when cropping.



OBJECT-FIT AND OBJECT-POSITION ON A 24 × 24 UNIT BOX

COMPILED CSS

```
.object-cover {
  object-fit: cover;
}
.object-\[25\%_75\%\] {
  object-position: 25% 75%;
}
.object-center {
  object-position: center;
}
```

PRACTICE CHECKPOINT

A **hero section** with a background image and gradient overlay, **glassmorphism cards** with backdrop blur, and **buttons with rings and coloured shadows**.

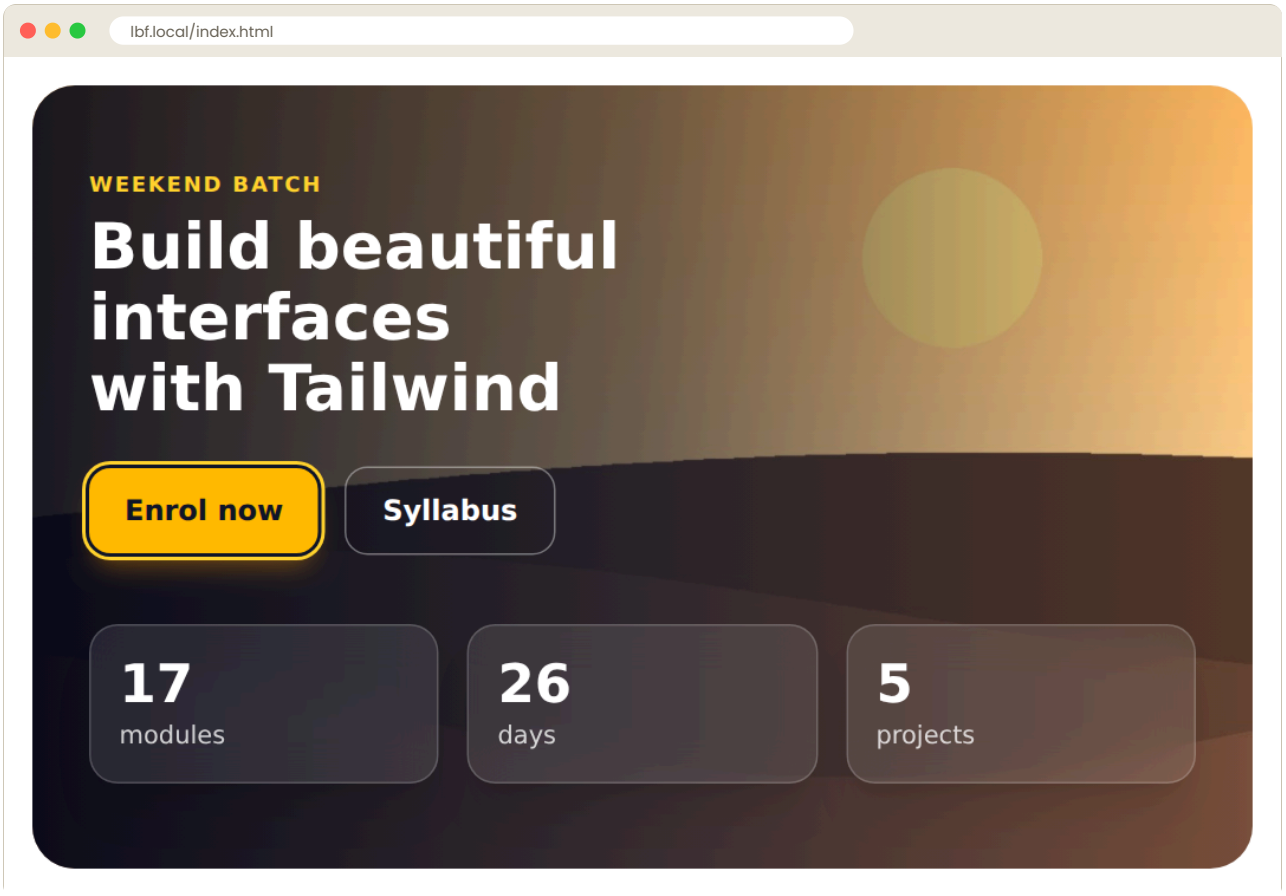
HERO.HTML: SAMPLE SOLUTION

HTML

```
<section class="relative overflow-hidden rounded-3xl bg-[url(mysuru-palace.jpg)] bg-cover bg-center">
  <!-- gradient overlay for text contrast -->
  <div class="absolute inset-0 bg-linear-to-r from-slate-950/90 via-slate-900/60 to-transparent">
</div>

  <div class="relative px-8 py-12">
    <p class="text-xs font-semibold uppercase tracking-widest text-amber-300">Weekend batch</p>
    <h1 class="mt-2 max-w-md text-4xl font-black text-white text-balance">Build beautiful interfaces
with Tailwind</h1>
    <div class="mt-6 flex gap-4">
      <a href="#" class="rounded-xl bg-amber-400 px-5 py-3 font-semibold text-slate-900 shadow-lg
shadow-amber-500/40 ring-2 ring-amber-300 ring-offset-2 ring-offset-slate-900">Enrol now</a>
      <a href="#" class="rounded-xl px-5 py-3 font-semibold text-white ring-1 ring-white/40 hover:bg-
white/10">Syllabus</a>
    </div>

    <div class="mt-10 grid grid-cols-3 gap-4">
      <div class="rounded-2xl border border-white/20 bg-white/10 p-4 text-white shadow-xl backdrop-
blur-md">
        <p class="text-3xl font-black">17</p><p class="text-sm text-white/80">modules</p>
      </div>
      <div class="rounded-2xl border border-white/20 bg-white/10 p-4 text-white shadow-xl backdrop-
blur-md">
        <p class="text-3xl font-black">26</p><p class="text-sm text-white/80">days</p>
      </div>
      <div class="rounded-2xl border border-white/20 bg-white/10 p-4 text-white shadow-xl backdrop-
blur-md">
        <p class="text-3xl font-black">5</p><p class="text-sm text-white/80">projects</p>
      </div>
    </div>
  </div>
</section>
```



THE HERO SECTION

Gradients are a direction plus `from-` / `via-` / `to-` stops; borders need an explicit colour in v4; rings, shadows and inset shadows compose instead of overriding; `backdrop-blur` plus a translucent background gives the glass effect; `object-cover` crops images to any box.

MODULE 06 · BEGINNER

State variants

Minimum duration **1 day** | Level **Beginner**

Variants apply a utility only in a certain state, by adding a prefix and a colon: hover:bg-indigo-700 applies only while the pointer is over the element. This module covers the variant syntax and stacking; interactive states (hover, focus, focus-visible, focus-within, active, visited, disabled); structural variants (first, last, odd, even, only, nth-*, empty); form variants (checked, required, invalid, user-invalid, placeholder-shown, placeholder, file); pseudo-elements (before, after, selection, marker, first-letter); styling based on other elements with group, named groups, peer, has-*, not-* and the * child variant; and attribute variants for ARIA, data attributes and open elements. Interactive screenshots were produced by really hovering, focusing, typing and clicking in Chromium.

IN THIS MODULE

6.1 Variant basics
6.2 Interactive states
6.3 Structural variants
6.4 Form variants
6.5 Pseudo-elements
6.6 Styling based on other elements
6.7 Attribute variants

6.1 Variant basics

6.1.1 Syntax: hover:bg-blue-600

A **variant** is a prefix that wraps a utility in a condition: a pseudo-class (`:hover`), a pseudo-element (`::before`), a media query (`md:`), an attribute selector (`aria-expanded:`) and so on. The utility after the colon is unchanged, so every utility works with every variant. In plain CSS you would write a separate rule for each state; with variants every state lives on the element itself.

ONE CLASS PER STATE

HTML

```
<button class="rounded-lg bg-blue-500 px-4 py-2 font-semibold text-white hover:bg-blue-700">Hover me</button>
```



SCREENSHOT TAKEN WITH THE POINTER OVER THE BUTTON: HOVER:BG-BLUE-700 HAS TAKEN EFFECT

COMPILED CSS

```
.bg-blue-500 {
  background-color: var(--color-blue-500);
}
.hover\:bg-blue-700 {
  &:hover {
    @media (hover: hover) {
      background-color: var(--color-blue-700);
    }
  }
}
```

Two details in the generated CSS are worth knowing. The colon is escaped in the selector (`.hover\:bg-blue-700:hover`) because `:` is special in CSS. And in v4 the `hover:` variant is wrapped in `@media (hover: hover)`, so hover styles apply only on devices with a real pointer. On phones, tapping a button no longer leaves it stuck in its hover colour.

6.1.2 Stacking variants md:hover:underline

Variants can be chained, and each one adds its condition. `md:hover:underline` underlines on hover only at the `md` breakpoint and above; `dark:hover:bg-slate-700` is a hover colour for dark mode; `group-hover:disabled:opacity-50` combines a parent state with the element's own. In v4 stacked variants apply **left to right**, like reading nested CSS: `md:hover:` means "inside the md media query, on hover". (v3 applied them right to left, which mattered for a few variants such as `*:`.)

COMPILED CSS

```
.md\:hover\:underline {
  @media (width >= 48rem) {
    &:hover {
      @media (hover: hover) {
        text-decoration-line: underline;
      }
    }
  }
}
.dark\:hover\:bg-slate-700 {
  @media (prefers-color-scheme: dark) {
    &:hover {
      @media (hover: hover) {
        background-color: var(--color-slate-700);
      }
    }
  }
}
```

Variant family	Examples	Covered in
Pseudo-classes	`hover:`, `focus-visible:`, `first:`, `checked:`	this module
Pseudo-elements	`before:`, `placeholder:`, `file:`, `selection:`	this module
Relationship	`group-*:`, `peer-*:`, `has-*:`, `not-*:`, `*:`	this module
Attributes	`aria-*:`, `data-*:`, `open:`, `rtl:`	this module
Media and features	`sm:` to `2xl:`, `@md:`, `motion-reduce:`, `print:`, `supports-[...]:`	Module 9
Theme	`dark:` and custom variants	Modules 10 and 14

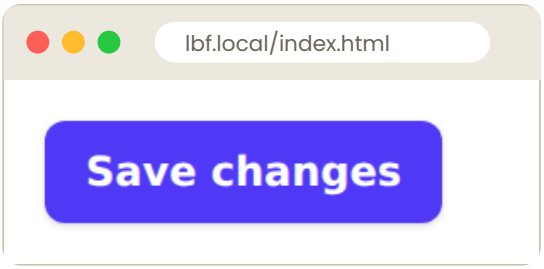
6.2 Interactive states

6.2.1 hover:, focus:, active:, visited:

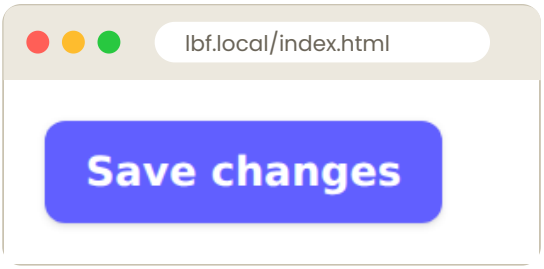
`hover:` applies while the pointer is over the element, `focus:` while it has focus (clicked or tabbed to), `active:` while it is being pressed, and `visited:` to links already visited (browsers only allow colour changes here, for privacy). A good button styles all of them.

A BUTTON WITH HOVER, ACTIVE AND FOCUS STYLES HTML

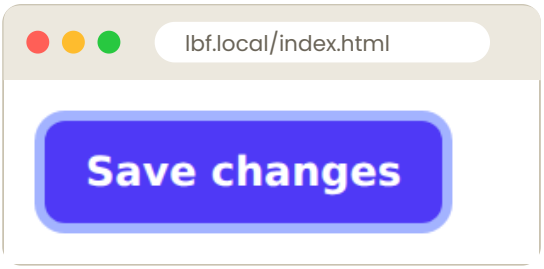
```
<button class="rounded-lg bg-indigo-600 px-4 py-2 font-semibold text-white shadow-sm transition hover:bg-indigo-500 active:scale-95 active:bg-indigo-700 focus:outline-hidden focus:ring-4 focus:ring-indigo-300">Save changes</button>
```



RESTING STATE



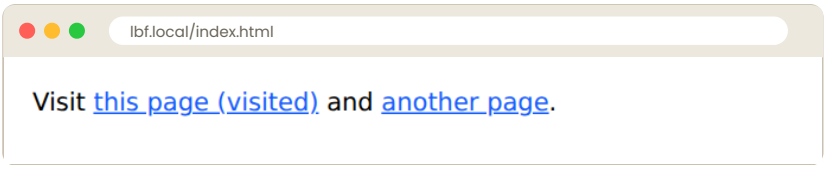
HOVER STATE: CAPTURED WITH THE POINTER OVER THE BUTTON



FOCUS STATE: CAPTURED AFTER FOCUSING THE BUTTON

INDEX.HTML HTML

```
<p class="text-sm">Visit <a href="index.html" class="text-blue-600 underline visited:text-purple-700">this page (visited)</a> and <a href="https://never-visited.example/" class="text-blue-600 underline visited:text-purple-700">another page</a>.</p>
```



VISITED: COLOURS THE LINK TO THE CURRENT PAGE (WHICH THE BROWSER HAS JUST VISITED) PURPLE

6.2.2 focus-visible: and focus-within:

`focus:` fires on every focus, including a mouse click, which is why designers often remove focus rings and hurt keyboard users. `focus-visible:` applies only when the browser decides a focus indicator is useful: when navigating with the keyboard, or always for text inputs. The accessible default is therefore `focus-visible:ring-*` (or outline) rather than `focus:ring-*`.



AFTER A MOUSE CLICK: FOCUSED, BUT FOCUS-VISIBLE DOES NOT MATCH, SO NO OUTLINE

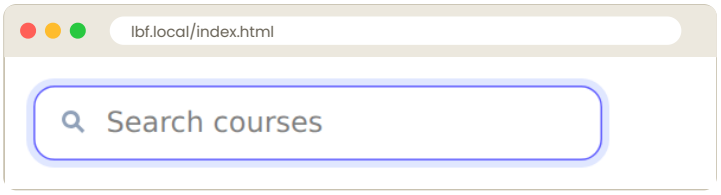


AFTER PRESSING TAB: FOCUS-VISIBLE MATCHES AND THE OUTLINE APPEARS

`focus-within:` styles a parent while **any** descendant has focus, for example highlighting a whole search bar when its input is focused.

FOCUS-WITHIN ON THE WRAPPER HTML

```
<label class="flex w-80 items-center gap-2 rounded-xl border border-slate-300 px-3 py-2 focus-within:border-indigo-500 focus-within:ring-4 focus-within:ring-indigo-100">
  <svg viewBox="0 0 20 20" class="size-5 fill-slate-400"><path d="M8 3a5 5 0 1 0 2.9 9.1l3.5 3.5 1.4-1.4-3.5-3.5A5 5 0 0 0 8 3Zm0 2a3 3 0 1 1 0 6 3 3 0 0 1 0-6Z"/></svg>
  <input class="flex-1 outline-hidden" placeholder="Search courses">
</label>
```



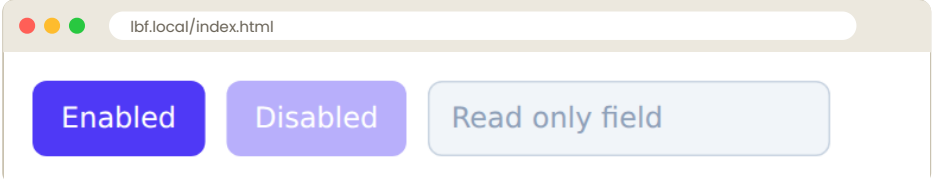
THE INPUT HAS FOCUS, SO THE WHOLE WRAPPER GETS THE BORDER AND RING

6.2.3 disabled:

`disabled:` targets form controls with the `disabled` attribute. A disabled button usually gets reduced opacity and a not-allowed cursor; and because hover styles still match on disabled elements, it is common to neutralise them too (`disabled:hover:bg-indigo-600`), or use `enabled:hover:` so hover only applies to enabled buttons.

DISABLED: AND ENABLED: HTML

```
<div class="flex gap-3">
  <button class="rounded-lg bg-indigo-600 px-4 py-2 text-white enabled:hover:bg-indigo-500
disabled:cursor-not-allowed disabled:opacity-40">Enabled</button>
  <button disabled class="rounded-lg bg-indigo-600 px-4 py-2 text-white enabled:hover:bg-indigo-500
disabled:cursor-not-allowed disabled:opacity-40">Disabled</button>
  <input disabled value="Read only field" class="rounded-lg border border-slate-300 px-3 py-2
disabled:bg-slate-100 disabled:text-slate-400">
</div>
```



HOW THE BROWSER DISPLAYS IT

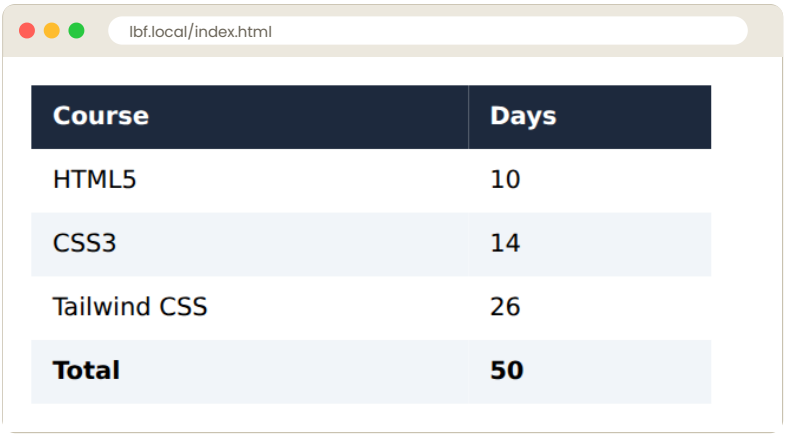
6.3 Structural variants

6.3.1 first:, last:, odd:, even:, only:

Structural variants select an element by its position among its siblings. They are applied to the **children** (often in a loop that renders the same markup for every item), not to the parent: `first:pt-0`, `last:border-0`, `odd:bg-white even:bg-slate-50` for zebra-striped rows, and `only:` for an element that has no siblings.

ODD:/EVEN: ZEBRA ROWS AND LAST: FOR THE TOTAL ROW HTML

```
<table class="w-96 text-left text-sm">
  <thead><tr class="bg-slate-800 text-white"><th class="px-3 py-2">Course</th><th class="px-3 py-
2">Days</th></tr></thead>
  <tbody>
    <tr class="odd:bg-white even:bg-slate-100"><td class="px-3 py-2">HTML5</td><td class="px-3 py-
2">10</td></tr>
    <tr class="odd:bg-white even:bg-slate-100"><td class="px-3 py-2">CSS3</td><td class="px-3 py-
2">14</td></tr>
    <tr class="odd:bg-white even:bg-slate-100"><td class="px-3 py-2">Tailwind CSS</td><td class="px-3
py-2">26</td></tr>
    <tr class="odd:bg-white even:bg-slate-100 last:font-semibold"><td class="px-3 py-2">Total</td><td
class="px-3 py-2">50</td></tr>
  </tbody>
</table>
```



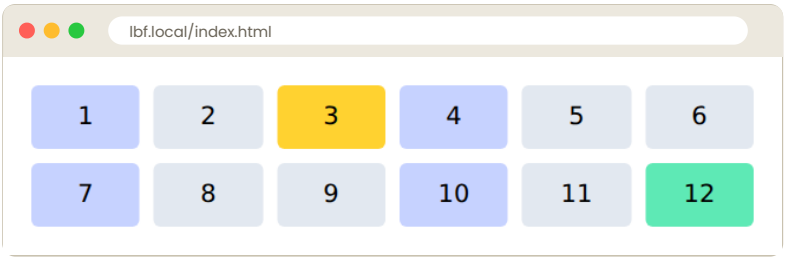
HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.first\:pt-0 {
  &:first-child {
    padding-top: calc(var(--spacing) * 0);
  }
}
.last\:font-semibold {
  &:last-child {
    --tw-font-weight: var(--font-weight-semibold);
    font-weight: var(--font-weight-semibold);
  }
}
.only\:mx-auto {
  &:only-child {
    margin-inline: auto;
  }
}
.odd\:bg-white {
  &:nth-child(odd) {
    background-color: var(--color-white);
  }
}
.even\:bg-slate-100 {
  &:nth-child(even) {
    background-color: var(--color-slate-100);
  }
}
```

6.3.2 nth-* variants

v4 adds `nth-3:` (exactly the third), `nth-[3n+1]:` (any formula), `nth-last-2:`, `nth-of-type-2:` and `nth-last-of-type-*:`, mapping to the CSS `:nth-child()` family.



NTH-[3N+1] IN INDIGO, NTH-3 IN AMBER, NTH-LAST-1 IN GREEN

EACH TILE

HTML

```
<div class="... nth-3:bg-amber-300 nth-[3n+1]:bg-indigo-200 nth-last-1:bg-emerald-300">1</div>
<!-- repeated 12 times -->
```

COMPILED CSS

```
.nth-3\:bg-amber-300 {
  &:nth-child(3) {
    background-color: var(--color-amber-300);
  }
}
.nth-\[3n\+1\]\:bg-indigo-200 {
  &:nth-child(3n+1) {
    background-color: var(--color-indigo-200);
  }
}
.nth-last-1\:bg-emerald-300 {
  &:nth-last-child(1) {
    background-color: var(--color-emerald-300);
  }
}
```

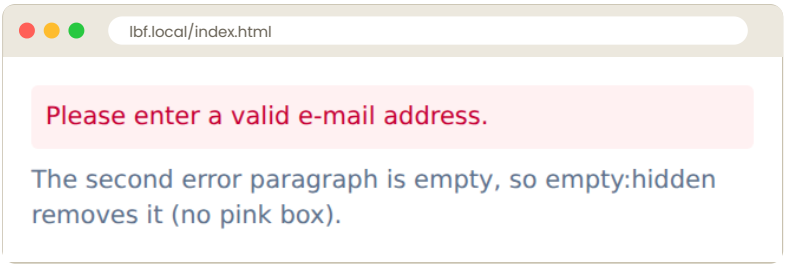
6.3.3 empty:

`empty:` matches an element with no children at all, not even text. It is handy to hide a container that a template sometimes leaves empty, such as an error message area: `empty:hidden`.

INDEX.HTML

HTML

```
<div class="space-y-2 text-sm">
  <p class="rounded bg-rose-50 p-2 text-rose-700 empty:hidden">Please enter a valid e-mail address.
</p>
  <p class="rounded bg-rose-50 p-2 text-rose-700 empty:hidden"></p>
  <p class="text-slate-500">The second error paragraph is empty, so empty:hidden removes it (no pink
box).</p>
</div>
```



EMPTY:HIDDEN

6.4 Form variants

6.4.1 checked:, required:, invalid:, user-invalid:, placeholder-shown:

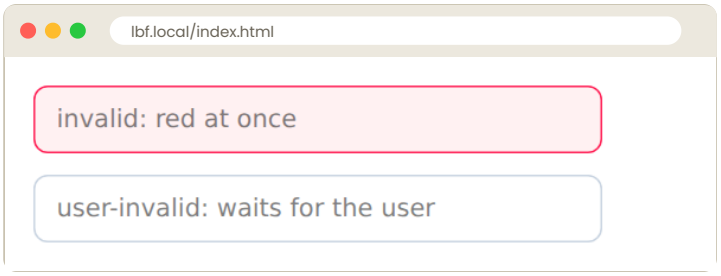
Variant	Matches
`checked:`	checked checkboxes and radio buttons
`indeterminate:`	checkboxes in the mixed state (set by JavaScript)
`required:` / `optional:`	inputs with or without the `required` attribute

Variant	Matches
`invalid:` / `valid:`	inputs failing or passing validation, **immediately**, even before the user types
`user-invalid:` / `user-valid:`	the same, but only after the user has interacted with the field (new in v4)
`placeholder-shown:`	inputs currently showing their placeholder (they are empty)
`read-only:`, `autofill:`, `in-range:`, `out-of-range:`	the matching CSS pseudo-classes

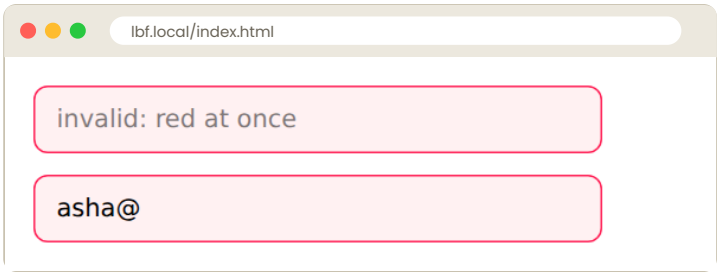
The difference between `invalid:` and `user-invalid:` matters a lot for user experience. An empty required field is invalid from the start, so `invalid:border-rose-500` paints a fresh form red before anyone has typed. `user-invalid:` waits until the user has changed the value and left the field:

TWO E-MAIL FIELDS HTML

```
<form class="w-80 space-y-3 text-sm">
  <input type="email" required placeholder="invalid: red at once" class="block w-full rounded-lg
border border-slate-300 px-3 py-2 invalid:border-rose-500 invalid:bg-rose-50">
  <input type="email" required placeholder="user-invalid: waits for the user" class="u block w-full
rounded-lg border border-slate-300 px-3 py-2 user-invalid:border-rose-500 user-invalid:bg-rose-50">
</form>
```



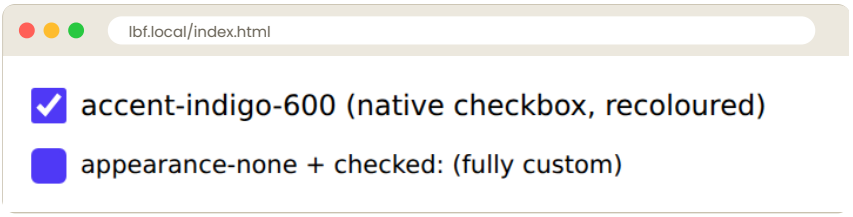
FRESH PAGE: THE FIRST FIELD IS ALREADY RED; THE SECOND WAITS



AFTER TYPING "ASHA@" IN THE SECOND FIELD AND LEAVING IT, USER-INVALID MATCHES

TWO WAYS TO STYLE A CHECKBOX HTML

```
<label class="flex items-center gap-2"><input type="checkbox" checked class="size-5 accent-indigo-
600"> accent-indigo-600 (native checkbox, recoloured)</label>
<label class="mt-3 flex items-center gap-2 text-sm"><input type="checkbox" checked class="size-5
appearance-none rounded border-2 border-slate-300 checked:border-indigo-600 checked:bg-indigo-600
checked:bg-[url('data:image/svg+xml;utf8,<svg xmlns=%22http://www.w3.org/2000/svg%22 viewBox=%220 0 16
16%22><path fill=%22white%22 d=%22M6.2 11.3 3 8.1l1.1-1.1 2.1 2.1 5-5L12.3 5z%22/></svg>')]">
appearance-none + checked: (fully custom)</label>
```



HOW THE BROWSER DISPLAYS IT

`accent-*` recolours native checkboxes, radios, range sliders and progress bars with one class. For complete control, remove the native look with `appearance-none` and draw the box yourself, using `checked:` for the ticked state.

INDEX.HTML HTML

```
<input placeholder="Your name" class="w-72 rounded-lg border-2 px-3 py-2 placeholder-shown:border-dashed placeholder-shown:border-slate-300 not-placeholder-shown:border-emerald-500">
<input value="Asha Rao" placeholder="Your name" class="mt-2 w-72 rounded-lg border-2 px-3 py-2 placeholder-shown:border-dashed placeholder-shown:border-slate-300 not-placeholder-shown:border-emerald-500">
```



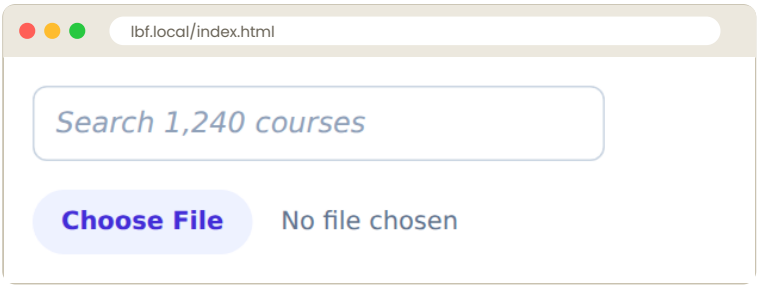
PLACEHOLDER-SHOWN: (EMPTY, DASHED) AND NOT-PLACEHOLDER-SHOWN: (FILLED, GREEN)

6.4.2 placeholder:, file:

`placeholder:` styles the placeholder text itself (`placeholder:text-slate-400 placeholder:italic`). `file:` styles the "Choose file" button inside `<input type="file">`, which otherwise cannot be styled.

INDEX.HTML HTML

```
<input placeholder="Search 1,240 courses" class="w-80 rounded-lg border border-slate-300 px-3 py-2 placeholder:italic placeholder:text-slate-400">
<input type="file" class="mt-4 block text-sm text-slate-500 file:mr-4 file:rounded-full file:border-0 file:bg-indigo-50 file:px-4 file:py-2 file:font-semibold file:text-indigo-700 hover:file:bg-indigo-100">
```



PLACEHOLDER: AND FILE:

6.5 Pseudo-elements

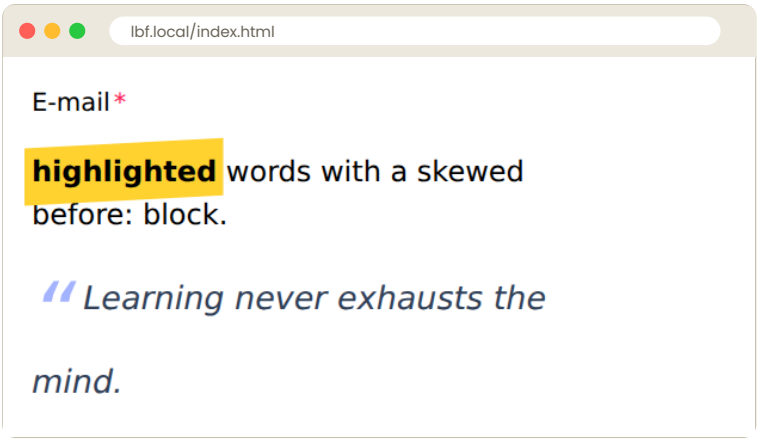
6.5.1 before: and after: with content-[...]

`before:` and `after:` create the `::before` and `::after` pseudo-elements. Tailwind automatically adds `content: var(--tw-content)` (an empty string by default), so a pseudo-element appears as soon as you give it any utility. Text is added with `content-['*']` (spaces as underscores: `content-['Required_field']`) or from an attribute with `content-[attr(data-label)]`.

INDEX.HTML

HTML

```
<label class="block text-sm font-medium after:ml-0.5 after:text-rose-500 after:content-['*']">E-
mail</label>
<p class="mt-4 w-72"><span class="relative inline-block before:absolute before:-inset-1 before:block
before:-skew-y-3 before:bg-amber-300"><span class="relative font-bold">highlighted</span></span> words
with a skewed before: block.</p>
<blockquote class="mt-5 w-80 text-lg italic text-slate-700 before:mr-1 before:align-[-0.4em]
before:text-5xl before:text-indigo-300 before:content-['\201C']">Learning never exhausts the mind.
</blockquote>
```



A REQUIRED-FIELD ASTERISK, A SKEWED HIGHLIGHT AND A DECORATIVE QUOTE MARK

COMPILED CSS

```
.after\:ml-0\.5 {
  &::after {
    content: var(--tw-content);
    margin-left: calc(var(--spacing) * 0.5);
  }
}
.after\:text-rose-500 {
  &::after {
    content: var(--tw-content);
    color: var(--color-rose-500);
  }
}
.after\:content-\['\*\'\] {
  &::after {
    --tw-content: '*';
    content: var(--tw-content);
  }
}
```

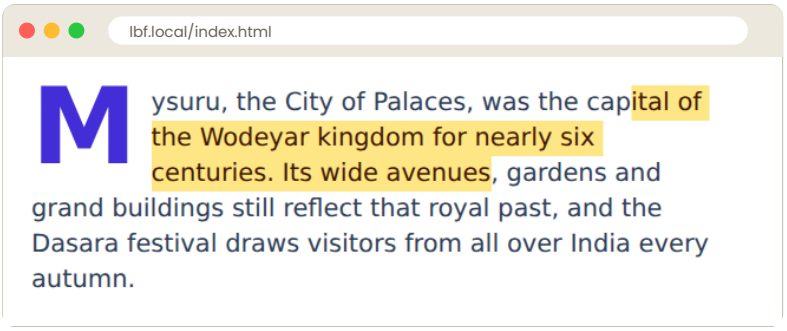
> **NOTE**
> Content added with `before:` / `after:` may be read by screen readers, but it cannot be selected or translated reliably. Use it for decoration, not for important information.

6.5.2 selection:, marker:, first-letter:

`selection:` styles text the user highlights (it can be set on a parent and is inherited, so `selection:bg-amber-200` on `<body>` brands the whole site). `marker:` styles list bullets (Module 4). `first-letter:` and `first-line:` style the first letter or line, for example a drop cap.

INDEX.HTML HTML

```
<p class="w-96 text-sm text-slate-700 selection:bg-amber-200 selection:text-amber-950 first-letter:float-left first-letter:mr-2 first-letter:text-6xl first-letter:leading-[0.8] first-letter:font-bold first-letter:text-indigo-700">Mysuru, the City of Palaces, was the capital of the Wodeyar kingdom for nearly six centuries. Its wide avenues, gardens and grand buildings still reflect that royal past, and the Dasara festival draws visitors from all over India every autumn.</p>
```



A DROP CAP WITH FIRST-LETTER: AND CUSTOM SELECTION COLOURS (PART OF THE TEXT IS SELECTED)

6.6 Styling based on other elements

6.6.1 Parent state with group and group-hover:

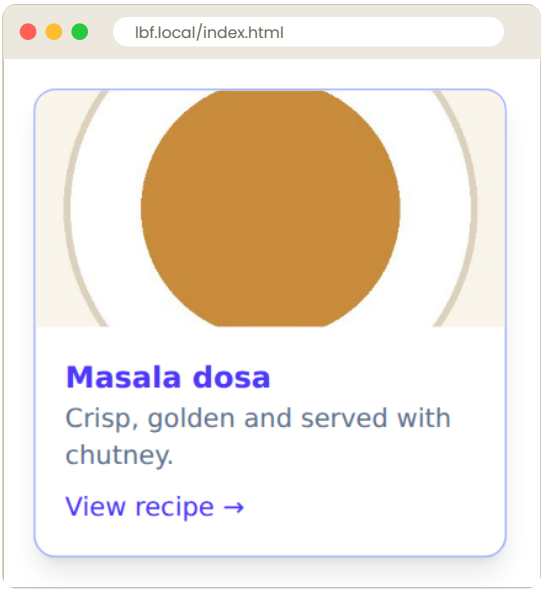
To style a child when its **parent** is hovered, mark the parent with the class `group` and use `group-hover:` on the child. Every pseudo-class variant has a group version: `group-focus:`, `group-focus-within:`, `group-active:`, `group-odd:`, `group-aria-expanded:`, `group-data-[state=open]:`. This is how whole cards react as one unit.

A CARD THAT REACTS AS A GROUP HTML

```
<a href="#" class="group block w-64 overflow-hidden rounded-xl border border-slate-200 bg-white hover:border-indigo-300 hover:shadow-lg">
  <div class="overflow-hidden"><img src="food-dosa.jpg" alt="" class="h-32 w-full object-cover transition duration-300 group-hover:scale-110"></div>
  <div class="p-4">
    <h3 class="font-semibold group-hover:text-indigo-600">Masala dosa</h3>
    <p class="text-sm text-slate-500">Crisp, golden and served with chutney.</p>
    <p class="mt-2 text-sm font-medium text-indigo-600 opacity-0 group-hover:opacity-100">View recipe &rarr;</p>
  </div>
</a>
```



CARD AT REST



CARD CAPTURED WHILE HOVERED: IMAGE ZOOMED, TITLE INDIGO, LINK VISIBLE

COMPILED CSS

```
.group-hover\:scale-110 {
  &:is(:where(.group):hover *) {
    @media (hover: hover) {
      --tw-scale-x: 110%;
      --tw-scale-y: 110%;
      --tw-scale-z: 110%;
      scale: var(--tw-scale-x) var(--tw-scale-y);
    }
  }
}
```

The generated selector is `.group-hover\:scale-110:is(:where(.group):hover *)`: "an element inside a hovered `.group`". The `group` class itself generates no CSS; it is only a marker.

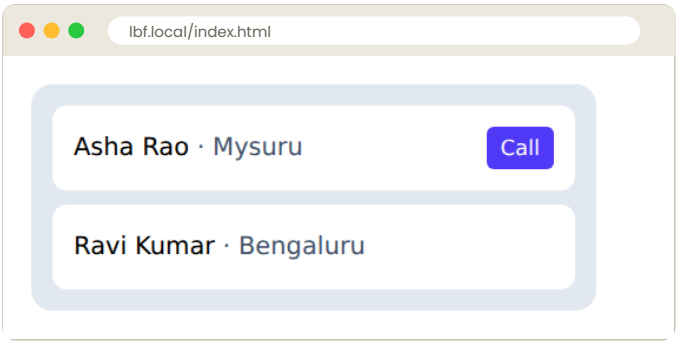
6.6.2 Named groups group/card

When groups are nested (a card inside a list item that is also a group), `group-hover:` reacts to **any** ancestor with `group`, which can trigger the wrong effect. Name the group with a slash, `group/card`, and

target it with `group-hover/card:`.

TWO NAMED GROUPS: THE LIST AND EACH ITEM HTML

```
<ul class="group/list w-80 space-y-2 rounded-xl bg-slate-100 p-3 hover:bg-slate-200">
  <li class="group/item flex items-center justify-between rounded-lg bg-white p-3 text-sm">
    <span>Asha Rao <span class="text-slate-400 group-hover/list:text-slate-600">· Mysuru</span></span>
    <a href="#" class="invisible rounded bg-indigo-600 px-2 py-1 text-xs text-white group-
hover/item:visible">Call</a>
  </li>
  <li class="group/item flex items-center justify-between rounded-lg bg-white p-3 text-sm">
    <span>Ravi Kumar <span class="text-slate-400 group-hover/list:text-slate-600">· Bengaluru</span>
</span>
    <a href="#" class="invisible rounded bg-indigo-600 px-2 py-1 text-xs text-white group-
hover/item:visible">Call</a>
  </li>
</ul>
```



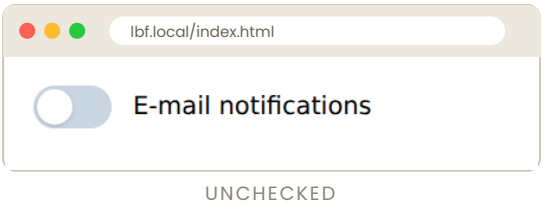
HOVERING THE FIRST ITEM: ONLY ITS OWN CALL BUTTON APPEARS (GROUP-HOVER/ITEM), WHILE BOTH LOCATIONS DARKEN (GROUP-HOVER/LIST)

6.6.3 Sibling state with peer and peer-checked:

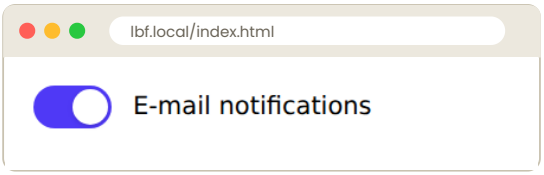
To style an element based on a **previous sibling**, mark the sibling with `peer` and use `peer-*:` variants. Because CSS can only select following siblings (`~`), the peer must come **before** the element being styled in the HTML. Typical uses are custom toggles (a hidden checkbox with `peer` followed by a styled track) and validation messages that appear below an invalid input. Peers can be named too: `peer/email` and `peer-invalid/email:`.

A TOGGLE SWITCH FROM A CHECKBOX (NO JAVASCRIPT) HTML

```
<label class="inline-flex cursor-pointer items-center gap-3">
  <input type="checkbox" class="peer sr-only">
  <span class="relative h-6 w-11 rounded-full bg-slate-300 transition peer-checked:bg-indigo-600
              after:absolute after:left-0.5 after:top-0.5 after:size-5 after:rounded-full after:bg-
white after:shadow after:transition
              peer-checked:after:translate-x-5 peer-focus-visible:ring-4 peer-focus-visible:ring-
indigo-200"></span>
  <span class="text-sm">E-mail notifications</span>
</label>
```



UNCHECKED



CHECKED AFTER A CLICK: PEER-CHECKED: MOVES THE KNOB AND COLOURS THE TRACK

AN ERROR MESSAGE THAT APPEARS ONLY WHEN THE INPUT IS INVALID

HTML

```
<label class="block w-80 text-sm">
  <span class="font-medium">E-mail</span>
  <input type="email" required class="peer mt-1 block w-full rounded-lg border border-slate-300 px-3
py-2 user-invalid:border-rose-500">
  <span class="mt-1 hidden text-rose-600 peer-user-invalid:block">Please enter a valid e-mail address.
</span>
</label>
```



AFTER TYPING "ASHA" AND LEAVING THE FIELD

6.6.4 has-* variants

`has-*:` uses the CSS `:has()` selector to style an element based on what it **contains**: `has-checked:bg-indigo-50` highlights a label whose radio button is checked, `has-[img]:p-0` removes padding from a card that contains an image. It also has `group-has-*:` and `peer-has-*:` forms. Before `:has()`, this "parent selector" needed JavaScript.

PLAN PICKER WITH HAS-CHECKED:

HTML

```
<fieldset class="grid w-[420px] grid-cols-3 gap-3 text-sm">
  <label class="rounded-xl border border-slate-200 p-3 has-checked:border-indigo-600 has-checked:bg-
indigo-50 has-checked:ring-2 has-checked:ring-indigo-200">
    <input type="radio" name="plan" class="sr-only"><b class="block">Starter</b>&#8377;0
  </label>
  <label class="rounded-xl border border-slate-200 p-3 has-checked:border-indigo-600 has-checked:bg-
indigo-50 has-checked:ring-2 has-checked:ring-indigo-200">
    <input type="radio" name="plan" class="sr-only" checked><b class="block">Pro</b>&#8377;499
  </label>
  <label class="rounded-xl border border-slate-200 p-3 has-checked:border-indigo-600 has-checked:bg-
indigo-50 has-checked:ring-2 has-checked:ring-indigo-200">
    <input type="radio" name="plan" class="sr-only"><b class="block">Team</b>&#8377;1,999
  </label>
</fieldset>
```



THE PRO LABEL CONTAINS THE CHECKED RADIO BUTTON

COMPILED CSS

```
.has-checked\:bg-indigo-50 {
  &:has(*:checked) {
    background-color: var(--color-indigo-50);
  }
}
.has-\[img\]\:p-0 {
  &:has(*:is(img)) {
    padding: calc(var(--spacing) * 0);
  }
}
```

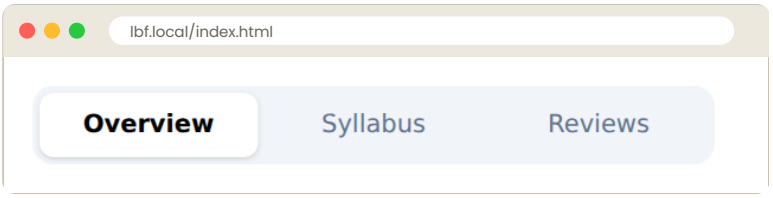
6.6.5 not-* variants

`not-*:` negates another variant or a selector: `not-first:border-t` (every item except the first), `not-disabled:hover:bg-indigo-500`, `not-last:mb-2`, `not-[.active]:opacity-60`, and even media variants such as `not-supports-[display:grid]:float-left`.

INDEX.HTML

HTML

```
<nav class="flex w-96 gap-1 rounded-xl bg-slate-100 p-1 text-sm">
  <a class="active flex-1 rounded-lg bg-white py-2 text-center font-semibold shadow-sm not-
[.active]:text-slate-500">Overview</a>
  <a class="flex-1 rounded-lg py-2 text-center not-[.active]:text-slate-500">Syllabus</a>
  <a class="flex-1 rounded-lg py-2 text-center not-[.active]:text-slate-500">Reviews</a>
</nav>
```



NOT-[.ACTIVE]: GREYS OUT EVERY TAB THAT IS NOT ACTIVE

COMPILED CSS

```
.not-first\:border-t {
  &:not(*:first-child) {
    border-top-style: var(--tw-border-style);
    border-top-width: 1px;
  }
}
.not-\[\.active\]\:text-slate-500 {
  &:not(*:is(.active)) {
    color: var(--color-slate-500);
  }
}
.not-disabled\:hover\:bg-indigo-500 {
  &:not(*:disabled) {
    &:hover {
      @media (hover: hover) {
        background-color: var(--color-indigo-500);
      }
    }
  }
}
```

6.6.6 Child selectors *:

The `*:` variant applies a utility to **every direct child** from the parent, which is convenient when you do not control the children's markup or want to avoid repeating classes: `*:rounded-full *:bg-slate-100 *:px-3` on a list of tags. `**:` targets all descendants. Classes set directly on a child will not override `*:` utilities, because `*:` is generated with a higher position in the output; use it when every child should look the same.

INDEX.HTML

HTML

```
<ul class="flex flex-wrap gap-2 text-sm *:rounded-full *:border *:border-indigo-200 *:bg-indigo-50
*:px-3 *:py-1 *:text-indigo-700">
  <li>HTML5</li><li>CSS3</li><li>Tailwind</li><li>React</li><li>Vue</li>
</ul>
```



FIVE TAGS STYLED FROM THE PARENT WITH *:

COMPILED CSS

```
.\*\:rounded-full {
  :is(& > *) {
    border-radius: calc(infinity * 1px);
  }
}
.\*\*\:text-sm {
  :is(& *) {
    font-size: var(--text-sm);
    line-height: var(--tw-leading, var(--text-sm--line-height));
  }
}
```

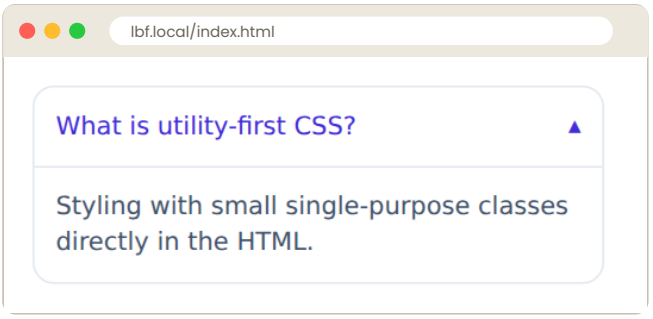
6.7 Attribute variants

6.7.1 aria-* and data-* variants

ARIA variants style elements by their accessibility state, keeping visual and assistive-technology states in sync: `aria-expanded:rotate-180`, `aria-selected:bg-white`, `aria-current:font-bold`, `aria-disabled:opacity-50`, `aria-checked:`, `aria-pressed:`, plus arbitrary ones like `aria-[sort=ascending]:`. **Data variants** do the same for custom attributes that JavaScript or a component library sets: `data-active:` (the attribute is present), `data-[state=open]:`, `data-[size=lg]:`. Libraries such as Radix and Headless UI expose their state this way (Module 15).

AN ACCORDION DRIVEN BY ARIA-EXPANDED AND DATA-OPEN HTML

```
<div class="w-80 rounded-xl border border-slate-200 text-sm">
  <button aria-expanded="false" class="flex w-full items-center justify-between p-3 font-medium aria-expanded:text-indigo-700"
          onclick="this.setAttribute('aria-expanded', this.getAttribute('aria-expanded') !== 'true'); this.nextElementSibling.toggleAttribute('data-open')">
    What is utility-first CSS?
    <span class="transition-transform group-aria-expanded:rotate-180 in-aria-expanded:rotate-180">&#9662;</span>
  </button>
  <p class="hidden border-t border-slate-200 p-3 text-slate-600 data-open:block">Styling with small single-purpose classes directly in the HTML.</p>
</div>
```



AFTER CLICKING THE QUESTION: ARIA-EXPANDED="TRUE" COLOURS AND ROTATES, DATA-OPEN SHOWS THE ANSWER

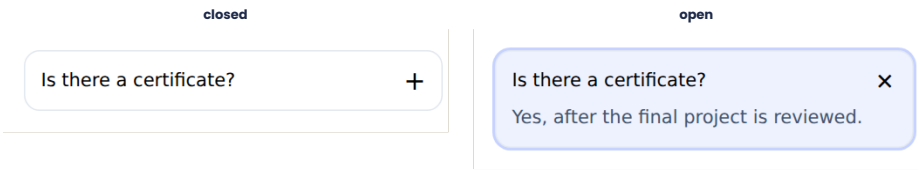
COMPILED CSS

```
.in-aria-expanded\:rotate-180 {
  :where(*[aria-expanded="true"]) & {
    rotate: 180deg;
  }
}
.aria-expanded\:text-indigo-700 {
  &[aria-expanded="true"] {
    color: var(--color-indigo-700);
  }
}
.data-open\:block {
  &[data-open] {
    display: block;
  }
}
.data-\[state\=open\]\:bg-white {
  &[data-state="open"] {
    background-color: var(--color-white);
  }
}
```

The arrow uses `in-aria-expanded:`, a v4 variant that matches when **any ancestor** has the state, without needing a `group` class (it works like `group-*` with an implicit group).

6.7.2 open: for details and dialog

`open:` matches `<details>` and `<dialog>` elements that have the `open` attribute, and in v4 also popovers that are showing (`:popover-open`). Combined with `group`, the summary arrow can rotate when the disclosure opens, entirely without JavaScript.



OPEN: ON DETAILS AND GROUP-OPEN: ON THE ICON

PRACTICE CHECKPOINT

A **sign-up form** with focus rings, invalid-state messages using `peer`, a **custom checkbox**, and **product cards** whose image zooms when the card is hovered using `group`.

SIGNUP.HTML: SAMPLE SOLUTION HTML

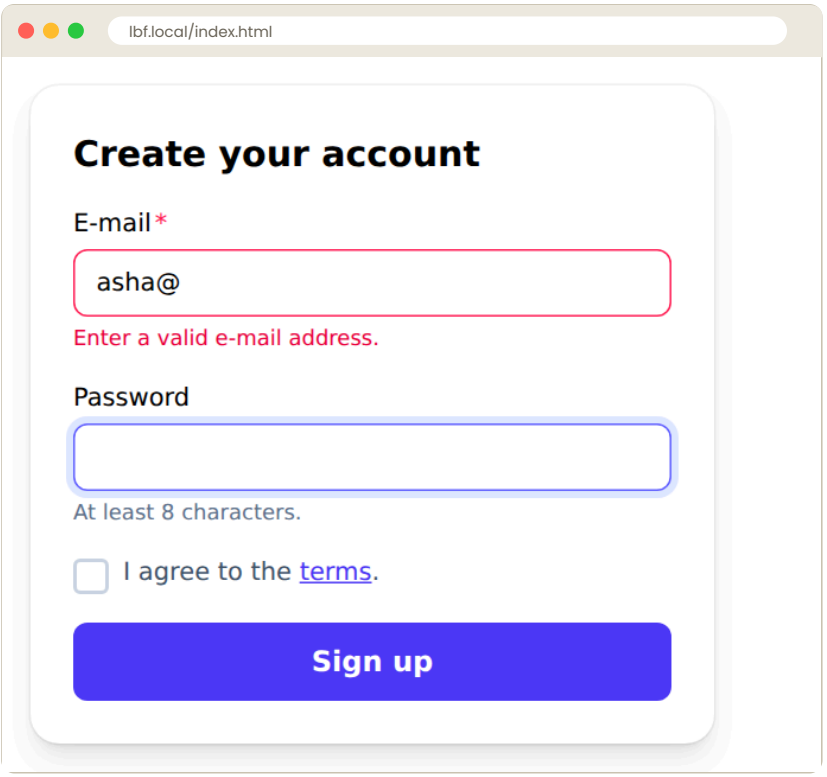
```
<form class="w-96 space-y-4 rounded-2xl bg-white p-6 shadow-lg ring-1 ring-black/5">
  <h2 class="text-xl font-bold">Create your account</h2>

  <label class="block text-sm">
    <span class="font-medium after:ml-0.5 after:text-rose-500 after:content-['*']">E-mail</span>
    <input type="email" required placeholder="you@example.in"
      class="peer mt-1 block w-full rounded-lg border border-slate-300 px-3 py-2 placeholder:text-
slate-400
             focus:outline-hidden focus-visible:border-indigo-500 focus-visible:ring-4 focus-
visible:ring-indigo-100
             user-invalid:border-rose-500 user-invalid:focus-visible:ring-rose-100">
    <span class="mt-1 hidden text-xs text-rose-600 peer-user-invalid:block">Enter a valid e-mail
address.</span>
  </label>

  <label class="block text-sm">
    <span class="font-medium">Password</span>
    <input type="password" required minlength="8"
      class="peer mt-1 block w-full rounded-lg border border-slate-300 px-3 py-2 focus:outline-hidden
focus-visible:border-indigo-500 focus-visible:ring-4 focus-visible:ring-indigo-100 user-
invalid:border-rose-500">
    <span class="mt-1 block text-xs text-slate-500 peer-user-invalid:text-rose-600">At least 8
characters.</span>
  </label>

  <label class="flex items-start gap-2 text-sm">
    <input type="checkbox" required class="peer mt-0.5 size-5 shrink-0 appearance-none rounded border-
2 border-slate-300 checked:border-indigo-600 checked:bg-indigo-600 focus-visible:ring-4 focus-
visible:ring-indigo-100">
    <span class="text-slate-600">I agree to the <a href="#" class="font-medium text-indigo-600
underline">terms</a>.</span>
  </label>

  <button class="w-full rounded-lg bg-indigo-600 py-2.5 font-semibold text-white hover:bg-indigo-500
active:bg-indigo-700 focus-visible:outline-3 focus-visible:outline-offset-2 focus-visible:outline-
indigo-600">Sign up</button>
</form>
```



AFTER TYPING "ASHA@" INTO THE E-MAIL FIELD AND PRESSING TAB: THE ERROR APPEARS AND THE PASSWORD FIELD SHOWS ITS FOCUS RING

PRODUCTS.HTML: SAMPLE SOLUTION HTML

```
<div class="grid grid-cols-3 gap-4">
  <a href="#" class="group overflow-hidden rounded-xl bg-white ring-1 ring-slate-200 hover:ring-
indigo-300 hover:shadow-lg">
    <div class="overflow-hidden"><img src="food-idli.jpg" alt="" class="aspect-square w-full object-
cover transition duration-500 group-hover:scale-110"></div>
    <div class="p-3"><h3 class="font-semibold group-hover:text-indigo-600">Idli</h3><p class="text-sm
text-slate-500">&#8377;40</p></div>
  </a>
  <a href="#" class="group overflow-hidden rounded-xl bg-white ring-1 ring-slate-200 hover:ring-
indigo-300 hover:shadow-lg">
    <div class="overflow-hidden"><img src="food-dosa.jpg" alt="" class="aspect-square w-full object-
cover transition duration-500 group-hover:scale-110"></div>
    <div class="p-3"><h3 class="font-semibold group-hover:text-indigo-600">Masala dosa</h3><p
class="text-sm text-slate-500">&#8377;80</p></div>
  </a>
  <a href="#" class="group overflow-hidden rounded-xl bg-white ring-1 ring-slate-200 hover:ring-
indigo-300 hover:shadow-lg">
    <div class="overflow-hidden"><img src="food-coffee.jpg" alt="" class="aspect-square w-full object-
cover transition duration-500 group-hover:scale-110"></div>
    <div class="p-3"><h3 class="font-semibold group-hover:text-indigo-600">Filter coffee</h3><p
class="text-sm text-slate-500">&#8377;25</p></div>
  </a>
</div>
```



THE MIDDLE CARD CAPTURED WHILE HOVERED: IMAGE ZOOMED AND TITLE HIGHLIGHTED

Variants are prefixes that wrap a utility in a condition. Use `focus-visible:` for focus rings, `user-invalid:` for friendly validation, `group` / `peer` (named when nested) for relationships, `has-*:` to style a parent by its content, and `aria-*:` / `data-*:` to keep styling in sync with component state.

LEVEL 2

Intermediate level

Outcome: build layouts with Flexbox and Grid, make them responsive with breakpoints and container queries, add dark mode, motion and reusable components.

6 MODULES · 9 DAYS MINIMUM

Module 07 · Layout, positioning and Flexbox	**2 days**
Module 08 · CSS Grid with Tailwind	**1 day**
Module 09 · Responsive design and container queries	**2 days**
Module 10 · Dark mode and theming basics	**1 day**
Module 11 · Transitions, transforms and animations	**1 day**
Module 12 · Building components and reusing styles	**2 days**

MODULE 07 · INTERMEDIATE

Layout, positioning and Flexbox

Minimum duration **2 days** | Level **Intermediate**

This module covers the utilities that give a page its structure: display types, centred containers, box sizing, visibility and screen-reader-only text; the five position values with inset and z-index utilities; overflow and scroll snapping; and the complete set of Flexbox utilities for direction, wrapping, alignment, gaps, growing and shrinking and order. It ends with the everyday patterns built from them: a navbar, centring anything, a sticky header and footer layout and a badge on an avatar. The CSS theory behind these properties is in the CSS3 book (Modules 7 and 10); here the focus is on the Tailwind classes.

IN THIS MODULE

7.1 Layout utilities
7.2 Positioning
7.3 Overflow
7.4 Flexbox
7.5 Common patterns

7.1 Layout utilities

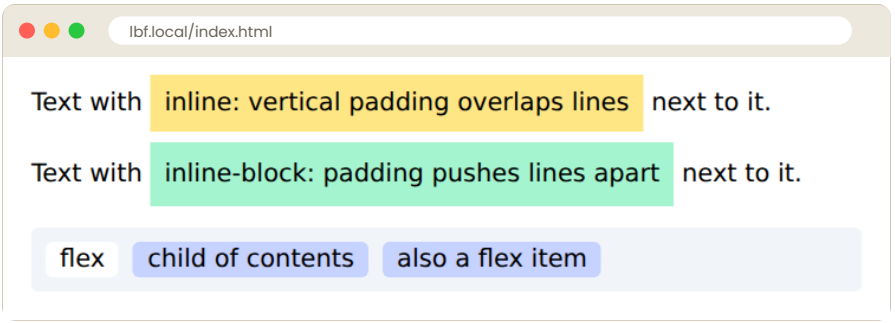
7.1.1 Display: block, inline-block, hidden, flex, grid, contents

Each display value is a single class named after the value: `block`, `inline`, `inline-block`, `flex`, `inline-flex`, `grid`, `inline-grid`, `table`, `flow-root`, `contents`, `list-item`, and `hidden` for `display: none`. Combined with breakpoints (`hidden md:block`) they show and hide whole sections responsively (Module 9).

Class	Behaviour
`block`	Starts on a new line, takes the full width
`inline-block`	Flows in a line of text but accepts width, height and vertical padding
`flex` / `inline-flex`	Flex container (block-level or inline-level), Section 7.4
`grid` / `inline-grid`	Grid container, Module 8
`contents`	The element's own box disappears; its children act as children of the grandparent
`flow-root`	Creates a new formatting context (contains floats and margins)
`hidden`	Not rendered at all, and hidden from screen readers

INLINE VS INLINE-BLOCK, AND CONTENTS HTML

```
<div class="space-y-3 text-sm">
  <p>Text with <span class="bg-amber-200 px-2 py-2">inline: vertical padding overlaps lines</span>
next to it.</p>
  <p>Text with <span class="inline-block bg-emerald-200 px-2 py-2">inline-block: padding pushes lines
apart</span> next to it.</p>
  <div class="flex gap-2 rounded bg-slate-100 p-2"><span class="rounded bg-white px-2">flex</span><div
class="contents"><span class="rounded bg-indigo-200 px-2">child of contents</span><span class="rounded
bg-indigo-200 px-2">also a flex item</span></div></div>
</div>
```



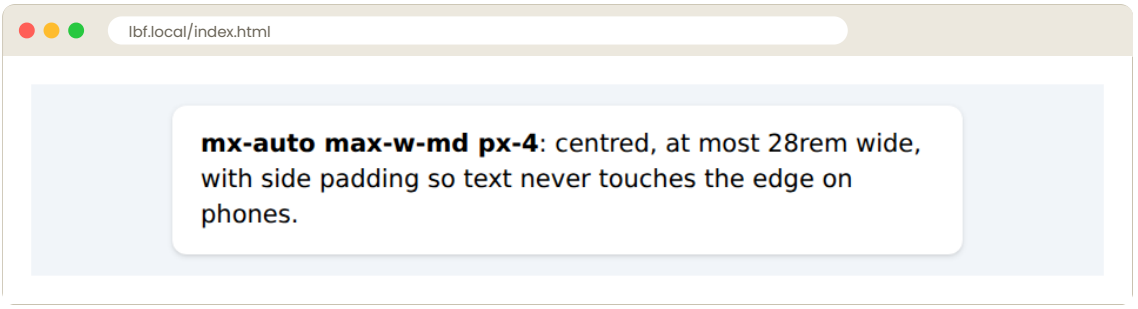
THE TWO INDIGO SPANS ARE INSIDE A CONTENTS WRAPPER, SO THEY BECOME FLEX ITEMS OF THE OUTER ROW

7.1.2 container and centred content with max-w-* and mx-auto

`container` sets `width: 100%` and a `max-width` equal to the current breakpoint (640px at `sm`, 768px at `md`, and so on). In v4 it no longer centres itself or adds padding, so it is normally written `container mx-auto px-4`. Many developers prefer a single fixed reading width instead: `mx-auto max-w-7xl px-4 sm:px-6 lg:px-8`, which is the pattern used by Tailwind's own templates. The `max-w-*` sizes come from the container scale (`max-w-sm` 24rem up to `max-w-7xl` 80rem) plus `max-w-prose` (65ch).

INDEX.HTML HTML

```
<div class="bg-slate-100 py-3">
  <div class="mx-auto max-w-md rounded-lg bg-white px-4 py-3 text-sm shadow-sm">
    <b>mx-auto max-w-md px-4</b>: centred, at most 28rem wide, with side padding so text never touches
the edge on phones.
  </div>
</div>
```



THE CENTRED CONTENT WRAPPER

COMPILED CSS

```
.container {
  width: 100%;
  @media (width >= 40rem) {
    max-width: 40rem;
  }
  @media (width >= 48rem) {
    max-width: 48rem;
  }
  @media (width >= 64rem) {
    max-width: 64rem;
  }
  @media (width >= 80rem) {
    max-width: 80rem;
  }
  @media (width >= 96rem) {
    max-width: 96rem;
  }
}
.max-w-7xl {
  max-width: var(--container-7xl);
}
.max-w-prose {
  max-width: 65ch;
}
```

TIP

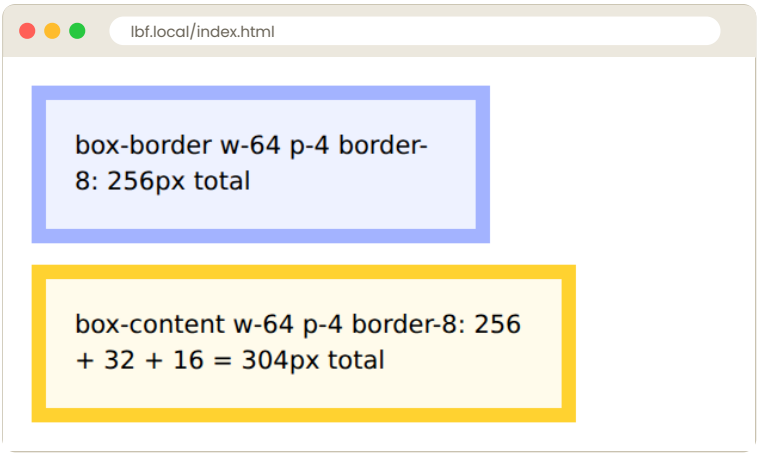
To make `container` centred and padded by default, as in v3, customise it with `@utility container { margin-inline: auto; padding-inline: 1rem; }` (Module 14).

7.1.3 box-border vs box-content

Preflight sets `box-sizing: border-box` on every element, so `w-64` is the total width including padding and border. `box-content` switches an element back to the CSS default, where width means the content only and padding and border are added on top. You rarely need it, except with third-party widgets that expect content-box sizing.

INDEX.HTML HTML

```
<div class="space-y-3 text-sm">
  <div class="box-border w-64 border-8 border-indigo-300 bg-indigo-50 p-4">box-border w-64 p-4 border-
8: 256px total</div>
  <div class="box-content w-64 border-8 border-amber-300 bg-amber-50 p-4">box-content w-64 p-4 border-
8: 256 + 32 + 16 = 304px total</div>
</div>
```



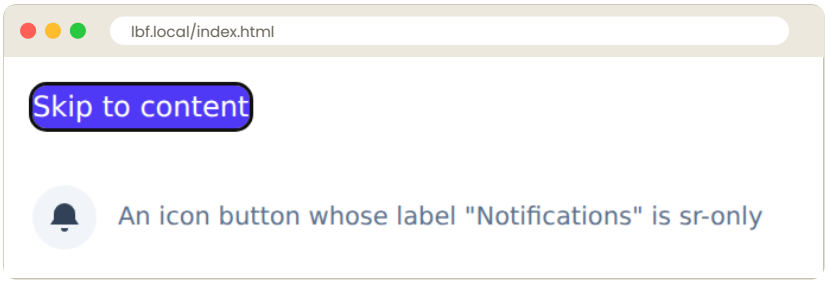
THE SAME CLASSES, TWO BOX-SIZING MODELS

7.1.4 Visibility and sr-only

`invisible` hides an element but keeps its space (`visibility: hidden`); `visible` undoes it; `collapse` hides table rows and columns. To hide something visually but keep it for **screen readers**, use `sr-only`, for example a label for an icon-only button. `not-sr-only` reverses it, often as `focus:not-sr-only` for a "Skip to content" link that appears when tabbed to.

A SKIP LINK AND AN ICON BUTTON HTML

```
<a href="#main" class="sr-only rounded-lg bg-indigo-600 px-4 py-2 text-white focus:not-sr-only
focus:absolute focus:left-4 focus:top-4">Skip to content</a>
<div class="mt-14 flex items-center gap-3">
  <button class="rounded-full bg-slate-100 p-2"><svg viewBox="0 0 20 20" class="size-5 fill-slate-
700"><path d="M10 2a6 6 0 0 0-6 6v3l-2 3h16l-2-3V8a6 6 0 0 0-6-6Zm0 16a2 2 0 0 0 2-2H8a2 2 0 0 0 2
2Z"/></svg><span class="sr-only">Notifications</span></button>
  <span class="text-sm text-slate-500">An icon button whose label "Notifications" is sr-only</span>
</div>
```



PRESSING TAB FOCUSES THE SKIP LINK, SO FOCUS:NOT-SR-ONLY MAKES IT VISIBLE AT THE TOP LEFT

COMPILED CSS

```
.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip-path: inset(50%);
  white-space: nowrap;
  border-width: 0;
}
```

7.2 Positioning

7.2.1 static, relative, absolute, fixed, sticky

Class	Positioned relative to	Takes space in the flow?
`static`	not positioned (the default)	yes
`relative`	its own normal position; also the anchor for absolute children	yes
`absolute`	the nearest positioned ancestor	no
`fixed`	the viewport (stays put when scrolling)	no
`sticky`	scrolls normally until it reaches its inset, then sticks within its parent	yes

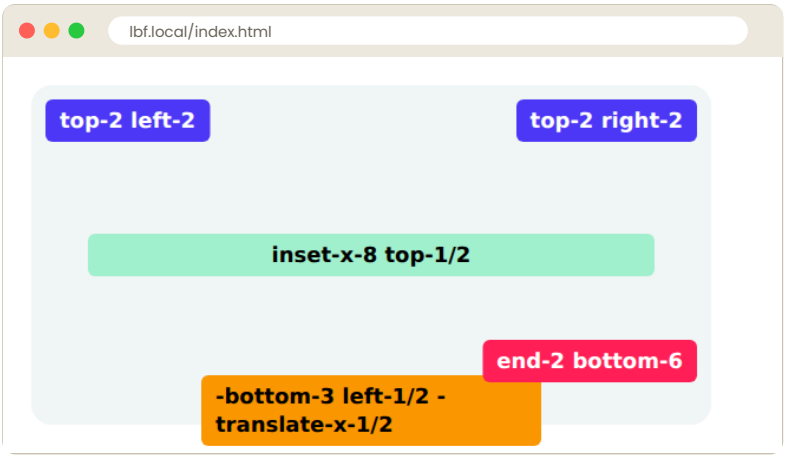
The most common combination is a `relative` parent with an `absolute` child: a badge on a card corner, a close button in a dialog, an overlay on a photo.

7.2.2 inset-*, top-*, right-*, bottom-*, left-*

Offsets use the spacing scale and fractions: `top-0`, `right-4`, `-top-2` (negative), `left-1/2`, `top-[72px]`. Shorthands: `inset-0` sets all four sides (an overlay that fills its parent), `inset-x-0` left and right, `inset-y-4` top and bottom, and logical `start-0` / `end-0`.

ABSOLUTE CHILDREN INSIDE A RELATIVE PARENT HTML

```
<div class="relative h-48 w-96 rounded-xl bg-slate-100 text-xs font-semibold">
  <span class="absolute left-2 top-2 rounded bg-indigo-600 px-2 py-1 text-white">top-2 left-2</span>
  <span class="absolute right-2 top-2 rounded bg-indigo-600 px-2 py-1 text-white">top-2 right-2</span>
  <span class="absolute -bottom-3 left-1/2 -translate-x-1/2 rounded bg-amber-500 px-2 py-1">-bottom-3
left-1/2 -translate-x-1/2</span>
  <span class="absolute inset-x-8 top-1/2 -translate-y-1/2 rounded bg-emerald-200 py-1 text-
center">inset-x-8 top-1/2</span>
  <span class="absolute bottom-6 end-2 rounded bg-rose-500 px-2 py-1 text-white">end-2 bottom-6</span>
</div>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

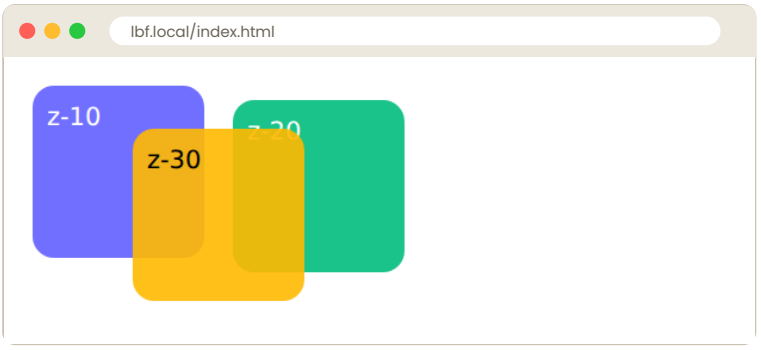
```
.inset-0 {
  inset: calc(var(--spacing) * 0);
}
.inset-x-8 {
  inset-inline: calc(var(--spacing) * 8);
}
.end-2 {
  inset-inline-end: calc(var(--spacing) * 2);
}
.top-1\/2 {
  top: calc(1/2 * 100%);
}
.-bottom-3 {
  bottom: calc(var(--spacing) * -3);
}
```

7.2.3 z-*

`z-0`, `z-10` … `z-50`, `z-auto`, negative `-z-10` and any number (`z-100`, `z-[999]`) set the stacking order of positioned elements. Keep a small project-wide convention, for example dropdowns `z-10`, sticky header `z-20`, modals `z-50`, so values do not escalate. Remember that z-index only compares elements inside the same stacking context; `isolate` creates a new one deliberately.

INDEX.HTML HTML

```
<div class="relative h-32 w-96">
  <div class="absolute left-0 top-0 z-10 size-24 rounded-xl bg-indigo-500/90 p-2 text-sm text-
white">z-10</div>
  <div class="absolute left-14 top-6 z-30 size-24 rounded-xl bg-amber-400/90 p-2 text-sm">z-30</div>
  <div class="absolute left-28 top-2 z-20 size-24 rounded-xl bg-emerald-500/90 p-2 text-sm text-
white">z-20</div>
</div>
```



HIGHER Z-INDEX PAINTS ON TOP REGARDLESS OF HTML ORDER

7.3 Overflow

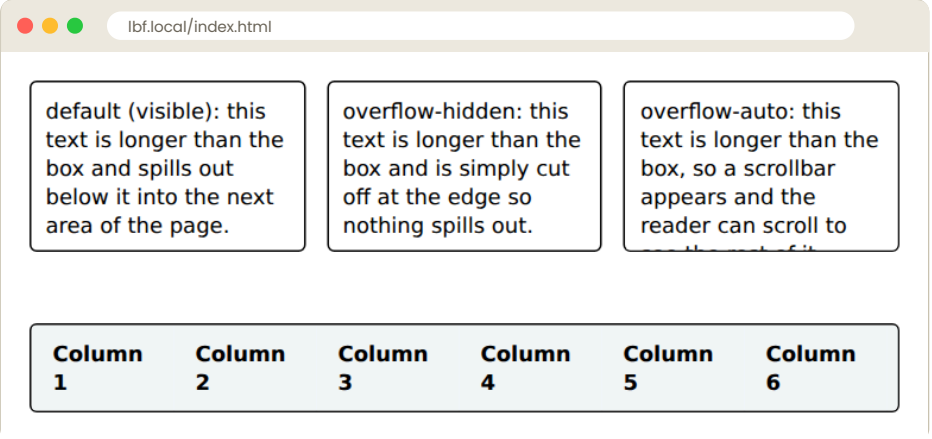
7.3.1 overflow-hidden, overflow-auto, overflow-x-scroll

`overflow-hidden` clips content (rounded image cards, zoom effects), `overflow-auto` adds scrollbars only when needed, `overflow-scroll` always shows them, and axis versions such as `overflow-x-auto` scroll one direction only, for wide tables on phones. `overflow-clip` clips without creating a scroll container, and `overscroll-contain` stops a scrolling panel from scrolling the page behind it when it reaches the end.

INDEX.HTML

HTML

```
<div class="grid grid-cols-3 gap-3 text-xs">
  <div class="h-24 rounded border bg-white p-2">default (visible): this text is longer than the box
and spills out below it into the next area of the page.</div>
  <div class="h-24 overflow-hidden rounded border bg-white p-2">overflow-hidden: this text is longer
than the box and is simply cut off at the edge so nothing spills out.</div>
  <div class="h-24 overflow-auto rounded border bg-white p-2">overflow-auto: this text is longer than
the box, so a scrollbar appears and the reader can scroll to see the rest of it.</div>
</div>
<div class="mt-10 overflow-x-auto rounded border"><table class="min-w-[640px] text-left text-xs"><tr
class="bg-slate-100"><th class="px-3 py-2">Column 1</th><th class="px-3 py-2">Column 2</th><th
class="px-3 py-2">Column 3</th><th class="px-3 py-2">Column 4</th><th class="px-3 py-2">Column 5</th>
<th class="px-3 py-2">Column 6</th><th class="px-3 py-2">Column 7</th><th class="px-3 py-2">Column
8</th></tr></table></div>
```



VISIBLE, HIDDEN AND AUTO; AND A WIDE TABLE IN OVERFLOW-X-AUTO

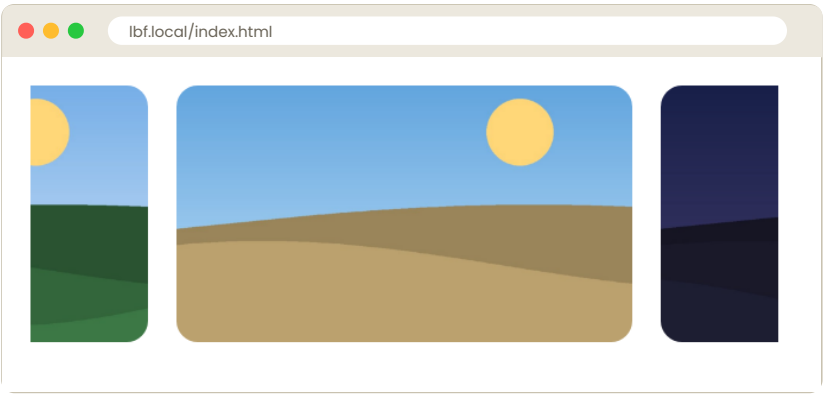
7.3.2 Scroll snapping: snap-x, snap-center

Scroll snapping makes a scroll container stop exactly on its items, like a carousel, with no JavaScript. Put `snap-x` (or `snap-y`) and `snap-mandatory` on the scroll container, and `snap-start`, `snap-center` or `snap-end` on the children. `scroll-px-4` and `scroll-mt-20` (scroll padding and margin) leave space for a sticky header when jumping to an anchor.

A SNAP CAROUSEL

HTML

```
<div class="flex w-[420px] snap-x snap-mandatory gap-4 overflow-x-auto pb-3 [scrollbar-width:thin]">
  <img src="hills.jpg" alt="" class="h-36 w-64 shrink-0 snap-center rounded-xl object-cover">
  <img src="beach.jpg" alt="" class="h-36 w-64 shrink-0 snap-center rounded-xl object-cover">
  <img src="night.jpg" alt="" class="h-36 w-64 shrink-0 snap-center rounded-xl object-cover">
  <img src="mysuru-palace.jpg" alt="" class="h-36 w-64 shrink-0 snap-center rounded-xl object-cover">
</div>
```



AFTER A SCRIPT SCROLLS 250PX, THE BROWSER SNAPS SO THE SECOND PHOTO SITS IN THE CENTRE

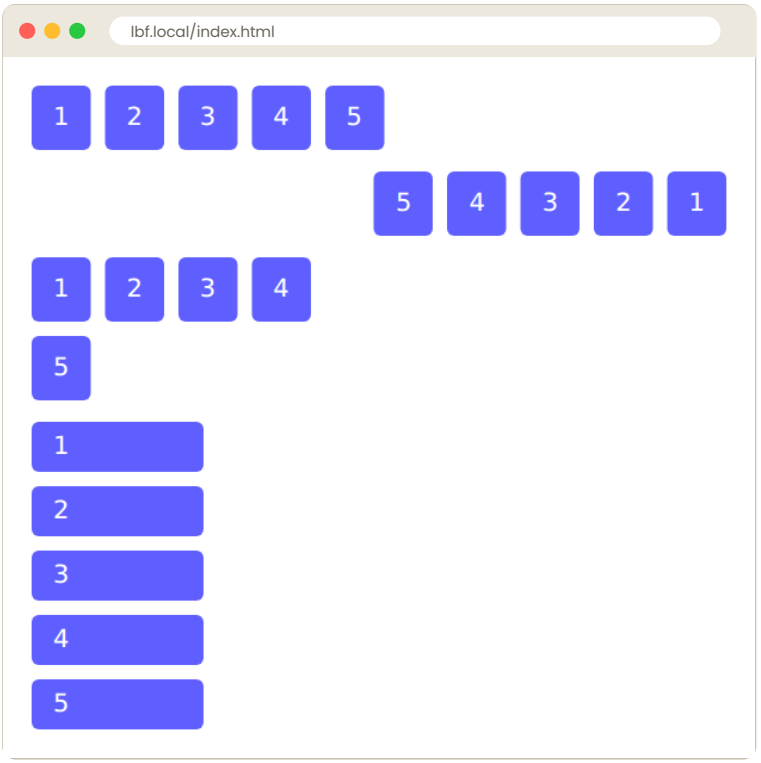
COMPILED CSS

```
.snap-x {
  scroll-snap-type: x var(--tw-scroll-snap-strictness);
}
.snap-mandatory {
  --tw-scroll-snap-strictness: mandatory;
}
.snap-center {
  scroll-snap-align: center;
}
.scroll-mt-20 {
  scroll-margin-top: calc(var(--spacing) * 20);
}
```

7.4 Flexbox

7.4.1 flex, flex-row, flex-col, flex-wrap

`flex` makes a flex container; its children line up in a row. The direction is `flex-row` (default), `flex-col`, `flex-row-reverse` or `flex-col-reverse`. Items stay on one line and shrink unless you add `flex-wrap`; `flex-wrap-reverse` and `flex-nowrap` also exist.



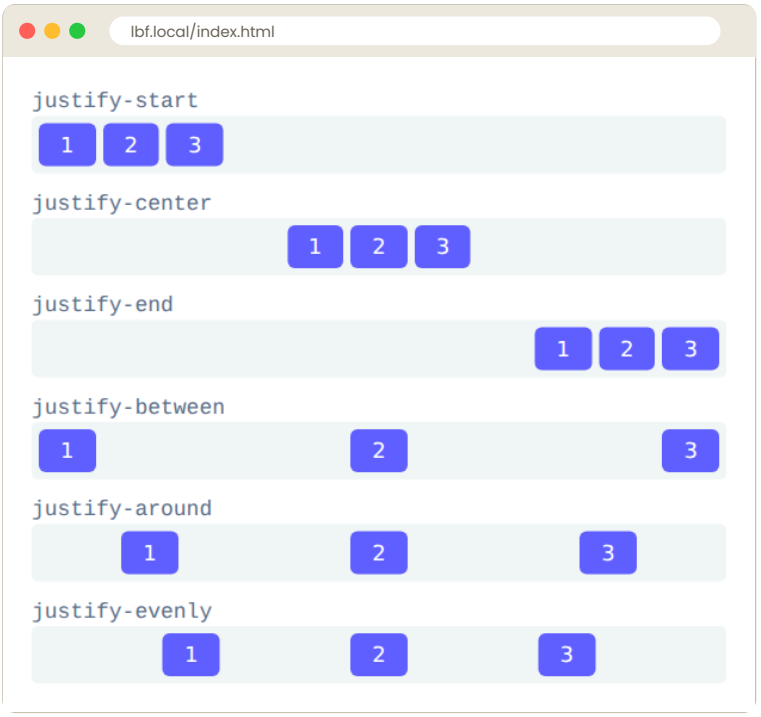
FLEX, FLEX-ROW-REVERSE, FLEX-WRAP (IN A NARROW BOX) AND FLEX-COL

FOUR CONTAINERS, THE SAME FIVE ITEMS HTML

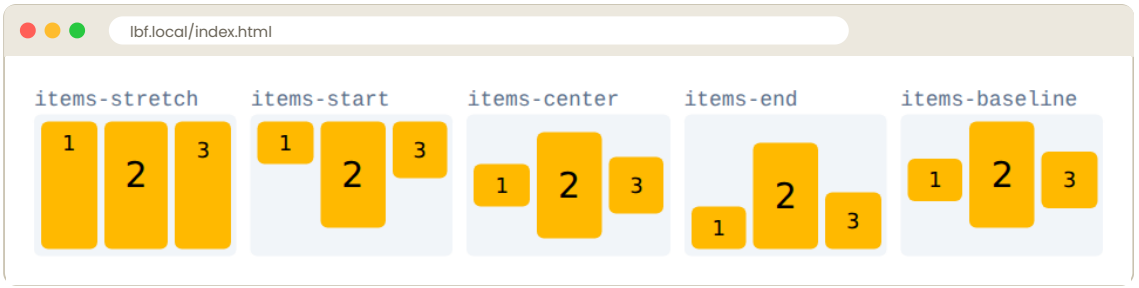
```
<div class="flex gap-2">...</div>
<div class="flex flex-row-reverse gap-2">...</div>
<div class="flex w-48 flex-wrap gap-2">...</div>
<div class="flex flex-col gap-2">...</div>
```

7.4.2 justify-* and items-*

`justify-*` distributes items along the **main axis** (horizontal in a row): `justify-start`, `justify-center`, `justify-end`, `justify-between`, `justify-around`, `justify-evenly`. `items-*` aligns them on the **cross axis**: `items-stretch` (default), `items-start`, `items-center`, `items-end`, `items-baseline`. In a `flex-col` container the axes swap.



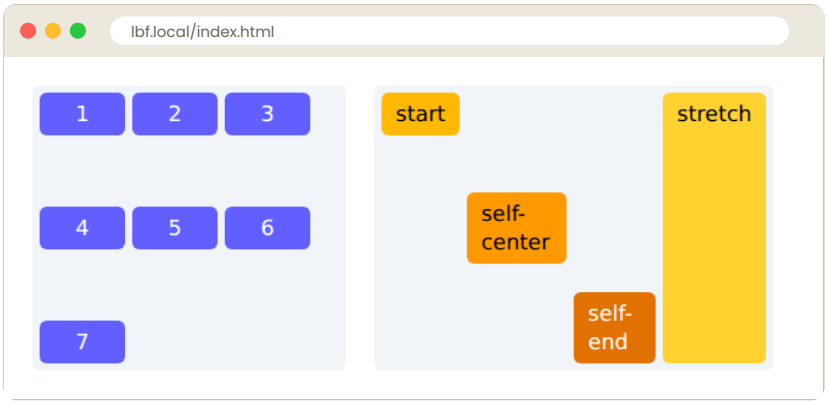
THE SIX JUSTIFY-CONTENT VALUES



THE FIVE ALIGN-ITEMS VALUES ON ITEMS OF DIFFERENT HEIGHTS

7.4.3 content-* and self-*

`content-*` (align-content) distributes the **lines** of a wrapping flex container on the cross axis, so it only has an effect with `flex-wrap` and spare height: `content-start`, `content-center`, `content-between` and so on. `self-*` overrides `items-*` for one item: `self-start`, `self-center`, `self-end`, `self-stretch`, `self-auto`.



CONTENT-BETWEEN ON WRAPPED LINES; SELF-* ON INDIVIDUAL ITEMS

7.4.4 gap-*

`gap-4` puts space between flex items (and grid cells) without adding it on the outer edges, and it works correctly when items wrap, which margins do not. `gap-x-*` and `gap-y-*` set the column and row gap separately: `flex flex-wrap gap-x-6 gap-y-2` for a tag list.

COMPILED CSS

```
.gap-4 {
  gap: calc(var(--spacing) * 4);
}
.gap-x-6 {
  column-gap: calc(var(--spacing) * 6);
}
.gap-y-2 {
  row-gap: calc(var(--spacing) * 2);
}
```

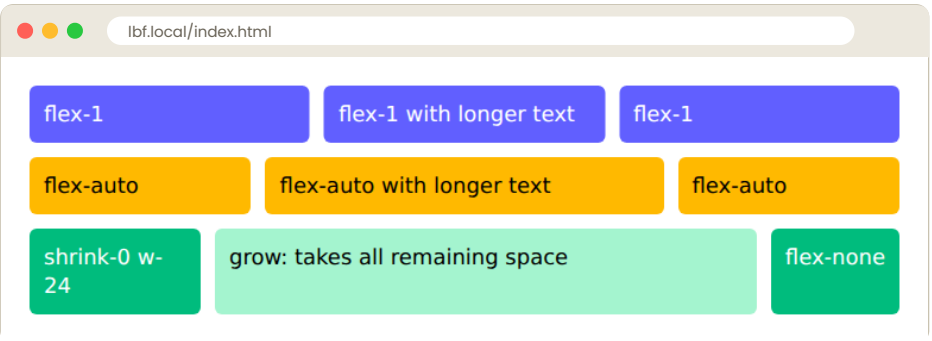
7.4.5 flex-1, flex-auto, flex-none, grow, shrink-0, basis-*

Class	CSS	Use
`flex-1`	flex: 1 (grow, shrink, basis 0%)	equal-width columns; fill remaining space
`flex-auto`	flex: auto (basis = content size)	grow, but bigger content gets more
`flex-initial`	flex: 0 auto	shrink but never grow (the CSS default)
`flex-none`	flex: none	fixed size: icons, avatars
`grow` / `grow-0`	flex-grow: 1 / 0	take up free space
`shrink-0`	flex-shrink: 0	never squash this item (logos, images)
`basis-64`, `basis-1/3`	flex-basis	starting size before growing and shrinking

INDEX.HTML

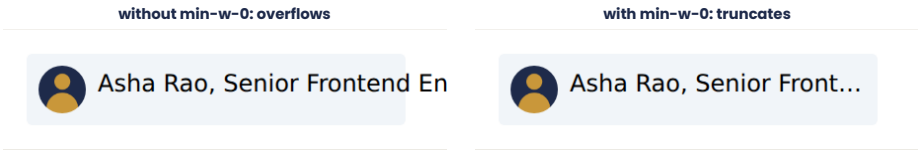
HTML

```
<div class="space-y-2 text-xs font-medium">
  <div class="flex gap-2"><div class="flex-1 rounded bg-indigo-500 p-2 text-white">flex-1</div><div
class="flex-1 rounded bg-indigo-500 p-2 text-white">flex-1 with longer text</div><div class="flex-1
rounded bg-indigo-500 p-2 text-white">flex-1</div></div>
  <div class="flex gap-2"><div class="flex-auto rounded bg-amber-400 p-2">flex-auto</div><div
class="flex-auto rounded bg-amber-400 p-2">flex-auto with longer text</div><div class="flex-auto
rounded bg-amber-400 p-2">flex-auto</div></div>
  <div class="flex gap-2"><div class="w-24 shrink-0 rounded bg-emerald-500 p-2 text-white">shrink-0 w-
24</div><div class="grow rounded bg-emerald-200 p-2">grow: takes all remaining space</div><div
class="flex-none rounded bg-emerald-500 p-2 text-white">flex-none</div></div>
</div>
```



FLEX-1 GIVES EQUAL WIDTHS; FLEX-AUTO SHARES SPACE BY CONTENT SIZE

A classic bug: text inside a flex item refuses to truncate or overflows the row. Flex items have `min-width: auto`, so they will not shrink below their content. Adding `min-w-0` to the item fixes it:



THE MIN-W-0 FIX

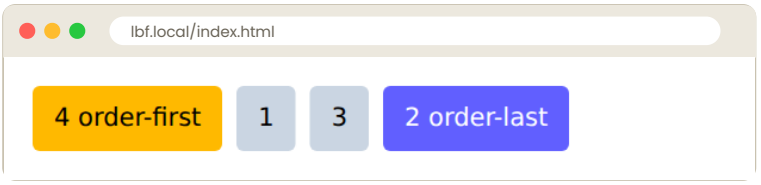
7.4.6 order-*

`order-1` to `order-12`, `order-first` (-9999), `order-last` (9999), `order-none` and negative values change the visual order of flex and grid items without changing the HTML. Use it carefully: keyboard focus and screen readers still follow the source order, so reorder only when the meaning does not change (for example placing an image after the text on mobile and before it on desktop: `order-last md:order-first`).

INDEX.HTML

HTML

```
<div class="flex gap-2 text-sm"><div class="rounded bg-slate-300 px-3 py-2">1</div><div class="order-
last rounded bg-indigo-500 px-3 py-2 text-white">2 order-last</div><div class="rounded bg-slate-300
px-3 py-2">3</div><div class="order-first rounded bg-amber-400 px-3 py-2">4 order-first</div></div>
```



SOURCE ORDER 1 2 3 4, DISPLAYED AS 4 1 3 2

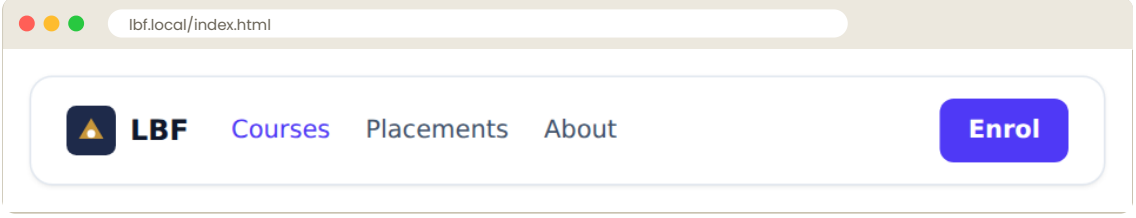
7.5 Common patterns

7.5.1 Navbar with logo, links and button

NAVBAR.HTML

HTML

```
<header class="flex items-center gap-6 rounded-xl bg-white px-5 py-3 shadow-sm ring-1 ring-slate-200">
  <a href="#" class="flex shrink-0 items-center gap-2 font-bold text-slate-900"><img src="logo.svg"
alt="" class="size-7"> LBF</a>
  <nav class="flex gap-5 text-sm text-slate-600">
    <a href="#" class="font-medium text-indigo-600" aria-current="page">Courses</a>
    <a href="#" class="hover:text-slate-900">Placements</a>
    <a href="#" class="hover:text-slate-900">About</a>
  </nav>
  <a href="#" class="ml-auto rounded-lg bg-indigo-600 px-4 py-2 text-sm font-semibold text-white
hover:bg-indigo-500">Enrol</a>
</header>
```



ML-AUTO ON THE BUTTON PUSHES IT TO THE FAR RIGHT

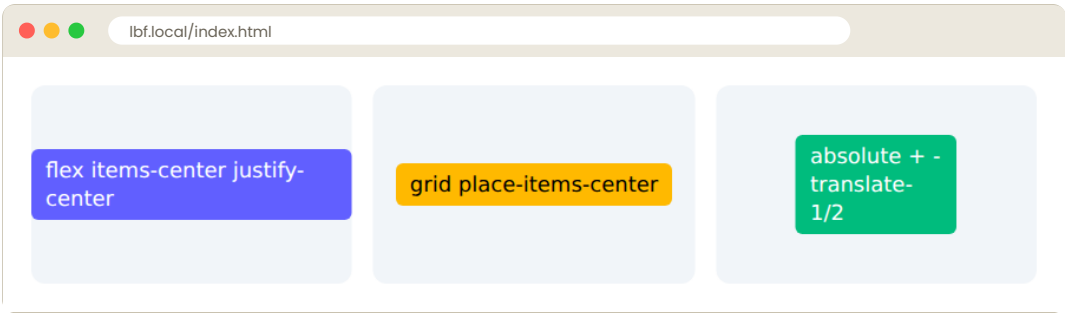
Three ideas carry this layout: `flex items-center` aligns everything on one line, `gap-6` spaces the groups, and `ml-auto` on the last item absorbs all free space to its left. `justify-between` works when there are exactly two or three groups.

7.5.2 Centring anything

Situation	Classes
A block horizontally	`mx-auto` plus a width or max-width
Text inside a box	`text-center`
Anything, both axes, with Flexbox	`flex items-center justify-center` on the parent
Anything, both axes, with Grid (shortest)	`grid place-items-center` on the parent
An overlay element	`absolute inset-0 m-auto size-12`, or `absolute left-1/2 top-1/2 -translate-1/2`
An icon next to text	`inline-flex items-center gap-1`

INDEX.HTML HTML

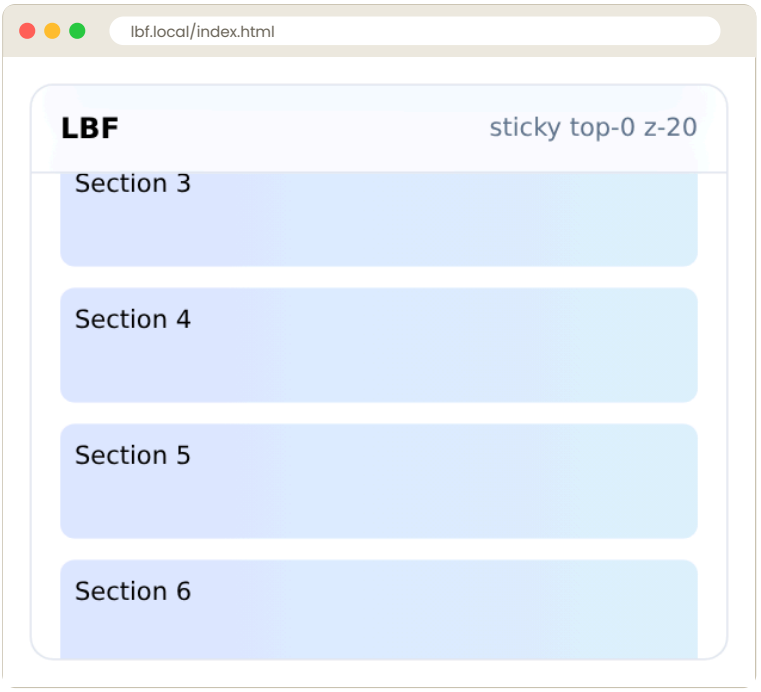
```
<div class="grid grid-cols-3 gap-3 text-xs">
  <div class="flex h-28 items-center justify-center rounded-lg bg-slate-100"><span class="rounded bg-indigo-500 px-2 py-1 text-white">flex items-center justify-center</span></div>
  <div class="grid h-28 place-items-center rounded-lg bg-slate-100"><span class="rounded bg-amber-400 px-2 py-1">grid place-items-center</span></div>
  <div class="relative h-28 rounded-lg bg-slate-100"><span class="absolute left-1/2 top-1/2 -translate-1/2 rounded bg-emerald-500 px-2 py-1 text-white">absolute + -translate-1/2</span></div>
</div>
```



THREE WAYS TO CENTRE IN BOTH DIRECTIONS

7.5.3 Sticky header and footer layout

A page shell where the footer sits at the bottom even on short pages: make the body a column (`flex min-h-dvh flex-col`), let `<main>` grow with `flex-1`, and make the header `sticky top-0 z-20` with a translucent, blurred background so content scrolls beneath it. The figure uses a 320px tall scrolling frame to show the header staying in place.



SCROLLED 180PX: THE CONTENT PASSES UNDER THE BLURRED, STICKY HEADER

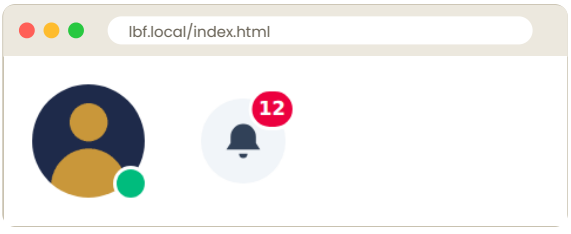
THE PAGE SHELL PATTERN HTML

```
<body class="flex min-h-dvh flex-col">
  <header class="sticky top-0 z-20 border-b border-slate-200 bg-white/80 backdrop-blur">...</header>
  <main class="flex-1">...</main>
  <footer class="bg-slate-900 text-slate-300">...</footer>
</body>
```

7.5.4 Badge positioned on an avatar

AVATAR-BADGE.HTML HTML

```
<div class="flex items-center gap-8">
  <div class="relative">
    <img src="avatar.png" alt="Asha Rao" class="size-16 rounded-full">
    <span class="absolute bottom-0 right-0 size-4 rounded-full bg-emerald-500 ring-2 ring-white"><span
class="sr-only">Online</span></span>
  </div>
  <div class="relative">
    <button class="rounded-full bg-slate-100 p-3"><svg viewBox="0 0 20 20" class="size-6 fill-slate-
700"><path d="M10 2a6 6 0 0 0-6 6v3l-2 3h16l-2-3V8a6 6 0 0 0-6-6Zm0 16a2 2 0 0 0 2-2H8a2 2 0 0 0 2
2Z"/></svg></button>
    <span class="absolute -right-1 -top-1 grid h-5 min-w-5 place-items-center rounded-full bg-rose-600
px-1 text-[11px] font-bold text-white ring-2 ring-white">12</span>
  </div>
</div>
```



STATUS DOT AND A COUNT BADGE

The parent is `relative`, the badge `absolute` with corner offsets (negative to hang over the edge). `ring-2 ring-white` draws a gap between badge and avatar that matches the background, and `min-w-5` keeps the count badge round for one digit while letting it widen for two.

PRACTICE CHECKPOINT

A **full page shell** with a sticky navbar, a hero with centred content, a row of feature cards and a footer, all with Flexbox utilities.

PAGE-SHELL.HTML: SAMPLE SOLUTION

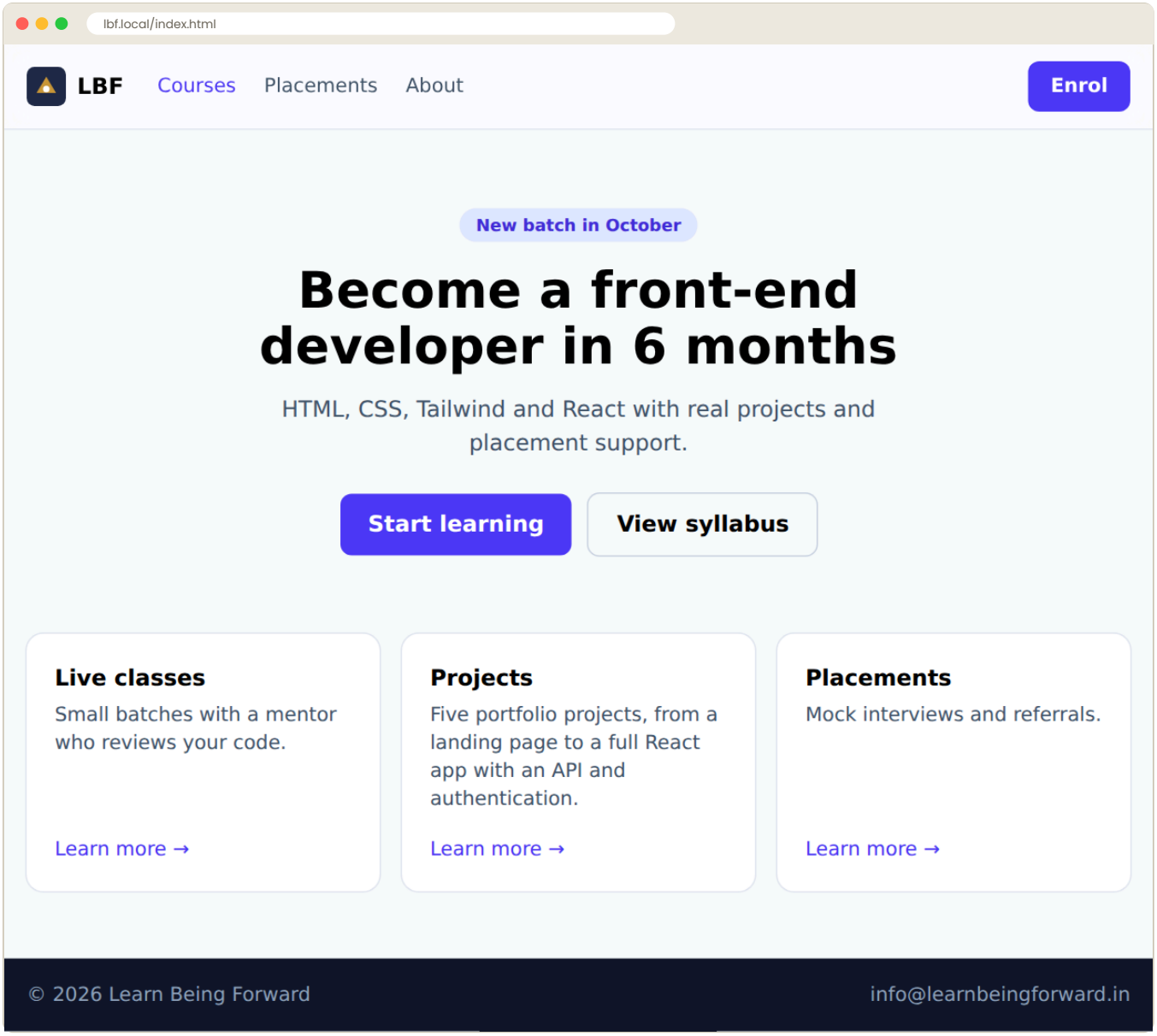
HTML

```
<div class="flex min-h-[640px] flex-col bg-slate-50">
  <header class="sticky top-0 z-20 border-b border-slate-200 bg-white/80 backdrop-blur">
    <div class="mx-auto flex max-w-5xl items-center gap-6 px-4 py-3">
      <a href="#" class="flex items-center gap-2 font-bold"><img src="logo.svg" alt="" class="size-7">
LBF</a>
      <nav class="flex gap-5 text-sm text-slate-600"><a href="#" class="text-indigo-600">Courses</a><a
href="#">Placements</a><a href="#">About</a></nav>
      <a href="#" class="ml-auto rounded-lg bg-indigo-600 px-4 py-2 text-sm font-semibold text-
white">Enrol</a>
    </div>
  </header>

  <main class="flex-1">
    <section class="flex flex-col items-center px-4 py-14 text-center">
      <p class="rounded-full bg-indigo-100 px-3 py-1 text-xs font-semibold text-indigo-700">New batch
in October</p>
      <h1 class="mt-4 max-w-xl text-4xl font-black text-balance">Become a front-end developer in 6
months</h1>
      <p class="mt-3 max-w-md text-slate-600">HTML, CSS, Tailwind and React with real projects and
placement support.</p>
      <div class="mt-6 flex gap-3">
        <a href="#" class="rounded-lg bg-indigo-600 px-5 py-2.5 font-semibold text-white">Start
learning</a>
        <a href="#" class="rounded-lg px-5 py-2.5 font-semibold ring-1 ring-slate-300">View
syllabus</a>
      </div>
    </section>

    <section class="mx-auto flex max-w-5xl gap-4 px-4 pb-12">
      <article class="flex flex-1 flex-col rounded-xl bg-white p-5 ring-1 ring-slate-200">
        <h3 class="font-semibold">Live classes</h3><p class="mt-1 flex-1 text-sm text-slate-600">Small
batches with a mentor who reviews your code.</p>
        <a href="#" class="mt-4 text-sm font-medium text-indigo-600">Learn more &rarr;</a>
      </article>
      <article class="flex flex-1 flex-col rounded-xl bg-white p-5 ring-1 ring-slate-200">
        <h3 class="font-semibold">Projects</h3><p class="mt-1 flex-1 text-sm text-slate-600">Five
portfolio projects, from a landing page to a full React app with an API and authentication.</p>
        <a href="#" class="mt-4 text-sm font-medium text-indigo-600">Learn more &rarr;</a>
      </article>
      <article class="flex flex-1 flex-col rounded-xl bg-white p-5 ring-1 ring-slate-200">
        <h3 class="font-semibold">Placements</h3><p class="mt-1 flex-1 text-sm text-slate-600">Mock
interviews and referrals.</p>
        <a href="#" class="mt-4 text-sm font-medium text-indigo-600">Learn more &rarr;</a>
      </article>
    </section>
  </main>

  <footer class="bg-slate-900 text-sm text-slate-400">
    <div class="mx-auto flex max-w-5xl items-center justify-between px-4 py-4"><span>&copy; 2026 Learn
Being Forward</span><span>info@learnbeingforward.in</span></div>
  </footer>
</div>
```



THE PAGE SHELL: THE CARD LINKS LINE UP AT THE BOTTOM BECAUSE EACH CARD IS A COLUMN WHOSE PARAGRAPH HAS FLEX-1

Layout in Tailwind is a handful of class groups: `relative` / `absolute` with `inset-*` for overlays, `flex items-center gap-*` for rows, `ml-auto` to push items apart, `flex-1` to fill space, `min-w-0` to let flex items shrink, and `mx-auto max-w-* px-4` for centred content.

MODULE 08 · INTERMEDIATE

CSS Grid with Tailwind

Minimum duration **1 day** | Level **Intermediate**

Grid utilities create two-dimensional layouts for galleries, dashboards and page sections. This module covers grid containers with column and row counts, gaps, spanning and explicit placement of items, auto-flow and implicit row sizing; arbitrary template grids, auto-fit responsive grids that need no breakpoints, and subgrid for aligning content across cards; the alignment utilities; and three complete layouts: a dashboard with a sidebar, a bento grid and an image gallery with featured items. Grid concepts are explained fully in the CSS3 book, Module 11.

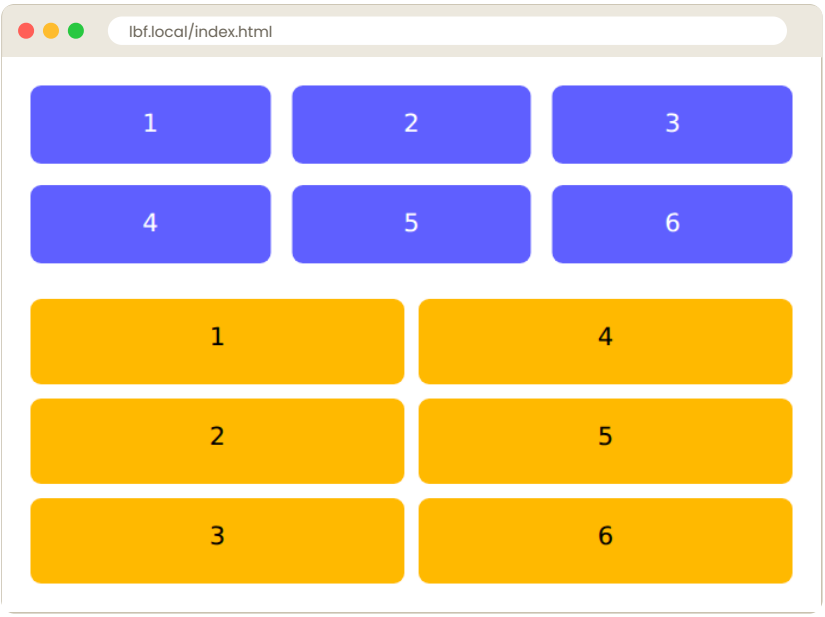
IN THIS MODULE

8.1 Grid basics
8.2 Placing items
8.3 Arbitrary and template grids
8.4 Alignment
8.5 Common grid layouts

8.1 Grid basics

8.1.1 grid, grid-cols-*, grid-rows-*

`grid` makes a grid container. `grid-cols-3` creates three equal columns, generated as `grid-template-columns: repeat(3, minmax(0, 1fr))`. The `minmax(0, 1fr)` (rather than plain `1fr`) lets columns shrink below their content, so a long word or wide image cannot stretch one column and break the grid. Any number works in v4 (`grid-cols-7`, `grid-cols-15`), plus `grid-cols-none` and `grid-cols-subgrid`. `grid-rows-*` does the same for rows, which is useful when the container has a fixed height.



GRID-COLS-3 (TOP); GRID-ROWS-3 WITH GRID-FLOW-COL IN A 160PX-TALL BOX (BOTTOM)

TWO GRIDS

HTML

```
<div class="grid grid-cols-3 gap-3">...6 items...</div>
<div class="grid h-40 grid-flow-col grid-rows-3 gap-2">...6 items...</div>
```

COMPILED CSS

```
.grid {
  display: grid;
}
.grid-cols-3 {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}
.grid-cols-7 {
  grid-template-columns: repeat(7, minmax(0, 1fr));
}
.grid-rows-3 {
  grid-template-rows: repeat(3, minmax(0, 1fr));
}
```

8.1.2 gap-*, gap-x-*, gap-y-*

`gap-4` sets the space between both rows and columns; `gap-x-8` (column gap) and `gap-y-2` (row gap) set them separately. Gaps never appear on the outside edges, so the grid aligns perfectly with surrounding content.

INDEX.HTML

HTML

```
<div class="grid grid-cols-4 gap-x-8 gap-y-2 rounded-lg bg-slate-100 p-2"><div class="rounded-md bg-indigo-500 text-white py-3 text-center text-sm">1</div><div class="rounded-md bg-indigo-500 text-white py-3 text-center text-sm">2</div><div class="rounded-md bg-indigo-500 text-white py-3 text-center text-sm">3</div><div class="rounded-md bg-indigo-500 text-white py-3 text-center text-sm">4</div><div class="rounded-md bg-indigo-500 text-white py-3 text-center text-sm">5</div><div class="rounded-md bg-indigo-500 text-white py-3 text-center text-sm">6</div><div class="rounded-md bg-indigo-500 text-white py-3 text-center text-sm">7</div><div class="rounded-md bg-indigo-500 text-white py-3 text-center text-sm">8</div></div>
```



GAP-X-8 GAP-Y-2: WIDE COLUMN GAPS, NARROW ROW GAPS

8.2 Placing items

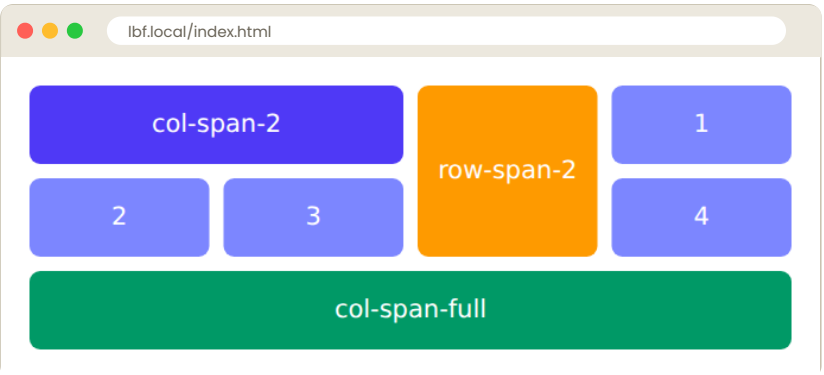
8.2.1 col-span-*, row-span-*

By default each item fills one cell. `col-span-2` makes an item span two columns, `row-span-2` two rows, and `col-span-full` the entire width (a heading or banner inside a grid). Spanning is the basis of magazine and bento layouts.

INDEX.HTML

HTML

```html
<div class="grid grid-cols-4 gap-2 text-center text-sm text-white">
  <div class="col-span-2 rounded-md bg-indigo-600 py-3">col-span-2</div>
  <div class="row-span-2 grid place-items-center rounded-md bg-amber-500">row-span-2</div>
  <div class="rounded-md bg-indigo-400 py-3">1</div>
  <div class="rounded-md bg-indigo-400 py-3">2</div>
  <div class="rounded-md bg-indigo-400 py-3">3</div>
  <div class="rounded-md bg-indigo-400 py-3">4</div>
  <div class="col-span-full rounded-md bg-emerald-600 py-3">col-span-full</div>
</div>
```



SPANNING COLUMNS AND ROWS

COMPILED CSS

```css
.col-span-2 {
  grid-column: span 2 / span 2;
}
.col-span-full {
  grid-column: 1 / -1;
}
.row-span-2 {
  grid-row: span 2 / span 2;
}
```

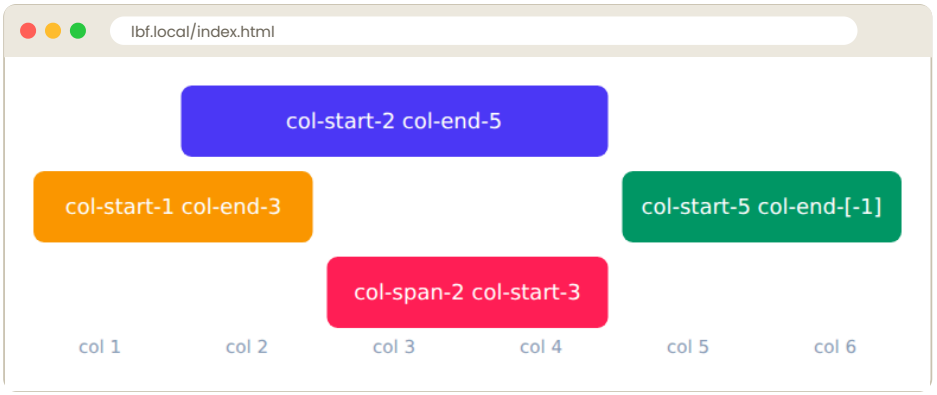
8.2.2 col-start-*, col-end-*

For exact placement, give the line numbers where an item starts and ends. Grid lines are numbered from 1 at the left edge; in a 6-column grid the right edge is line 7, and `-1` always means the last line. `col-start-2 col-end-5` covers columns 2 to 4. Rows work the same with `row-start-*` and `row-end-*`. v4 also has the shorthands `col-[2/5]` and `row-[1/span_2]` via arbitrary values.

INDEX.HTML

HTML

```html
<div class="relative grid grid-cols-6 gap-2 text-center text-xs text-white">
  <div class="col-start-2 col-end-5 rounded-md bg-indigo-600 py-3">col-start-2 col-end-5</div>
  <div class="col-start-1 col-end-3 rounded-md bg-amber-500 py-3">col-start-1 col-end-3</div>
  <div class="col-start-5 col-end-[-1] rounded-md bg-emerald-600 py-3">col-start-5 col-end-[-1]</div>
  <div class="col-span-2 col-start-3 rounded-md bg-rose-500 py-3">col-span-2 col-start-3</div>
</div>
<div class="mt-1 grid grid-cols-6 gap-2 text-center text-[10px] text-slate-400"><span>col 1</span>
<span>col 2</span><span>col 3</span><span>col 4</span><span>col 5</span><span>col 6</span></div>
```



PLACING ITEMS BY GRID LINE NUMBERS

8.2.3 grid-flow-* and auto-rows-*

`grid-flow-row` (default) fills items row by row; `grid-flow-col` fills column by column; `grid-flow-dense` lets later small items fill holes left by earlier large ones, which can reorder items visually. Rows created automatically (beyond any `grid-rows-*`) are sized by `auto-rows-*`: `auto-rows-fr` (equal heights), `auto-rows-min`, `auto-rows-max` or `auto-rows-[120px]`; `auto-cols-*` does the same for implicit columns.



AUTO-ROWS-[48PX] WITH AND WITHOUT DENSE PACKING

8.3 Arbitrary and template grids

8.3.1 grid-cols-[200px_1fr]

Any `grid-template-columns` value can be written in brackets, with underscores for spaces: `grid-cols-[200px_1fr]` (a fixed sidebar and flexible content), `grid-cols-[auto_1fr_auto]` (a media object), `grid-cols-[1fr_minmax(0,65ch)_1fr]` (a centred reading column with full-bleed sides). Named areas also work through arbitrary properties: `[grid-template-areas:'head_head'_'side_main']` together with `[grid-area:side]` on children.

INDEX.HTML

HTML

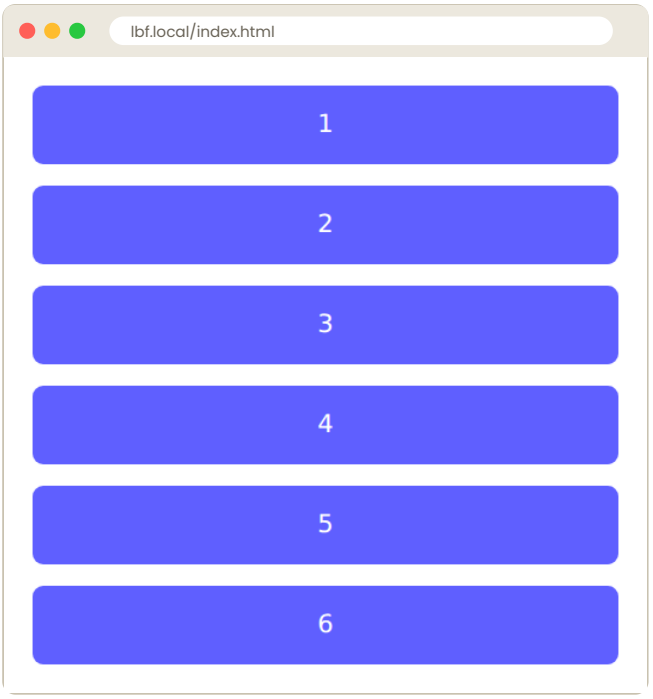
```
<div class="grid grid-cols-[200px_1fr] gap-3 text-sm">
  <aside class="rounded-lg bg-slate-800 p-3 text-slate-200">200px sidebar</aside>
  <main class="rounded-lg bg-slate-100 p-3">1fr: the content takes the rest of the width</main>
</div>
<div class="mt-4 grid grid-cols-[auto_1fr_auto] items-center gap-3 rounded-lg bg-white p-3 text-sm
ring-1 ring-slate-200">
  <img src="avatar.png" alt="" class="size-10 rounded-full">
  <div><b>Asha Rao</b><p class="text-slate-500">grid-cols-[auto_1fr_auto]: a media object</p></div>
  <button class="rounded-lg bg-indigo-600 px-3 py-1.5 text-white">Follow</button>
</div>
```



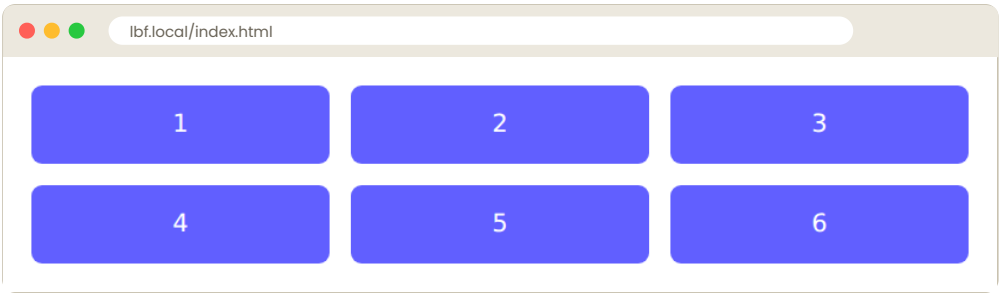
ARBITRARY COLUMN TEMPLATES

8.3.2 Auto-fit responsive grids grid-cols-[repeat(auto-fit,minmax(16rem,1fr))]

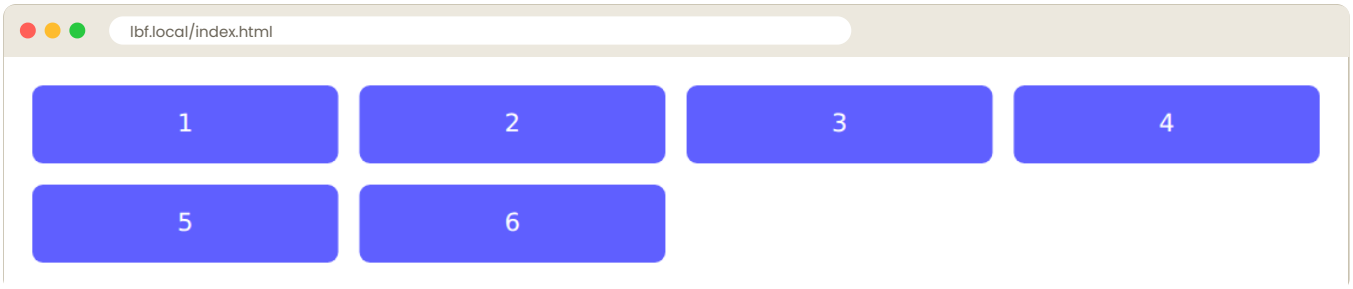
The most useful grid in practice needs **no breakpoints**: `grid-cols-[repeat(auto-fit,minmax(16rem,1fr))]` fits as many columns of at least 16rem as there is room for, and stretches them to fill the row. Wrapping the minimum in `min()`, `minmax(min(16rem,100%),1fr)`, stops the grid overflowing on screens narrower than 16rem. `auto-fill` instead of `auto-fit` keeps empty tracks, so a few items do not stretch to fill a wide row.



360PX: 2 COLUMNS



560PX: 3 COLUMNS



760PX: 4 COLUMNS. ONE CLASS, NO MEDIA QUERIES: THE COLUMN COUNT FOLLOWS THE AVAILABLE WIDTH

THE AUTO-FIT GRID HTML

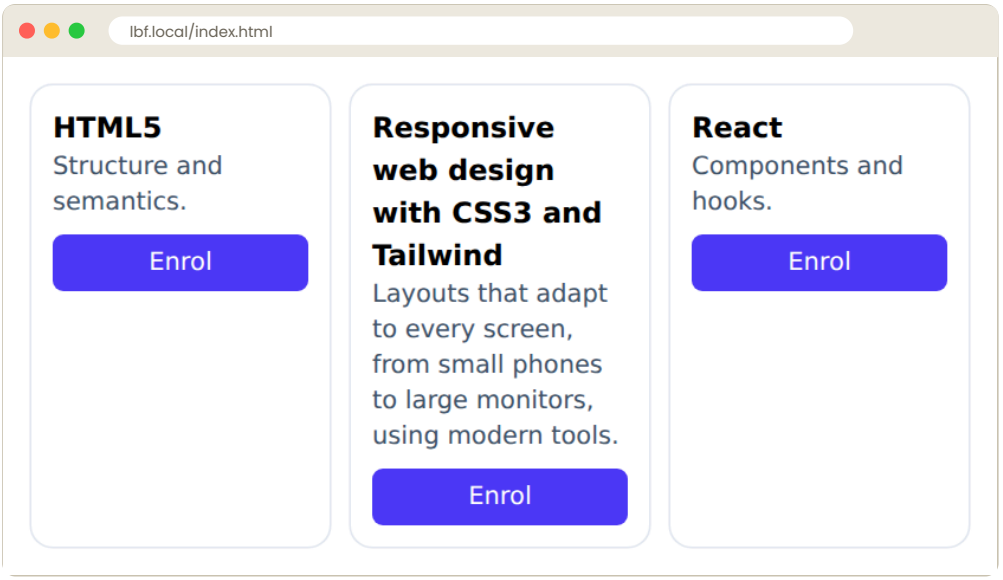
```
<div class="grid grid-cols-[repeat(auto-fit,minmax(min(10rem,100%),1fr))] gap-3">
  ...
</div>
```

TIP

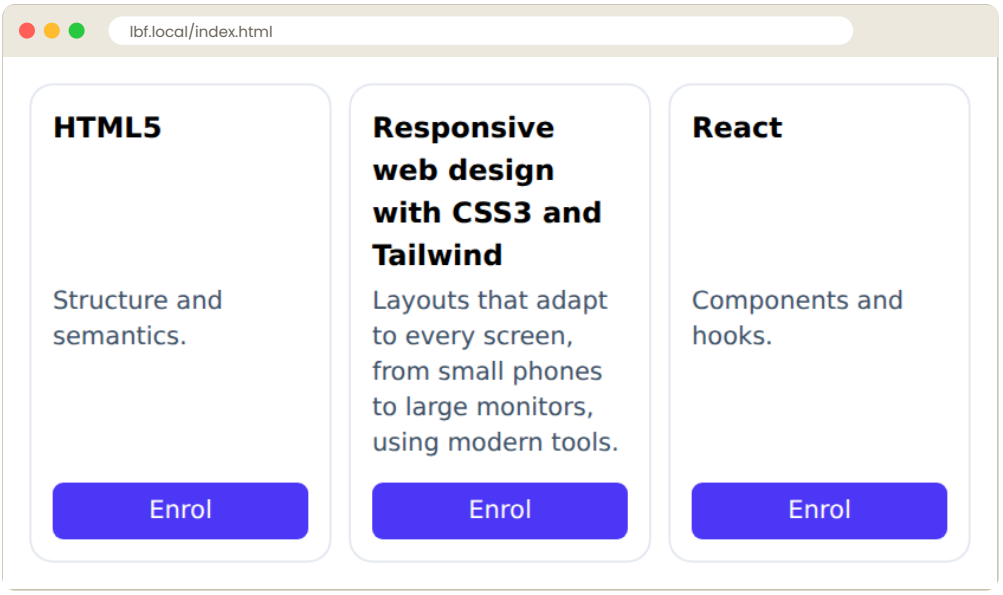
Because the class is long, it is a good candidate for a custom utility: `@utility grid-auto-fit { grid-template-columns: repeat(auto-fit, minmax(min(var(--min, 16rem), 100%), 1fr)); }` (Module 14).

8.3.3 Subgrid grid-cols-subgrid

Cards in a row often have titles and descriptions of different lengths, so their buttons end up at different heights. With **subgrid**, each card becomes a grid that borrows its **rows** from the parent: give the parent rows for every card part, make each card `row-span-3 grid grid-rows-subgrid`, and every title, text and button lines up across all cards.



WITHOUT SUBGRID: EACH CARD IS A FLEX COLUMN, SO TEXT AND BUTTONS SIT AT DIFFERENT HEIGHTS



WITH GRID-ROWS-SUBGRID: TITLES, DESCRIPTIONS AND BUTTONS SHARE THE PARENT ROWS AND LINE UP

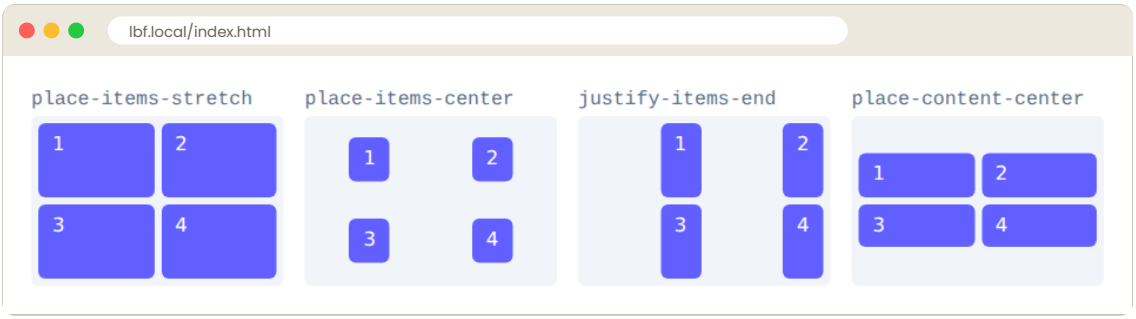
THE SUBGRID PATTERN HTML

```
<div class="grid grid-cols-3 grid-rows-[auto_1fr_auto] gap-x-3">
  <article class="row-span-3 grid grid-rows-subgrid gap-y-1 ...">
    <h3>...</h3> <p>...</p> <a>Enrol</a>
  </article>
  ...
</div>
```

8.4 Alignment

8.4.1 place-items-*, place-content-*, justify-items-*

Utility group	Aligns	Axis
`justify-items-*`	each item inside its cell	inline (horizontal)
`items-*`	each item inside its cell	block (vertical)
`place-items-*`	both at once (`place-items-center`)	both
`justify-self-*` , `self-*` , `place-self-*`	a single item	one or both
`justify-content-*` (`justify-*`)	the whole grid tracks inside a larger container	inline
`content-*` , `place-content-*`	the whole grid tracks	block / both



ITEM ALIGNMENT VERSUS CONTENT (TRACK) ALIGNMENT

8.5 Common grid layouts

8.5.1 Dashboard with sidebar

A dashboard shell is a two-column grid with a fixed sidebar (`grid-cols-[220px_1fr]`) and a full-height page (`min-h-dvh`). Inside the main area, the header and content stack with `grid-rows-[auto_1fr]`. The checkpoint at the end of this module builds the full version.

THE DASHBOARD SKELETON HTML

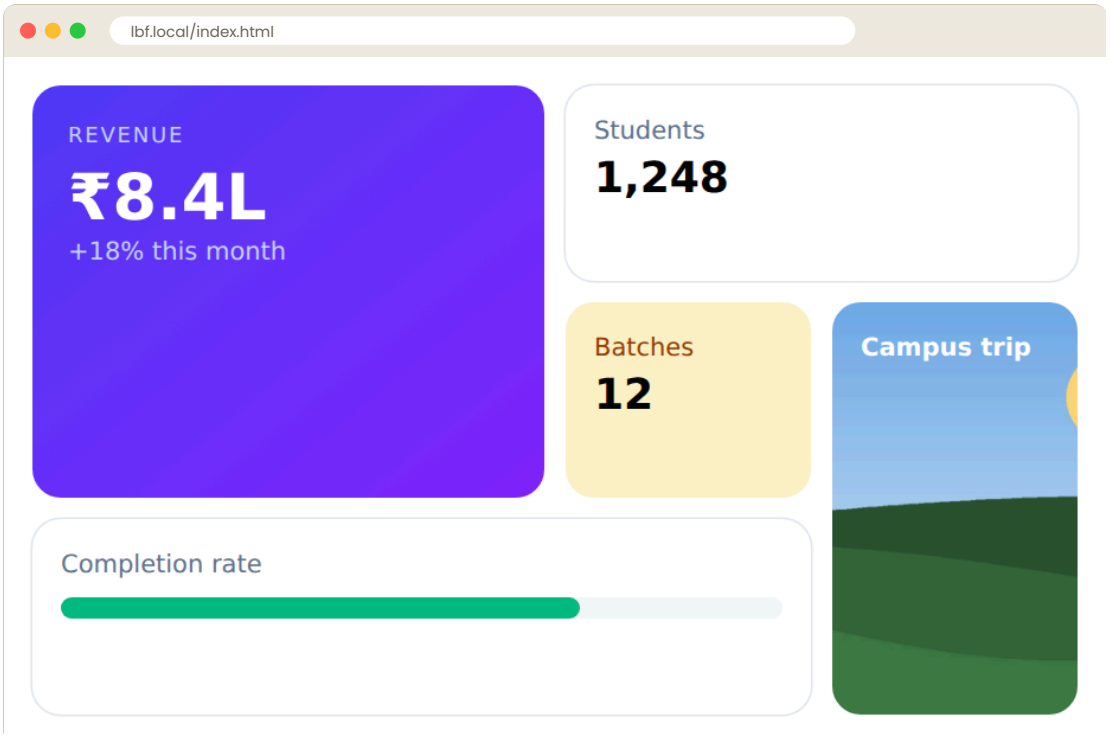
```
<div class="grid min-h-dvh grid-cols-[220px_1fr]">
  <aside class="bg-slate-900 text-slate-300">...navigation...</aside>
  <div class="grid grid-rows-[auto_1fr]">
    <header class="border-b border-slate-200">...</header>
    <main class="p-6">...</main>
  </div>
</div>
```

8.5.2 Bento grid

A **bento grid** (named after the compartmented lunch box) is a grid of tiles of different sizes, popular on product pages and dashboards. It is built from a fixed column count, a fixed row height (`auto-rows-[110px]`) and a mix of `col-span-*` and `row-span-*` tiles.

BENTO.HTML HTML

```
<div class="grid grid-cols-4 auto-rows-[110px] gap-3 text-sm">
  <div class="col-span-2 row-span-2 rounded-2xl bg-linear-to-br from-indigo-600 to-violet-600 p-5
text-white">
    <p class="text-xs uppercase tracking-widest text-indigo-200">Revenue</p><p class="mt-2 text-4xl
font-black">&amp;#8377;8.4L</p><p class="text-indigo-200">+18% this month</p>
  </div>
  <div class="col-span-2 rounded-2xl bg-white p-4 ring-1 ring-slate-200"><p class="text-slate-
500">Students</p><p class="text-2xl font-bold">1,248</p></div>
  <div class="rounded-2xl bg-amber-100 p-4"><p class="text-amber-800">Batches</p><p class="text-2xl
font-bold">12</p></div>
  <div class="row-span-2 rounded-2xl bg-[url(hills.jpg)] bg-cover bg-center p-4 font-semibold text-
white">Campus trip</div>
  <div class="col-span-3 rounded-2xl bg-white p-4 ring-1 ring-slate-200"><p class="text-slate-
500">Completion rate</p><div class="mt-2 h-3 rounded-full bg-slate-100"><div class="h-3 w-[72%]
rounded-full bg-emerald-500"></div></div></div>
</div>
```



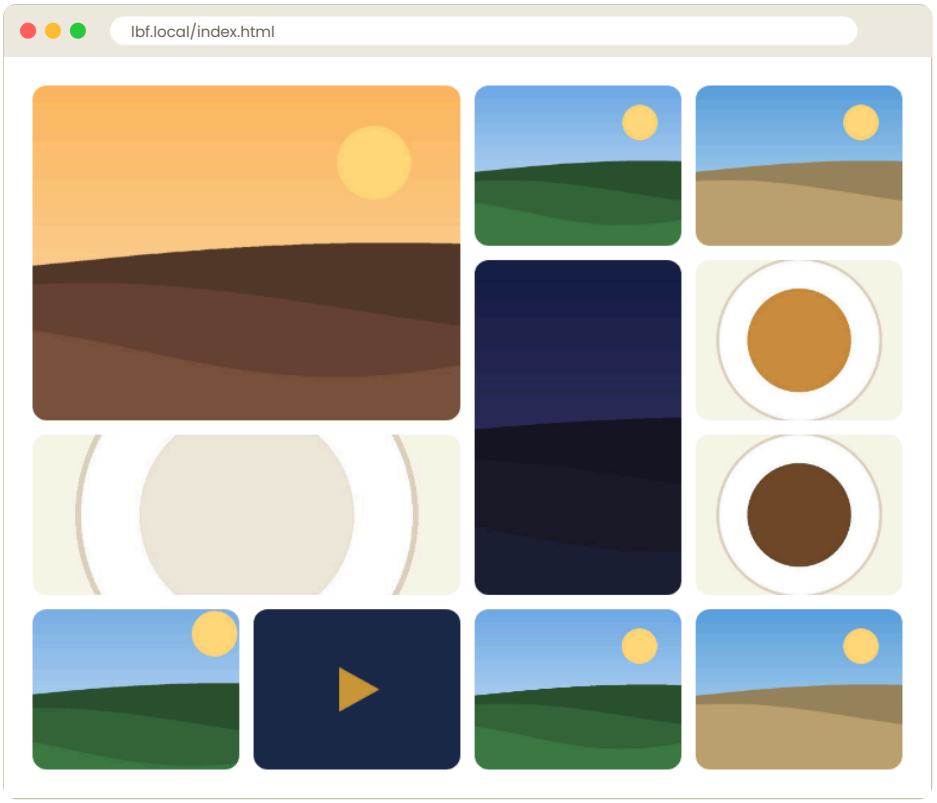
A BENTO GRID FROM COL-SPAN AND ROW-SPAN TILES

8.5.3 Image gallery with featured items

GALLERY.HTML

HTML

```
<div class="grid grid-cols-4 auto-rows-[90px] grid-flow-dense gap-2"><img src="mysuru-palace.jpg"
alt="" class="h-full w-full rounded-lg object-cover col-span-2 row-span-2"><img src="hills.jpg" alt=""
class="h-full w-full rounded-lg object-cover "><img src="beach.jpg" alt="" class="h-full w-full
rounded-lg object-cover "><img src="night.jpg" alt="" class="h-full w-full rounded-lg object-cover
row-span-2"><img src="food-dosa.jpg" alt="" class="h-full w-full rounded-lg object-cover "><img
src="food-idli.jpg" alt="" class="h-full w-full rounded-lg object-cover col-span-2"><img src="food-
coffee.jpg" alt="" class="h-full w-full rounded-lg object-cover "><img src="hills-square.jpg" alt=""
class="h-full w-full rounded-lg object-cover "><img src="poster.jpg" alt="" class="h-full w-full
rounded-lg object-cover "><img src="hills-800.jpg" alt="" class="h-full w-full rounded-lg object-cover
"><img src="beach.jpg" alt="" class="h-full w-full rounded-lg object-cover "></div>
```



FEATURED PHOTOS SPAN TWO CELLS; GRID-FLOW-DENSE PACKS THE REST INTO THE GAPS

Every image uses `h-full w-full object-cover` so it fills its cell whatever its original shape. Featured images get `col-span-2 row-span-2`, and `grid-flow-dense` avoids holes. Because dense packing can change the visual order, it suits galleries where order does not matter.

PRACTICE CHECKPOINT

An **admin dashboard** with a sidebar, stat cards and a bento-style analytics grid.

DASHBOARD.HTML: SAMPLE SOLUTION

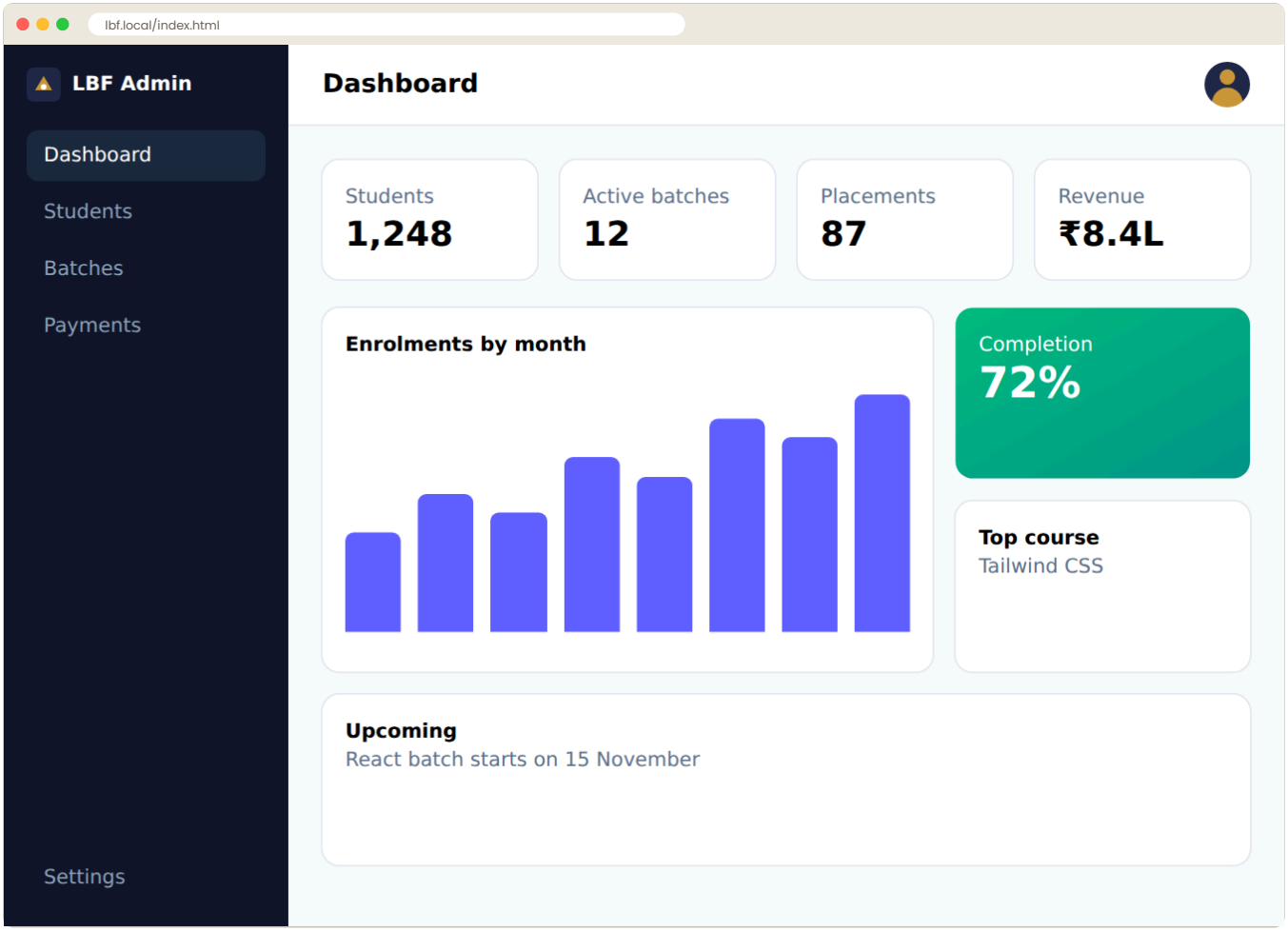
HTML

```
<div class="grid min-h-[620px] grid-cols-[200px_1fr] bg-slate-50 text-sm">
  <aside class="flex flex-col gap-1 bg-slate-900 p-4 text-slate-400">
    <a href="#" class="mb-4 flex items-center gap-2 font-bold text-white"><img src="logo.svg" alt=""
class="size-6"> LBF Admin</a>
    <a href="#" class="rounded-lg bg-slate-800 px-3 py-2 text-white">Dashboard</a>
    <a href="#" class="rounded-lg px-3 py-2 hover:bg-slate-800">Students</a>
    <a href="#" class="rounded-lg px-3 py-2 hover:bg-slate-800">Batches</a>
    <a href="#" class="rounded-lg px-3 py-2 hover:bg-slate-800">Payments</a>
    <a href="#" class="mt-auto rounded-lg px-3 py-2 hover:bg-slate-800">Settings</a>
  </aside>

  <div class="grid grid-rows-[auto_1fr]">
    <header class="flex items-center justify-between border-b border-slate-200 bg-white px-6 py-3">
      <h1 class="text-lg font-bold">Dashboard</h1>
      <img src="avatar.png" alt="" class="size-8 rounded-full">
    </header>

    <main class="space-y-5 p-6">
      <section class="grid grid-cols-4 gap-4">
        <div class="rounded-xl bg-white p-4 ring-1 ring-slate-200"><p class="text-slate-
500">Students</p><p class="text-2xl font-bold">1,248</p></div>
        <div class="rounded-xl bg-white p-4 ring-1 ring-slate-200"><p class="text-slate-500">Active
batches</p><p class="text-2xl font-bold">12</p></div>
        <div class="rounded-xl bg-white p-4 ring-1 ring-slate-200"><p class="text-slate-
500">Placements</p><p class="text-2xl font-bold">87</p></div>
        <div class="rounded-xl bg-white p-4 ring-1 ring-slate-200"><p class="text-slate-
500">Revenue</p><p class="text-2xl font-bold">&amp;#8377;8.4L</p></div>
      </section>

      <section class="grid grid-cols-3 auto-rows-[120px] gap-4">
        <div class="col-span-2 row-span-2 rounded-xl bg-white p-4 ring-1 ring-slate-200">
          <p class="font-semibold">Enrolments by month</p>
          <div class="mt-4 flex h-44 items-end gap-3"><div class="flex-1 rounded-t-md bg-indigo-500"
style="height:40%"></div><div class="flex-1 rounded-t-md bg-indigo-500" style="height:55%"></div><div
class="flex-1 rounded-t-md bg-indigo-500" style="height:48%"></div><div class="flex-1 rounded-t-md bg-
indigo-500" style="height:70%"></div><div class="flex-1 rounded-t-md bg-indigo-500"
style="height:62%"></div><div class="flex-1 rounded-t-md bg-indigo-500" style="height:85%"></div><div
class="flex-1 rounded-t-md bg-indigo-500" style="height:78%"></div><div class="flex-1 rounded-t-md bg-
indigo-500" style="height:95%"></div></div>
        </div>
        <div class="rounded-xl bg-linear-to-br from-emerald-500 to-teal-600 p-4 text-white">
<p>Completion</p><p class="text-3xl font-black">72%</p></div>
        <div class="rounded-xl bg-white p-4 ring-1 ring-slate-200"><p class="font-semibold">Top
course</p><p class="text-slate-500">Tailwind CSS</p></div>
        <div class="col-span-3 rounded-xl bg-white p-4 ring-1 ring-slate-200"><p class="font-
semibold">Upcoming</p><p class="text-slate-500">React batch starts on 15 November</p></div>
      </section>
    </main>
  </div>
</div>
```



THE ADMIN DASHBOARD

Use `grid-cols-N` for equal columns, arbitrary templates such as `grid-cols-[220px_1fr]` for page shells, `repeat(auto-fit,minmax(...))` for responsive card grids without breakpoints, spans for bento and gallery layouts, and `grid-rows-subgrid` to align the parts of sibling cards.

MODULE 09 · INTERMEDIATE

Responsive design and container queries

Minimum duration **2 days** | Level **Intermediate**

Tailwind is mobile-first: unprefixed utilities apply at every width, and breakpoint prefixes apply from that width upward. This module covers the default breakpoints and how to read md:flex, targeting a range with max-* variants and custom breakpoints; the everyday responsive patterns (stacking columns, responsive type and spacing, showing and hiding, a mobile menu, responsive images); container queries with @container, size variants and named containers, and when to choose them over breakpoints; the other media variants for reduced motion, print, orientation and touch devices; and how to test all of this in browser DevTools. Every multi-width figure is the same HTML rendered at real viewport widths in Chromium.

IN THIS MODULE

9.1 Mobile-first breakpoints
9.2 Responsive patterns
9.3 Container queries
9.4 Other media variants
9.5 Testing responsiveness in DevTools

9.1 Mobile-first breakpoints

9.1.1 Default breakpoints: sm, md, lg, xl, 2xl

Prefix	Minimum width	CSS generated	Typical devices
`sm:`	40rem (640px)	`@media (width >= 40rem)`	large phones in landscape
`md:`	48rem (768px)	`@media (width >= 48rem)`	tablets in portrait
`lg:`	64rem (1024px)	`@media (width >= 64rem)`	tablets in landscape, small laptops
`xl:`	80rem (1280px)	`@media (width >= 80rem)`	laptops and desktops
`2xl:`	96rem (1536px)	`@media (width >= 96rem)`	large desktop monitors

The breakpoints are theme variables (`--breakpoint-md: 48rem`), so they can be changed or extended in `@theme`: `--breakpoint-3xl: 120rem` creates a `3xl:` variant, and `--breakpoint-xs: 30rem` adds a smaller one (Module 13). They are written in rem so that they scale if the user increases the browser's default font size.

COMPILED CSS

```
.sm\:flex {
  @media (width >= 40rem) {
    display: flex;
  }
}
.md\:flex {
  @media (width >= 48rem) {
    display: flex;
  }
}
.lg\:flex {
  @media (width >= 64rem) {
    display: flex;
  }
}
.xl\:flex {
  @media (width >= 80rem) {
    display: flex;
  }
}
.\32 xl\:flex {
  @media (width >= 96rem) {
    display: flex;
  }
}
```

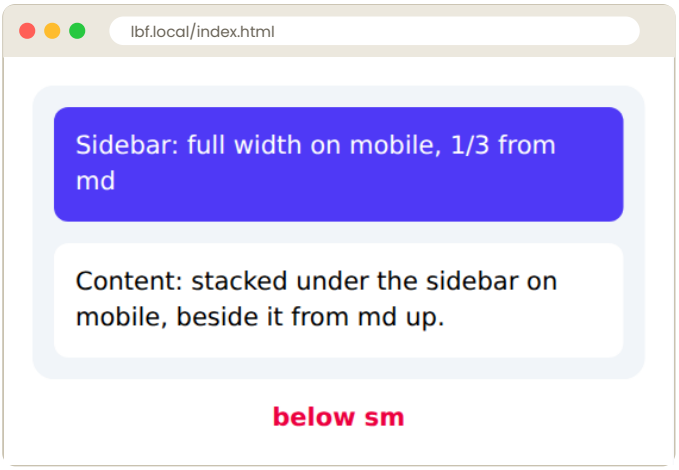
9.1.2 How md:flex means "from medium screens up"

There is no "mobile" prefix. Classes without a prefix are the mobile styles and apply everywhere; each breakpoint prefix **adds or overrides** from its width **upward**. So `flex-col md:flex-row` reads "a column on all screens, and a row from 768px up". The most common mistake is writing `sm:` to target phones: `sm:text-center` centres text on everything from 640px up, but **not** on small phones.

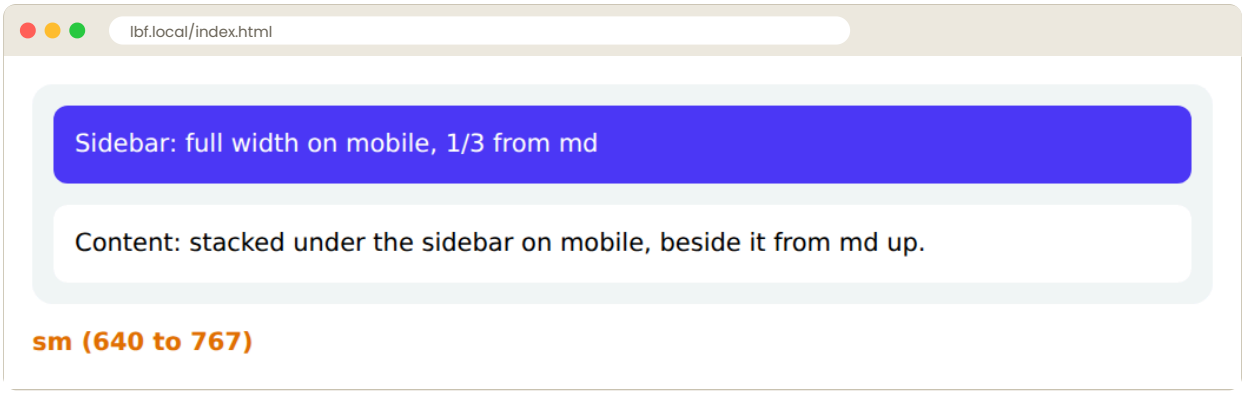
FLEX-COL MD:FLEX-ROW

HTML

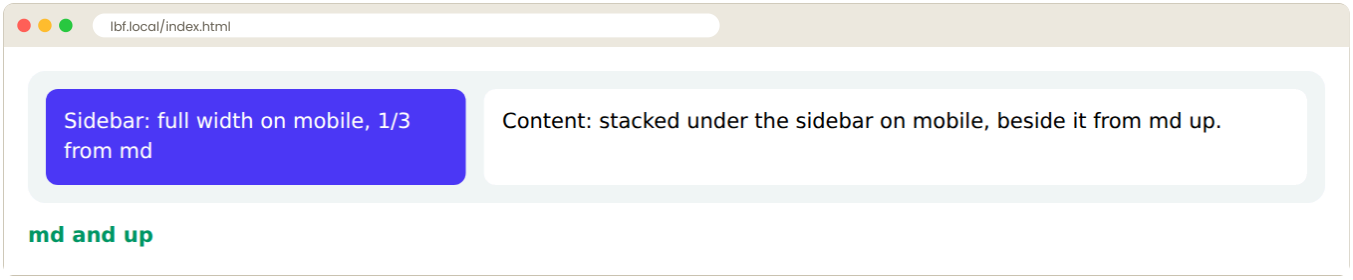
```
<div class="flex flex-col gap-3 rounded-xl bg-slate-100 p-3 text-sm md:flex-row">
  <div class="rounded-lg bg-indigo-600 p-3 text-white md:w-1/3">Sidebar: full width on mobile, 1/3
from md</div>
  <div class="flex-1 rounded-lg bg-white p-3">Content: stacked under the sidebar on mobile, beside it
from md up.</div>
</div>
<p class="mt-3 text-center text-sm font-semibold sm:text-left">
  <span class="text-rose-600 sm:hidden">below sm</span><span class="hidden text-amber-600 sm:inline
md:hidden">sm (640 to 767)</span><span class="hidden text-emerald-600 md:inline">md and up</span>
</p>
```



375PX



700PX



900PX. THE SAME HTML AT THREE VIEWPORT WIDTHS

COMPILED CSS

```
.md\:w-1\/3 {
  @media (width >= 48rem) {
    width: calc(1/3 * 100%);
  }
}
.md\:flex-row {
  @media (width >= 48rem) {
    flex-direction: row;
  }
}
```

9.1.3 Targeting a range with max-* variants (md:max-lg:)

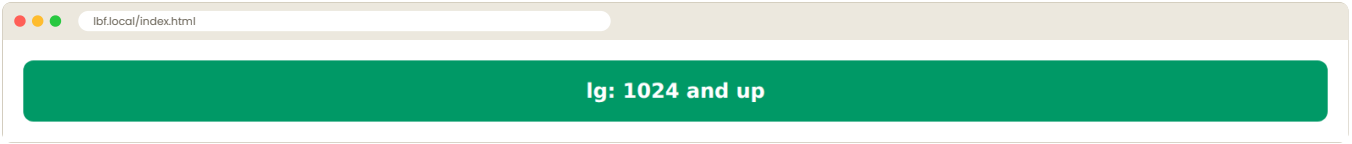
Every breakpoint has a `max-*` twin that applies **below** that width: `max-md:` means "width under 768px". Stacking a min and a max variant targets a range: `md:max-lg:` applies only from 768px up to 1023px (tablets). Use ranges sparingly; mobile-first overrides are usually simpler to maintain.



360PX



800PX



1060PX. MAX-MD:, MD:MAX-LG: AND LG: AS THREE EXCLUSIVE RANGES

COMPILED CSS

```
.max-md\:bg-rose-500 {
  @media (width < 48rem) {
    background-color: var(--color-rose-500);
  }
}
.md\:max-lg\:inline {
  @media (width >= 48rem) {
    @media (width < 64rem) {
      display: inline;
    }
  }
}
.md\:max-lg\:bg-amber-500 {
  @media (width >= 48rem) {
    @media (width < 64rem) {
      background-color: var(--color-amber-500);
    }
  }
}
```

9.1.4 Arbitrary breakpoints min-[900px]:

For a one-off width that is not worth adding to the theme, use `min-[900px]:` or `max-[600px]:`. Any media condition also works through the arbitrary variant syntax: `[@media(min-aspect-ratio:16/9)]:aspect-video`. If the same arbitrary breakpoint appears in several places, define it in the theme instead.

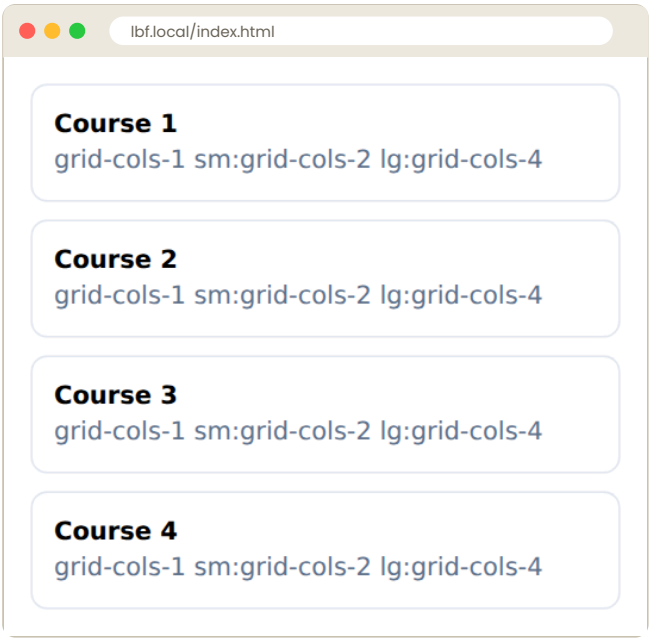
COMPILED CSS

```
.max-\[600px\]\:text-sm {
  @media (width < 600px) {
    font-size: var(--text-sm);
    line-height: var(--tw-leading, var(--text-sm--line-height));
  }
}
.min-\[900px\]\:grid-cols-3 {
  @media (width >= 900px) {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }
}
.\[\@media\(min-aspect-ratio\:16\/9\)\]\:aspect-video {
  @media (min-aspect-ratio:16/9) {
    aspect-ratio: var(--aspect-video);
  }
}
```

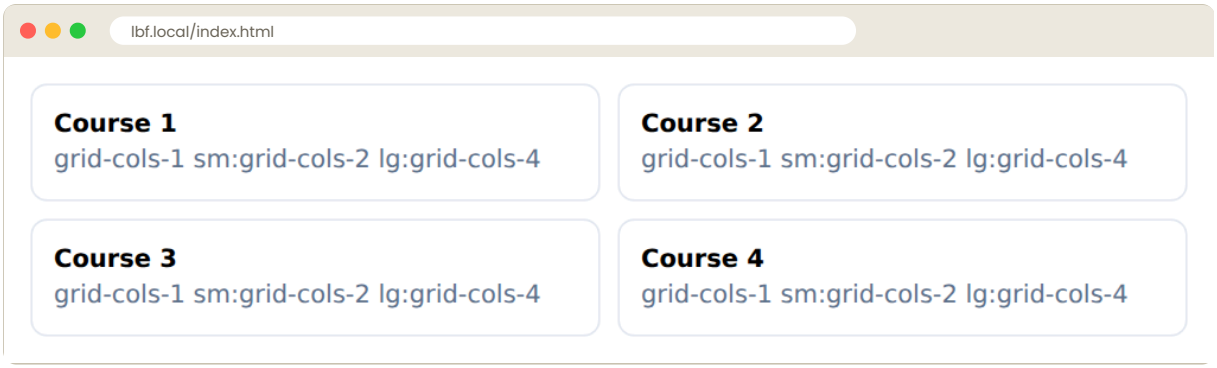
9.2 Responsive patterns

9.2.1 Stacking columns on mobile, side by side on desktop

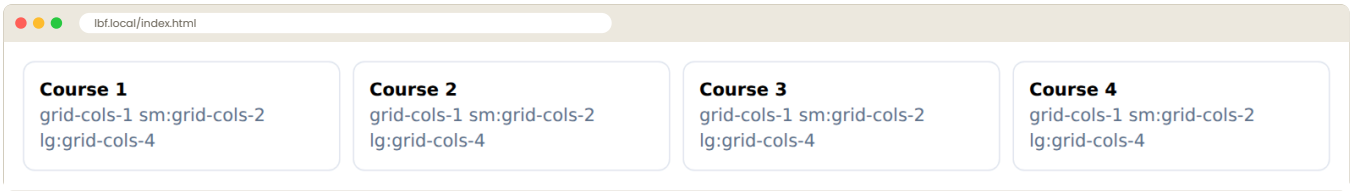
Three ways, depending on the layout: `flex flex-col md:flex-row` for two or three panels; `grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4` for a card grid with a known number of columns at each size; and the auto-fit grid from Module 8 when no specific counts are needed.



360PX: 1 COLUMN



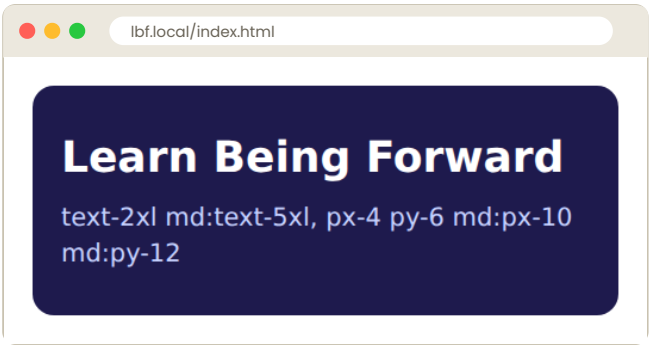
680PX: 2 COLUMNS



1060PX: 4 COLUMNS. A RESPONSIVE CARD GRID

9.2.2 Responsive typography and spacing

Headings and section padding usually grow with the screen: `text-3xl md:text-5xl`, `py-12 md:py-24`, `px-4 sm:px-6 lg:px-8`. An alternative to several steps is a fluid value that scales smoothly with the viewport: `text-[clamp(1.875rem,1rem+4vw,3.75rem)]`. Keep body text at 16px or larger on mobile; only headings need to change much.



360PX



820PX. THE HEADING, TEXT AND PADDING SCALE UP AT MD

9.2.3 Showing and hiding elements by screen size

`hidden md:block` hides an element on small screens and shows it from md up; `md:hidden` does the opposite. Pick the right display value for the visible state (`md:flex` for a flex row, `md:inline` for inline text).

Hidden content is still downloaded and is removed from the accessibility tree while hidden, so avoid duplicating large sections in two versions; prefer one flexible layout.



360PX: MENU BUTTON



800PX: FULL NAVIGATION. HIDDEN MD:FLEX FOR THE LINKS, MD:HIDDEN FOR THE BUTTON

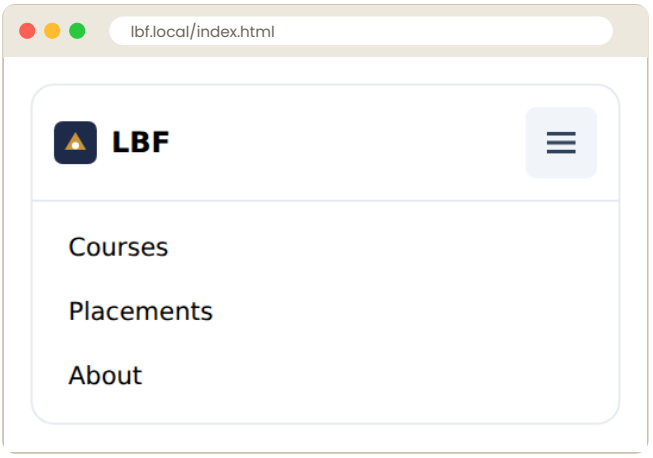
9.2.4 Responsive navigation with a mobile menu

A mobile menu needs a little JavaScript to toggle state, but the styling stays in Tailwind. The cleanest approach is to toggle an ARIA attribute and let variants do the rest: the button gets `aria-expanded` (which screen readers announce), and the menu panel is shown with a `group-aria-expanded:` or `peer-aria-expanded:` variant, or with a `data-open` attribute.

NAVBAR.HTML: A DATA-OPEN ATTRIBUTE ON THE HEADER DRIVES THE PANEL HTML

```
<header data-open="false" class="group rounded-xl bg-white ring-1 ring-slate-200">
  <div class="flex items-center justify-between p-3">
    <a href="#" class="flex items-center gap-2 font-bold"><img src="logo.svg" alt="" class="size-6">
LBF</a>
    <nav class="hidden gap-5 text-sm md:flex"><a href="#">Courses</a><a href="#">Placements</a><a
href="#">About</a></nav>
    <button id="menu-btn" aria-expanded="false" aria-controls="mobile-menu" class="rounded-md p-2
hover:bg-slate-100 md:hidden">
      <span class="sr-only">Open menu</span>
      <svg viewBox="0 0 24 24" class="size-6 stroke-slate-700" fill="none" stroke-width="2"><path
d="M4 7h16M4 12h16M4 17h16"/></svg>
    </button>
  </div>
  <nav id="mobile-menu" class="hidden flex-col border-t border-slate-200 p-2 text-sm group-data-
[open=true]:flex md:hidden">
    <a href="#" class="rounded-md px-3 py-2 hover:bg-slate-100">Courses</a>
    <a href="#" class="rounded-md px-3 py-2 hover:bg-slate-100">Placements</a>
    <a href="#" class="rounded-md px-3 py-2 hover:bg-slate-100">About</a>
  </nav>
</header>

<script>
  const btn = document.getElementById('menu-btn');
  btn.addEventListener('click', () => {
    const open = btn.getAttribute('aria-expanded') === 'true';
    btn.setAttribute('aria-expanded', String(!open));
    btn.closest('header').dataset.open = String(!open);
  });
</script>
```



AT 360PX AFTER TAPPING THE MENU BUTTON: DATA-OPEN="TRUE" MAKES GROUP-DATA-[OPEN=TRUE]:FLEX SHOW THE PANEL



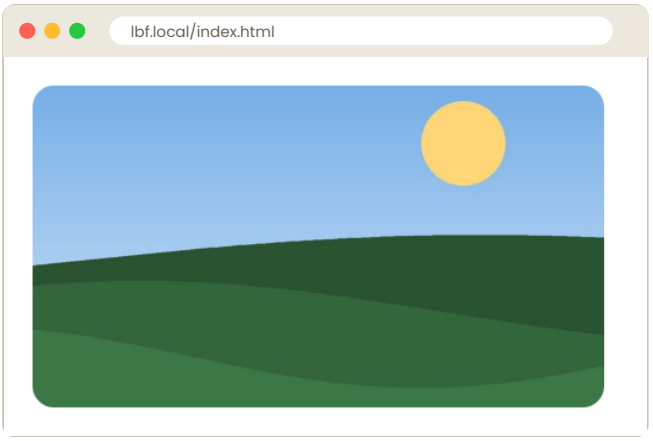
AT 800PX: THE INLINE LINKS SHOW AND THE BUTTON AND PANEL ARE HIDDEN BY MD:HIDDEN

9.2.5 Responsive images

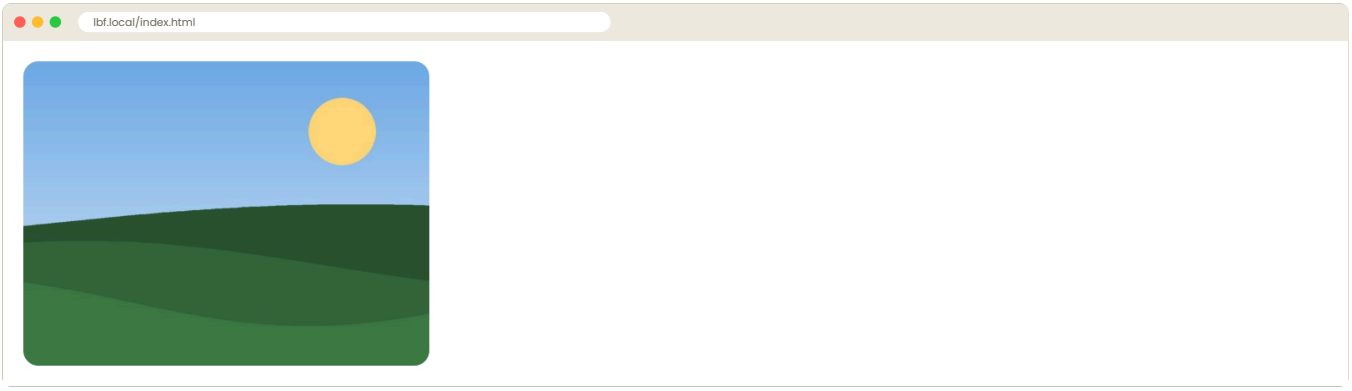
Images need two things to be responsive. Preflight already makes them `max-w-full h-auto` and `display: block`, so they never overflow. Beyond that, size them with utilities (`w-full aspect-video object-cover`) and let the browser choose the right file with `srcset` and `sizes` (HTML5 book, Module 6). The `sizes` attribute should mirror your breakpoint classes.

A RESPONSIVE IMAGE WHOSE SIZES MATCH GRID-COLS-1 SM:GRID-COLS-2 LG:GRID-COLS-3 HTML

```
<img
  src="hills-800.jpg"
  srcset="hills-400.jpg 400w, hills-800.jpg 800w, hills-1200.jpg 1200w"
  sizes="(width >= 64rem) 33vw, (width >= 40rem) 50vw, 100vw"
  alt="Green hills under an evening sky"
  class="aspect-video w-full rounded-xl object-cover lg:aspect-[4/3]">
```



360PX: ASPECT-VIDEO



1060PX: LG:ASPECT-[4/3]. THE IMAGE CHANGES SHAPE AT LG

9.3 Container queries

9.3.1 @container class on the parent

Breakpoints respond to the **viewport**. A card placed in a narrow sidebar on a wide screen still sees a wide viewport, so it cannot adapt. **Container queries** let an element respond to the width of a parent instead. In v4 they are built in (no plugin): mark the parent with `@container`, which sets `container-type: inline-size`, and use `@`-prefixed variants on its descendants.

COMPILED CSS

```
.\@container {
  container-type: inline-size;
}
.\@md\:flex {
  @container (width >= 28rem) {
    display: flex;
  }
}
```

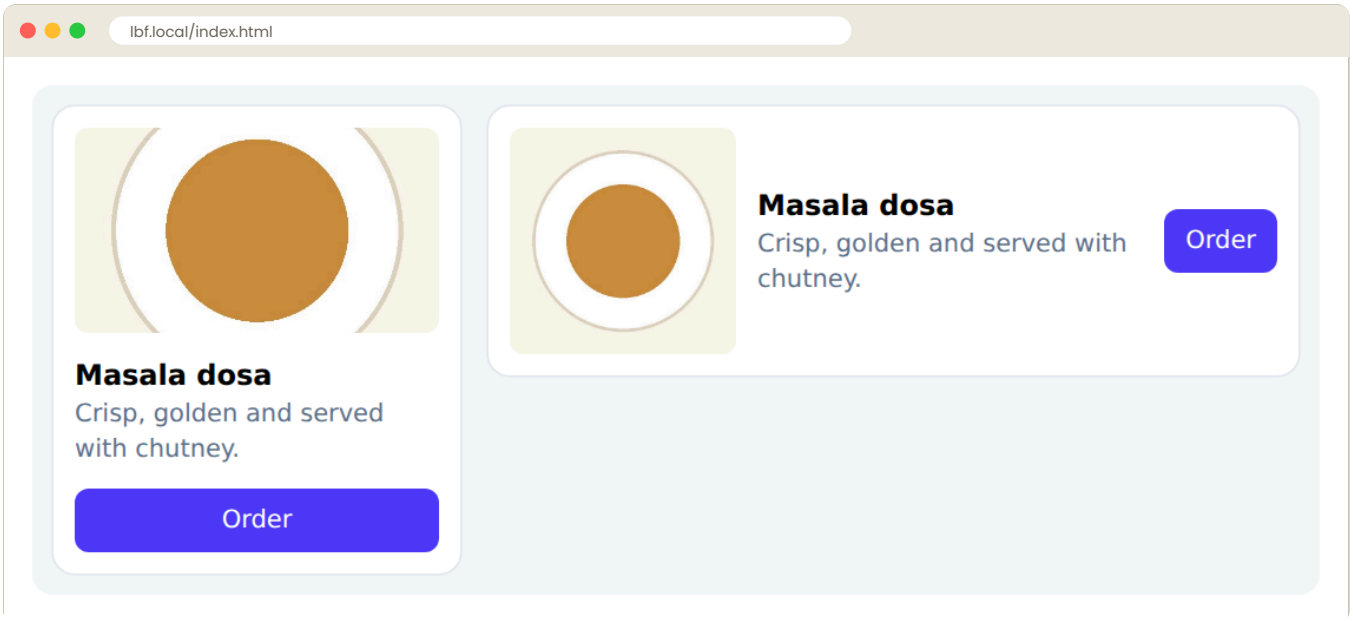
9.3.2 @sm:, @md:, @lg: variants on children

Variant	Container width	Variant	Container width
`@3xs:`	16rem (256px)	`@lg:`	32rem (512px)
`@2xs:`	18rem (288px)	`@xl:`	36rem (576px)
`@xs:`	20rem (320px)	`@2xl:`	42rem (672px)
`@sm:`	24rem (384px)	`@3xl:` … `@7xl:`	48rem … 80rem
`@md:`	28rem (448px)	`@min-[400px]:`, `@max-md:`	arbitrary and max ranges

The figure below places the **same card markup** in a narrow sidebar and a wide main column on **one page** at one viewport width. With breakpoints both cards would look identical; with container queries each adapts to its own space.

PRODUCT-CARD.HTML: ONE COMPONENT HTML

```html
<div class="@container">
  <article class="flex flex-col gap-3 rounded-xl bg-white p-3 ring-1 ring-slate-200 @md:flex-row
@md:items-center">
    <img src="food-dosa.jpg" alt="" class="aspect-video w-full rounded-lg object-cover @md:aspect-
square @md:w-32">
    <div class="@md:flex-1">
      <h3 class="font-semibold @lg:text-xl">Masala dosa</h3>
      <p class="text-sm text-slate-500">Crisp, golden and served with chutney.</p>
      <p class="mt-1 hidden text-xs text-indigo-600 @lg:block">@lg: extra detail when there is
room</p>
    </div>
    <button class="rounded-lg bg-indigo-600 px-3 py-2 text-sm text-white">Order</button>
  </article>
</div>
```



ONE VIEWPORT (760PX), TWO CONTAINER WIDTHS: THE SIDEBAR CARD STACKS, THE MAIN CARD GOES HORIZONTAL

9.3.3 Named containers

When containers are nested, an `@md:` variant responds to the **nearest** container. To respond to a specific outer one, name it with a slash, `@container/main`, and use the name in the variant: `@lg/main:grid-cols-3`. Names are also useful for documentation: a reader immediately sees which box the layout depends on.

COMPILED CSS

```css
.\@container\/main {
  container-type: inline-size;
  container-name: main;
}
.\@lg\/main\:grid-cols-3 {
  @container main (width >= 32rem) {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }
}
```

9.3.4 When to use container queries vs breakpoints

Use breakpoints (md:, lg:) for	Use container queries (@md:, @lg:) for
the overall page layout: number of page columns, header and navigation	reusable components: cards, media objects, widgets, form groups
showing and hiding page-level elements (sidebar, mobile menu)	components placed in sidebars, modals, grids or dashboards of varying widths
typography and spacing that relate to the screen	design-system components used by many teams in unknown layouts
print, orientation and device-feature styles	anything that should look right wherever it is dropped

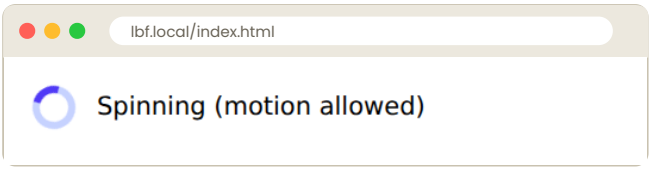
TIP

A good rule: **pages use breakpoints, components use container queries**. A component that only depends on its own width can be dropped into any layout and any framework without changes.

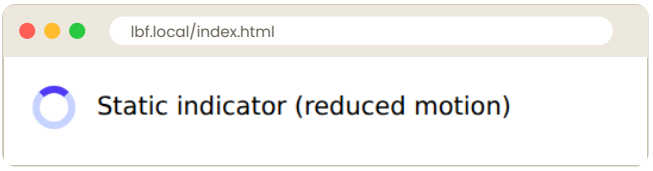
9.4 Other media variants

9.4.1 motion-reduce: and motion-safe:

Some users turn on "reduce motion" in their operating system because animation can trigger dizziness or nausea. `motion-reduce:` applies when that setting is on (`prefers-reduced-motion: reduce`), and `motion-safe:` applies only when it is off. Either remove motion in reduce mode (`animate-spin motion-reduce:animate-none`) or add it only in safe mode (`motion-safe:animate-bounce`). Module 11 returns to this.



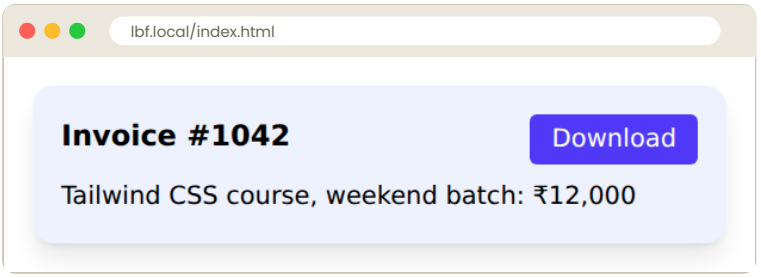
DEFAULT SETTINGS: MOTION-SAFE:ANIMATE-SPIN IS ACTIVE



EMULATING PREFERS-REDUCED-MOTION: REDUCE: NO ANIMATION, DIFFERENT LABEL

9.4.2 print:

`print:` applies only when the page is printed or saved as PDF: hide navigation and buttons (`print:hidden`), remove backgrounds and shadows to save ink, and show link URLs. Use `print:block` for content that should appear only on paper, such as a footer with the page address.



ON SCREEN



EMULATING PRINT MEDIA: BUTTON BAR HIDDEN, PRINT HEADING SHOWN, NO BACKGROUND OR SHADOW

9.4.3 portrait: and landscape:

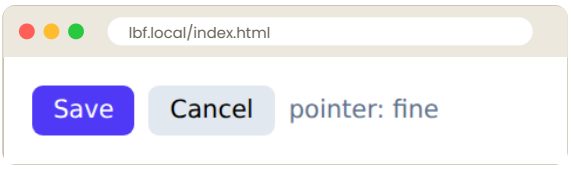
`portrait:` and `landscape:` match the viewport orientation (height greater than width, or the reverse). They are mostly used for full-screen experiences such as games, video players and kiosk screens, for example a "rotate your phone" hint: `hidden portrait:block`.

COMPILED CSS

```
.hidden {
  display: none;
}
.portrait\:block {
  @media (orientation: portrait) {
    display: block;
  }
}
.landscape\:grid-cols-2 {
  @media (orientation: landscape) {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}
```

9.4.4 pointer-coarse: for touch devices

v4 adds `pointer-fine:` (mouse or trackpad), `pointer-coarse:` (finger on a touchscreen) and `pointer-none:`, plus `any-pointer-*:` versions. They describe the **input**, not the screen size, which makes them the right tool for touch-friendly sizing: `pointer-coarse:py-3` enlarges tap targets to the recommended 44 to 48px only when a finger is used.



DESKTOP WITH A MOUSE (POINTER: FINE)



TOUCH DEVICE EMULATION (POINTER: COARSE): LARGER TAP TARGETS

COMPILED CSS

```
.pointer-coarse\:py-3 {
  @media (pointer: coarse) {
    padding-block: calc(var(--spacing) * 3);
  }
}
.pointer-fine\:py-1 {
  @media (pointer: fine) {
    padding-block: calc(var(--spacing) * 1);
  }
}
```

9.5 Testing responsiveness in DevTools

Check every responsive page at real sizes before calling it done. In Chrome, Edge or Firefox DevTools:

1. Open **Device Toolbar** (Ctrl+Shift+M / Cmd+Shift+M) and choose **Responsive**. Drag the width slowly from 320px to 1600px and watch for overflow, cramped text and awkward wrapping between breakpoints, not just at them.
2. Use the **media query bar** (enable it from the toolbar menu): it shows coloured bands for every `@media` rule in your CSS; clicking a band jumps to that width.
3. Try real device presets (an iPhone SE at 375px is a good smallest case) and rotate between portrait and landscape.
4. Open the **Rendering** panel to emulate `prefers-reduced-motion`, `prefers-color-scheme`, print media and forced colours.
5. In the **Elements** panel, a container shows a `container` badge; selecting a child shows which `@container` rules apply.
6. Finally, test on a real phone over your local network (Vite prints a Network URL with `--host`); touch, scrolling and the on-screen keyboard behave differently from emulation.

WATCH OUT

A horizontal scrollbar on mobile almost always means one element is wider than the viewport. In DevTools run `[...document.querySelectorAll('*')].filter(e => e.scrollWidth > innerWidth)` in the Console to find it; the usual culprits are fixed widths (`w-[600px]`), long words or URLs, and wide tables without `overflow-x-auto`.

PRACTICE CHECKPOINT

A **fully responsive landing page** with a mobile menu, and a **product card** that changes layout based on its container width.

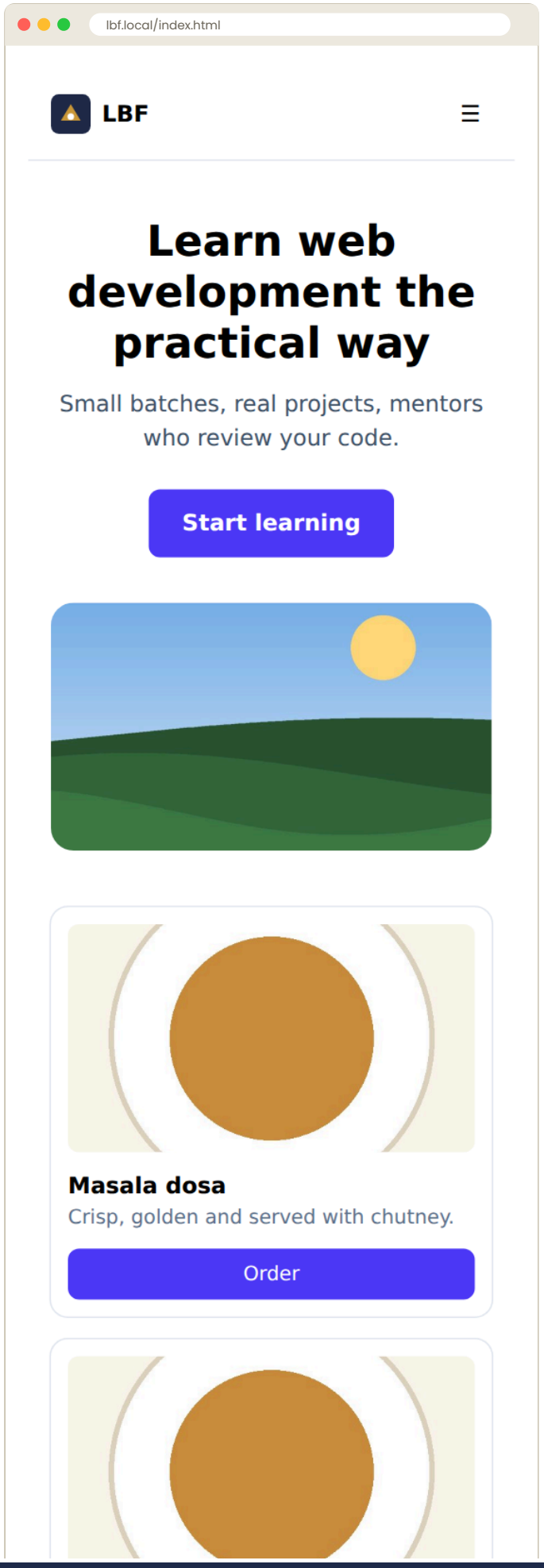
LANDING.HTML: SAMPLE SOLUTION

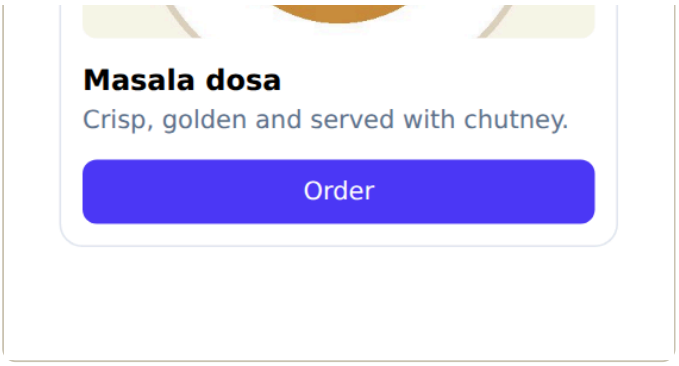
HTML

```
<div class="min-h-[400px] bg-white">
  <header data-open="false" class="group border-b border-slate-200">
    <div class="mx-auto flex max-w-5xl items-center justify-between px-4 py-3">
      <a href="#" class="flex items-center gap-2 font-bold"><img src="logo.svg" alt="" class="size-7">
LBF</a>
      <nav class="hidden gap-6 text-sm md:flex"><a href="#">Courses</a><a href="#">Placements</a><a
href="#">About</a></nav>
      <a href="#" class="hidden rounded-lg bg-indigo-600 px-4 py-2 text-sm font-semibold text-white
md:block">Enrol</a>
      <button id="menu-btn" aria-expanded="false" class="rounded-md p-2 md:hidden"><span class="sr-
only">Menu</span>&#9776;</button>
    </div>
    <nav class="hidden flex-col gap-1 px-4 pb-3 text-sm group-data-[open=true]:flex md:hidden">
      <a href="#" class="rounded-md px-3 py-2 hover:bg-slate-100">Courses</a><a href="#"
class="rounded-md px-3 py-2 hover:bg-slate-100">Placements</a>
      <a href="#" class="mt-1 rounded-lg bg-indigo-600 px-3 py-2 text-center font-semibold text-
white">Enrol</a>
    </nav>
  </header>

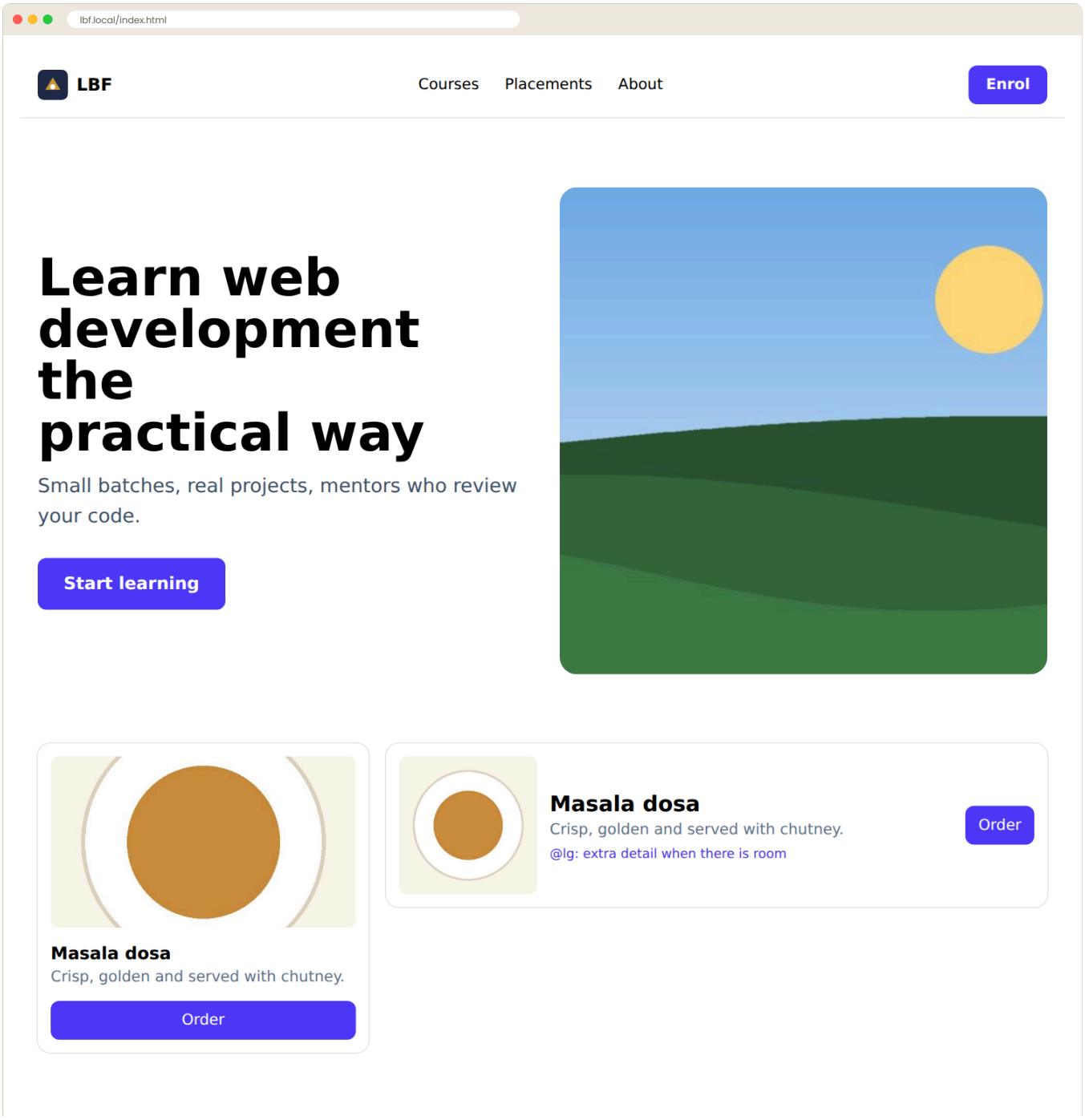
  <main class="mx-auto max-w-5xl px-4">
    <section class="grid items-center gap-8 py-10 md:grid-cols-2 md:py-16">
      <div class="text-center md:text-left">
        <h1 class="text-3xl font-black text-balance md:text-5xl">Learn web development the practical
way</h1>
        <p class="mt-3 text-slate-600 md:text-lg">Small batches, real projects, mentors who review
your code.</p>
        <a href="#" class="mt-6 inline-block rounded-lg bg-indigo-600 px-6 py-3 font-semibold text-
white">Start learning</a>
      </div>
      <img src="hills-800.jpg" alt="" class="aspect-video w-full rounded-2xl object-cover md:aspect-
square">
    </section>

    <section class="grid gap-4 pb-12 sm:grid-cols-[1fr_2fr]">
      <!-- the same card in a narrow and a wide column -->
      <div class="@container">...product card...</div>
      <div class="@container">...product card...</div>
    </section>
  </main>
</div>
<script>
  const btn = document.getElementById('menu-btn');
  btn.addEventListener('click', () => {
    const open = btn.getAttribute('aria-expanded') === 'true';
    btn.setAttribute('aria-expanded', String(!open));
    btn.closest('header').dataset.open = String(!open);
  });
</script>
```

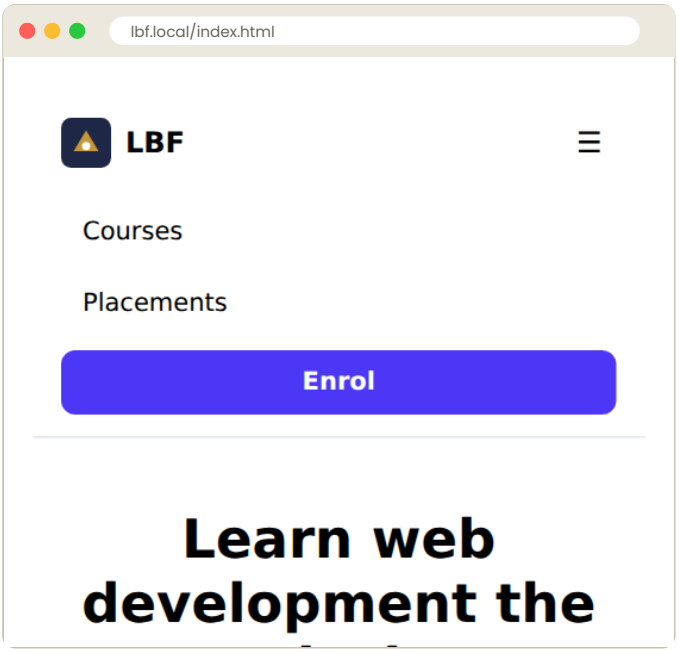

lbf.local/index.html
LBF
Learn web development the practical way
Small batches, real projects, mentors who review your code.
Start learning
Masala dosa
Crisp, golden and served with chutney.
Order



375PX



1000PX. THE LANDING PAGE AT PHONE AND LAPTOP WIDTHS: NAVIGATION, HERO ORDER, IMAGE SHAPE AND THE TWO PRODUCT CARDS ALL ADAPT



AT 375PX WITH THE MENU OPENED

Write mobile styles without a prefix and add `sm:`, `md:`, `lg:` overrides upward; use `max-*` only for true ranges; use breakpoints for page layout and `@container` with `@md:`-style variants for reusable components; and respect user settings with `motion-reduce:`, `print:` and `pointer-coarse:`.

MODULE 10 · INTERMEDIATE

Dark mode and theming basics

Minimum duration **1 day** | Level **Intermediate**

Tailwind makes dark mode a matter of adding dark: variants. This module covers the default dark: variant driven by the operating-system preference; switching to manual control with a class or data attribute through @custom-variant; remembering the user's choice in localStorage and avoiding a flash of the wrong theme on page load; designing good dark interfaces (colours, contrast, images and shadows); and semantic theme colours built on CSS variables, which reduce dark mode to a single set of variable overrides and make multiple themes possible. Dark screenshots were taken with Chromium emulating prefers-color-scheme: dark or with the class applied.

IN THIS MODULE

10.1 Dark mode
10.2 Designing for dark mode
10.3 Theme colours with CSS variables

10.1 Dark mode

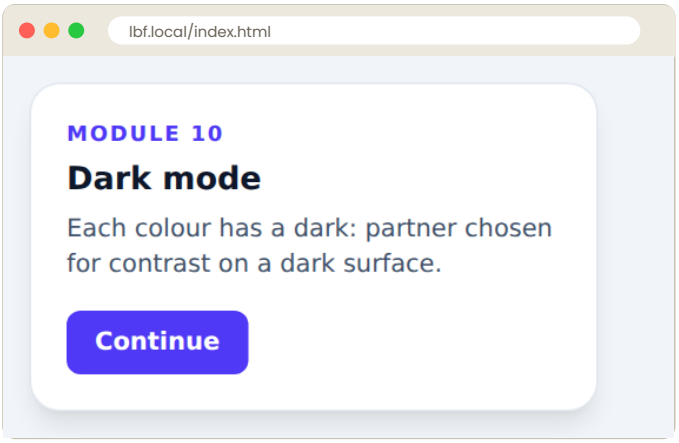
10.1.1 dark: variant using the system preference

Out of the box, `dark:` is a media-query variant: `dark:bg-slate-900` applies when the operating system or browser is set to a dark appearance (`prefers-color-scheme: dark`). Nothing else is needed; the page follows the user's system setting automatically, including scheduled switches at sunset. Write the light styles first, then add a `dark:` partner for each colour.

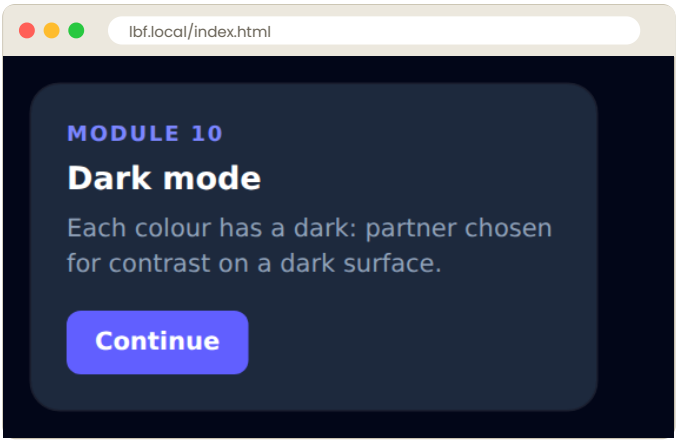
CARD.HTML

HTML

```
<article class="w-80 rounded-2xl bg-white p-5 shadow-lg ring-1 ring-slate-200 dark:bg-slate-800
dark:shadow-none dark:ring-white/10">
  <p class="text-xs font-semibold uppercase tracking-widest text-indigo-600 dark:text-indigo-
400">Module 10</p>
  <h3 class="mt-1 text-lg font-bold text-slate-900 dark:text-white">Dark mode</h3>
  <p class="mt-1 text-sm text-slate-600 dark:text-slate-400">Each colour has a dark: partner chosen
for contrast on a dark surface.</p>
  <button class="mt-4 rounded-lg bg-indigo-600 px-4 py-2 text-sm font-semibold text-white hover:bg-
indigo-500 dark:bg-indigo-500 dark:hover:bg-indigo-400">Continue</button>
</article>
```



SYSTEM SET TO LIGHT



CHROMIUM EMULATING PREFERS-COLOR-SCHEME: DARK: THE SAME HTML

COMPILED CSS

```
.dark\:bg-slate-800 {
  @media (prefers-color-scheme: dark) {
    background-color: var(--color-slate-800);
  }
}
.dark\:text-white {
  @media (prefers-color-scheme: dark) {
    color: var(--color-white);
  }
}
```

NOTE

Also set `color-scheme` so that native parts of the page (scrollbars, form controls, the default canvas) match: `<html class="scheme-light dark:scheme-dark">`, or `scheme-light-dark` to follow the system. Without it, a dark page can still show a bright white scrollbar and date picker.

10.1.2 Manual toggling with a class or data attribute using @custom-variant

Most sites also offer a toggle so the user can override the system. To do that, redefine the `dark` variant in your CSS with `@custom-variant`, so that it depends on a class (or data attribute) on the `<html>` element instead of the media query. Every `dark:` class in the project switches over; the HTML does not change.

APP.CSS: CLASS STRATEGY CSS

```css
@import "tailwindcss";

/* dark: now means "inside an element with class dark" */
@custom-variant dark (&:where(.dark, .dark *));
```

APP.CSS: DATA-ATTRIBUTE STRATEGY CSS

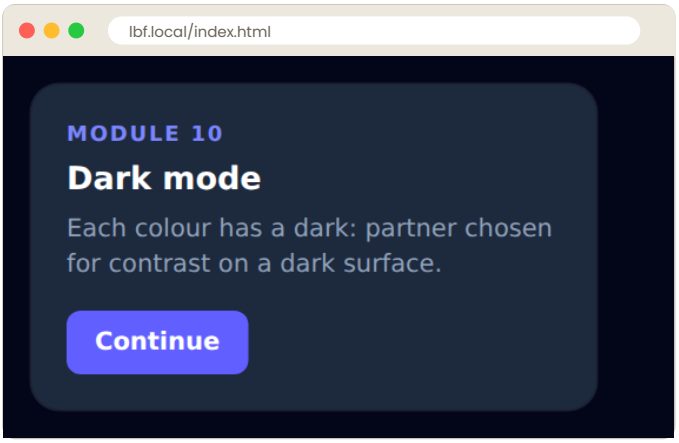
```css
@import "tailwindcss";

/* or: driven by data-theme="dark" on <html> */
@custom-variant dark (&:where([data-theme=dark], [data-theme=dark] *));
```

COMPILED CSS

```css
.dark\:bg-slate-800 {
  &:where(.dark, .dark *) {
    background-color: var(--color-slate-800);
  }
}
```

The generated selector uses `:where()`, which has zero specificity, so `dark:` utilities do not unexpectedly beat other classes. Now dark mode is on only when `class="dark"` is present, whatever the system says:



CLASS="DARK" ON THE WRAPPER SWITCHES THE CARD, WITH THE SYSTEM IN LIGHT MODE

10.1.3 Saving the user's choice with localStorage and JavaScript

A good toggle offers three choices: Light, Dark and System. Store the explicit choice in `localStorage`; when the user picks System, remove the stored value and follow `matchMedia('(prefers-color-scheme: dark)')`, listening for changes so the page switches if the system does.

THE THREE-WAY TOGGLE (ARIA-PRESSED SHOWS THE CURRENT CHOICE) HTML

```html
<div class="inline-flex rounded-full bg-slate-200 p-1 text-sm dark:bg-slate-800">
  <button data-set-theme="light" class="rounded-full px-3 py-1 text-slate-600 aria-pressed:bg-white
aria-pressed:text-slate-900 aria-pressed:shadow dark:text-slate-400 dark:aria-pressed:bg-slate-600
dark:aria-pressed:text-white">Light</button>
  <button data-set-theme="dark" class="rounded-full px-3 py-1 text-slate-600 aria-pressed:bg-white
aria-pressed:text-slate-900 aria-pressed:shadow dark:text-slate-400 dark:aria-pressed:bg-slate-600
dark:aria-pressed:text-white">Dark</button>
  <button data-set-theme="system" class="rounded-full px-3 py-1 text-slate-600 aria-pressed:bg-white
aria-pressed:text-slate-900 aria-pressed:shadow dark:text-slate-400 dark:aria-pressed:bg-slate-600
dark:aria-pressed:text-white">System</button>
</div>
```

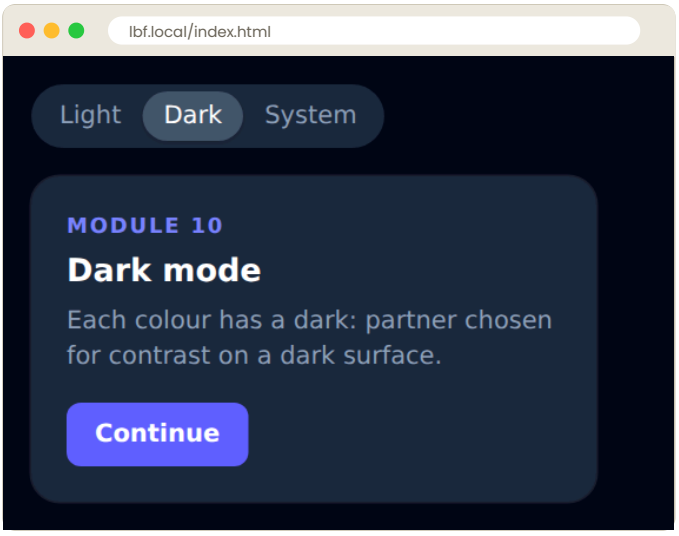
THEME.JS HTML

```
<script>
  const root = document.documentElement;
  const media = matchMedia('(prefers-color-scheme: dark)');

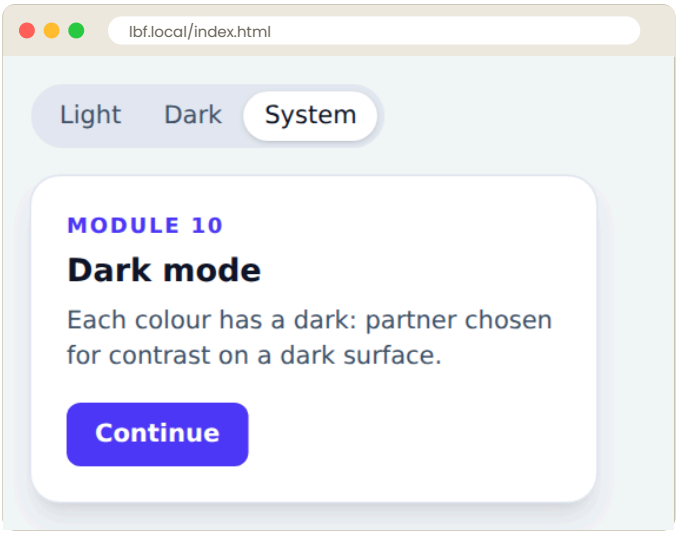
  function apply() {
    const choice = localStorage.getItem('theme');          // 'light' | 'dark' | null (= system)
    const dark = choice === 'dark' || (choice === null && media.matches);
    root.classList.toggle('dark', dark);
    document.querySelectorAll('[data-set-theme]').forEach(b =>
      b.setAttribute('aria-pressed', String(b.dataset.setTheme === (choice ?? 'system'))));
  }

  document.querySelectorAll('[data-set-theme]').forEach(btn =>
    btn.addEventListener('click', () => {
      const v = btn.dataset.setTheme;
      if (v === 'system') localStorage.removeItem('theme'); else localStorage.setItem('theme', v);
      apply();
    }));

  media.addEventListener('change', apply);   // follow the OS while on "system"
  apply();
</script>
```



AFTER CLICKING DARK: THE CLASS IS ADDED AND "DARK" IS SAVED IN LOCALSTORAGE



AFTER CLICKING SYSTEM WITH THE OS IN LIGHT MODE: THE STORED VALUE IS REMOVED AND THE PAGE FOLLOWS THE SYSTEM

10.1.4 Avoiding a flash of the wrong theme

If the script above runs at the end of `<body>` (or in a deferred bundle), the browser first paints the page in light mode and then switches, producing a white flash for dark-mode users on every page load. The fix is a tiny **blocking inline script in the `<head>`**, before the stylesheet is used, that sets the class immediately. Keep the full toggle logic in your normal script.

FLASH PREVENTION IN THE HEAD HTML

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <script>
    // runs before the first paint: no flash
    if (localStorage.theme === 'dark' ||
        (!('theme' in localStorage) && matchMedia('(prefers-color-scheme: dark)').matches)) {
      document.documentElement.classList.add('dark');
    }
  </script>
  <link rel="stylesheet" href="/app.css">
</head>
```

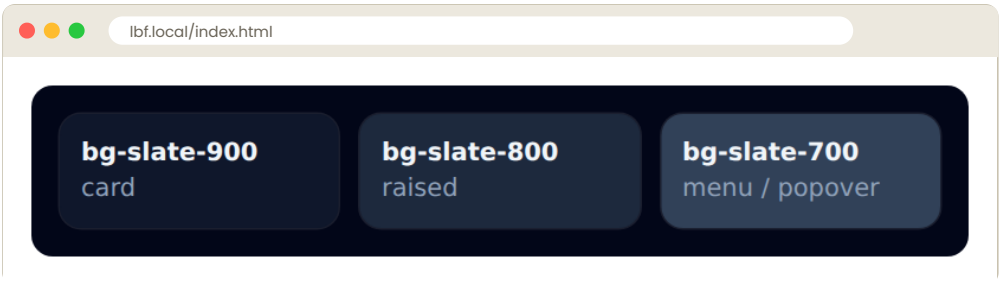
- Keep it inline and small; it must not wait for a network request.
- With server rendering (Next.js, Nuxt), the server does not know the stored choice, so add `suppressHydrationWarning` to `<html>` in React, or use a cookie so the server can render the right class. Libraries such as `next-themes` package exactly this logic (Module 15).
- Add `<meta name="color-scheme" content="light dark">` so the browser's default background is dark before CSS arrives.

10.2 Designing for dark mode

10.2.1 Choosing colours and contrast

Dark mode is not "invert the colours". Good dark interfaces follow a few rules:

Rule	In Tailwind terms
Avoid pure black backgrounds; use very dark greys or tinted navies, which are easier on the eyes and show elevation	`dark:bg-slate-900` / `dark:bg-slate-950` rather than `dark:bg-black`
Avoid pure white text on dark; slightly dimmed text reduces glare	`dark:text-slate-100` for headings, `dark:text-slate-400` for body
Show elevation with lighter surfaces, not shadows	page `slate-950`, card `slate-900`, raised card `slate-800`
Desaturate and lighten accent colours, because saturated colours vibrate on dark backgrounds	`indigo-600` becomes `dark:indigo-400` for text, `dark:indigo-500` for buttons
Replace borders with subtle translucent white	`dark:border-white/10`, `dark:ring-white/10`
Re-check contrast for every pair; 4.5:1 for normal text still applies	test both modes, not just light



ELEVATION IN DARK MODE: EACH LEVEL IS A LIGHTER SURFACE ON BG-SLATE-950

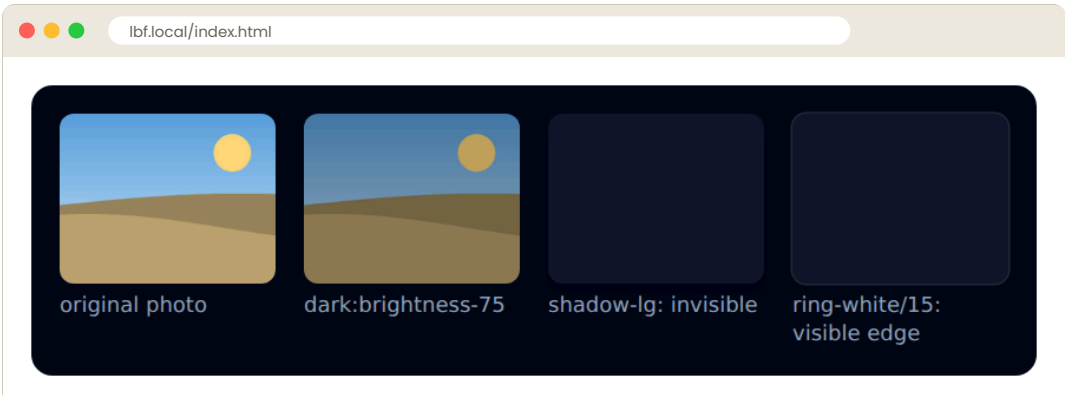
10.2.2 Images and shadows in dark mode

Shadows barely show on dark backgrounds, so replace them with a ring or a lighter surface: `shadow-lg dark:shadow-none dark:ring-1 dark:ring-white/10`. Large bright photos can dazzle in a dark page; dim them slightly with `dark:brightness-90` or `dark:opacity-90`. Logos and diagrams with dark lines on transparent backgrounds may disappear; provide a light version with `<picture>`, or invert simple monochrome icons with `dark:invert`.

SWAPPING IMAGES FOR DARK MODE HTML

```
<picture>
  <source srcset="logo-light-text.svg" media="(prefers-color-scheme: dark)">
  <img src="logo-dark-text.svg" alt="Learn Being Forward">
</picture>

<!-- with a class toggle, swap with utilities instead -->
<img src="logo-dark-text.svg" alt="Learn Being Forward" class="dark:hidden">
<img src="logo-light-text.svg" alt="" class="hidden dark:block">
```



DIMMING PHOTOS AND REPLACING SHADOWS WITH RINGS (DARK PREFERENCE EMULATED)

10.3 Theme colours with CSS variables

10.3.1 Semantic colour names (primary, surface, muted)

Writing `bg-white dark:bg-slate-900` on every element works, but it doubles the class lists and spreads the theme across hundreds of files. The scalable alternative is **semantic colours**: name colours by their **role** (surface, text, muted, primary, border) and define each one as a CSS variable whose value changes in dark mode. Register them in `@theme inline` so they become utilities such as `bg-surface` and `text-muted`.

NOTE

The keyword `inline` matters here. With a plain `@theme`, Tailwind defines `--color-surface: var(--surface)` on `:root`, where `var(--surface)` is resolved once, so overriding `--surface` on a nested element (a `.dark` section or a `data-theme` wrapper) would have no effect. With `@theme inline` the utilities use the value directly, `background-color: var(--surface)`, which is resolved on each element and therefore follows the nearest override.

APP.CSS CSS

```css
@import "tailwindcss";

@custom-variant dark (&:where(.dark, .dark *));

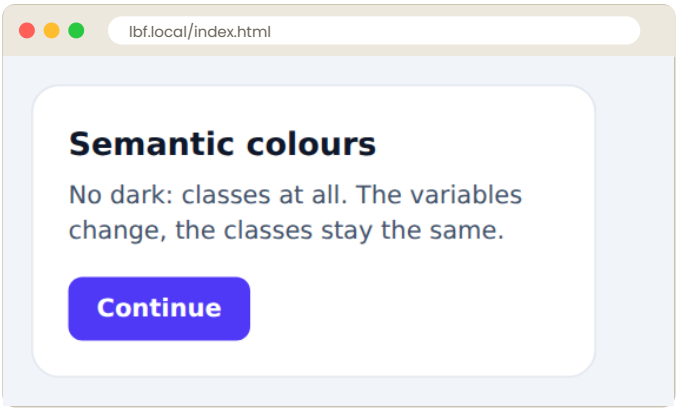
@theme inline {
  --color-page: var(--page);
  --color-surface: var(--surface);
  --color-ink: var(--ink);
  --color-muted: var(--muted);
  --color-line: var(--line);
  --color-primary: var(--primary);
  --color-on-primary: var(--on-primary);
}

:root {
  --page: var(--color-slate-100);
  --surface: var(--color-white);
  --ink: var(--color-slate-900);
  --muted: var(--color-slate-600);
  --line: var(--color-slate-200);
  --primary: var(--color-indigo-600);
  --on-primary: var(--color-white);
}

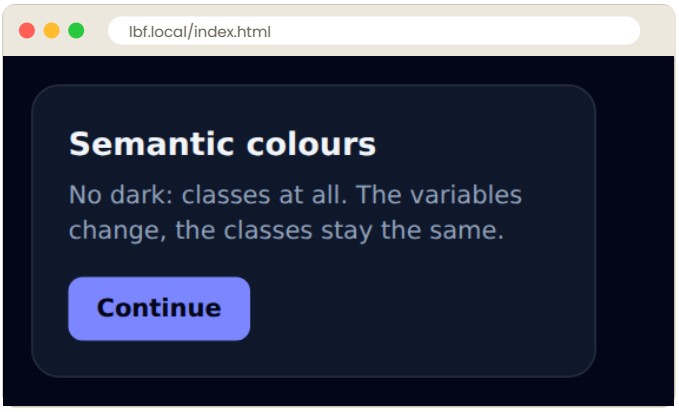
.dark {
  --page: var(--color-slate-950);
  --surface: var(--color-slate-900);
  --ink: var(--color-slate-100);
  --muted: var(--color-slate-400);
  --line: oklch(1 0 0 / 0.1);
  --primary: var(--color-indigo-400);
  --on-primary: var(--color-slate-950);
}
```

INDEX.HTML HTML

```html
<div class="bg-page p-4">
  <article class="w-80 rounded-2xl border border-line bg-surface p-5">
    <h3 class="text-lg font-bold text-ink">Semantic colours</h3>
    <p class="mt-1 text-sm text-muted">No dark: classes at all. The variables change, the classes stay the same.</p>
    <button class="mt-4 rounded-lg bg-primary px-4 py-2 text-sm font-semibold text-on-primary">Continue</button>
  </article>
</div>
```



LIGHT



THE SAME MARKUP INSIDE CLASS="DARK": ONLY THE VARIABLE VALUES CHANGED

COMPILED CSS

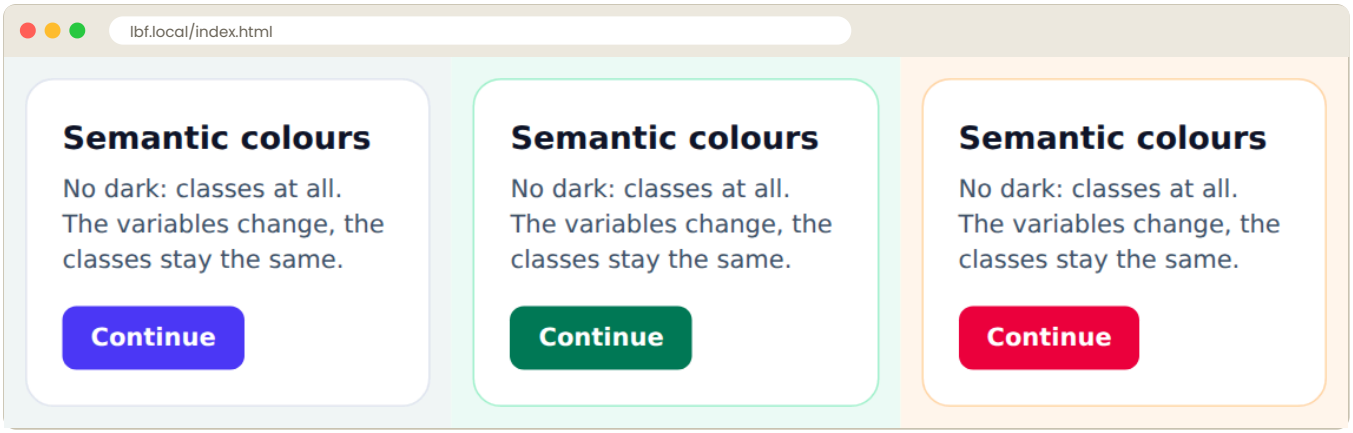
```
.bg-primary {
  background-color: var(--primary);
}
.bg-surface {
  background-color: var(--surface);
}
.text-muted {
  color: var(--muted);
}
```

TIP

The opacity modifier still works with semantic colours (`bg-primary/10`), because Tailwind 4 implements it with `color-mix()` , which accepts any colour value, including variables.

10.3.2 Switching themes by changing variables

Once components only use semantic names, a **theme** is just a set of variable values. Dark mode is one such set; brand themes (a different colour per course, per customer or per seasonal campaign) are others. Scope them with a data attribute and any part of the page can use a different theme, even several themes side by side.



ONE COMPONENT, THREE THEMES CHOSEN BY DATA-THEME ON A WRAPPER

EXTRA THEMES OVERRIDE ONLY THE VARIABLES THEY CHANGE

CSS

```css
[data-theme="forest"] {
  --page: var(--color-emerald-50);
  --line: var(--color-emerald-200);
  --primary: var(--color-emerald-700);
}
[data-theme="sunset"] {
  --page: var(--color-orange-50);
  --line: var(--color-orange-200);
  --primary: var(--color-rose-600);
}
```

PRACTICE CHECKPOINT

Add a **light/dark toggle** to the landing page that **remembers the user's choice** and uses **semantic theme colours**.

APP.CSS: SAMPLE SOLUTION

CSS

```css
@import "tailwindcss";

@custom-variant dark (&:where(.dark, .dark *));

@theme inline {
  --color-page: var(--page);
  --color-surface: var(--surface);
  --color-ink: var(--ink);
  --color-muted: var(--muted);
  --color-line: var(--line);
  --color-primary: var(--primary);
  --color-on-primary: var(--on-primary);
}

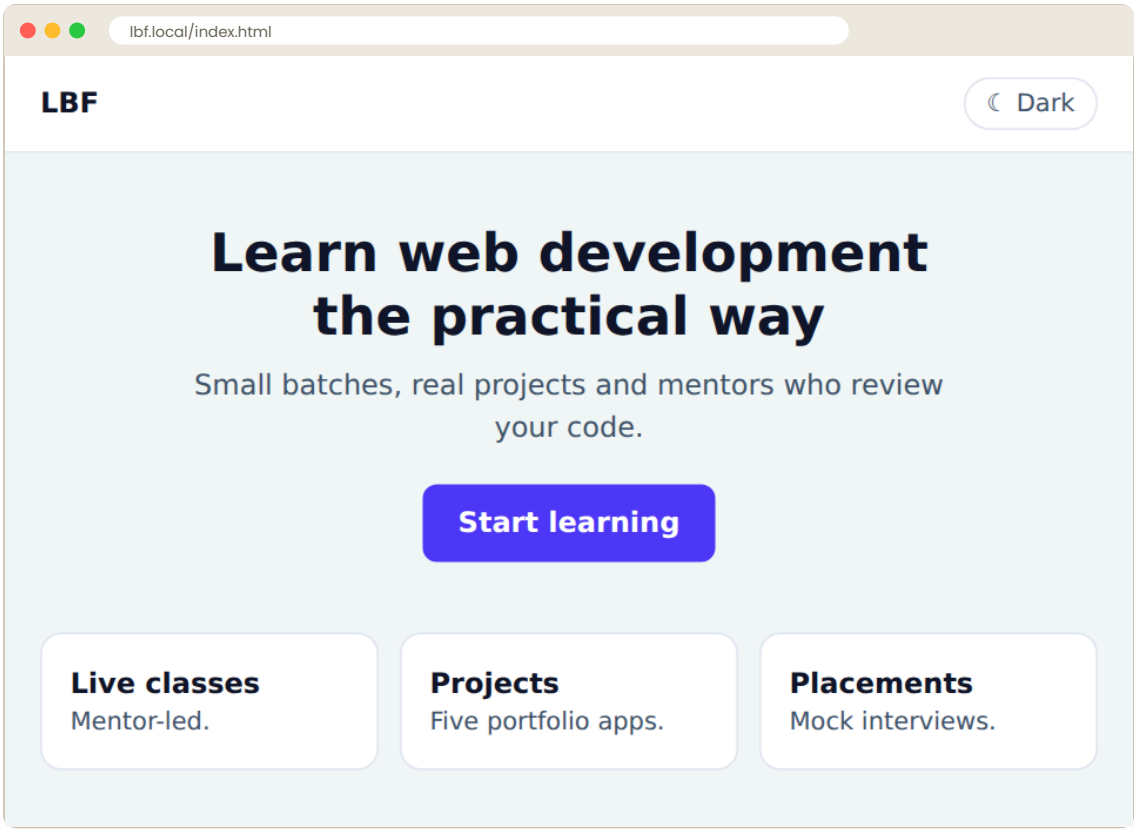
:root {
  --page: var(--color-slate-100);
  --surface: var(--color-white);
  --ink: var(--color-slate-900);
  --muted: var(--color-slate-600);
  --line: var(--color-slate-200);
  --primary: var(--color-indigo-600);
  --on-primary: var(--color-white);
}

.dark {
  --page: var(--color-slate-950);
  --surface: var(--color-slate-900);
  --ink: var(--color-slate-100);
  --muted: var(--color-slate-400);
  --line: oklch(1 0 0 / 0.1);
  --primary: var(--color-indigo-400);
  --on-primary: var(--color-slate-950);
}
```

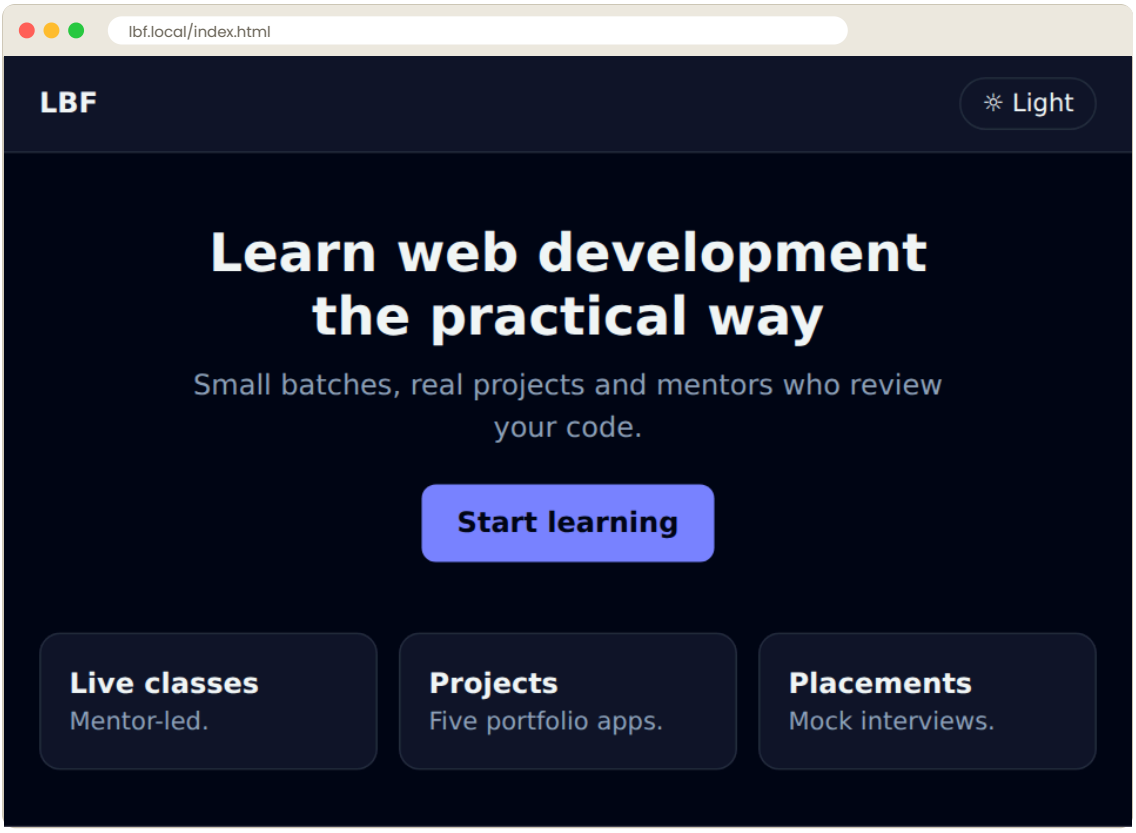
INDEX.HTML: SAMPLE SOLUTION (PLUS THE HEAD SCRIPT FROM 10.1.4)

HTML

```
<div class="bg-page text-ink">
  <header class="flex items-center justify-between border-b border-line bg-surface px-5 py-3">
    <a href="#" class="font-bold">LBF</a>
    <button id="theme" aria-label="Toggle dark mode" class="rounded-full border border-line px-3 py-1
text-sm text-muted hover:text-ink">
      <span class="dark:hidden">&#9790; Dark</span><span class="hidden dark:inline">&#9788;
Light</span>
    </button>
  </header>
  <section class="px-5 py-10 text-center">
    <h1 class="text-3xl font-black text-balance">Learn web development the practical way</h1>
    <p class="mx-auto mt-2 max-w-md text-muted">Small batches, real projects and mentors who review
your code.</p>
    <a href="#" class="mt-5 inline-block rounded-lg bg-primary px-5 py-2.5 font-semibold text-on-
primary">Start learning</a>
  </section>
  <section class="grid grid-cols-3 gap-3 px-5 pb-8">
    <div class="rounded-xl border border-line bg-surface p-4"><b>Live classes</b><p class="text-sm
text-muted">Mentor-led.</p></div>
    <div class="rounded-xl border border-line bg-surface p-4"><b>Projects</b><p class="text-sm text-
muted">Five portfolio apps.</p></div>
    <div class="rounded-xl border border-line bg-surface p-4"><b>Placements</b><p class="text-sm text-
muted">Mock interviews.</p></div>
  </section>
</div>
<script>
  document.getElementById('theme').addEventListener('click', () => {
    const dark = document.documentElement.classList.toggle('dark');
    localStorage.setItem('theme', dark ? 'dark' : 'light');
  });
</script>
```



LIGHT MODE



AFTER CLICKING THE TOGGLE: CLASS="DARK" ON THE HTML ELEMENT, SAVED TO LOCALSTORAGE

Use plain `dark:` to follow the system; redefine it with `@custom-variant dark (&:where(.dark, .dark *))` for a toggle; store the choice in localStorage and set the class in a blocking head script to avoid a flash; and prefer semantic colour variables so dark mode and extra themes are just variable overrides.

MODULE 11 · INTERMEDIATE

Transitions, transforms and animations

Minimum duration **1 day** | Level **Intermediate**

Tailwind provides utilities for smooth state changes and simple motion. This module covers transition utilities with duration, easing and delay; 2D transforms (scale, rotate, translate, skew) and their origin; 3D transforms with perspective, rotate-x/y, transform-3d and backface-hidden; the four built-in animations, custom keyframes and animations defined in the theme, and entry animations with the starting: variant; and reducing motion for users who ask for it. Because motion cannot be printed, transitions and animations are shown as frame strips: Chromium paused the real animation at the stated times.

IN THIS MODULE

11.1 Transitions
11.2 Transforms
11.3 Animations
11.4 Accessibility

11.1 Transitions

11.1.1 transition, transition-colors, transition-all

A transition animates the change between two states, such as the normal and hover colours of a button. Add a transition utility to the element (not to the `hover:` state), and every change to the listed properties animates instead of jumping.

Class	Properties transitioned
`transition`	colours, opacity, shadow, transform, translate/scale/rotate, filters (the sensible default)
`transition-colors`	color, background-color, border-color, outline-color, text-decoration-color, fill, stroke, gradient stops
`transition-opacity`	opacity
`transition-shadow`	box-shadow
`transition-transform`	transform, translate, scale, rotate
`transition-all`	every animatable property (convenient, but can animate things you did not intend, such as layout changes)
`transition-none`	no transition
`transition-discrete`	allows discrete properties such as `display` to transition (used with `starting:`, Section 11.3.3)

COMPILED CSS

```
.transition-colors {
  transition-property: color, background-color, border-color, outline-color, text-decoration-color,
fill, stroke, --tw-gradient-from, --tw-gradient-via, --tw-gradient-to;
  transition-timing-function: var(--tw-ease, var(--default-transition-timing-function));
  transition-duration: var(--tw-duration, var(--default-transition-duration));
}
```

Notice that duration and easing come from variables with defaults: 150ms and `cubic-bezier(0.4, 0, 0.2, 1)`. A bare `transition` is therefore already a short, natural-feeling transition.

A BUTTON THAT LIFTS AND CHANGES COLOUR

HTML

```
<button class="rounded-lg bg-indigo-600 px-5 py-2.5 font-semibold text-white shadow-md transition
duration-500 hover:-translate-y-1 hover:bg-amber-500 hover:shadow-xl hover:shadow-amber-500/40">Hover
me</button>
```



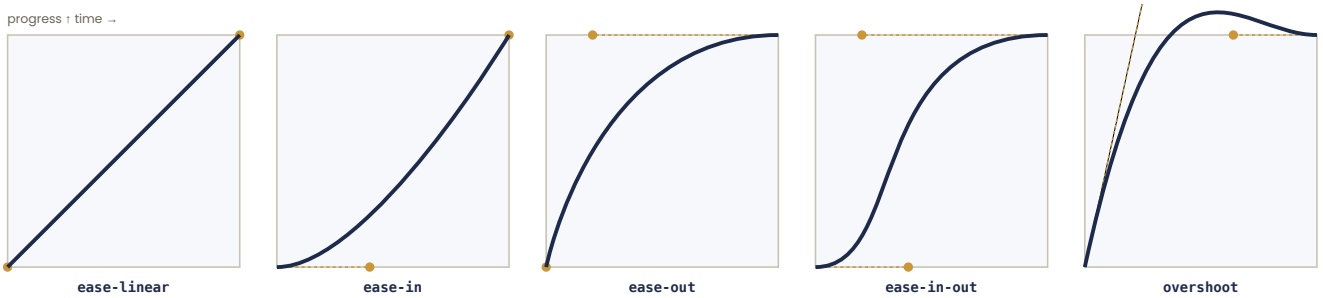
THE TRANSITION CAPTURED AT 0, 125, 250 AND 500 MS (DURATION-500): COLOUR, POSITION AND SHADOW MOVE TOGETHER

NOTE

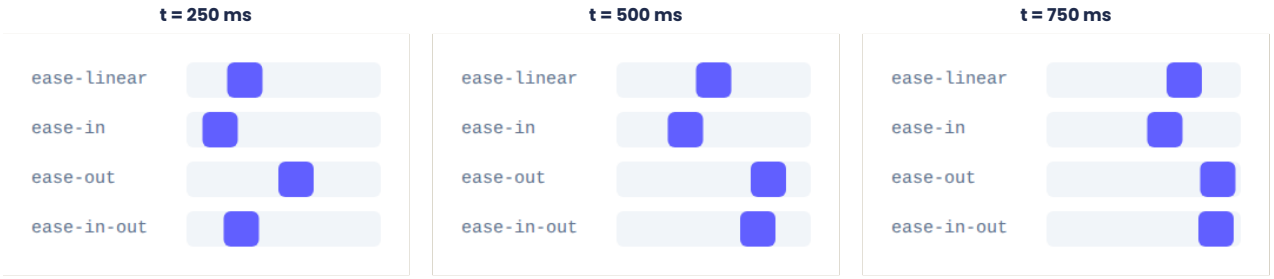
To capture the frames, the hover classes were swapped for an equivalent class-triggered variant and the browser's animations were paused at each time. In a real page, the transition plays when the pointer enters the button and reverses when it leaves.

11.1.2 duration-*, ease-*, delay-*

`duration-75` to `duration-1000` (any number of milliseconds in v4, e.g. `duration-250`) set how long the transition takes; `delay-*` waits before starting. Easing utilities set the speed curve: `ease-linear`, `ease-in` (starts slowly; good for elements leaving), `ease-out` (starts fast and settles; good for elements entering), `ease-in-out`, and arbitrary curves such as `ease-[cubic-bezier(0.34,1.56,0.64,1)]` for a slight overshoot. Most interface transitions feel right between 150 and 300ms.



TAILWIND'S EASING CURVES (VALUES FROM THE V4 THEME) AND AN ARBITRARY OVERSHOOT CURVE



FOUR SQUARES MOVING FOR 1000 MS WITH DIFFERENT EASING, CAPTURED AT 250, 500 AND 750 MS

COMPILED CSS

```
.delay-150 {
  transition-delay: 150ms;
}
.duration-300 {
  --tw-duration: 300ms;
  transition-duration: 300ms;
}
.ease-out {
  --tw-ease: var(--ease-out);
  transition-timing-function: var(--ease-out);
}
```

11.2 Transforms

11.2.1 scale-*, rotate-*, translate-*, skew-*

Transforms move, resize and rotate an element **visually** without affecting layout, and they are cheap to animate because the browser can run them on the GPU. In v4 each is its own CSS property (`scale`, `rotate`, `translate`) where possible, so they combine freely without overwriting each other.

Utilities	Examples
Scale	`scale-95`, `scale-110`, `scale-x-50`, `-scale-x-100` (mirror), `scale-[1.03]`
Rotate	`rotate-45`, `-rotate-6`, `rotate-[17deg]`
Translate	`translate-x-4`, `-translate-y-1`, `translate-x-1/2`, `-translate-1/2` (both axes), `translate-y-full`
Skew	`skew-x-12`, `-skew-y-3`



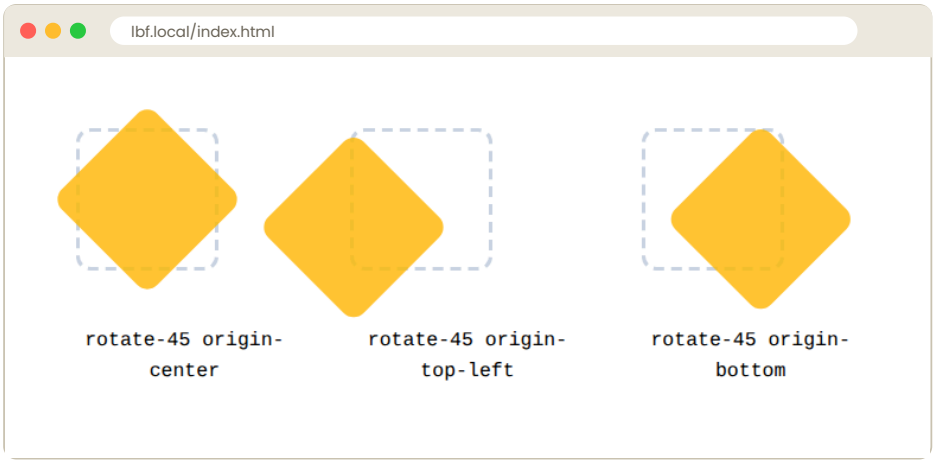
EACH TRANSFORM COMPARED WITH THE ORIGINAL POSITION (DASHED)

COMPILED CSS

```
.translate-x-4 {
  --tw-translate-x: calc(var(--spacing) * 4);
  translate: var(--tw-translate-x) var(--tw-translate-y);
}
.scale-75 {
  --tw-scale-x: 75%;
  --tw-scale-y: 75%;
  --tw-scale-z: 75%;
  scale: var(--tw-scale-x) var(--tw-scale-y);
}
.rotate-12 {
  rotate: 12deg;
}
.skew-x-12 {
  --tw-skew-x: skewX(12deg);
  transform: var(--tw-rotate-x,) var(--tw-rotate-y,) var(--tw-rotate-z,) var(--tw-skew-x,) var(--tw-
skew-y,);
}
```

11.2.2 origin-*

The **transform origin** is the fixed point a transform happens around. The default is the centre; `origin-top-left`, `origin-bottom`, `origin-[25%_75%]` and so on move it. A dropdown that scales in from `origin-top-right` appears to grow out of its button.



THE SAME ROTATION AROUND THREE DIFFERENT ORIGINS

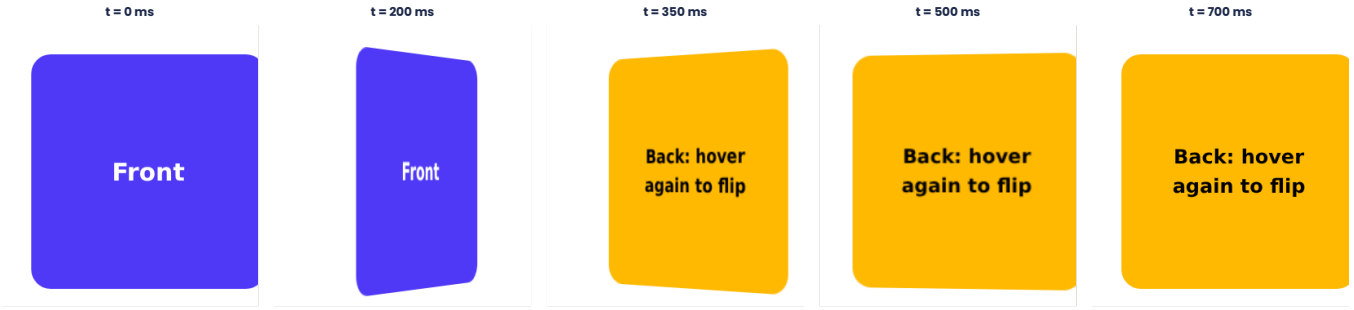
11.2.3 3D transforms: perspective-*, rotate-x-*, transform-3d, backface-hidden

v4 adds 3D transform utilities: `rotate-x-*`, `rotate-y-*`, `rotate-z-*`, `translate-z-*` and `scale-z-*`. For depth to be visible, the **parent** needs a perspective (`perspective-normal` = 500px, `perspective-dramatic` = 100px, `perspective-distant` = 1200px, or arbitrary values). `transform-3d` (`transform-style: preserve-3d`) keeps children in the same 3D space, and `backface-hidden` hides an element when it is turned away from the viewer. These are the ingredients of a flip card:

FLIP-CARD.HTML

HTML

```
<div class="group size-48 perspective-distant">
  <div class="relative size-full transition duration-700 transform-3d group-hover:rotate-y-180">
    <div class="absolute inset-0 grid place-items-center rounded-2xl bg-indigo-600 text-xl font-bold
text-white backface-hidden">Front</div>
    <div class="absolute inset-0 grid place-items-center rounded-2xl bg-amber-400 p-4 text-center
font-semibold rotate-y-180 backface-hidden">Back: hover again to flip</div>
  </div>
</div>
```



THE CARD FLIPPING, CAPTURED AT 0, 200, 350, 500 AND 700 MS: THE FRONT DISAPPEARS AT 90 DEGREES AND THE BACK APPEARS

COMPILED CSS

```
.rotate-y-180 {
  --tw-rotate-y: rotateY(180deg);
  transform: var(--tw-rotate-x,) var(--tw-rotate-y,) var(--tw-rotate-z,) var(--tw-skew-x,) var(--tw-
skew-y,);
}
.backface-hidden {
  backface-visibility: hidden;
}
.perspective-distant {
  perspective: var(--perspective-distant);
}
.transform-3d {
  transform-style: preserve-3d;
}
```

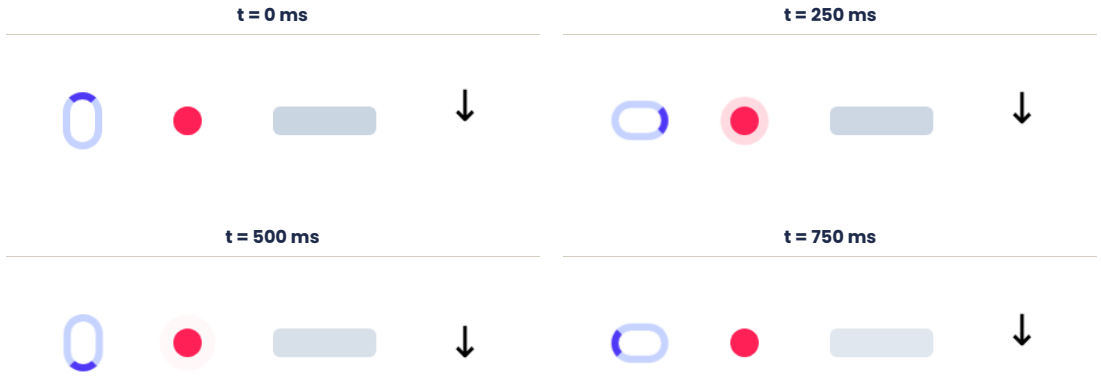
11.3 Animations

11.3.1 Built-in: animate-spin, animate-ping, animate-pulse, animate-bounce

Class	Animation	Typical use
`animate-spin`	`spin 1s linear infinite`	loading spinners
`animate-ping`	`ping 1s cubic-bezier(0, 0, 0.2, 1) infinite`	notification dots, "live" indicators (scales up and fades)
`animate-pulse`	`pulse 2s cubic-bezier(0.4, 0, 0.6, 1) infinite`	skeleton loaders (fades in and out)
`animate-bounce`	`bounce 1s infinite`	"scroll down" arrows
`animate-none`	none	turning an animation off (e.g. with `motion-reduce:`)

SPIN, PING, PULSE AND BOUNCE HTML

```
<div class="flex items-center gap-10 p-4">
  <span class="size-8 animate-spin rounded-full border-4 border-indigo-200 border-t-indigo-600">
</span>
  <span class="relative flex size-4"><span class="absolute inline-flex size-full animate-ping rounded-
full bg-rose-400 opacity-75"></span><span class="relative inline-flex size-4 rounded-full bg-rose-
500"></span></span>
  <span class="block h-4 w-24 animate-pulse rounded bg-slate-300"></span>
  <span class="animate-bounce text-2xl">&darr;</span>
</div>
```



ALL FOUR ANIMATIONS PAUSED AT THE SAME MOMENTS

11.3.2 Defining custom keyframes and animations in the theme

Custom animations go in `@theme`: a `--animate-*` variable creates the utility, and the matching `@keyframes` can be written **inside** `@theme` so that it is only included in the output when the animation is actually used.

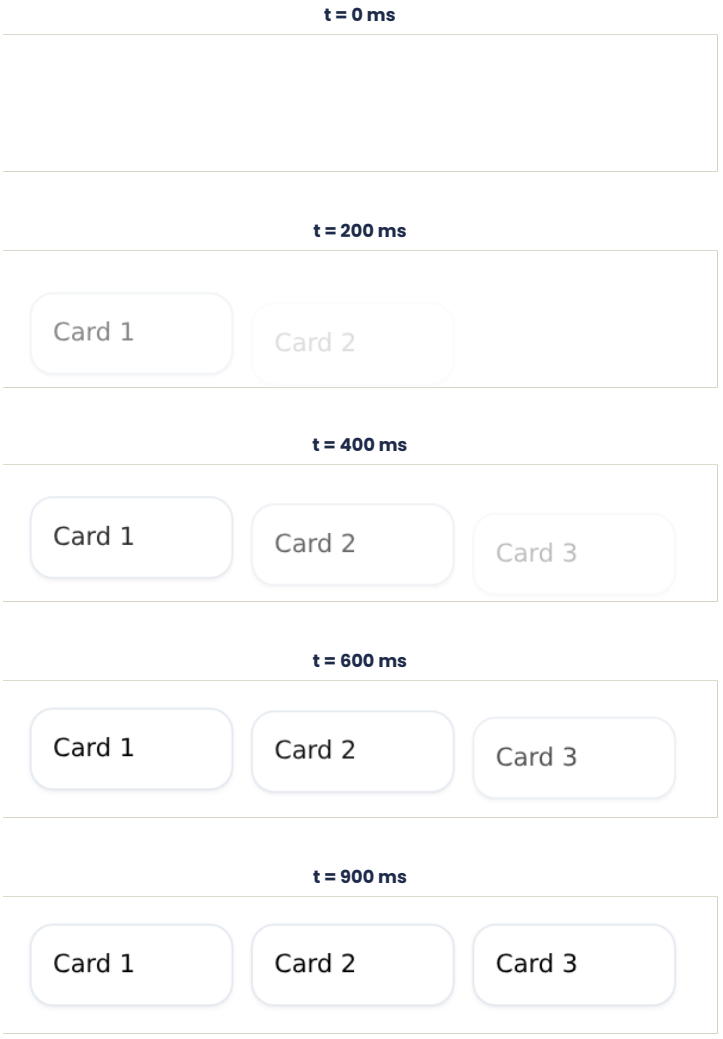
THREE CARDS FADING IN WITH STAGGERED DELAYS HTML

```
<div class="flex gap-3">
  <div class="w-28 animate-fade-in-up rounded-xl bg-white p-3 text-sm shadow ring-1 ring-slate-200
[animation-delay:0ms]">Card 1</div>
  <div class="w-28 animate-fade-in-up rounded-xl bg-white p-3 text-sm shadow ring-1 ring-slate-200
[animation-delay:150ms]">Card 2</div>
  <div class="w-28 animate-fade-in-up rounded-xl bg-white p-3 text-sm shadow ring-1 ring-slate-200
[animation-delay:300ms]">Card 3</div>
</div>
```

APP.CSS (AFTER @IMPORT "TAILWINDCSS") CSS

```
@theme {
  --animate-fade-in-up: fade-in-up 0.6s ease-out both;
  --animate-wiggle: wiggle 0.4s ease-in-out 3;

  @keyframes fade-in-up {
    from { opacity: 0; translate: 0 1rem; }
    to   { opacity: 1; translate: 0 0; }
  }
  @keyframes wiggle {
    0%, 100% { rotate: -4deg; }
    50%      { rotate: 4deg; }
  }
}
```



CAPTURED AT 0, 200, 400, 600 AND 900 MS: EACH CARD STARTS 150 MS AFTER THE PREVIOUS ONE

COMPILED CSS

```
.animate-fade-in-up {
  animation: var(--animate-fade-in-up);
}
```

TIP

Staggering is done here with an arbitrary property (`[animation-delay:150ms]`); Tailwind's `delay-*` utilities set `transition-delay` , not the animation delay. You can also register a custom utility for it (Module 14).

11.3.3 Entry animations with starting:

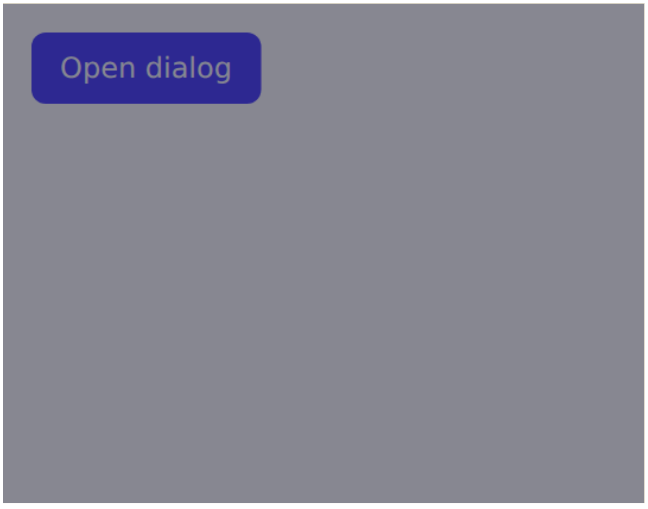
Elements that appear (a dialog opening, a popover, an element switched from `hidden` to `block`) used to need JavaScript to animate in, because a transition needs a "before" state. The `starting:` variant uses the CSS `@starting-style` rule to provide that state: styles in `starting:` apply only for the first frame after the element is rendered, and a transition takes it to its normal state. For `display` changes add `transition-discrete`.

DIALOG.HTML HTML

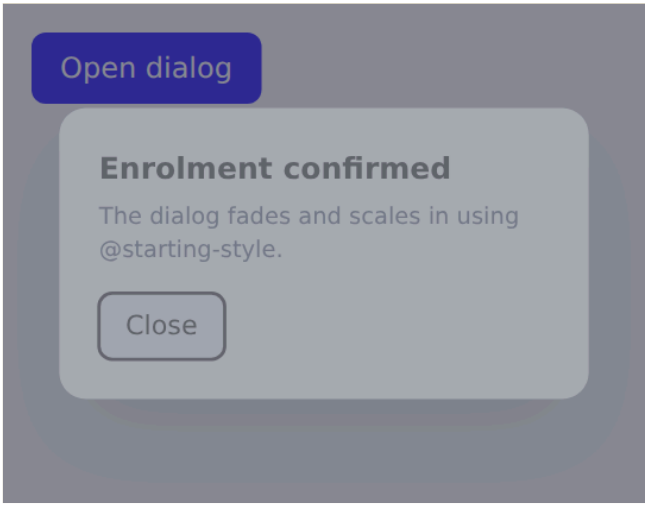
```
<button onclick="document.querySelector('dialog').showModal()" class="rounded-lg bg-indigo-600 px-4
py-2 text-white">Open dialog</button>

<dialog class="m-auto w-80 rounded-2xl p-6 shadow-2xl backdrop:bg-slate-900/50
               opacity-100 scale-100 transition-all transition-discrete duration-300
               starting:open:opacity-0 starting:open:scale-90">
  <h2 class="text-lg font-bold">Enrolment confirmed</h2>
  <p class="mt-1 text-sm text-slate-600">The dialog fades and scales in using @starting-style.</p>
  <form method="dialog"><button class="mt-4 rounded-lg bg-slate-100 px-4 py-2">Close</button></form>
</dialog>
```

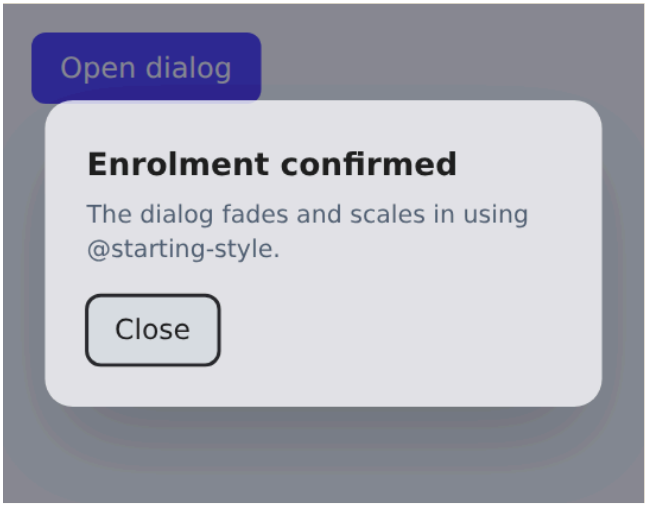
t = 0 ms



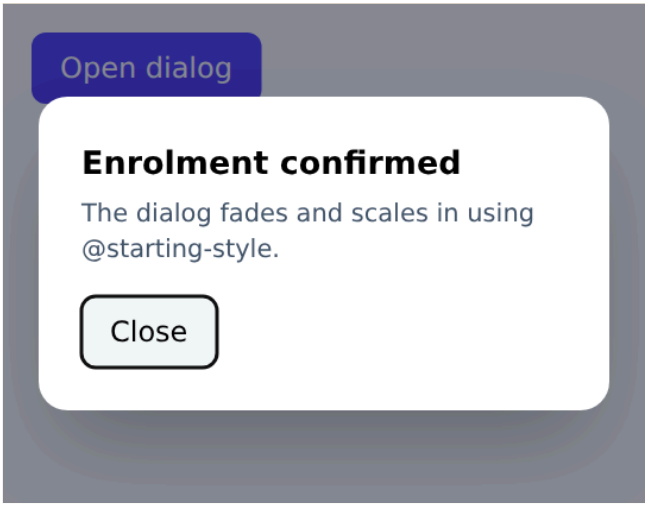
t = 80 ms



t = 150 ms



t = 300 ms



CAPTURED 0, 80, 150 AND 300 MS AFTER SHOWMODAL(): STARTING:OPEN: PROVIDES THE INITIAL OPACITY AND SCALE

COMPILED CSS

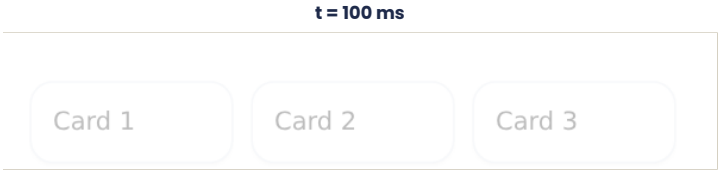
```
.transition-discrete {
  transition-behavior: allow-discrete;
}
.starting\:open\:opacity-0 {
  @starting-style {
    &:is([open], :popover-open, :open) {
      opacity: 0%;
    }
  }
}
```

11.4 Accessibility

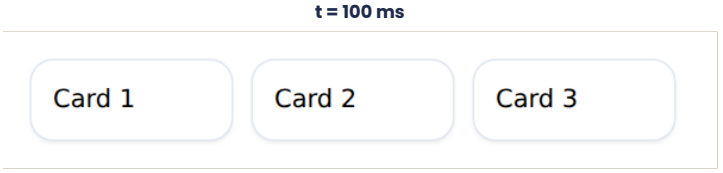
11.4.1 Reducing motion with motion-reduce:

Large movement, parallax, zooming and continuous animation can cause dizziness, nausea or migraines for people with vestibular disorders, who often enable "reduce motion" in their system settings. Respect it in one of two ways:

- **Opt out**: keep the motion and remove it for those users: `transition hover:-translate-y-1 motion-reduce:transition-none motion-reduce:hover:translate-y-0`.
- **Opt in**: add motion only when the user has not asked for less: `motion-safe:animate-fade-in-up`. Opt-in is safer for decorative animations.
- Reduced motion does not mean **no** feedback: replace movement with a quick opacity or colour change, and keep essential indicators such as a loading spinner (or swap it for a pulse).



DEFAULT: 100 MS IN, THE CARDS ARE STILL FADING IN



WITH PREFERS-REDUCED-MOTION: REDUCE, MOTION-SAFE: DOES NOT APPLY: THE CARDS ARE SIMPLY THERE

PRACTICE CHECKPOINT

Buttons with smooth hover transitions, a **loading skeleton** with `animate-pulse`, a **3D flip card** and a **custom fade-in animation**.

APP.CSS CSS

```css
@import "tailwindcss";

@theme {
  --animate-fade-in-up: fade-in-up 0.6s ease-out both;
  --animate-wiggle: wiggle 0.4s ease-in-out 3;

  @keyframes fade-in-up {
    from { opacity: 0; translate: 0 1rem; }
    to   { opacity: 1; translate: 0 0; }
  }
  @keyframes wiggle {
    0%, 100% { rotate: -4deg; }
    50%      { rotate: 4deg; }
  }
}
```

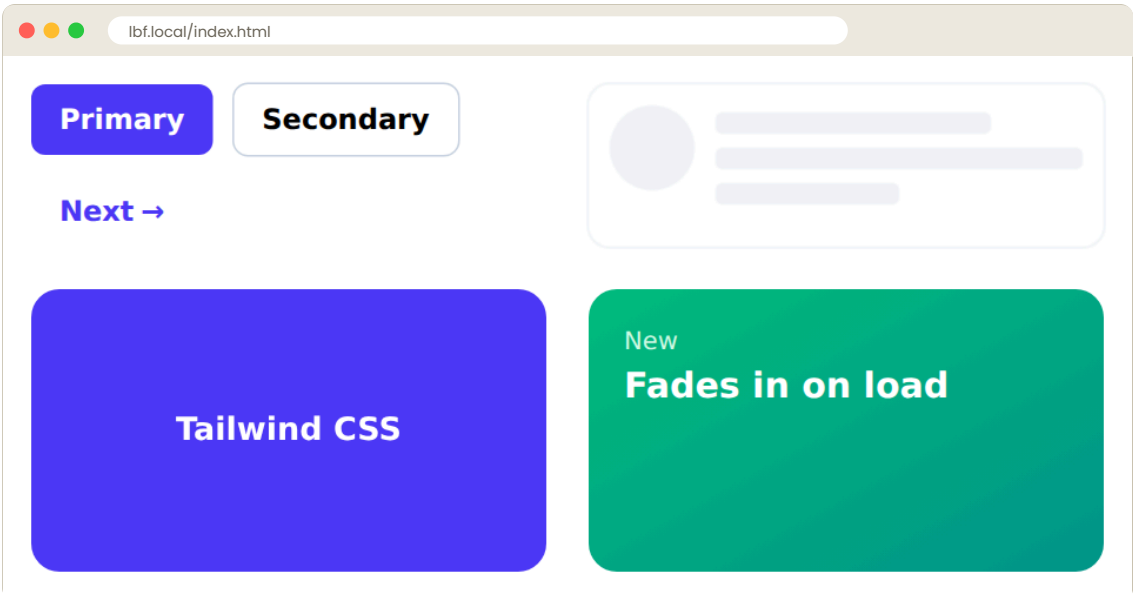
MOTION.HTML: SAMPLE SOLUTION HTML

```html
<div class="grid grid-cols-2 gap-6">
  <!-- 1. buttons -->
  <div class="flex flex-wrap items-start gap-3">
    <button class="rounded-lg bg-indigo-600 px-4 py-2 font-semibold text-white transition hover:bg-indigo-500 active:scale-95 motion-reduce:transition-none">Primary</button>
    <button class="rounded-lg px-4 py-2 font-semibold ring-1 ring-slate-300 transition-colors hover:bg-slate-100">Secondary</button>
    <button class="group flex items-center gap-1 rounded-lg px-4 py-2 font-semibold text-indigo-600">
      Next <span class="transition-transform group-hover:translate-x-1 motion-reduce:transition-none">&rarr;</span>
    </button>
  </div>

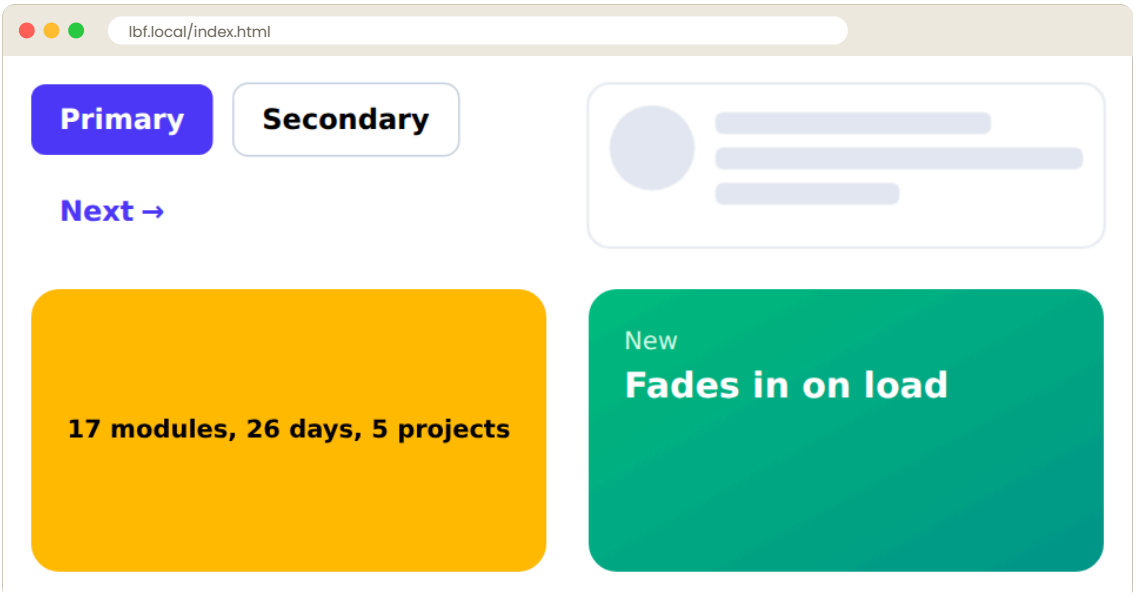
  <!-- 2. skeleton -->
  <div class="flex animate-pulse gap-3 rounded-xl p-3 ring-1 ring-slate-200 motion-reduce:animate-none" aria-busy="true" aria-label="Loading">
    <div class="size-12 rounded-full bg-slate-200"></div>
    <div class="flex-1 space-y-2 py-1"><div class="h-3 w-3/4 rounded bg-slate-200"></div><div class="h-3 rounded bg-slate-200"></div><div class="h-3 w-1/2 rounded bg-slate-200"></div></div>
  </div>

  <!-- 3. flip card -->
  <div class="group h-40 perspective-distant">
    <div class="relative size-full transition duration-700 transform-3d group-hover:rotate-y-180 motion-reduce:transition-none">
      <div class="absolute inset-0 grid place-items-center rounded-2xl bg-indigo-600 text-lg font-bold text-white backface-hidden">Tailwind CSS</div>
      <div class="absolute inset-0 grid place-items-center rounded-2xl bg-amber-400 p-4 text-center text-sm font-semibold rotate-y-180 backface-hidden">17 modules, 26 days, 5 projects</div>
    </div>
  </div>

  <!-- 4. custom fade-in -->
  <div class="motion-safe:animate-fade-in-up rounded-2xl bg-linear-to-br from-emerald-500 to-teal-600 p-5 text-white">
    <p class="text-sm text-emerald-100">New</p><p class="text-xl font-bold">Fades in on load</p>
  </div>
</div>
```



THE FINISHED PAGE (STATIC STATE); APP.CSS CONTAINS THE @THEME BLOCK FROM 11.3.2



HOVERING THE FLIP CARD, CAPTURED AFTER THE 700 MS TRANSITION

Put `transition` (plus `duration-*` and `ease-*`) on the element and the changed values on its `hover:` / `focus:` state; use transforms for cheap motion and the 3D utilities for flips; define custom animations with `--animate-*` and `@keyframes` inside `@theme`; use `starting:` for entry effects; and guard motion with `motion-safe:` or `motion-reduce:`.

MODULE 12 · INTERMEDIATE

Building components and reusing styles

Minimum duration **2 days** | Level **Intermediate**

Long class lists are normal in Tailwind; this module shows the right ways to reuse them without duplicating code. It covers thinking in components (loops, template partials and framework components, plus multi-cursor editing); extracting with CSS through @apply and @layer components, and when that is appropriate; a catalogue of common UI components (buttons, cards, navbars and sidebars, forms, modals and dropdowns, tables, alerts, badges and toasts, tabs and accordions, pricing tables and testimonials); the official Forms plugin; and the accessibility details every component needs. All components were compiled and rendered for real.

IN THIS MODULE

12.1 Thinking in components
12.2 Extracting with CSS
12.3 Common UI components
12.4 The Forms plugin
12.5 Accessibility for components

12.1 Thinking in components

12.1.1 Reuse through loops and framework components (the preferred way)

The first reaction to a long class list repeated on ten buttons is to want a `.btn` class. Tailwind's recommendation is different: **do not repeat the markup in the first place**. If the same element appears many times, it should be generated once by a loop, a template partial or a component, and the classes then live in exactly one place.

A LOOP IN PLAIN JAVASCRIPT

javascript

```
const courses = [
  { name: 'HTML5', days: 10 },
  { name: 'CSS3', days: 14 },
  { name: 'Tailwind CSS', days: 26 },
];

// the card classes are written once, however many courses there are
list.innerHTML = courses.map(c => `
  <li class="rounded-xl bg-white p-4 ring-1 ring-slate-200">
    <h3 class="font-semibold">${c.name}</h3>
    <p class="text-sm text-slate-500">${c.days} days</p>
  </li>`).join('');
```

A REACT COMPONENT (REPRESENTATIVE; SEE MODULE 15) javascript

```
// Button.jsx: the classes live in one component
export function Button({ variant = 'primary', children, ...props }) {
  const styles = {
    primary:   'bg-indigo-600 text-white hover:bg-indigo-500',
    secondary: 'bg-white text-slate-900 ring-1 ring-slate-300 hover:bg-slate-50',
  };
  return (
    <button className={`rounded-lg px-4 py-2 text-sm font-semibold ${styles[variant]}`} {...props}>
      {children}
    </button>
  );
}

// usage everywhere else is short
<Button>Save</Button>
<Button variant="secondary">Cancel</Button>
```

The same idea applies in every tool: a Blade, Jinja, Handlebars or Nunjucks partial, an Astro, Vue, Svelte or Angular component, or a server-side helper. The component is the abstraction; the classes are its implementation detail. This keeps the advantages of utilities (styles visible in the markup, no naming, safe deletion) while removing the repetition.

> **NOTE**
> Notice that the variant classes are written out **in full** as strings (`'bg-indigo-600 ...'`). Never build class names from pieces such as `` `bg-${color}-600` `` : Tailwind scans the source as text and cannot see a class that only exists after the code runs (Module 2).

12.1.2 Multi-cursor editing for repeated markup

When repetition is local, for example five navigation links in one file, extracting a component can be overkill. Every modern editor supports **multi-cursor editing** to change them all at once:

Action	VS Code (Windows / Linux)	VS Code (macOS)
Select the next occurrence of the selection	Ctrl+D	Cmd+D
Select all occurrences	Ctrl+Shift+L	Cmd+Shift+L
Add a cursor above / below	Ctrl+Alt+Up / Down	Cmd+Option+Up / Down
Add a cursor where you click	Alt+Click	Option+Click

> **TIP**
> Install the official **Tailwind CSS IntelliSense** extension: it autocompletes class names, shows the generated CSS on hover, warns about conflicting classes (`p-2 p-4`) and works with multi-cursor edits. Pair it with the Prettier plugin `prettier-plugin-tailwindcss` , which sorts classes into a consistent order automatically (Module 16).

12.2 Extracting with CSS

12.2.1 @apply and when it is appropriate

`@apply` inlines utility classes into your own CSS rule. It is useful in a few specific places: styling HTML you cannot add classes to (Markdown output, a CMS, a third-party widget), overriding styles of a library component, and small, truly global primitives in a project without a component system (a server-rendered site with plain templates). Variants work inside it.

APP.CSS CSS

```css
@import "tailwindcss";

@layer components {
  .btn {
    @apply inline-flex items-center gap-2 rounded-lg px-4 py-2 text-sm font-semibold
           transition focus-visible:outline-2 focus-visible:outline-offset-2;
  }
  .btn-primary {
    @apply bg-indigo-600 text-white hover:bg-indigo-500 focus-visible:outline-indigo-600;
  }
  .btn-secondary {
    @apply bg-white text-slate-900 ring-1 ring-slate-300 hover:bg-slate-50;
  }
}
```

INDEX.HTML HTML

```html
<div class="flex gap-3">
  <button class="btn btn-primary">Save</button>
  <button class="btn btn-secondary">Cancel</button>
  <button class="btn btn-primary px-8">Wide (utility wins)</button>
</div>
```



CLASSES BUILT WITH @APPLY

The compiled output shows that `@apply` simply copies the declarations of each utility into the rule; there is no runtime cost and no extra class in the HTML:

COMPILED CSS (components layer)

```
.btn {
  display: inline-flex;
  align-items: center;
  gap: calc(var(--spacing) * 2);
  border-radius: var(--radius-lg);
  padding-inline: calc(var(--spacing) * 4);
  padding-block: calc(var(--spacing) * 2);
  font-size: var(--text-sm);
  line-height: var(--tw-leading, var(--text-sm--line-height));
  --tw-font-weight: var(--font-weight-semibold);
  font-weight: var(--font-weight-semibold);
  transition-property: color, background-color, border-color, outline-color, text-decoration-color,
fill, stroke, --tw-gradient-from, --tw-gradient-via, --tw-gradient-to, opacity, box-shadow, transform,
translate, scale, rotate, filter, -webkit-backdrop-filter, backdrop-filter, display, content-
visibility, overlay, pointer-events;
  transition-timing-function: var(--tw-ease, var(--default-transition-timing-function));
  transition-duration: var(--tw-duration, var(--default-transition-duration));
  &:focus-visible {
    outline-style: var(--tw-outline-style);
    outline-width: 2px;
  }
  &:focus-visible {
    outline-offset: 2px;
  }
}
.btn-primary {
  background-color: var(--color-indigo-600);
  color: var(--color-white);
  &:hover {
    @media (hover: hover) {
      background-color: var(--color-indigo-500);
    }
  }
  &:focus-visible {
    outline-color: var(--color-indigo-600);
  }
}
.btn-secondary {
  background-color: var(--color-white);
  color: var(--color-slate-900);
  --tw-ring-shadow: var(--tw-ring-inset,) 0 0 0 calc(1px + var(--tw-ring-offset-width)) var(--tw-ring-
color, currentcolor);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow), var(--tw-ring-offset-shadow), var(-
-tw-ring-shadow), var(--tw-shadow);
  --tw-ring-color: var(--color-slate-300);
  &:hover {
    @media (hover: hover) {
      background-color: var(--color-slate-50);
    }
  }
}
```

WATCH OUT

Do not use `@apply` just to make HTML look tidy. Each extracted class needs a name, hides its styles in another file, can no longer be deleted safely and grows the CSS, which are exactly the problems utility-first CSS removes. If you find yourself writing `.card-title`, `.card-body` and `.card-footer`, you want a component, not `@apply`.

In v4, a separate stylesheet (a Vue or Svelte `<style>` block, a CSS module) does not automatically know your theme. To use `@apply` or theme variables there, import the theme without emitting any CSS using

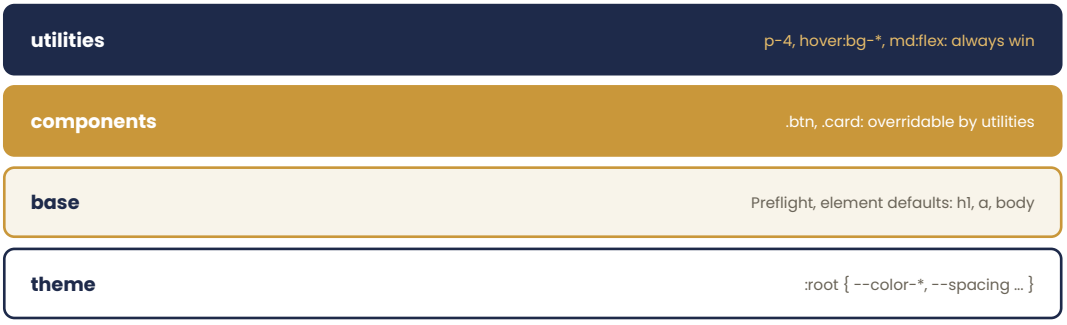
`@reference` (Module 14):

COMPONENT-SCOPED CSS USING @REFERENCE HTML

```
<style scoped>
  @reference "../app.css";
  h1 { @apply text-2xl font-bold text-indigo-900; }
</style>
```

12.2.2 @layer components

Tailwind's output is organised in cascade layers: `theme`, `base`, `components` and `utilities`, in that order. Rules in a later layer always beat rules in an earlier one, regardless of specificity. Putting your `.btn` classes in `@layer components` therefore guarantees that a utility on the same element still wins, as the third button above shows: `px-8` overrides the padding from `.btn`. Write `@layer base` for element defaults (headings, links) and `@layer components` for classes; anything that should behave like a utility (with variants) should be a custom `@utility` instead (Module 14).



TAILWIND'S CASCADE LAYERS: HIGHER LAYERS OVERRIDE LOWER ONES

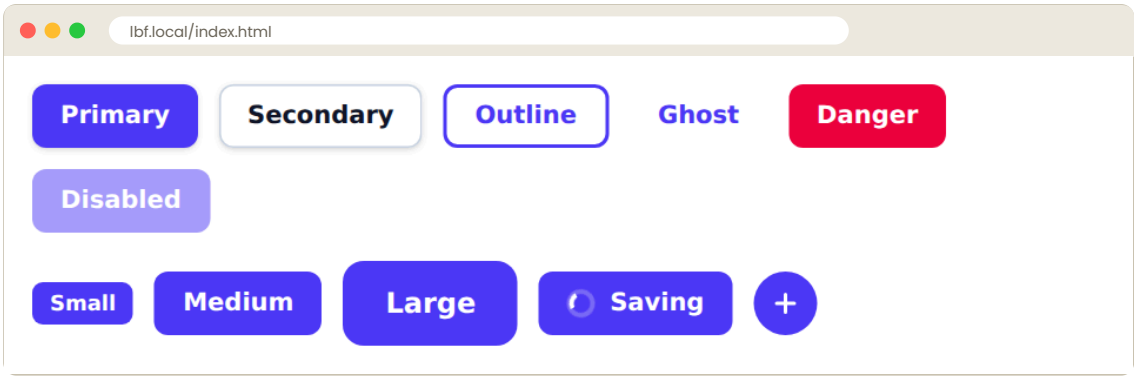
12.3 Common UI components

The following components form a small UI kit. Each one is short, accessible and built only with utilities; copy them into your own components and adjust colours through the theme.

12.3.1 Buttons (primary, secondary, outline, sizes)

BUTTONS.HTML HTML

```
<div class="space-y-4">
  <div class="flex flex-wrap items-center gap-3">
    <button class="rounded-lg bg-indigo-600 px-4 py-2 text-sm font-semibold text-white shadow-sm
hover:bg-indigo-500 focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-
indigo-600">Primary</button>
    <button class="rounded-lg bg-white px-4 py-2 text-sm font-semibold text-slate-900 shadow-sm ring-1
ring-slate-300 ring-inset hover:bg-slate-50">Secondary</button>
    <button class="rounded-lg border-2 border-indigo-600 px-4 py-1.5 text-sm font-semibold text-
indigo-600 hover:bg-indigo-50">Outline</button>
    <button class="rounded-lg px-4 py-2 text-sm font-semibold text-indigo-600 hover:bg-indigo-
50">Ghost</button>
    <button class="rounded-lg bg-rose-600 px-4 py-2 text-sm font-semibold text-white hover:bg-rose-
500">Danger</button>
    <button disabled class="rounded-lg bg-indigo-600 px-4 py-2 text-sm font-semibold text-white
disabled:cursor-not-allowed disabled:opacity-50">Disabled</button>
  </div>
  <div class="flex flex-wrap items-center gap-3">
    <button class="rounded-md bg-indigo-600 px-2.5 py-1 text-xs font-semibold text-
white">Small</button>
    <button class="rounded-lg bg-indigo-600 px-4 py-2 text-sm font-semibold text-
white">Medium</button>
    <button class="rounded-xl bg-indigo-600 px-6 py-3 text-base font-semibold text-
white">Large</button>
    <button class="inline-flex items-center gap-2 rounded-lg bg-indigo-600 px-4 py-2 text-sm font-
semibold text-white">
      <svg class="size-4 animate-spin" viewBox="0 0 24 24" fill="none"><circle cx="12" cy="12" r="10"
stroke="currentColor" stroke-width="4" class="opacity-25"/><path d="M4 12a8 8 0 0 1 8-8"
stroke="currentColor" stroke-width="4"/></svg> Saving
    </button>
    <button class="rounded-full bg-indigo-600 p-2 text-white" aria-label="Add"><svg class="size-5"
viewBox="0 0 20 20" fill="currentColor"><path d="M10 4a1 1 0 0 1 1 1v4h4a1 1 0 0 1 0 2h-4v4a1 1 0 0 1-
2 0v-4H5a1 1 0 0 1 0-2h4V5a1 1 0 0 1 1-1Z"/></svg></button>
  </div>
</div>
```



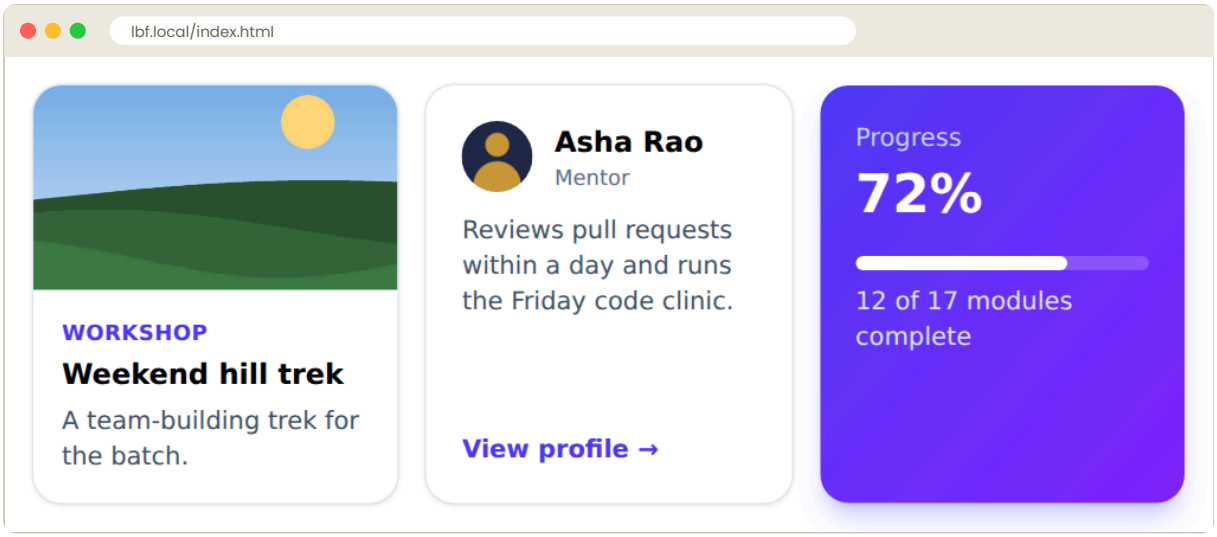
BUTTON VARIANTS, SIZES, A LOADING STATE AND AN ICON BUTTON

12.3.2 Cards

CARDS.HTML

HTML

```
<div class="grid grid-cols-3 gap-4">
  <article class="overflow-hidden rounded-2xl bg-white shadow-sm ring-1 ring-slate-200">
    <img src="hills.jpg" alt="" class="aspect-video w-full object-cover">
    <div class="p-4">
      <p class="text-xs font-semibold uppercase tracking-wide text-indigo-600">Workshop</p>
      <h3 class="mt-1 font-semibold">Weekend hill trek</h3>
      <p class="mt-1 text-sm text-slate-600">A team-building trek for the batch.</p>
    </div>
  </article>
  <article class="flex flex-col rounded-2xl bg-white p-5 shadow-sm ring-1 ring-slate-200">
    <div class="flex items-center gap-3"><img src="avatar.png" alt="" class="size-10 rounded-full">
<div><p class="font-semibold">Asha Rao</p><p class="text-xs text-slate-500">Mentor</p></div></div>
    <p class="mt-3 flex-1 text-sm text-slate-600">Reviews pull requests within a day and runs the
Friday code clinic.</p>
    <a href="#" class="mt-4 text-sm font-semibold text-indigo-600">View profile &rarr;</a>
  </article>
  <article class="rounded-2xl bg-linear-to-br from-indigo-600 to-violet-600 p-5 text-white shadow-lg
shadow-indigo-500/30">
    <p class="text-sm text-indigo-200">Progress</p>
    <p class="mt-1 text-3xl font-black">72%</p>
    <div class="mt-4 h-2 rounded-full bg-white/20"><div class="h-2 w-[72%] rounded-full bg-white">
</div></div>
    <p class="mt-2 text-sm text-indigo-100">12 of 17 modules complete</p>
  </article>
</div>
```



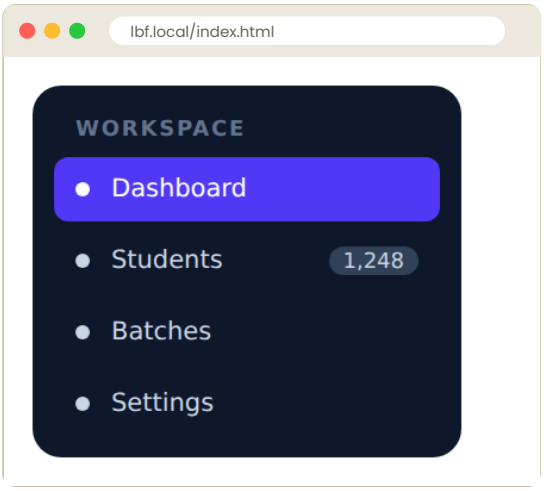
AN IMAGE CARD, A PROFILE CARD AND A STAT CARD

12.3.3 Navbars and sidebars

The navbar was built in Module 7 and made responsive in Module 9. A sidebar navigation uses a vertical list of links with an active state marked by `aria-current="page"`, which is both announced by screen readers and usable as a styling hook through `aria-[current=page]:`.

SIDEBAR.HTML HTML

```
<nav class="w-60 rounded-2xl bg-slate-900 p-3 text-sm" aria-label="Main">
  <p class="px-3 pb-2 pt-1 text-xs font-semibold uppercase tracking-widest text-slate-
500">Workspace</p>
  <ul class="space-y-1">
    <li><a href="#" aria-current="page" class="flex items-center gap-3 rounded-lg px-3 py-2 text-
slate-300 hover:bg-white/5 aria-[current=page]:bg-indigo-600 aria-[current=page]:text-white"><span
class="size-2 rounded-full bg-current"></span>Dashboard</a></li>
    <li><a href="#" class="flex items-center gap-3 rounded-lg px-3 py-2 text-slate-300 hover:bg-
white/5 aria-[current=page]:bg-indigo-600 aria-[current=page]:text-white"><span class="size-2 rounded-
full bg-current"></span>Students<span class="ml-auto rounded-full bg-slate-700 px-2 text-
xs">1,248</span></a></li>
    <li><a href="#" class="flex items-center gap-3 rounded-lg px-3 py-2 text-slate-300 hover:bg-
white/5 aria-[current=page]:bg-indigo-600 aria-[current=page]:text-white"><span class="size-2 rounded-
full bg-current"></span>Batches</a></li>
    <li><a href="#" class="flex items-center gap-3 rounded-lg px-3 py-2 text-slate-300 hover:bg-
white/5 aria-[current=page]:bg-indigo-600 aria-[current=page]:text-white"><span class="size-2 rounded-
full bg-current"></span>Settings</a></li>
  </ul>
</nav>
```



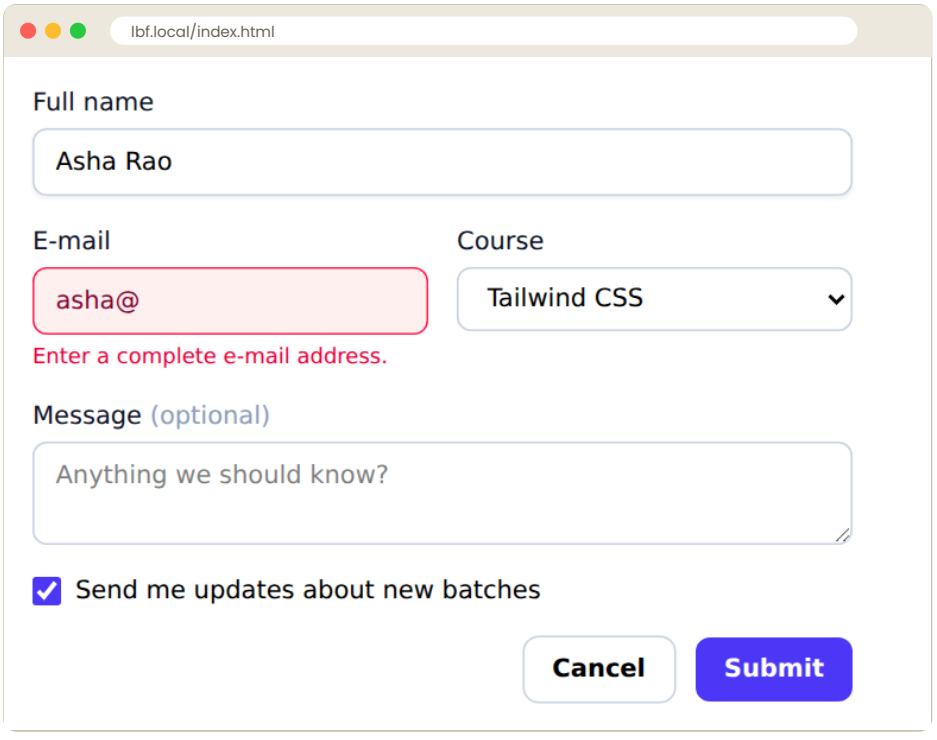
SIDEBAR NAVIGATION WITH THE ACTIVE ITEM MARKED BY ARIA-CURRENT

12.3.4 Forms, inputs and validation messages

FORM.HTML

HTML

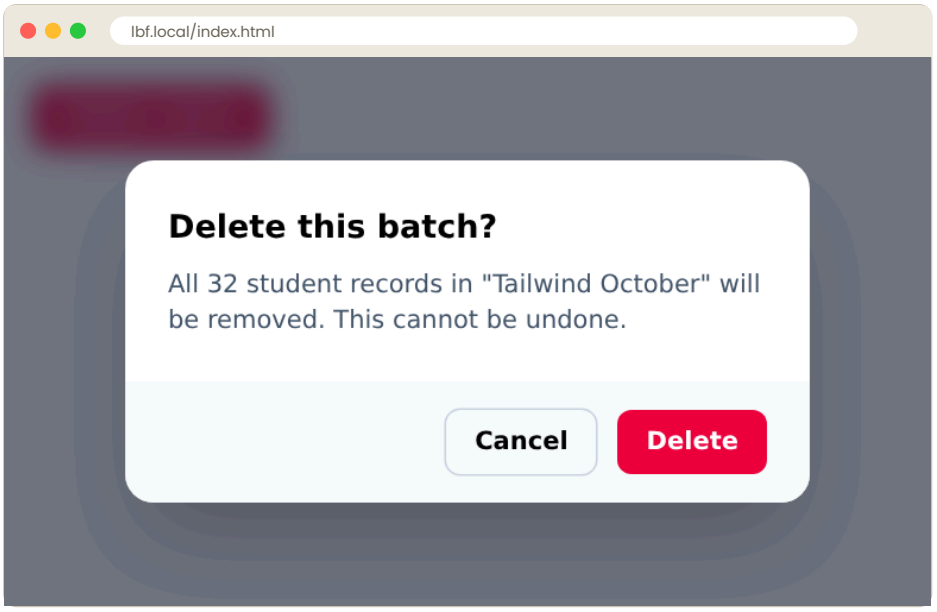
```
<form class="grid w-[460px] grid-cols-2 gap-4 text-sm">
  <label class="col-span-2 block">
    <span class="font-medium text-slate-900">Full name</span>
    <input type="text" value="Asha Rao" class="mt-1 block w-full rounded-lg border border-slate-300
px-3 py-2 shadow-xs focus:border-indigo-500 focus:outline-hidden focus:ring-4 focus:ring-indigo-100">
  </label>
  <label class="block">
    <span class="font-medium text-slate-900">E-mail</span>
    <input type="email" value="asha@" aria-invalid="true" aria-describedby="email-error"
      class="mt-1 block w-full rounded-lg border border-rose-500 bg-rose-50 px-3 py-2 text-rose-900
focus:outline-hidden focus:ring-4 focus:ring-rose-100">
    <span id="email-error" class="mt-1 block text-xs text-rose-600">Enter a complete e-mail address.
</span>
  </label>
  <label class="block">
    <span class="font-medium text-slate-900">Course</span>
    <select class="mt-1 block w-full rounded-lg border border-slate-300 bg-white px-3 py-2">
<option>Tailwind CSS</option><option>React</option></select>
  </label>
  <label class="col-span-2 block">
    <span class="font-medium text-slate-900">Message <span class="font-normal text-slate-400">
(optional)</span></span>
    <textarea rows="2" class="mt-1 block w-full rounded-lg border border-slate-300 px-3 py-2"
placeholder="Anything we should know?"></textarea>
  </label>
  <label class="col-span-2 flex items-center gap-2"><input type="checkbox" checked class="size-4
rounded accent-indigo-600"> Send me updates about new batches</label>
  <div class="col-span-2 flex justify-end gap-3">
    <button type="button" class="rounded-lg px-4 py-2 font-semibold ring-1 ring-slate-
300">Cancel</button>
    <button class="rounded-lg bg-indigo-600 px-4 py-2 font-semibold text-white">Submit</button>
  </div>
</form>
```



A FORM WITH AN ERROR STATE CONNECTED THROUGH ARIA-INVALID AND ARIA-DESCRIBEDBY

12.3.5 Modals and dropdowns

Use the native elements: `<dialog>` with `showModal()` gives a modal with focus trapping, the Escape key and a `::backdrop` (styled with `backdrop:`) for free; the `popover` attribute gives a dropdown that closes on outside click and Escape, with no JavaScript at all.



AFTER CLICKING THE BUTTON: THE NATIVE MODAL WITH A BLURRED BACKDROP

MODAL.HTML HTML

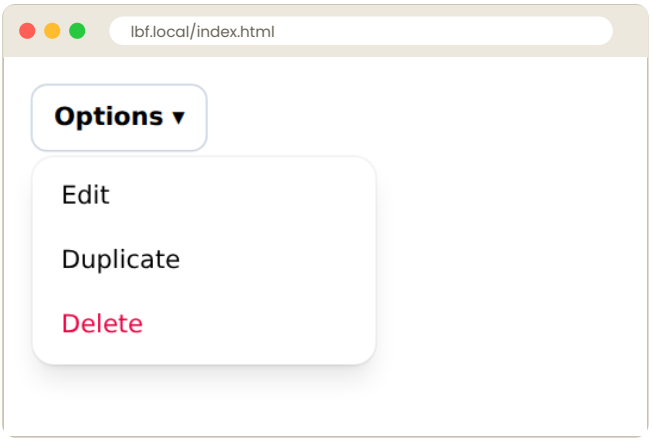
```
<button onclick="document.getElementById('confirm').showModal()" class="rounded-lg bg-rose-600 px-4 py-2 text-sm font-semibold text-white">Delete batch</button>

<dialog id="confirm" class="m-auto w-96 rounded-2xl p-0 shadow-2xl backdrop:bg-slate-900/60 backdrop:backdrop-blur-sm">
  <div class="p-6">
    <h2 class="text-lg font-bold">Delete this batch?</h2>
    <p class="mt-2 text-sm text-slate-600">All 32 student records in "Tailwind October" will be removed. This cannot be undone.</p>
  </div>
  <form method="dialog" class="flex justify-end gap-3 bg-slate-50 px-6 py-4">
    <button class="rounded-lg px-4 py-2 text-sm font-semibold ring-1 ring-slate-300" autofocus>Cancel</button>
    <button value="delete" class="rounded-lg bg-rose-600 px-4 py-2 text-sm font-semibold text-white">Delete</button>
  </form>
</dialog>
```

DROPDOWN.HTML HTML

```
<button popovertarget="menu" class="inline-flex items-center gap-1 rounded-lg bg-white px-3 py-2 text-sm font-semibold ring-1 ring-slate-300">Options &#9662;</button>

<div id="menu" popover class="m-0 mt-1 w-48 rounded-xl bg-white p-1 text-sm shadow-lg ring-1 ring-black/5 [position-area:bottom_span-right]">
  <a href="#" class="block rounded-lg px-3 py-2 hover:bg-slate-100">Edit</a>
  <a href="#" class="block rounded-lg px-3 py-2 hover:bg-slate-100">Duplicate</a>
  <a href="#" class="block rounded-lg px-3 py-2 text-rose-600 hover:bg-rose-50">Delete</a>
</div><div class="h-36"></div>
```



A POPOVER DROPDOWN, OPENED BY CLICKING THE BUTTON (NO JAVASCRIPT)

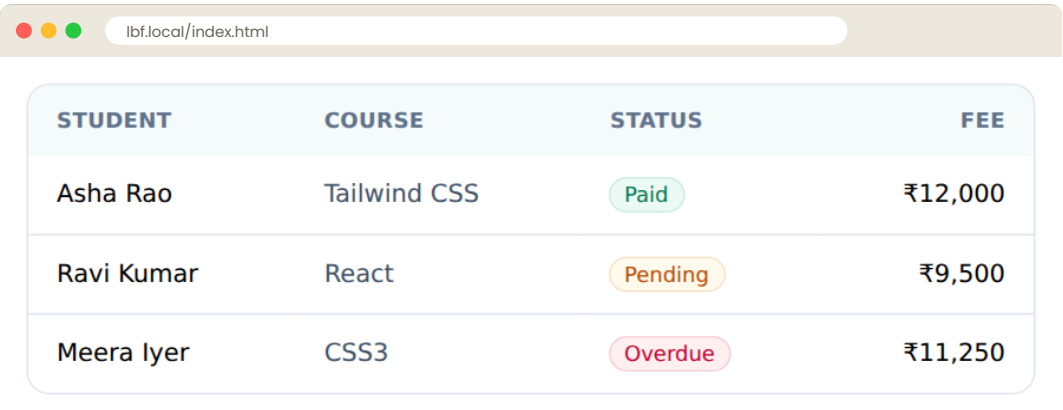
NOTE

`[position-area:bottom_span-right]` uses CSS anchor positioning (supported in Chromium-based browsers) to place the popover under its button. Where it is not supported, give the popover explicit `top` / `left` values or use a positioning library such as Floating UI.

12.3.6 Tables

TABLE.HTML HTML

```
<div class="overflow-hidden rounded-xl ring-1 ring-slate-200">
  <table class="w-full text-left text-sm">
    <thead class="bg-slate-50 text-xs uppercase tracking-wide text-slate-500">
      <tr><th class="px-4 py-3">Student</th><th class="px-4 py-3">Course</th><th class="px-4 py-
3">Status</th><th class="px-4 py-3 text-right">Fee</th></tr>
    </thead>
    <tbody class="divide-y divide-slate-200 bg-white">
      <tr class="hover:bg-slate-50"><td class="px-4 py-3 font-medium">Asha Rao</td><td class="px-4 py-
3 text-slate-600">Tailwind CSS</td><td class="px-4 py-3"><span class="rounded-full bg-emerald-50 px-2
py-0.5 text-xs font-medium text-emerald-700 ring-1 ring-emerald-600/20">Paid</span></td><td class="px-
4 py-3 text-right tabular-nums">&amp;#8377;12,000</td></tr>
      <tr class="hover:bg-slate-50"><td class="px-4 py-3 font-medium">Ravi Kumar</td><td class="px-4
py-3 text-slate-600">React</td><td class="px-4 py-3"><span class="rounded-full bg-amber-50 px-2 py-0.5
text-xs font-medium text-amber-700 ring-1 ring-amber-600/20">Pending</span></td><td class="px-4 py-3
text-right tabular-nums">&amp;#8377;9,500</td></tr>
      <tr class="hover:bg-slate-50"><td class="px-4 py-3 font-medium">Meera Iyer</td><td class="px-4
py-3 text-slate-600">CSS3</td><td class="px-4 py-3"><span class="rounded-full bg-rose-50 px-2 py-0.5
text-xs font-medium text-rose-700 ring-1 ring-rose-600/20">Overdue</span></td><td class="px-4 py-3
text-right tabular-nums">&amp;#8377;11,250</td></tr>
    </tbody>
  </table>
</div>
```



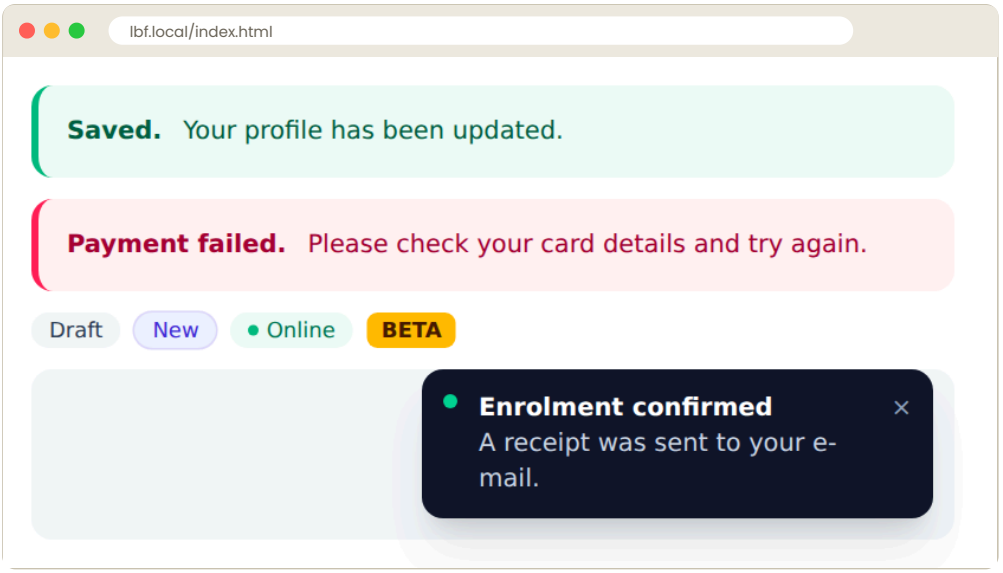
A DATA TABLE: DIVIDE-Y ROWS, STATUS BADGES AND TABULAR-NUMS FOR ALIGNED FIGURES

12.3.7 Alerts, badges and toasts

FEEDBACK.HTML

HTML

```
<div class="w-[520px] space-y-3 text-sm">
  <div role="status" class="flex gap-3 rounded-xl border-l-4 border-emerald-500 bg-emerald-50 p-4
text-emerald-800"><b>Saved.</b> Your profile has been updated.</div>
  <div role="alert" class="flex gap-3 rounded-xl border-l-4 border-rose-500 bg-rose-50 p-4 text-rose-
800"><b>Payment failed.</b> Please check your card details and try again.</div>
  <div class="flex flex-wrap gap-2">
    <span class="rounded-full bg-slate-100 px-2.5 py-0.5 text-xs font-medium text-slate-
700">Draft</span>
    <span class="rounded-full bg-indigo-50 px-2.5 py-0.5 text-xs font-medium text-indigo-700 ring-1
ring-indigo-600/20">New</span>
    <span class="inline-flex items-center gap-1 rounded-full bg-emerald-50 px-2.5 py-0.5 text-xs font-
medium text-emerald-700"><span class="size-1.5 rounded-full bg-emerald-500"></span>Online</span>
    <span class="rounded-md bg-amber-400 px-2 py-0.5 text-xs font-bold uppercase text-amber-
950">Beta</span>
  </div>
  <div class="relative h-24 rounded-xl bg-slate-100">
    <div role="status" aria-live="polite" class="absolute bottom-3 right-3 flex w-72 items-start gap-3
rounded-xl bg-slate-900 p-3 text-white shadow-xl">
      <span class="mt-0.5 size-2 shrink-0 rounded-full bg-emerald-400"></span>
      <div class="flex-1"><p class="font-semibold">Enrolment confirmed</p><p class="text-slate-300">A
receipt was sent to your e-mail.</p></div>
      <button class="text-slate-400 hover:text-white" aria-label="Dismiss">&amp;times;</button>
    </div>
  </div>
</div>
```



SUCCESS AND ERROR ALERTS, BADGES, AND A TOAST IN THE CORNER OF ITS CONTAINER

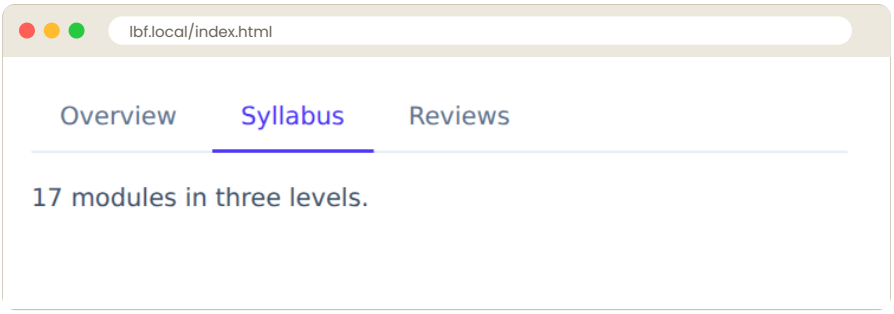
Use `role="alert"` only for urgent messages (it interrupts the screen reader) and `role="status"` or `aria-live="polite"` for confirmations and toasts. A real toast is `fixed bottom-4 right-4 z-50` and is removed after a few seconds by JavaScript; keep it visible long enough to read and let users dismiss it.

12.3.8 Tabs and accordions

TABS.HTML

HTML

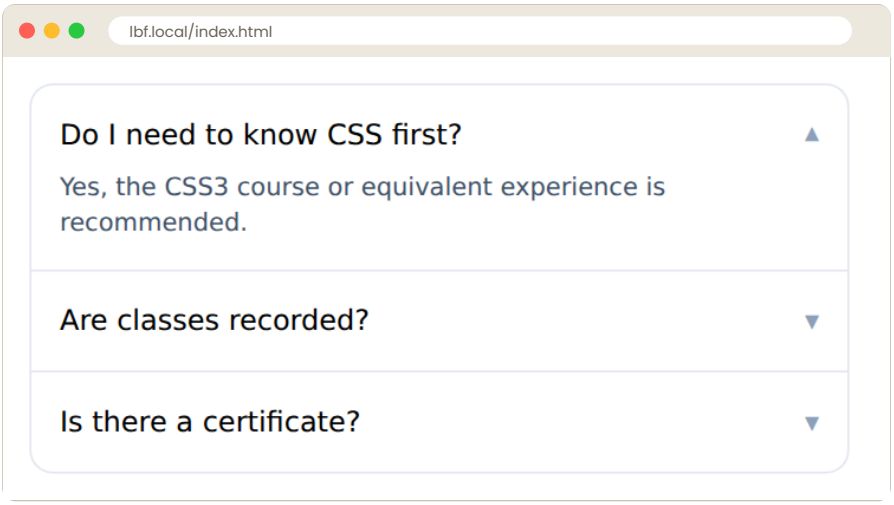
```
<div class="w-[460px]">
  <div role="tablist" aria-label="Course details" class="flex gap-1 border-b border-slate-200 text-sm
font-medium">
    <button role="tab" aria-selected="true" aria-controls="p1" class="-mb-px border-b-2 border-
transparent px-4 py-2 text-slate-500 hover:text-slate-900 aria-selected:border-indigo-600 aria-
selected:text-indigo-600">Overview</button>
    <button role="tab" aria-selected="false" aria-controls="p2" class="-mb-px border-b-2 border-
transparent px-4 py-2 text-slate-500 hover:text-slate-900 aria-selected:border-indigo-600 aria-
selected:text-indigo-600">Syllabus</button>
    <button role="tab" aria-selected="false" aria-controls="p3" class="-mb-px border-b-2 border-
transparent px-4 py-2 text-slate-500 hover:text-slate-900 aria-selected:border-indigo-600 aria-
selected:text-indigo-600">Reviews</button>
  </div>
  <div id="p1" role="tabpanel" class="py-4 text-sm text-slate-600">A 26-day course covering Tailwind
CSS v4 from setup to production.</div>
  <div id="p2" role="tabpanel" hidden class="py-4 text-sm text-slate-600">17 modules in three levels.
</div>
  <div id="p3" role="tabpanel" hidden class="py-4 text-sm text-slate-600">4.8 out of 5 from 320
students.</div>
</div>
<script>
  document.querySelectorAll('[role=tab]').forEach(tab => tab.addEventListener('click', () => {
    document.querySelectorAll('[role=tab]').forEach(t => t.setAttribute('aria-selected', t === tab));
    document.querySelectorAll('[role=tabpanel]').forEach(p => p.hidden = p.id !==
tab.getAttribute('aria-controls'));
  }));
</script>
```



AFTER CLICKING "SYLLABUS": ARIA-SELECTED DRIVES THE ACTIVE STYLE

FAQ.HTML HTML

```
<div class="w-[460px] divide-y divide-slate-200 rounded-xl ring-1 ring-slate-200">
  <details class="group p-4" open>
    <summary class="flex cursor-pointer list-none items-center justify-between font-medium [&::-
webkit-details-marker]:hidden">Do I need to know CSS first?<span class="text-slate-400 transition
group-open:rotate-180">&#9662;</span></summary>
    <p class="mt-2 text-sm text-slate-600">Yes, the CSS3 course or equivalent experience is
recommended.</p>
  </details>
  <details class="group p-4">
    <summary class="flex cursor-pointer list-none items-center justify-between font-medium [&::-
webkit-details-marker]:hidden">Are classes recorded?<span class="text-slate-400 transition group-
open:rotate-180">&#9662;</span></summary>
    <p class="mt-2 text-sm text-slate-600">Every live class is recorded and available for a year.</p>
  </details>
  <details class="group p-4">
    <summary class="flex cursor-pointer list-none items-center justify-between font-medium [&::-
webkit-details-marker]:hidden">Is there a certificate?<span class="text-slate-400 transition group-
open:rotate-180">&#9662;</span></summary>
    <p class="mt-2 text-sm text-slate-600">Yes, after the final project review.</p>
  </details>
</div>
```



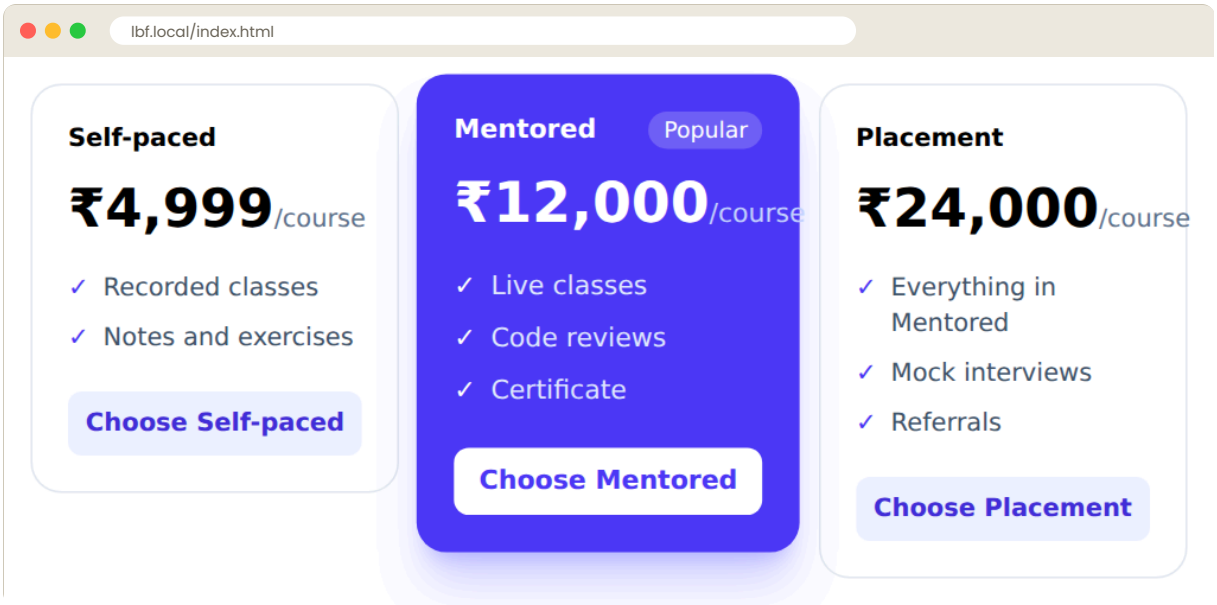
AN ACCORDION FROM NATIVE DETAILS ELEMENTS

12.3.9 Pricing tables and testimonial sections

PRICING.HTML

HTML

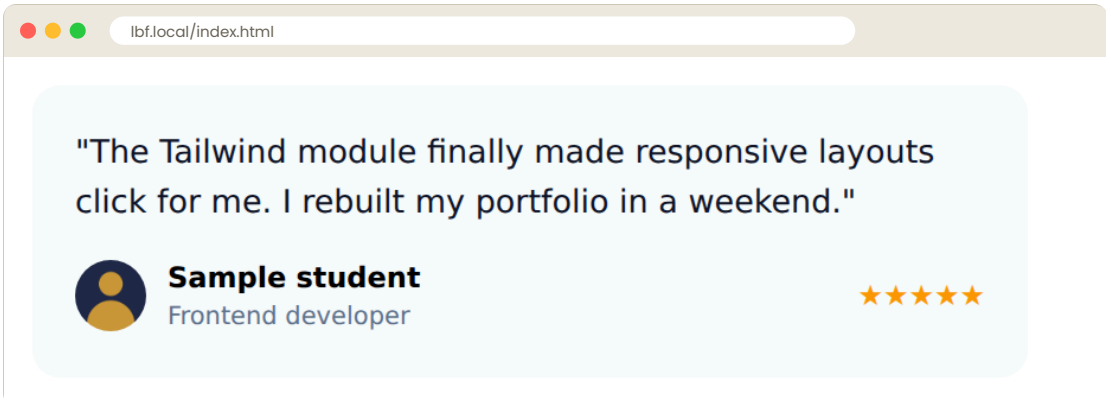
```
<div class="grid grid-cols-3 items-start gap-4 text-sm">
  <div class="rounded-2xl p-5 bg-white ring-1 ring-slate-200">
    <div class="flex items-center justify-between"><h3 class="font-semibold">Self-paced</h3></div>
    <p class="mt-3"><span class="text-3xl font-black">&#8377;4,999</span><span class="text-slate-
500">/course</span></p>
    <ul class="mt-4 space-y-2 text-slate-600"><li class="flex gap-2"><span class="text-indigo-
600">&#10003;</span>Recorded classes</li><li class="flex gap-2"><span class="text-indigo-600">&#10003;
</span>Notes and exercises</li></ul>
    <a href="#" class="mt-5 block rounded-lg py-2 text-center font-semibold bg-indigo-50 text-indigo-
700 hover:bg-indigo-100">Choose Self-paced</a>
  </div>
  <div class="rounded-2xl p-5 bg-indigo-600 text-white shadow-xl shadow-indigo-500/30 ring-0 scale-
105">
    <div class="flex items-center justify-between"><h3 class="font-semibold">Mentored</h3><span
class="rounded-full bg-white/20 px-2 py-0.5 text-xs">Popular</span></div>
    <p class="mt-3"><span class="text-3xl font-black">&#8377;12,000</span><span class="text-indigo-
200">/course</span></p>
    <ul class="mt-4 space-y-2 text-indigo-100"><li class="flex gap-2"><span class="text-
white">&#10003;</span>Live classes</li><li class="flex gap-2"><span class="text-white">&#10003;
</span>Code reviews</li><li class="flex gap-2"><span class="text-white">&#10003;
</span>Certificate</li></ul>
    <a href="#" class="mt-5 block rounded-lg py-2 text-center font-semibold bg-white text-indigo-
600">Choose Mentored</a>
  </div>
  <div class="rounded-2xl p-5 bg-white ring-1 ring-slate-200">
    <div class="flex items-center justify-between"><h3 class="font-semibold">Placement</h3></div>
    <p class="mt-3"><span class="text-3xl font-black">&#8377;24,000</span><span class="text-slate-
500">/course</span></p>
    <ul class="mt-4 space-y-2 text-slate-600"><li class="flex gap-2"><span class="text-indigo-
600">&#10003;</span>Everything in Mentored</li><li class="flex gap-2"><span class="text-indigo-
600">&#10003;</span>Mock interviews</li><li class="flex gap-2"><span class="text-indigo-600">&#10003;
</span>Referrals</li></ul>
    <a href="#" class="mt-5 block rounded-lg py-2 text-center font-semibold bg-indigo-50 text-indigo-
700 hover:bg-indigo-100">Choose Placement</a>
  </div>
</div>
```



A THREE-TIER PRICING TABLE WITH THE RECOMMENDED PLAN HIGHLIGHTED

TESTIMONIAL.HTML HTML

```
<figure class="w-[560px] rounded-2xl bg-slate-50 p-6">
  <blockquote class="text-lg font-medium text-slate-900">"The Tailwind module finally made responsive layouts click for me. I rebuilt my portfolio in a weekend."</blockquote>
  <figcaption class="mt-4 flex items-center gap-3">
    <img src="avatar.png" alt="" class="size-10 rounded-full">
    <div><p class="font-semibold">Sample student</p><p class="text-sm text-slate-500">Frontend developer</p></div>
    <p class="ml-auto text-amber-500" aria-label="5 out of 5 stars">&#9733;&#9733;&#9733;&#9733;&#9733;</p>
  </figcaption>
</figure>
```



A TESTIMONIAL LAYOUT (PLACEHOLDER TEXT: USE REAL, ATTRIBUTED QUOTES ON A LIVE SITE)

12.4 The Forms plugin

12.4.1 @tailwindcss/forms for consistent form controls

Browsers style form controls differently, and Preflight resets only part of that. The official **@tailwindcss/forms** plugin adds a consistent, easily overridden base style for text inputs, selects (with a custom arrow), textareas, checkboxes and radios, so that utilities like `rounded-md`, `border-slate-300` and `text-indigo-600` (which colours checked boxes) work as expected.

APP.CSS CSS

```
@import "tailwindcss";
@plugin "@tailwindcss/forms";

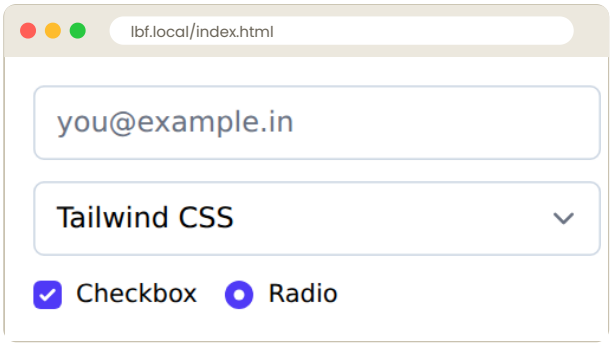
/* or only provide .form-input, .form-select ... classes instead of global styles */
/* @plugin "@tailwindcss/forms" { strategy: "class"; } */
```

CONTROLS.HTML HTML

```
<div class="grid w-80 gap-3 text-sm">
  <input type="email" placeholder="you@example.in" class="rounded-md border-slate-300 focus:border-indigo-500 focus:ring-indigo-500">
  <select class="rounded-md border-slate-300"><option>Tailwind CSS</option></select>
  <div class="flex gap-4">
    <label class="flex items-center gap-2"><input type="checkbox" checked class="rounded text-indigo-600"> Checkbox</label>
    <label class="flex items-center gap-2"><input type="radio" name="r" checked class="text-indigo-600"> Radio</label>
  </div>
</div>
```



PREFLIGHT ONLY: THE UTILITIES HAVE LITTLE VISIBLE EFFECT ON THE NATIVE CONTROLS



WITH @TAILWINDCSS/FORMS: BORDERED INPUTS, A CUSTOM SELECT ARROW AND COLOURABLE CHECKBOX AND RADIO

12.5 Accessibility for components

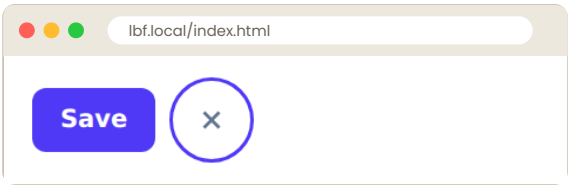
12.5.1 Focus styles, sr-only labels and ARIA attributes

Utilities make it easy to style components, but accessibility depends on the HTML. A checklist for every component:

Requirement	How
Every interactive element is reachable and usable with the keyboard	use `<button>` and `<a href>`, not clickable `<div>`s; native `<dialog>`, `<details>`, `popover`
Focus is always visible	`focus-visible:outline-2 focus-visible:outline-offset-2` (or a ring); never remove outlines without a replacement
Icon-only controls have a name	`<span class="sr-only">Close</span>` or `aria-label="Close"`; decorative icons get `aria-hidden="true"`
Form fields have labels and linked errors	`<label>` around or `for`; `aria-invalid` and `aria-describedby` for errors
State is exposed, not only shown	`aria-expanded`, `aria-selected`, `aria-current`, `aria-pressed`, styled with the matching variants
Colour is not the only signal	pair colours with text or icons ("Overdue" badge, error message text)
Contrast meets 4.5:1 for text	check grey text such as `text-slate-400` on white, which fails for small text
Touch targets are large enough	at least 44 by 44px: `min-h-11 min-w-11` or padding; `pointer-coarse:` to enlarge on touch

KEYBOARD FOCUS ON AN ICON BUTTON HTML

```
<div class="flex items-center gap-3">
  <button class="rounded-lg bg-indigo-600 px-4 py-2 text-sm font-semibold text-white focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-indigo-600">Save</button>
  <button class="grid size-10 place-items-center rounded-full text-slate-500 hover:bg-slate-100 focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-indigo-600">
    <svg aria-hidden="true" class="size-5" viewBox="0 0 20 20" fill="currentColor"><path d="M6.3 5 5 6.3 8.7 10 5 13.7 6.3 15 10 11.3 13.7 15 15 13.7 11.3 10 15 6.3 13.7 5 10 8.7z"/></svg>
    <span class="sr-only">Close</span>
  </button>
</div>
```



AFTER PRESSING TAB TWICE: THE CLOSE BUTTON SHOWS A CLEAR FOCUS OUTLINE; ITS NAME COMES FROM THE SR-ONLY SPAN

PRACTICE CHECKPOINT

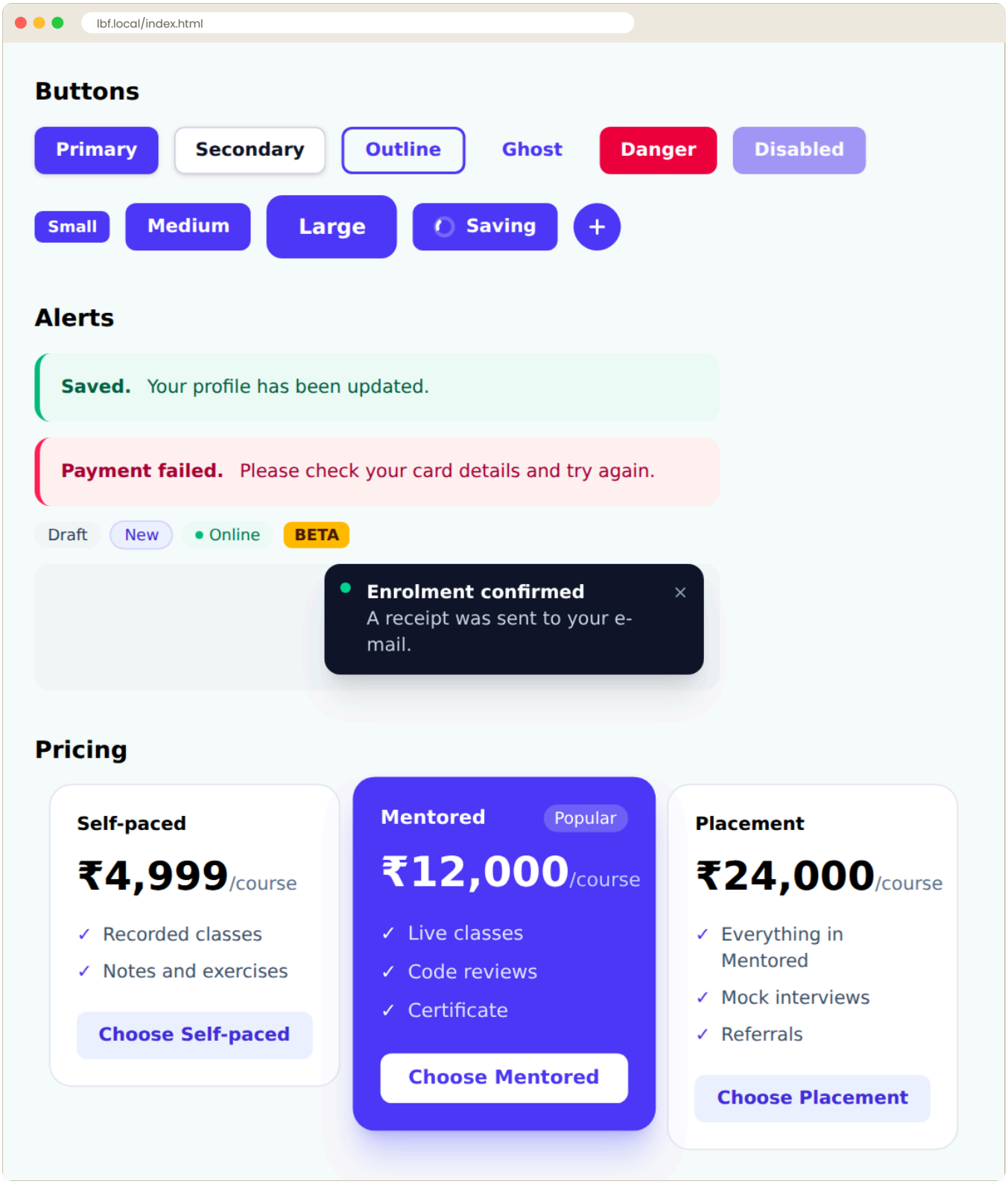
A **small UI kit page** showing buttons, cards, a form, a modal, tabs, alerts and a pricing table, all built with Tailwind.

The UI kit page is the set of components from Section 12.3 placed on one page under headings, with the modal opened from a button. Its structure:

UI-KIT.HTML: SAMPLE SOLUTION (STRUCTURE) HTML

```
<body class="bg-slate-50 text-slate-900">
  <header class="sticky top-0 z-20 border-b border-slate-200 bg-white/80 backdrop-blur">
    <div class="mx-auto flex max-w-6xl items-center justify-between px-6 py-3">
      <b>LBF UI kit</b>
      <nav class="flex gap-4 text-sm"><a href="#buttons">Buttons</a><a href="#cards">Cards</a><a href="#forms">Forms</a> ...</nav>
    </div>
  </header>

  <main class="mx-auto max-w-6xl space-y-16 px-6 py-12">
    <section id="buttons" class="scroll-mt-20">
      <h2 class="mb-4 text-xl font-bold">Buttons</h2>
      <!-- 12.3.1 -->
    </section>
    <section id="cards" class="scroll-mt-20"> <!-- 12.3.2 --> </section>
    <section id="forms" class="scroll-mt-20"> <!-- 12.3.4, with @tailwindcss/forms --> </section>
    <section id="overlays" class="scroll-mt-20"> <!-- 12.3.5 modal and dropdown --> </section>
    <section id="tabs" class="scroll-mt-20"> <!-- 12.3.8 --> </section>
    <section id="feedback" class="scroll-mt-20"> <!-- 12.3.7 --> </section>
    <section id="pricing" class="scroll-mt-20"> <!-- 12.3.9 --> </section>
  </main>
</body>
```



PART OF THE FINISHED UI KIT PAGE

Remove repetition with loops and components first; use `@apply` inside `@layer components` only for markup you cannot control or tiny global primitives; build components from native elements (`dialog`, `details`, `popover`) and expose their state with ARIA attributes that double as styling hooks.

LEVEL 3

Advanced level

Outcome: customise the theme with @theme, extend Tailwind with custom utilities, variants and plugins, use it inside frameworks and ship production-quality projects.

5 MODULES · 10 DAYS MINIMUM

Module 13 · Customising the theme	**2 days**
Module 14 · Directives, custom utilities, variants and plugins	**2 days**
Module 15 · Tailwind with frameworks and the ecosystem	**2 days**
Module 16 · Performance, accessibility and best practices	**1 day**
Module 17 · Projects and interview preparation	**3 days**

MODULE 13 · ADVANCED

Customising the theme

Minimum duration **2 days** | Level **Advanced**

Tailwind 4 uses CSS-first configuration: the design system is defined in CSS with the @theme directive. This module explains theme variables, their namespaces and exactly how they generate utility classes; extending the theme with brand colours, fonts and breakpoints, overriding or removing defaults, and @theme inline for variables that reference other variables; reading theme values in custom CSS and in JavaScript; organising design tokens in a shared theme file and supporting several brands; and the legacy tailwind.config.js file with @config. The generated CSS shown throughout is the real compiler output.

IN THIS MODULE

13.1 Theme variables
13.2 Extending and overriding
13.3 Using theme values elsewhere
13.4 Design systems
13.5 Legacy JavaScript configuration

13.1 Theme variables

13.1.1 The @theme directive

`@theme` looks like a block of CSS custom properties, and it is: every variable in it is emitted on `:root` so you can use it anywhere. But it does a second job that a normal `:root` block cannot: it tells Tailwind which **utility classes** to generate. A variable named `--color-mint-500` makes `bg-mint-500`, `text-mint-500`, `border-mint-500`, `fill-mint-500` and every other colour utility exist.

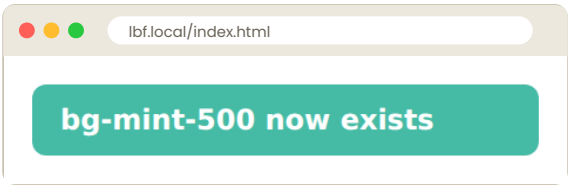
APP.CSS — CSS

```css
@import "tailwindcss";

@theme {
  --color-mint-500: oklch(0.72 0.11 178);
}
```

INDEX.HTML — HTML

```html
<p class="rounded-lg bg-mint-500 px-4 py-2 font-semibold text-white">bg-mint-500 now exists</p>
```



HOW THE BROWSER DISPLAYS IT

COMPILED CSS

```
.border-mint-500 {
  border-color: var(--color-mint-500);
}
.bg-mint-500 {
  background-color: var(--color-mint-500);
}
.text-mint-500 {
  color: var(--color-mint-500);
}
```

COMPILED CSS (theme layer)

```
:root, :host {
  --font-sans: ui-sans-serif, system-ui, sans-serif, 'Apple Color Emoji', 'Segoe UI Emoji', 'Segoe UI
Symbol',
  'Noto Color Emoji';
  --font-mono: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, 'Liberation Mono', 'Courier
New',
  monospace;
  --default-font-family: var(--font-sans);
  --default-mono-font-family: var(--font-mono);
  --color-mint-500: oklch(0.72 0.11 178);
}
```

	:root { --x: … }	@theme { --x: … }
Creates a CSS variable	yes	yes (on `:root`, in the `theme` layer)
Creates utility classes	no	yes, if the name is in a theme namespace
Allowed where	anywhere, inside any selector	top level only, not nested in selectors or media queries
Emitted when unused	always	only variables that are used (by a utility or by your CSS), unless `static` is set

13.1.2 Theme namespaces: --color-*, --font-*, --text-*, --spacing, --breakpoint-*, --radius-*, --shadow-*, --animate-*

The prefix of a variable decides which utilities it feeds. The main namespaces:

Namespace	Generates	Example variable	Example class
`--color-*`	all colour utilities	`--color-brand-600`	`bg-brand-600`
`--font-*`	font-family	`--font-display`	`font-display`
`--text-*`	font-size (+ line height)	`--text-hero`	`text-hero`
`--font-weight-*`	font-weight	`--font-weight-book`	`font-book`
`--tracking-*` / `--leading-*`	letter spacing / line height	`--tracking-label`	`tracking-label`
`--spacing`	the base unit for all spacing and sizing	`--spacing: 0.25rem`	`p-4`, `w-13`

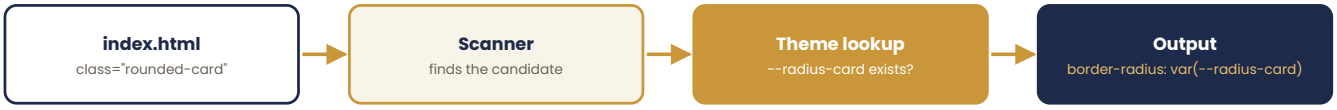
Namespace	Generates	Example variable	Example class
`--spacing-*`	named spacing values	`--spacing-gutter`	`px-gutter`
`--breakpoint-*`	responsive variants	`--breakpoint-3xl`	`3xl:flex`
`--container-*`	container query variants and max-w-*	`--container-card`	`@card:`, `max-w-card`
`--radius-*`	border radius	`--radius-card`	`rounded-card`
`--shadow-*`, `--inset-shadow-*`, `--drop-shadow-*`, `--text-shadow-*`	shadows	`--shadow-soft`	`shadow-soft`
`--blur-*`	blur and backdrop blur	`--blur-glass`	`backdrop-blur-glass`
`--ease-*`	timing functions	`--ease-bounce`	`ease-bounce`
`--animate-*`	animations	`--animate-wiggle`	`animate-wiggle`
`--perspective-*`, `--aspect-*`	3D perspective, aspect ratios	`--aspect-photo`	`aspect-photo`

A size in the `--text-*` namespace can carry its own line height, letter spacing and weight through double-dash sub-properties, exactly like the built-in sizes:

COMPILED CSS
```
.text-hero {
  font-size: var(--text-hero);
  line-height: var(--tw-leading, var(--text-hero--line-height));
  letter-spacing: var(--tw-tracking, var(--text-hero--letter-spacing));
  font-weight: var(--tw-font-weight, var(--text-hero--font-weight));
}
```

13.1.3 How theme variables generate utility classes

Tailwind scans your templates for candidate class names, then looks each one up. For `rounded-card` it checks whether the `rounded` utility accepts the value `card`: it looks for `--radius-card` in the theme. If it exists, the class is generated, referring to the variable rather than copying its value. That is why changing a variable (in the theme, in a media query or on a wrapper element) restyles every use of the class.



FROM A CLASS NAME IN THE HTML TO A RULE IN THE CSS

COMPILED CSS
```
.rounded-card {
  border-radius: var(--radius-card);
}
.rounded-t-card {
  border-top-left-radius: var(--radius-card);
  border-top-right-radius: var(--radius-card);
}
```

`rounded-unknown` produced nothing: there is no `--radius-unknown` variable, so it is not a valid class, and Tailwind silently ignores it. This is how a typo in a class name usually shows up: the class simply has no effect. The IntelliSense extension flags such classes in the editor.

13.2 Extending and overriding

13.2.1 Adding brand colours (for example Deep Indigo and Gold)

Add brand colours as new variables next to the default palette. Giving each brand colour a full shade scale (50 to 950) lets designers use tints for backgrounds and darker shades for text and hover states, exactly like built-in colours. Writing colours in `oklch()` makes building a scale easy: keep the hue, vary lightness and chroma.

APP.CSS

CSS

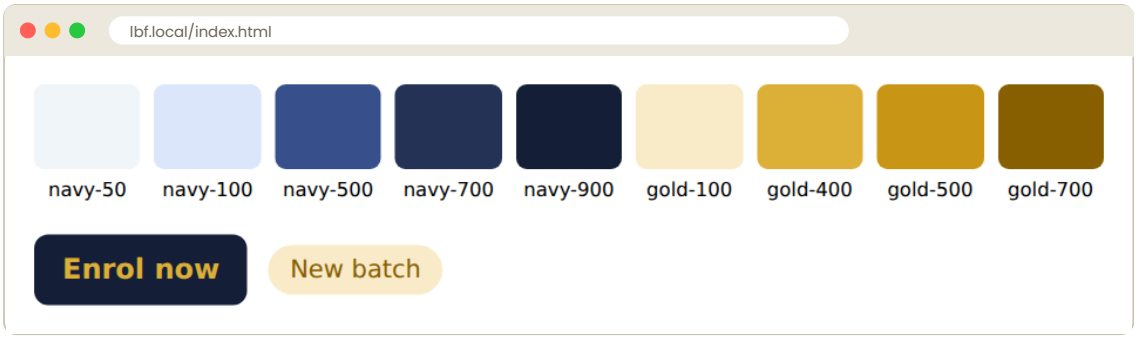
```
@import "tailwindcss";

@theme {
  --color-navy-50:  oklch(0.97 0.01 265);
  --color-navy-100: oklch(0.93 0.03 265);
  --color-navy-500: oklch(0.45 0.10 265);
  --color-navy-700: oklch(0.33 0.07 265);
  --color-navy-900: oklch(0.25 0.05 265);   /* Deep Indigo #1C2B4A */
  --color-gold-100: oklch(0.95 0.05 90);
  --color-gold-400: oklch(0.78 0.14 88);
  --color-gold-500: oklch(0.71 0.14 86);    /* Gold #C9A227 */
  --color-gold-700: oklch(0.52 0.11 80);
}
```

INDEX.HTML

HTML

```
<div class="flex gap-2 text-[11px]"><div class="w-16"><div class="h-12 rounded-md bg-navy-50"></div><p
class="mt-1 text-center">navy-50</p></div><div class="w-16"><div class="h-12 rounded-md bg-navy-100">
</div><p class="mt-1 text-center">navy-100</p></div><div class="w-16"><div class="h-12 rounded-md bg-
navy-500"></div><p class="mt-1 text-center">navy-500</p></div><div class="w-16"><div class="h-12
rounded-md bg-navy-700"></div><p class="mt-1 text-center">navy-700</p></div><div class="w-16"><div
class="h-12 rounded-md bg-navy-900"></div><p class="mt-1 text-center">navy-900</p></div><div class="w-
16"><div class="h-12 rounded-md bg-gold-100"></div><p class="mt-1 text-center">gold-100</p></div><div
class="w-16"><div class="h-12 rounded-md bg-gold-400"></div><p class="mt-1 text-center">gold-400</p>
</div><div class="w-16"><div class="h-12 rounded-md bg-gold-500"></div><p class="mt-1 text-
center">gold-500</p></div><div class="w-16"><div class="h-12 rounded-md bg-gold-700"></div><p
class="mt-1 text-center">gold-700</p></div></div>
<div class="mt-4 flex items-center gap-3">
  <button class="rounded-lg bg-navy-900 px-4 py-2 font-semibold text-gold-400 hover:bg-navy-700">Enrol
now</button>
  <span class="rounded-full bg-gold-100 px-3 py-1 text-sm font-medium text-gold-700">New batch</span>
</div>
```



THE BRAND SCALES AND TWO COMPONENTS USING THEM

TIP

Use the exact brand value for the shade most used as "the" brand colour and derive the rest. Check text contrast for each pair you actually use (gold on navy, navy on gold-100), as in Module 3.

13.2.2 Adding custom fonts

Fonts were introduced in Module 4. In a design system, register a role-based name (`--font-display` for headings, `--font-sans` for body) rather than a font name, so that changing the font later does not mean renaming classes across the project.

ROLE-BASED FONT NAMES CSS

```
@font-face { font-family: "Poppins"; src: url("/fonts/Poppins-Regular.woff2") format("woff2"); font-
weight: 400; font-display: swap; }
@font-face { font-family: "Poppins"; src: url("/fonts/Poppins-Bold.woff2") format("woff2"); font-
weight: 700; font-display: swap; }

@theme {
  --font-sans: "Poppins", ui-sans-serif, system-ui, sans-serif;   /* body text everywhere */
  --font-display: "Lora", ui-serif, Georgia, serif;              /* class: font-display */
}
```

13.2.3 Custom breakpoints

Breakpoints are `--breakpoint-*` variables. Adding one creates a responsive variant, and changing one moves every class that uses it. Use rem so they respect the user's font size. Tailwind sorts breakpoints by size automatically, so a new `xs` works before `sm`.

COMPILED CSS

```
.xs\:grid-cols-2 {
  @media (width >= 30rem) {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}
.md\:grid-cols-3 {
  @media (width >= 50rem) {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }
}
.\33 xl\:grid-cols-6 {
  @media (width >= 120rem) {
    grid-template-columns: repeat(6, minmax(0, 1fr));
  }
}
```

13.2.4 Overriding or removing the defaults

Redefining a variable **overrides** it (`--color-indigo-600: #4338ca;`). To **remove** a whole namespace, set it to `initial`: `--color-*: initial;` deletes the entire default palette, leaving only the colours you define afterwards. `--*: initial;` removes every default and gives a completely custom design system. This prevents developers from reaching for off-brand colours such as `bg-pink-500`.

COMPILED CSS
```
.bg-ink {
  background-color: var(--color-ink);
}
.bg-paper {
  background-color: var(--color-paper);
}
.text-gold {
  color: var(--color-gold);
}
```

Only the three brand classes were generated; `bg-pink-500` and `text-slate-700` no longer exist. Override other defaults the same way: `--radius-lg: 0.75rem;` changes every `rounded-lg`, and `--spacing: 0.3rem;` changes the whole spacing scale at once.

13.2.5 @theme inline and referencing other variables

When a theme variable refers to another variable, the order of resolution matters. With a plain `@theme`, `--font-sans: var(--font-inter)` is defined on `:root` and resolved there, so if `--font-inter` is only set lower down (for example by Next.js's font loader on `<body>`), the reference fails. `@theme inline` tells Tailwind to put the **value itself** into each utility instead of a reference to the theme variable, so it resolves on the element that uses the class.

COMPILED CSS
```
.bg-surface {
  background-color: var(--surface);
}
.font-sans {
  font-family: var(--font-inter);
}
```

Compare: without `inline` the class would be `background-color: var(--color-surface)`; with it, `var(--surface)`. Module 10 used this for semantic colours that change per section. Two more modifiers exist: `@theme static` emits every variable even if unused (useful when JavaScript reads them), and `@theme default` marks values that can be overridden by later themes.

13.3 Using theme values elsewhere

13.3.1 Theme values as CSS variables in custom CSS

Because every theme value is a real CSS variable, custom CSS, inline styles and arbitrary values can use the design tokens directly. This keeps hand-written CSS on-brand:

APP.CSS

CSS

```css
@import "tailwindcss";

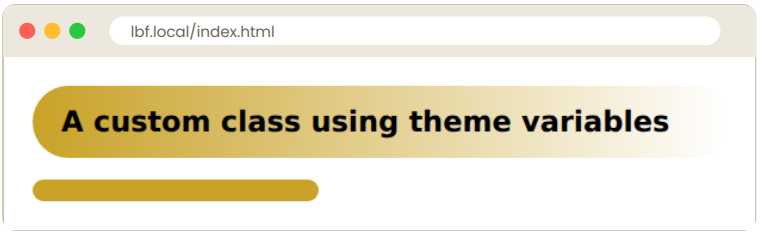
@theme { --color-gold: #c9a227; --radius-card: 1.25rem; }

.ribbon {
  background: linear-gradient(90deg, var(--color-gold), transparent);
  border-radius: var(--radius-card);
  padding: --spacing(2) --spacing(4);     /* the spacing function, Module 14 */
}
```

INDEX.HTML

HTML

```html
<div class="ribbon font-semibold">A custom class using theme variables</div>
<div class="mt-3 h-3 w-[calc(var(--spacing)*40)] rounded-full bg-(--color-gold)"></div>
```



CUSTOM CSS AND ARBITRARY VALUES READING THE THEME

13.3.2 Theme values in JavaScript

In v4 there is no JavaScript theme object to import; the theme lives in CSS. JavaScript reads it the same way it reads any CSS variable, with `getComputedStyle`. This is useful for chart libraries, canvas drawing and animation libraries that need actual colour values:

READING THEME VALUES IN JAVASCRIPT

javascript

```javascript
const styles = getComputedStyle(document.documentElement);
const gold = styles.getPropertyValue('--color-gold').trim();
const md = styles.getPropertyValue('--breakpoint-md').trim();
console.log(gold, md, matchMedia(`(width >= ${md})`).matches);
```

COMPUTED IN CHROMIUM

```
--color-gold = #c9a227
--breakpoint-md = 48rem
matches md: true
```

The page used @theme static so that both variables are emitted even though no utility references --breakpoint-md directly (breakpoint values are written into the media queries).

NOTE

Only variables that are emitted can be read. A theme value that no class uses is dropped from the CSS; mark the block `@theme static` (or use the value in a class) if JavaScript needs it.

13.4 Design systems

13.4.1 Design tokens and a shared theme file

Design tokens are the named decisions of a design system: colours, type sizes, spacing, radii, shadows, motion. In Tailwind 4 they are naturally a CSS file containing `@theme`. Put that file in one place (a shared package in a monorepo, or a `theme.css` imported by every app) and every project gets the same utilities. Design tools such as Figma can export tokens to this format with plugins.

THEME.CSS — CSS

```css
/* packages/lbf-theme/theme.css: the single source of truth */
@theme {
  --color-*: initial;
  --color-ink: #1c2b4a;   --color-gold: #c9a227;   --color-paper: #f7f3ea;
  --font-sans: "Poppins", sans-serif;
  --radius-card: 1.25rem;
  --shadow-card: 0 1px 2px rgb(28 43 74 / 0.06), 0 8px 24px rgb(28 43 74 / 0.08);
}
```

TWO APPS SHARING IT — CSS

```css
/* apps/website/src/app.css */
@import "tailwindcss";
@import "@lbf/theme/theme.css";

/* apps/admin/src/app.css */
@import "tailwindcss";
@import "@lbf/theme/theme.css";
@theme { --color-danger: #dc2626; }   /* app-specific additions */
```

13.4.2 Multiple themes (brands or tenants)

One code base serving several brands (white-label products, a SaaS where each customer has its own colours) keeps the **same class names** and swaps the variable values. Because utilities reference theme variables, redefining them under a selector restyles everything inside it:

APP.CSS — CSS

```css
@import "tailwindcss";

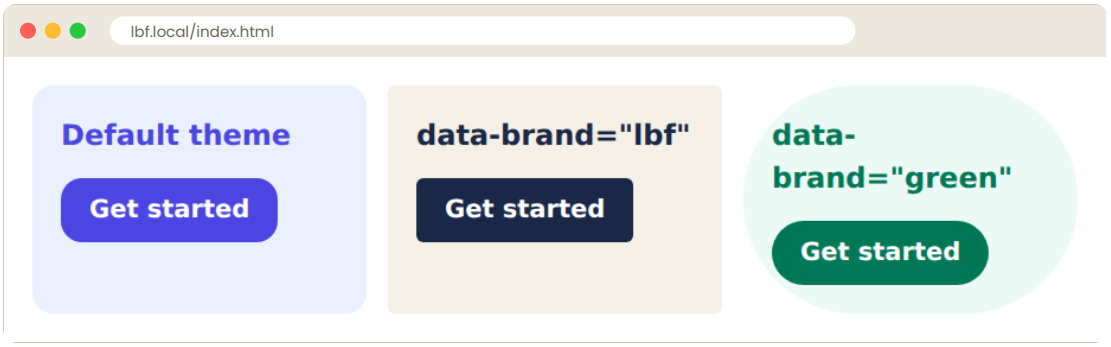
@theme {
  --color-brand: #4f46e5;
  --color-brand-soft: #eef2ff;
  --radius-ui: 0.75rem;
}

[data-brand="lbf"] {
  --color-brand: #1c2b4a;
  --color-brand-soft: #f7f3ea;
  --radius-ui: 0.25rem;
}
[data-brand="green"] {
  --color-brand: #047857;
  --color-brand-soft: #ecfdf5;
  --radius-ui: 9999px;
}
```

INDEX.HTML

HTML

```
<div class="grid grid-cols-3 gap-3">
  <div  class="rounded-(--radius-ui) bg-brand-soft p-4">
    <p class="font-semibold text-brand">Default theme</p>
    <button class="mt-3 rounded-(--radius-ui) bg-brand px-4 py-2 text-sm font-semibold text-white">Get
started</button>
  </div>
  <div data-brand="lbf" class="rounded-(--radius-ui) bg-brand-soft p-4">
    <p class="font-semibold text-brand">data-brand="lbf"</p>
    <button class="mt-3 rounded-(--radius-ui) bg-brand px-4 py-2 text-sm font-semibold text-white">Get
started</button>
  </div>
  <div data-brand="green" class="rounded-(--radius-ui) bg-brand-soft p-4">
    <p class="font-semibold text-brand">data-brand="green"</p>
    <button class="mt-3 rounded-(--radius-ui) bg-brand px-4 py-2 text-sm font-semibold text-white">Get
started</button>
  </div>
</div>
```



IDENTICAL MARKUP, THREE BRANDS

13.5 Legacy JavaScript configuration

13.5.1 tailwind.config.js in older projects and @config

Tailwind 3 was configured in a JavaScript file. Many existing projects still have one, and v4 can load it for compatibility with the `@config` directive, so a project can upgrade the engine first and migrate the configuration later. The automatic upgrade tool (`npx @tailwindcss/upgrade`, Module 16) converts most configurations to `@theme` for you.

A V3 CONFIGURATION FILE

javascript

```
// tailwind.config.js (v3 style)
module.exports = {
  content: ['./src/**/*.{html,js,jsx}'],
  theme: {
    extend: {
      colors: { navy: { 900: '#1c2b4a' }, gold: '#c9a227' },
      fontFamily: { display: ['Lora', 'serif'] },
      borderRadius: { card: '1.25rem' },
    },
  },
  plugins: [require('@tailwindcss/typography')],
};
```

USING IT FROM V4

CSS

```css
@import "tailwindcss";
@config "../tailwind.config.js";   /* load the legacy file */
```

THE SAME CONFIGURATION, CSS-FIRST

CSS

```css
@import "tailwindcss";
@plugin "@tailwindcss/typography";

@theme {
  --color-navy-900: #1c2b4a;
  --color-gold: #c9a227;
  --font-display: "Lora", serif;
  --radius-card: 1.25rem;
}
```

v3 config key	v4 equivalent
`content: [...]`	automatic detection; `@source` for extra paths (Module 14)
`theme.extend.colors`	`--color-*` in `@theme`
`theme.colors` (replace)	`--color-*: initial;` then your colours
`theme.screens`	`--breakpoint-*`
`darkMode: 'class'`	`@custom-variant dark (&:where(.dark, .dark *));`
`plugins: [require(...)]`	`@plugin "..."`
`safelist`	`@source inline("...")`
`prefix: 'tw-'`	`@import "tailwindcss" prefix(tw);` (classes become `tw:flex`)

NOTE

Options removed in v4 such as `corePlugins`, `safelist` and `separator` are not supported through `@config`. A configuration file that relies on them must be migrated to CSS.

PRACTICE CHECKPOINT

Build a **Learn Being Forward brand theme** with custom colours, fonts, radius, shadows and breakpoints, then restyle the UI kit with it.

LBF-THEME.CSS: SAMPLE SOLUTION

CSS

```css
@import "tailwindcss";

@font-face { font-family: "Poppins"; src: url("fonts/Poppins-Regular.ttf"); font-weight: 400; }
@font-face { font-family: "Poppins"; src: url("fonts/Poppins-Bold.ttf"); font-weight: 700; }
@font-face { font-family: "Lora"; src: url("fonts/Lora-Variable.ttf"); font-weight: 400 700; }

@theme {
  /* colours: brand scales plus neutrals, defaults removed */
  --color-*: initial;
  --color-white: #fff;
  --color-ink: #1c2b4a;
  --color-ink-700: #2d4170;
  --color-ink-soft: #e8ecf5;
  --color-gold: #c9a227;
  --color-gold-soft: #f6eccb;
  --color-paper: #f7f3ea;
  --color-line: #e2dccd;
  --color-muted: #6a6458;
  --color-danger: #b42318;

  /* type */
  --font-sans: "Poppins", ui-sans-serif, system-ui, sans-serif;
  --font-display: "Lora", ui-serif, Georgia, serif;

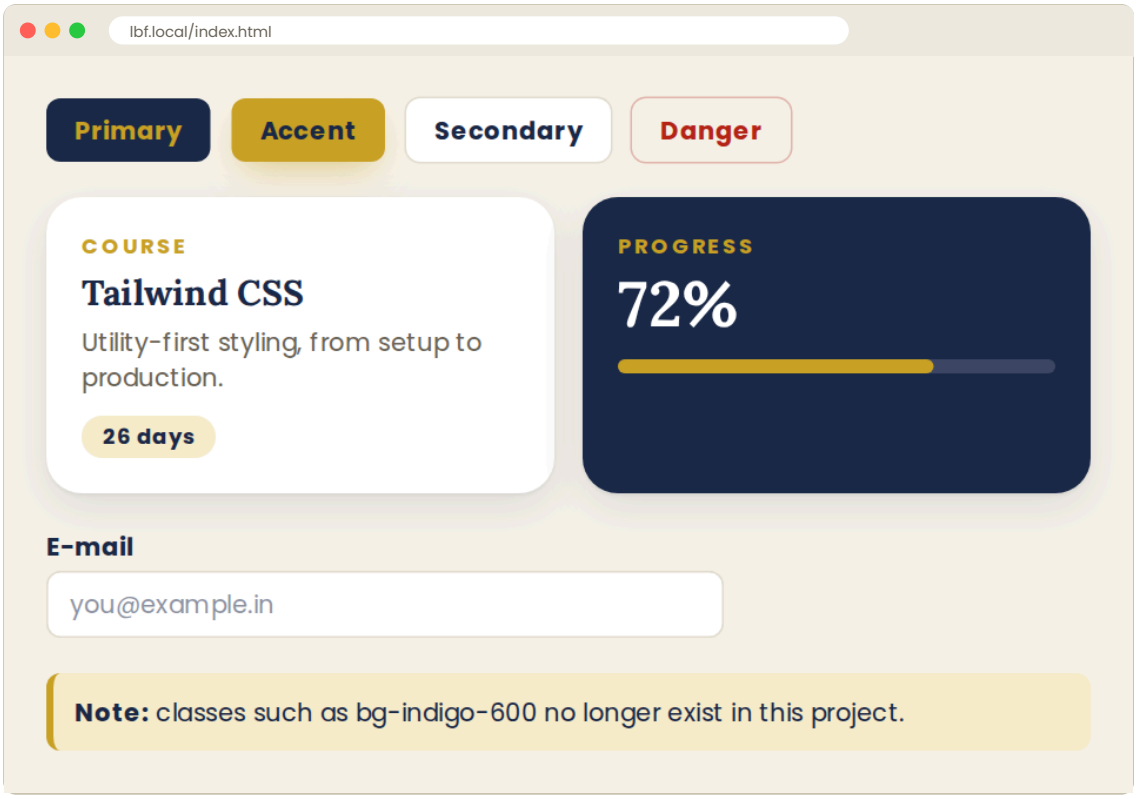
  /* shape and depth */
  --radius-ui: 0.5rem;
  --radius-card: 1.25rem;
  --shadow-card: 0 1px 2px rgb(28 43 74 / 0.06), 0 8px 24px rgb(28 43 74 / 0.10);
  --shadow-gold: 0 8px 20px -6px rgb(201 162 39 / 0.55);

  /* breakpoints */
  --breakpoint-xs: 30rem;
  --breakpoint-3xl: 120rem;
}
```

UI-KIT.HTML: RESTYLED WITH THE BRAND THEME

HTML

```
<div class="space-y-5 bg-paper p-6 text-ink">
  <div class="flex flex-wrap gap-3">
    <button class="rounded-ui bg-ink px-4 py-2 text-sm font-bold text-gold hover:bg-ink-
700">Primary</button>
    <button class="rounded-ui bg-gold px-4 py-2 text-sm font-bold text-ink shadow-
gold">Accent</button>
    <button class="rounded-ui bg-white px-4 py-2 text-sm font-bold ring-1 ring-
line">Secondary</button>
    <button class="rounded-ui px-4 py-2 text-sm font-bold text-danger ring-1 ring-
danger/30">Danger</button>
  </div>
  <div class="grid gap-4 xs:grid-cols-2">
    <article class="rounded-card bg-white p-5 shadow-card">
      <p class="text-xs font-bold uppercase tracking-widest text-gold">Course</p>
      <h3 class="mt-1 font-display text-xl font-bold">Tailwind CSS</h3>
      <p class="mt-1 text-sm text-muted">Utility-first styling, from setup to production.</p>
      <span class="mt-3 inline-block rounded-full bg-gold-soft px-3 py-1 text-xs font-bold text-
ink">26 days</span>
    </article>
    <article class="rounded-card bg-ink p-5 text-white shadow-card">
      <p class="text-xs font-bold uppercase tracking-widest text-gold">Progress</p>
      <p class="mt-1 font-display text-4xl font-bold">72%</p>
      <div class="mt-3 h-2 rounded-full bg-white/15"><div class="h-2 w-[72%] rounded-full bg-gold">
</div></div>
    </article>
  </div>
  <label class="block max-w-sm text-sm">
    <span class="font-bold">E-mail</span>
    <input type="email" placeholder="you@example.in" class="mt-1 block w-full rounded-ui border
border-line bg-white px-3 py-2 focus:border-ink focus:outline-hidden focus:ring-4 focus:ring-gold/30">
  </label>
  <div class="rounded-ui border-l-4 border-gold bg-gold-soft p-3 text-sm"><b>Note:</b> classes such as
bg-indigo-600 no longer exist in this project.</div>
</div>
```



THE UI KIT WITH THE LEARN BEING FORWARD THEME

`@theme` variables are CSS variables that also generate utilities by namespace; extend with new names, override by redefining, remove with `--namespace-*: initial`, use `@theme inline` when a value references another variable, read tokens in JavaScript with `getComputedStyle`, and share one theme file across projects or swap variable values per brand.

MODULE 14 · ADVANCED

Directives, custom utilities, variants and plugins

Minimum duration **2 days** | Level **Advanced**

This module covers extending Tailwind itself. It explains every directive and function in a v4 stylesheet (@import, @theme, @source, @utility, @variant, @custom-variant, @apply, @reference, --alpha() and --spacing()); controlling which files are scanned and safelisting classes; writing simple and functional custom utilities; creating custom variants for themes and states; loading official and third-party plugins and the outline of writing one; and how Tailwind's cascade layers work together with your own native CSS. All examples are compiled with Tailwind v4.1.13.

IN THIS MODULE

14.1 Directives and functions
14.2 Content sources
14.3 Custom utilities
14.4 Custom variants
14.5 Plugins
14.6 Native CSS with Tailwind

14.1 Directives and functions

A Tailwind 4 stylesheet is plain CSS plus a small set of at-rules (directives) and functions that the compiler understands. The table is a map of this module:

Directive / function	Purpose	Section
`@import "tailwindcss"`	load the framework (theme, Preflight, utilities)	14.1.1
`@theme`	define design tokens that generate utilities	Module 13
`@source`	add, exclude or inline content sources	14.2
`@utility`	define a custom utility that works with variants	14.1.2, 14.3
`@custom-variant`	define a custom variant	14.1.3, 14.4
`@variant`	apply a variant inside your own CSS	14.1.3
`@apply`	inline utilities into a rule	14.1.4
`@reference`	import theme and utilities for reference without emitting CSS	14.1.5
`@plugin`	load a JavaScript plugin	14.5
`@config`	load a legacy JavaScript configuration	Module 13
`--alpha()`, `--spacing()`	functions for colour opacity and the spacing scale	14.1.6

14.1.1 @import, @theme, @source

`@import "tailwindcss";` is shorthand for three imports, one per layer. Knowing this lets you leave out a part, for example Preflight when adding Tailwind to an existing site whose base styles must not change:

THE EXPANSION CSS

```css
/* what @import "tailwindcss" expands to */
@layer theme, base, components, utilities;
@import "tailwindcss/theme.css" layer(theme);
@import "tailwindcss/preflight.css" layer(base);
@import "tailwindcss/utilities.css" layer(utilities);
```

LEAVING OUT PREFLIGHT CSS

```css
/* Tailwind without Preflight, e.g. inside an existing site */
@layer theme, base, components, utilities;
@import "tailwindcss/theme.css" layer(theme);
@import "tailwindcss/utilities.css" layer(utilities);
```

`@import` also takes options: `@import "tailwindcss" prefix(tw);` prefixes every class (`tw:flex`, `tw:hover:bg-sky-600`) to avoid clashes with another CSS framework, and `@import "tailwindcss" important;` marks every utility `!important` (a last resort when fighting legacy CSS). `@theme` was covered in Module 13 and `@source` follows in Section 14.2.

COMPILED CSS

```css
.tw\:flex {
  display: flex;
}
.tw\:hover\:bg-sky-600 {
  &:hover {
    @media (hover: hover) {
      background-color: var(--tw-color-sky-600);
    }
  }
}
```

14.1.2 @utility for custom utilities

`@utility` registers a new utility class. Unlike a normal CSS class, it is placed in the `utilities` layer, sorted with the other utilities, generated only when used, and it works with **every variant**: `hover:`, `md:`, `dark:`, `group-hover:`. That makes it the right tool whenever you want "one more utility" rather than a component.

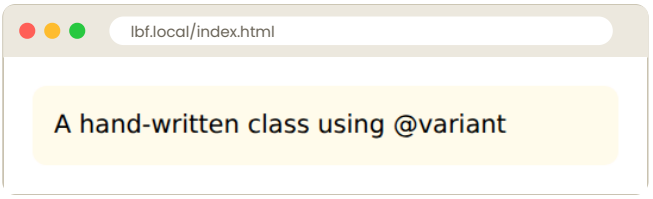
COMPILED CSS

```css
.content-auto {
  content-visibility: auto;
  contain-intrinsic-size: auto 500px;
}
.lg\:content-auto {
  @media (width >= 64rem) {
    content-visibility: auto;
    contain-intrinsic-size: auto 500px;
  }
}
```

14.1.3 @variant and @custom-variant

`@custom-variant` creates a new variant (Section 14.4 and Module 10). `@variant` does the opposite job: it lets you use an **existing** variant inside your own CSS, so hand-written rules can react to dark mode,

breakpoints or states using the same definitions as the utilities:



HOW THE BROWSER DISPLAYS IT

USING VARIANTS IN CUSTOM CSS CSS

```css
@custom-variant dark (&:where(.dark, .dark *));

.prose-note {
  background: var(--color-amber-50);
  @variant dark {
    background: var(--color-amber-950);
  }
  @variant md {
    padding: --spacing(6);
  }
}
```

COMPILED CSS

```css
.prose-note {
  background: var(--color-amber-50);
  &:where(.dark, .dark *) {
    background: var(--color-amber-950);
  }
  @media (width >= 48rem) {
    padding: calc(var(--spacing) * 6);
  }
}
```

14.1.4 @apply

`@apply` inlines utilities into a rule (Module 12 covered when to use it). In v4 it accepts any utility, including custom ones defined with `@utility` and variants (`@apply hover:bg-indigo-500 md:px-6`). It cannot be used on a class that is itself a component class: apply utilities, not other `@layer components` classes.

STYLING MARKDOWN OUTPUT YOU DO NOT CONTROL CSS

```css
@utility content-auto { content-visibility: auto; }

.markdown h2 { @apply mt-8 text-2xl font-bold text-slate-900 content-auto; }
.markdown a  { @apply font-medium text-indigo-600 underline-offset-2 hover:underline; }
```

COMPILED CSS

```
.markdown h2 {
  margin-top: calc(var(--spacing) * 8);
  font-size: var(--text-2xl);
  line-height: var(--tw-leading, var(--text-2xl--line-height));
  --tw-font-weight: var(--font-weight-bold);
  font-weight: var(--font-weight-bold);
  color: var(--color-slate-900);
  content-visibility: auto;
}
.markdown a {
  --tw-font-weight: var(--font-weight-medium);
  font-weight: var(--font-weight-medium);
  color: var(--color-indigo-600);
  text-underline-offset: 2px;
  &:hover {
    @media (hover: hover) {
      text-decoration-line: underline;
    }
  }
}
```

14.1.5 @reference for using Tailwind inside component styles

Framework components often have their own stylesheet (a Vue or Svelte `<style>` block, an Angular component `styleUrl`, a CSS module). Each is compiled **separately**, so it does not know your theme, and `@apply bg-brand` fails with "Cannot apply unknown utility class". Importing the main stylesheet would duplicate all of Tailwind into every component. `@reference` solves this: it loads the theme, custom utilities and variants **for reference only**, emitting nothing.

HEADER.VUE — HTML

```
<template>
  <h1 class="title">Hello</h1>
</template>

<style scoped>
  @reference "../../app.css";   /* your main CSS file: brings in @theme, @utility, @custom-variant */

  .title {
    @apply text-3xl font-bold text-brand;
  }
  .title:hover {
    color: var(--color-gold);      /* theme variables also work */
  }
</style>
```

TIP

If a component only needs theme variables (not `@apply`), skip `@reference` entirely and use `var(--color-brand)`: the variables are global CSS custom properties.

14.1.6 --alpha() and --spacing() functions

Two functions bring the shorthand of utilities into custom CSS. `--alpha(colour / opacity)` adjusts a colour's opacity (compiled to `color-mix()`), and `--spacing(n)` multiplies the spacing unit (compiled to `calc(var(--spacing) * n)`). Both also work inside arbitrary values: `p-[--spacing(5)]`.



HOW THE BROWSER DISPLAYS IT

FUNCTIONS IN CUSTOM CSS CSS

```css
.badge {
  background: --alpha(var(--color-indigo-600) / 15%);
  color: var(--color-indigo-700);
  padding: --spacing(1) --spacing(3);
  border-radius: --spacing(10);
}
```

COMPILED CSS

```css
.badge {
  background: color-mix(in srgb, oklch(51.1% 0.262 276.966) 15%, transparent);
  @supports (color: color-mix(in lab, red, red)) {
    background: color-mix(in oklab, var(--color-indigo-600) 15%, transparent);
  }
  color: var(--color-indigo-700);
  padding: calc(var(--spacing) * 1) calc(var(--spacing) * 3);
  border-radius: calc(var(--spacing) * 10);
}
```

14.2 Content sources

14.2.1 Registering extra sources with @source

Tailwind 4 finds your templates automatically: it scans the project from the current directory, skipping files listed in `.gitignore`, binary files, CSS files and lock files. Use `@source` when classes live somewhere it does not look, typically a UI library inside `node_modules` or a sibling package in a monorepo. Paths are relative to the CSS file.

ADDING AND EXCLUDING SOURCES CSS

```css
@import "tailwindcss";

@source "../node_modules/@lbf/ui";          /* a component library that ships Tailwind classes */
@source "../../packages/shared/src";         /* a sibling package in a monorepo */
@source not "../src/legacy";                 /* exclude a folder (v4.1+) */
```

EXPLICIT SOURCES ONLY CSS

```css
@import "tailwindcss" source(none);   /* turn off automatic detection */
@source "../src";                        /* scan only these folders */
@source "../content";
```

14.2.2 Safelisting classes with @source inline()

Classes that never appear in a scanned file (class names stored in a database or a CMS, for example a colour chosen per course) will not be generated. `@source inline()` forces them into the output. It supports brace expansion to generate many classes compactly, including variants; `@source not inline()` blocks classes from ever being generated.

GENERATED SELECTORS (FROM THE COMPILED CSS)

```
12 classes generated from an empty page:
  .bg-amber-100, .bg-amber-600, .bg-emerald-100
  .bg-emerald-600, .bg-indigo-100, .bg-indigo-600
  .hover:bg-amber-100, .hover:bg-amber-600, .hover:bg-emerald-100
  .hover:bg-emerald-600, .hover:bg-indigo-100, .hover:bg-indigo-600
```

That one line produced twelve classes: three colours, two shades, with and without `hover:`, although the scanned HTML contained none of them. Use safelisting sparingly, because every inline class is always in the CSS; mapping database values to full class names in code (Module 15) is usually better.

14.3 Custom utilities

14.3.1 Simple and functional utilities

A **simple** utility has a fixed name and fixed declarations, like `content-auto` above. A **functional** utility takes a value, written with a `-*` suffix in its name; the special `--value()` function says where the value comes from: a theme namespace, a bare number, an arbitrary value in brackets, or a list of keywords. `--modifier()` reads an optional slash modifier.

THREE FUNCTIONAL UTILITIES

CSS

```css
@theme {
  --tab-size-github: 8;
}

/* tab-2, tab-4, tab-github, tab-[12] */
@utility tab-* {
  tab-size: --value(--tab-size-*, integer, [integer]);
}

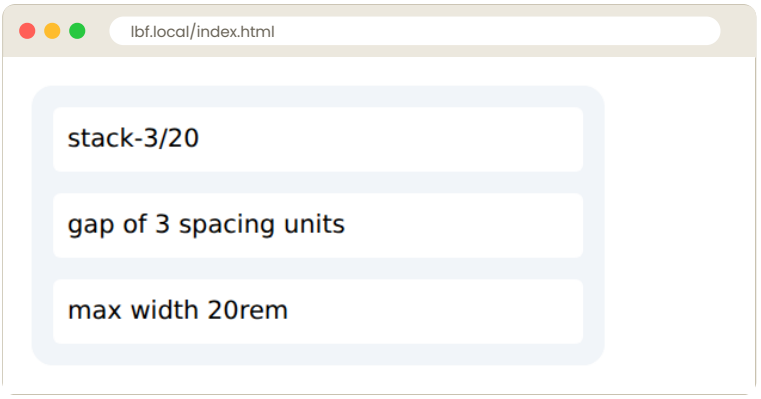
/* line-clamp style: scrollbar-thin, scrollbar-none */
@utility scrollbar-* {
  scrollbar-width: --value("thin", "none", "auto");
}

/* stack-4 gives vertical gap in spacing units; stack-4/12 adds a max width in rem */
@utility stack-* {
  display: flex;
  flex-direction: column;
  gap: --spacing(--value(integer));
  max-width: calc(--modifier(integer) * 1rem);
}
```

COMPILED CSS

```
.stack-6\/30 {
  display: flex;
  flex-direction: column;
  gap: calc(var(--spacing) * 6);
  max-width: calc(30 * 1rem);
}
.stack-4 {
  display: flex;
  flex-direction: column;
  gap: calc(var(--spacing) * 4);
}
.scrollbar-none {
  scrollbar-width: none;
}
.tab-4 {
  tab-size: 4;
}
.tab-\[12\] {
  tab-size: 12;
}
.tab-github {
  tab-size: var(--tab-size-github);
}
.md\:scrollbar-thin {
  @media (width >= 48rem) {
    scrollbar-width: thin;
  }
}
```

Each class was generated from the value in its name. Values that do not match any accepted type, such as `tab-wide`, produce no CSS, just like built-in utilities. Functional utilities let a design system expose a whole family of classes with one definition.



A CUSTOM FUNCTIONAL UTILITY IN USE

14.4 Custom variants

14.4.1 Theme variants, state variants

`@custom-variant name (selector);` creates a variant from a selector, where `&` is the element. For more complex cases (media queries, several selectors) use the block form with `@slot` marking where the utility's declarations go.

THREE CUSTOM VARIANTS CSS

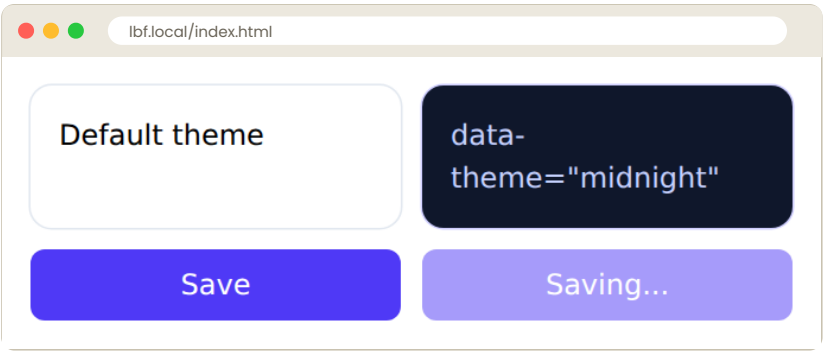
```css
/* a theme variant: anything inside data-theme="midnight" */
@custom-variant theme-midnight (&:where([data-theme=midnight], [data-theme=midnight] *));

/* a state variant from an attribute set by JavaScript */
@custom-variant loading (&[aria-busy=true]);

/* block form: hover only with a mouse AND outside reduced motion */
@custom-variant fancy-hover {
  @media (hover: hover) and (prefers-reduced-motion: no-preference) {
    &:hover {
      @slot;
    }
  }
}
```

COMPILED CSS

```css
.theme-midnight\:bg-slate-900 {
  &:where([data-theme=midnight], [data-theme=midnight] *) {
    background-color: var(--color-slate-900);
  }
}
.loading\:opacity-50 {
  &[aria-busy=true] {
    opacity: 50%;
  }
}
.fancy-hover\:-translate-y-0\.5 {
  @media (hover: hover) and (prefers-reduced-motion: no-preference) {
    &:hover {
      --tw-translate-y: calc(var(--spacing) * -0.5);
      translate: var(--tw-translate-x) var(--tw-translate-y);
    }
  }
}
```



THEME-MIDNIGHT: AND LOADING: IN USE

14.5 Plugins

14.5.1 Official plugins: Typography, Forms

Tailwind Labs maintains a small set of official plugins: **@tailwindcss/typography** (the `prose` classes, Module 4) and **@tailwindcss/forms** (form resets, Module 12). Container queries and aspect ratio, which were plugins in v3, are now built into the core. The standalone CLI bundles the official plugins, so they work without npm.

14.5.2 Loading plugins with @plugin

In v4, JavaScript plugins are loaded from CSS with `@plugin`, by package name or relative path. Options are passed in a block.

LOADING PLUGINS CSS

```css
@import "tailwindcss";

@plugin "@tailwindcss/typography";
@plugin "@tailwindcss/forms" {
  strategy: "class";          /* only .form-input, .form-select ... */
}
@plugin "daisyui";            /* a third-party plugin (Module 15) */
@plugin "./plugins/scrollbar.js";
```

> **NOTE**
> Many things that needed a plugin in v3 are now plain CSS in v4: custom utilities (`@utility`), variants (`@custom-variant`) and theme values (`@theme`). Reach for a JavaScript plugin only when you need logic, such as generating utilities from a data file.

14.5.3 Writing a simple plugin (overview)

A plugin is a JavaScript function from `tailwindcss/plugin` that receives helper functions: `addUtilities` and `matchUtilities` (static and value-based utilities), `addComponents`, `addBase`, `addVariant` and `theme()` to read theme values. The v3 plugin API still works in v4.

A PLUGIN WITH A STATIC AND A VALUE-BASED UTILITY (REPRESENTATIVE) javascript

```javascript
// plugins/scrollbar.js
import plugin from 'tailwindcss/plugin';

export default plugin(function ({ addUtilities, matchUtilities, theme }) {
  addUtilities({
    '.scrollbar-hidden': {
      'scrollbar-width': 'none',
      '&::-webkit-scrollbar': { display: 'none' },
    },
  });

  // scrollbar-thumb-indigo-500, scrollbar-thumb-[#c9a227] ...
  matchUtilities(
    { 'scrollbar-thumb': (value) => ({ 'scrollbar-color': `${value} transparent` }) },
    { values: theme('colors'), type: 'color' },
  );
});
```

For a single project, the same result is simpler in CSS: `@utility scrollbar-hidden { ... }` and `@utility scrollbar-thumb-* { scrollbar-color: --value(--color-*, [color]) transparent; }`. Write a JavaScript plugin when the utilities must be shared as an npm package or generated programmatically.

14.6 Native CSS with Tailwind

14.6.1 Cascade layers in Tailwind (theme, base, components, utilities)

Tailwind 4 is built on native CSS **cascade layers**. The order is declared once, `@layer theme, base, components, utilities;`, and a rule in a later layer beats any rule in an earlier layer, whatever the

specificity. Two consequences matter in practice:

- Your own CSS written **outside** any layer beats every Tailwind layer, because unlayered styles have the highest priority. Put custom CSS in `@layer base` or `@layer components` if utilities should still be able to override it.
- Third-party CSS (a date picker, a map widget) can be imported into a layer to control its priority: `@import "datepicker.css" layer(components);`.

APP.CSS CSS

```css
@import "tailwindcss";

.unlayered { color: crimson; }

@layer components {
  .card-title { color: navy; font-weight: 700; }
}
```

INDEX.HTML HTML

```html
<p class="card-title text-emerald-600">@layer components .card-title + text-emerald-600: the utility wins (green)</p>
<p class="unlayered mt-2 text-emerald-600">unlayered .unlayered + text-emerald-600: unlayered CSS wins (red)</p>
```



LAYER ORDER DECIDES, NOT SPECIFICITY OR SOURCE ORDER

14.6.2 Writing base styles

Base styles are defaults for plain elements: headings, links, body text, focus rings. Add them in `@layer base` so that utilities on any element can still override them. Use theme variables and `--spacing()` to stay on the design system.

APP.CSS CSS

```css
@import "tailwindcss";

@layer base {
  h1 { font-size: var(--text-3xl); font-weight: 800; line-height: 1.15; }
  h2 { font-size: var(--text-xl); font-weight: 700; margin-top: --spacing(6); }
  a  { color: var(--color-indigo-700); text-decoration: underline; text-underline-offset: 3px; }
  :focus-visible { outline: 2px solid var(--color-indigo-600); outline-offset: 2px; }
  ::selection { background: var(--color-amber-200); }
}
```

INDEX.HTML HTML

```html
<h1>Plain h1, styled by @layer base</h1>
<h2>Plain h2</h2>
<p class="mt-2">A paragraph with <a href="#">a default link</a> and <a href="#" class="text-rose-600 no-underline">a link overridden by utilities</a>.</p>
```



BASE STYLES FOR UNCLASSED ELEMENTS, STILL OVERRIDABLE BY UTILITIES

PRACTICE CHECKPOINT

Create a custom **content-auto** utility, a **theme-midnight** custom variant, and a **scrollbar-hiding** utility, then use them on a page.

APP.CSS: SAMPLE SOLUTION CSS

```css
@import "tailwindcss";

@utility content-auto {
  content-visibility: auto;
  contain-intrinsic-size: auto 400px;
}

@utility scrollbar-hidden {
  scrollbar-width: none;
  &::-webkit-scrollbar { display: none; }
}

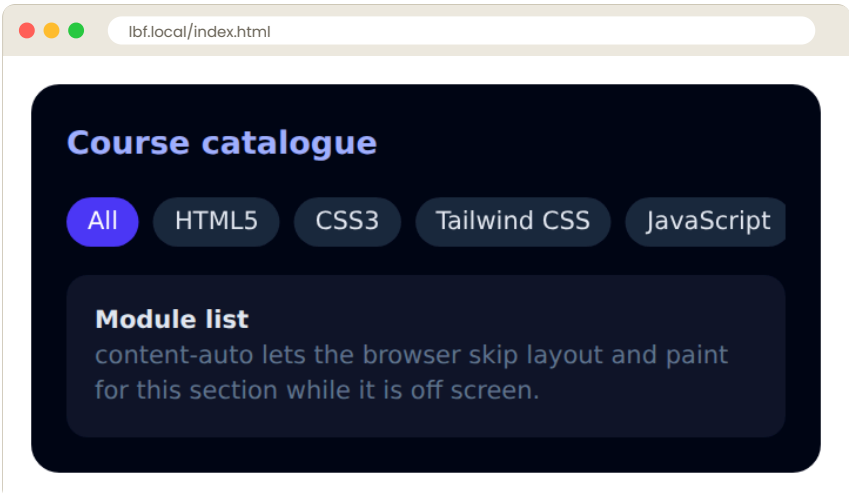
@custom-variant theme-midnight (&:where([data-theme=midnight], [data-theme=midnight] *));
```

CATALOGUE.HTML: SAMPLE SOLUTION HTML

```
<div data-theme="midnight" class="space-y-4 rounded-2xl bg-white p-5 theme-midnight:bg-slate-950
theme-midnight:text-slate-200">
  <h2 class="text-lg font-bold theme-midnight:text-indigo-300">Course catalogue</h2>

  <!-- horizontally scrolling chips without a visible scrollbar -->
  <div class="scrollbar-hidden flex gap-2 overflow-x-auto text-sm">
    <span class="shrink-0 rounded-full bg-indigo-600 px-3 py-1 text-white">All</span>
    <span class="shrink-0 rounded-full bg-slate-100 px-3 py-1 theme-midnight:bg-slate-
800">HTML5</span>
    <span class="shrink-0 rounded-full bg-slate-100 px-3 py-1 theme-midnight:bg-slate-800">CSS3</span>
    <span class="shrink-0 rounded-full bg-slate-100 px-3 py-1 theme-midnight:bg-slate-800">Tailwind
CSS</span>
    <span class="shrink-0 rounded-full bg-slate-100 px-3 py-1 theme-midnight:bg-slate-
800">JavaScript</span>
    <span class="shrink-0 rounded-full bg-slate-100 px-3 py-1 theme-midnight:bg-slate-
800">TypeScript</span>
    <span class="shrink-0 rounded-full bg-slate-100 px-3 py-1 theme-midnight:bg-slate-
800">React</span>
    <span class="shrink-0 rounded-full bg-slate-100 px-3 py-1 theme-midnight:bg-slate-
800">Angular</span>
  </div>

  <!-- long sections skip rendering work until scrolled near -->
  <section class="content-auto rounded-xl bg-slate-50 p-4 text-sm theme-midnight:bg-slate-900">
    <h3 class="font-semibold">Module list</h3>
    <p class="text-slate-500">content-auto lets the browser skip layout and paint for this section
while it is off screen.</p>
  </section>
</div>
```



THE PAGE IN THE MIDNIGHT THEME: CHIPS SCROLL SIDEWAYS WITH NO SCROLLBAR

COMPILED CSS

```
.scrollbar-hidden {
  scrollbar-width: none;
  &::-webkit-scrollbar {
    display: none;
  }
}
.content-auto {
  content-visibility: auto;
  contain-intrinsic-size: auto 400px;
}
.theme-midnight\:bg-slate-950 {
  &:where([data-theme=midnight], [data-theme=midnight] *) {
    background-color: var(--color-slate-950);
  }
}
```

Use `@utility` for new utilities (simple or functional with `--value()`), `@custom-variant` for new variants, `@variant` to use variants in your own CSS, `@reference` in component stylesheets, `@source` to control scanning and safelisting, `@plugin` for JavaScript plugins, and cascade layers to decide how your CSS and Tailwind's interact.

MODULE 15 · ADVANCED

Tailwind with frameworks and the ecosystem

Minimum duration **2 days** | Level **Advanced**

Most Tailwind code lives inside JavaScript frameworks; this module covers the patterns and libraries used in professional projects. It shows how Tailwind is set up and used in React and Next.js, Vue and Nuxt, Angular and Svelte; how to build class names dynamically and safely with clsx, tailwind-merge and cva, and why string-concatenated class names break; the main component libraries (shadcn/ui, Headless UI, Radix UI, daisyUI, Flowbite and Tailwind Plus); and the icon sets designed for Tailwind. Framework code in this module is representative: it shows the official setup and patterns rather than output captured from a running framework. The class-merging behaviour of clsx, tailwind-merge and cva is reproduced exactly as those libraries document it.

IN THIS MODULE

15.1 Tailwind with frameworks
15.2 Dynamic classes
15.3 Component libraries
15.4 Icons and assets

15.1 Tailwind with frameworks

In every framework the integration has the same two parts: a **build plugin** that compiles your CSS (the Vite plugin `@tailwindcss/vite` or the PostCSS plugin `@tailwindcss/postcss`), and a **CSS file** with `@import "tailwindcss";` that is loaded once by the app. After that, classes are written in the framework's templates. Tailwind's automatic source detection finds `.jsx`, `.tsx`, `.vue`, `.svelte` and `.html` files without configuration.

Framework	Build integration	Class attribute	Where the CSS is imported
React (Vite)	`@tailwindcss/vite`	`className`	`src/index.css` in `main.jsx`
Next.js	`@tailwindcss/postcss`	`className`	`app/globals.css` in `app/layout.tsx`
Vue (Vite)	`@tailwindcss/vite`	`class`, `:class`	`src/style.css` in `main.js`
Nuxt	`@tailwindcss/vite` in `nuxt.config`	`class`, `:class`	`css: ['~/assets/css/main.css']`
Angular	`@tailwindcss/postcss` (`.postcssrc.json`)	`class`, `[class]`, `[ngClass]`	`src/styles.css`
SvelteKit	`@tailwindcss/vite`	`class`, `class:name`	`src/app.css` in `+layout.svelte`

15.1.1 React and Next.js

REACT WITH VITE: INSTALL shell

```
npm create vite@latest my-app -- --template react
cd my-app
npm install tailwindcss @tailwindcss/vite
```

VITE.CONFIG.JS javascript

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

SRC/INDEX.CSS (IMPORTED IN MAIN.JSX) CSS

```
/* src/index.css */
@import "tailwindcss";
```

A REACT COMPONENT (CLASSNAME, CONDITIONAL CLASSES) javascript

```
export default function CourseCard({ title, days, featured }) {
  return (
    <article className={`rounded-xl p-5 ring-1 ${featured ? 'bg-indigo-600 text-white ring-indigo-600'
: 'bg-white ring-slate-200'}`}>
      <h3 className="font-semibold">{title}</h3>
      <p className={featured ? 'text-indigo-100' : 'text-slate-500'}>{days} days</p>
    </article>
  );
}
```

Next.js (App Router) uses the PostCSS plugin. `create-next-app` offers Tailwind during setup and generates this for you:

NEXT.JS SETUP javascript

```
npm install tailwindcss @tailwindcss/postcss postcss

// postcss.config.mjs
export default { plugins: { '@tailwindcss/postcss': {} } };

/* app/globals.css */
@import "tailwindcss";

// app/layout.tsx
import './globals.css';
```

Next.js's font loader (`next/font`) exposes a font as a CSS variable on `<html>` or `<body>`; connect it to the theme with `@theme inline { --font-sans: var(--font-inter); }` (Module 13). For dark mode with a toggle, the `next-themes` package implements the localStorage and flash-prevention logic from Module 10.

15.1.2 Vue and Nuxt

VUE WITH VITE shell

```shell
npm create vue@latest my-app
cd my-app
npm install tailwindcss @tailwindcss/vite
# add tailwindcss() to plugins in vite.config.js, then in src/style.css:
#   @import "tailwindcss";
```

COURSECARD.VUE HTML

```HTML
<script setup>
defineProps({ title: String, days: Number, featured: Boolean });
</script>

<template>
  <article
    class="rounded-xl p-5 ring-1"
    :class="featured ? 'bg-indigo-600 text-white ring-indigo-600' : 'bg-white ring-slate-200'">
    <h3 class="font-semibold">{{ title }}</h3>
    <p :class="{ 'text-indigo-100': featured, 'text-slate-500': !featured }">{{ days }} days</p>
  </article>
</template>
```

Vue merges a static `class` with a bound `:class`, which accepts a string, an array or an object of `{ class: condition }`. Remember `@reference` (Module 14) if a component's `<style>` block uses `@apply`.

NUXT javascript

```javascript
// nuxt.config.ts
import tailwindcss from '@tailwindcss/vite';

export default defineNuxtConfig({
  css: ['~/assets/css/main.css'],     // contains @import "tailwindcss";
  vite: { plugins: [tailwindcss()] },
});
```

15.1.3 Angular

ANGULAR SETUP shell

```shell
ng new my-app --style css
cd my-app
npm install tailwindcss @tailwindcss/postcss postcss

# .postcssrc.json
{ "plugins": { "@tailwindcss/postcss": {} } }

# src/styles.css
@import "tailwindcss";
```

AN ANGULAR COMPONENT

typescript

```
@Component({
  selector: 'app-course-card',
  template: `
    <article class="rounded-xl p-5 ring-1"
             [class]="featured ? 'bg-indigo-600 text-white' : 'bg-white ring-slate-200'">
      <h3 class="font-semibold">{{ title }}</h3>
      <p [class.text-indigo-100]="featured" [class.text-slate-500]="!featured">{{ days }} days</p>
    </article>`,
})
export class CourseCardComponent {
  @Input() title = '';
  @Input() days = 0;
  @Input() featured = false;
}
```

Angular's `[class.name]="condition"` binding is handy for single classes but cannot contain characters such as `/` or `:` reliably (`[class.hover:bg-indigo-500]`); use `[ngClass]` or a string binding for those. Component `styleUrl` files that use `@apply` need `@reference`.

15.1.4 Svelte (overview)

COURSECARD.SVELTE (SVELTE 5)

HTML

```
<script>
  let { title, days, featured = false } = $props();
</script>

<article class={['rounded-xl p-5 ring-1', featured ? 'bg-indigo-600 text-white' : 'bg-white ring-
slate-200']}>
  <h3 class="font-semibold">{title}</h3>
  <p class:text-slate-500={!featured}>{days} days</p>
</article>
```

SvelteKit uses the Vite plugin like React and Vue. Svelte 5.16+ accepts arrays and objects in `class` (built-in clsx-style merging), and the `class:name={condition}` directive toggles one class.

15.2 Dynamic classes

15.2.1 Conditional classes with clsx

Template-string conditionals get unreadable quickly. **clsx** (and the similar `classnames`) builds a class string from strings, arrays and objects, dropping falsy values:

CLSX

javascript

```
import clsx from 'clsx';

clsx('rounded-lg px-4 py-2', isActive && 'bg-indigo-600 text-white', { 'opacity-50': disabled });
// isActive = true,  disabled = false  ->  'rounded-lg px-4 py-2 bg-indigo-600 text-white'
// isActive = false, disabled = true   ->  'rounded-lg px-4 py-2 opacity-50'
```

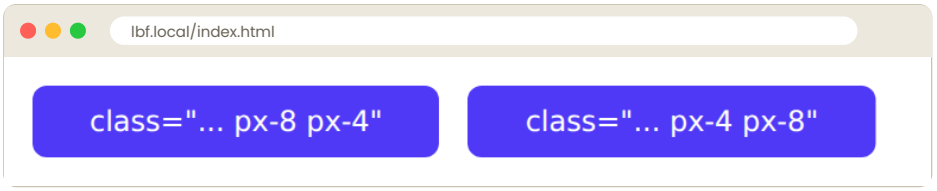
15.2.2 Merging conflicting classes with tailwind-merge

A reusable component often accepts a `className` prop so callers can adjust it. If the component has `px-4` and the caller passes `px-8`, both classes end up in the attribute, and which one wins depends on their

order in the **generated CSS**, not in the attribute. The following figure shows exactly that problem with real output:

INDEX.HTML HTML

```html
<div class="flex gap-4">
  <button class="rounded-lg bg-indigo-600 px-8 px-4 py-2 text-white">class="... px-8 px-4"</button>
  <button class="rounded-lg bg-indigo-600 px-4 px-8 py-2 text-white">class="... px-4 px-8"</button>
</div>
```



BOTH BUTTONS GET THE SAME PADDING: THE ATTRIBUTE ORDER IS IRRELEVANT; PX-8 WINS IN BOTH BECAUSE IT COMES LATER IN THE STYLESHEET

tailwind-merge understands Tailwind's class groups and removes conflicting earlier classes, so the last one passed always wins. The common `cn()` helper combines it with clsx:

LIB/UTILS.JS: THE CN() HELPER javascript

```javascript
import { clsx } from 'clsx';
import { twMerge } from 'tailwind-merge';

export function cn(...inputs) {
  return twMerge(clsx(inputs));
}

twMerge('px-4 py-2 bg-indigo-600', 'px-8');           // 'py-2 bg-indigo-600 px-8'
twMerge('p-4', 'px-2');                               // 'p-4 px-2'   (px refines p, both kept)
twMerge('px-2', 'p-4');                               // 'p-4'        (p overrides px)
twMerge('text-sm text-slate-500', 'text-rose-600');   // 'text-sm text-rose-600' (size and colour are
different groups)
twMerge('hover:bg-indigo-500', 'hover:bg-rose-500');  // 'hover:bg-rose-500'
```

ACCEPTING OVERRIDES SAFELY javascript

```javascript
function Button({ className, ...props }) {
  return <button className={cn('rounded-lg bg-indigo-600 px-4 py-2 text-white', className)} {...props}
/>;
}

<Button className="px-8 bg-rose-600">Delete</Button>   // px-8 and bg-rose-600 reliably win
```

15.2.3 Component variants with cva (class-variance-authority)

cva describes a component's styling as a base plus named **variants** with options, default values and **compound variants** (classes that apply only to a combination). It returns a function that produces the class string, keeping all style decisions in one typed object.

BUTTON.JS WITH CVA

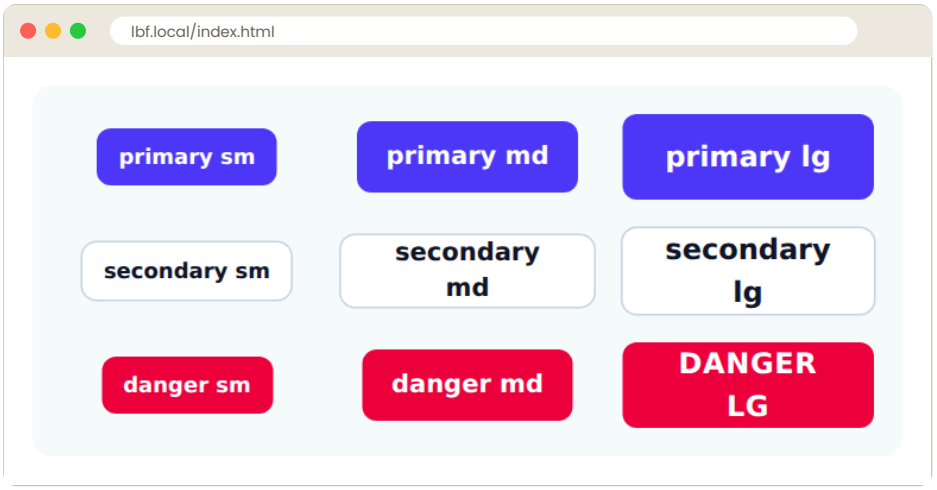
javascript

```
import { cva } from 'class-variance-authority';

export const button = cva(
  'inline-flex items-center justify-center rounded-lg font-semibold transition focus-visible:outline-2
focus-visible:outline-offset-2',
  {
    variants: {
      intent: {
        primary:   'bg-indigo-600 text-white hover:bg-indigo-500 focus-visible:outline-indigo-600',
        secondary: 'bg-white text-slate-900 ring-1 ring-slate-300 hover:bg-slate-50',
        danger:    'bg-rose-600 text-white hover:bg-rose-500',
      },
      size: {
        sm: 'h-8 px-3 text-xs',
        md: 'h-10 px-4 text-sm',
        lg: 'h-12 px-6 text-base',
      },
    },
    compoundVariants: [
      { intent: 'danger', size: 'lg', class: 'uppercase tracking-wide' },
    ],
    defaultVariants: { intent: 'primary', size: 'md' },
  },
);

button();                                  // base + primary + md
button({ intent: 'danger', size: 'lg' });   // base + danger + lg + 'uppercase tracking-wide'
```

The resulting buttons, rendered from the exact class strings cva produces for each call:



EVERY INTENT AND SIZE COMBINATION; THE COMPOUND VARIANT MAKES DANGER + LG UPPERCASE

15.2.4 Why dynamic class strings break and how to map values to full class names

Tailwind never runs your code. It reads source files as **plain text** and generates CSS for every string that looks like a class. A class assembled at runtime from pieces does not exist anywhere as text, so no CSS is generated for it:

THE BUG

javascript

```
// BROKEN: 'bg-emerald-600' never appears in the source
const color = course.color;                    // 'emerald' from the database
<span className={`bg-${color}-600 text-white`}>{course.name}</span>
```

COMPILING THE BROKEN COMPONENT WITH THE REAL CLI terminal

```
tailwindcss -i input.css -o out.css 2>/dev/null
echo "bg-emerald-600 rules generated: $(grep -c 'bg-emerald-600' out.css)"
sed -n '/@layer utilities/,$p' out.css
```

TERMINAL

```
$ tailwindcss -i input.css -o out.css 2>/dev/null
$ echo "bg-emerald-600 rules generated: $(grep -c 'bg-emerald-600' out.css)"
bg-emerald-600 rules generated: 0
$ sed -n '/@layer utilities/,$p' out.css
@layer utilities {
  .text-white {
    color: var(--color-white);
  }
}
```

THE FIX: MAP VALUES TO COMPLETE CLASS NAMES javascript

```
// FIXED: every possible class is written out in full
const badgeColors = {
  emerald: 'bg-emerald-600 text-white',
  amber:   'bg-amber-500 text-amber-950',
  rose:    'bg-rose-600 text-white',
};

<span className={badgeColors[course.color] ?? 'bg-slate-600 text-white'}>{course.name}</span>
```

Other correct options: pass the value as a CSS variable and use one static class (`style={{ '--c': course.hex }}` with `bg-(--c)`, Module 3); or, when the values come from outside the code base, safelist them with `@source inline()` (Module 14).

15.3 Component libraries

Component libraries for Tailwind fall into three families, which differ in who owns the markup and the styles:

Family	Examples	You get	You own
Copy-in components	shadcn/ui, Tailwind Plus	source code added to your project	everything: edit freely
Headless (unstyled) libraries	Headless UI, Radix UI, React Aria	behaviour and accessibility, no styles	all styling, via utilities and data/ARIA variants
Class-based plugins	daisyUI, Flowbite	component classes such as `btn`, `card`	markup; styles via theme and overrides

15.3.1 shadcn/ui

shadcn/ui is not installed as a dependency. A CLI **copies** each component's source (built on Radix UI primitives, styled with Tailwind, variants via cva, merging via `cn()`) into your project, where you own and edit it. Its theming uses semantic CSS variables (`--primary`, `--background`, `--muted`…) mapped through `@theme inline`, the pattern from Module 10.

ADDING COMPONENTS shell

```
npx shadcn@latest init          # sets up cn(), CSS variables and components.json
npx shadcn@latest add button card dialog input
```

USING SHADCN/UI COMPONENTS (REPRESENTATIVE) javascript

```
import { Button } from '@/components/ui/button';
import { Card, CardHeader, CardTitle, CardContent } from '@/components/ui/card';

export default function Pricing() {
  return (
    <Card className="w-80">
      <CardHeader><CardTitle>Mentored plan</CardTitle></CardHeader>
      <CardContent className="space-y-4">
        <p className="text-3xl font-black">&#8377;12,000</p>
        <Button className="w-full">Enrol</Button>
        <Button variant="outline" className="w-full">Talk to us</Button>
      </CardContent>
    </Card>
  );
}
```

15.3.2 Headless UI and Radix UI

Headless libraries implement the hard parts of interactive components (keyboard navigation, focus management, ARIA roles and states, closing on outside click) and leave the look entirely to you. They expose state through data attributes, which Tailwind's `data-*` variants style directly:

HEADLESS UI (REACT; THERE IS ALSO A VUE VERSION) javascript

```
import { Menu, MenuButton, MenuItem, MenuItems } from '@headlessui/react';

<Menu>
  <MenuButton className="rounded-lg bg-white px-3 py-2 ring-1 ring-slate-300 data-open:ring-indigo-
500">Options</MenuButton>
  <MenuItems anchor="bottom end" className="mt-1 w-48 rounded-xl bg-white p-1 shadow-lg ring-1 ring-
black/5">
    <MenuItem><a href="/edit" className="block rounded-lg px-3 py-2 data-focus:bg-slate-100">Edit</a>
</MenuItem>
    <MenuItem><a href="/delete" className="block rounded-lg px-3 py-2 text-rose-600 data-focus:bg-
rose-50">Delete</a></MenuItem>
  </MenuItems>
</Menu>
```

RADIX UI PRIMITIVES javascript

```
import * as Dialog from '@radix-ui/react-dialog';

<Dialog.Root>
  <Dialog.Trigger className="rounded-lg bg-indigo-600 px-4 py-2 text-white">Open</Dialog.Trigger>
  <Dialog.Portal>
    <Dialog.Overlay className="fixed inset-0 bg-black/50 data-[state=open]:animate-fade-in" />
    <Dialog.Content className="fixed left-1/2 top-1/2 w-96 -translate-1/2 rounded-2xl bg-white p-6
shadow-2xl">
      <Dialog.Title className="text-lg font-bold">Confirm</Dialog.Title>
      <Dialog.Close className="mt-4 rounded-lg px-4 py-2 ring-1 ring-slate-300">Close</Dialog.Close>
    </Dialog.Content>
  </Dialog.Portal>
</Dialog.Root>
```

15.3.3 daisyUI and Flowbite

daisyUI is a Tailwind plugin that adds semantic component classes (`btn`, `btn-primary`, `card`, `modal`, `tabs`) and many ready-made themes switched with `data-theme`. You still use utilities for layout and adjustments. It suits prototypes, admin panels and teams that prefer shorter markup. **Flowbite** provides HTML component snippets plus a small JavaScript library for interactive parts, with versions for React, Vue, Angular and Svelte.

APP.CSS WITH DAISYUI CSS

```css
@import "tailwindcss";
@plugin "daisyui" {
  themes: light --default, dark --prefersdark, cupcake;
}
```

DAISYUI MARKUP HTML

```html
<button class="btn btn-primary">Enrol</button>
<button class="btn btn-outline btn-sm">Details</button>

<div class="card w-80 bg-base-100 shadow-xl">
  <div class="card-body">
    <h2 class="card-title">Tailwind CSS</h2>
    <p>26 days, 5 projects.</p>
    <div class="card-actions justify-end"><button class="btn btn-primary">Start</button></div>
  </div>
</div>
```

15.3.4 Tailwind Plus templates (overview)

Tailwind Plus (formerly Tailwind UI) is the commercial collection from the makers of Tailwind: hundreds of professionally designed components (marketing sections, application UI, e-commerce) as copy-and-paste HTML, React and Vue code, plus complete site templates. Like shadcn/ui, you own the copied code. Its components are a useful reference for well-structured, accessible Tailwind markup even if you do not use them directly; the free **Catalyst** and **Headless UI** projects come from the same team.

15.4 Icons and assets

15.4.1 Heroicons and Lucide

Heroicons (by the Tailwind team) and **Lucide** are open-source SVG icon sets designed to be styled with utilities. Because the icons use `currentColor` for their strokes or fills, a text colour class colours them, and `size-*` sizes them. Both offer React and Vue packages and plain SVG files.

ICON COMPONENTS javascript

```javascript
npm install @heroicons/react      # or: npm install lucide-react

import { BellIcon } from '@heroicons/react/24/outline';
import { Search } from 'lucide-react';

<BellIcon className="size-6 text-slate-500" aria-hidden="true" />
<Search className="size-5 text-indigo-600" strokeWidth={2.5} />
```

INLINE HEROICONS SVG (BELL-OUTLINE AND PLUS) HTML

```
<div class="flex items-center gap-6">
  <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24" stroke-width="1.5"
stroke="currentColor" class="size-8 text-slate-500" aria-hidden="true">
    <path stroke-linecap="round" stroke-linejoin="round" d="M14.857 17.082a23.848 23.848 0 0 0 5.454-
1.31A8.967 8.967 0 0 1 18 9.75V9A6 6 0 0 0 6 9v.75a8.967 8.967 0 0 1-2.312 6.022c1.733.64 3.56 1.085
5.455 1.31m5.714 0a24.255 24.255 0 0 1-5.714 0m5.714 0a3 3 0 1 1-5.714 0"/>
  </svg>
  <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24" stroke-width="1.5"
stroke="currentColor" class="size-8 text-indigo-600 hover:text-indigo-400" aria-hidden="true">
    <path stroke-linecap="round" stroke-linejoin="round" d="M14.857 17.082a23.848 23.848 0 0 0 5.454-
1.31A8.967 8.967 0 0 1 18 9.75V9A6 6 0 0 0 6 9v.75a8.967 8.967 0 0 1-2.312 6.022c1.733.64 3.56 1.085
5.455 1.31m5.714 0a24.255 24.255 0 0 1-5.714 0m5.714 0a3 3 0 1 1-5.714 0"/>
  </svg>
  <button class="inline-flex items-center gap-2 rounded-lg bg-indigo-600 px-4 py-2 text-sm font-
semibold text-white">
    <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24" stroke-width="2"
stroke="currentColor" class="size-4" aria-hidden="true"><path stroke-linecap="round" stroke-
linejoin="round" d="M12 4.5v15m7.5-7.5h-15"/></svg>
    New batch
  </button>
</div>
```



THE SAME ICON IN TWO COLOURS, AND INSIDE A BUTTON: CURRENTCOLOR FOLLOWS THE TEXT COLOUR

TIP

Mark decorative icons `aria-hidden="true"`. When an icon is the only content of a button, give the button an accessible name (`aria-label` or an `sr-only` span, Module 12).

PRACTICE CHECKPOINT

A **React Button component** with size and variant props built with **cva** and **tailwind-merge**, plus **one page built with shadcn/ui components**.

BUTTON WITH CVA AND TAILWIND-MERGE: SAMPLE SOLUTION

javascript

```
// src/lib/utils.js
import { clsx } from 'clsx';
import { twMerge } from 'tailwind-merge';
export const cn = (...inputs) => twMerge(clsx(inputs));

// src/components/Button.jsx
import { cva } from 'class-variance-authority';
import { cn } from '../lib/utils';

const buttonVariants = cva(
  'inline-flex items-center justify-center gap-2 rounded-lg font-semibold transition ' +
  'focus-visible:outline-2 focus-visible:outline-offset-2 disabled:pointer-events-none
disabled:opacity-50',
  {
    variants: {
      variant: {
        primary:   'bg-indigo-600 text-white hover:bg-indigo-500 focus-visible:outline-indigo-600',
        secondary: 'bg-white text-slate-900 ring-1 ring-slate-300 hover:bg-slate-50',
        ghost:     'text-indigo-600 hover:bg-indigo-50',
        danger:    'bg-rose-600 text-white hover:bg-rose-500 focus-visible:outline-rose-600',
      },
      size: {
        sm: 'h-8 px-3 text-xs',
        md: 'h-10 px-4 text-sm',
        lg: 'h-12 px-6 text-base',
        icon: 'size-10',
      },
    },
    defaultVariants: { variant: 'primary', size: 'md' },
  },
);

export function Button({ variant, size, className, ...props }) {
  return <button className={cn(buttonVariants({ variant, size }), className)} {...props} />;
}

// usage
<Button>Save</Button>
<Button variant="secondary" size="sm">Cancel</Button>
<Button variant="danger" className="w-full">Delete batch</Button>   // w-full merged safely
```

A SHADCN/UI PAGE: SAMPLE SOLUTION (REPRESENTATIVE)

javascript

```
// app/enrol/page.jsx: a page assembled from shadcn/ui components
import { Card, CardHeader, CardTitle, CardDescription, CardContent, CardFooter } from
'@/components/ui/card';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import { Button } from '@/components/ui/button';
import { Tabs, TabsList, TabsTrigger, TabsContent } from '@/components/ui/tabs';

export default function EnrolPage() {
  return (
    <main className="mx-auto max-w-md p-6">
      <Tabs defaultValue="enrol">
        <TabsList className="grid w-full grid-cols-2">
          <TabsTrigger value="enrol">Enrol</TabsTrigger>
          <TabsTrigger value="login">Log in</TabsTrigger>
        </TabsList>
        <TabsContent value="enrol">
          <Card>
            <CardHeader>
              <CardTitle>Join the Tailwind batch</CardTitle>
              <CardDescription>Classes start on 3 November.</CardDescription>
            </CardHeader>
            <CardContent className="space-y-3">
              <div className="space-y-1"><Label htmlFor="name">Name</Label><Input id="name" /></div>
              <div className="space-y-1"><Label htmlFor="email">E-mail</Label><Input id="email"
type="email" /></div>
            </CardContent>
            <CardFooter><Button className="w-full">Enrol</Button></CardFooter>
          </Card>
        </TabsContent>
        <TabsContent value="login">...</TabsContent>
      </Tabs>
    </main>
  );
}
```

Every framework uses the same two pieces (a Vite or PostCSS plugin and `@import "tailwindcss"`); build class strings with clsx, resolve overrides with tailwind-merge, describe component variants with cva, always write complete class names, and choose between copy-in (shadcn/ui), headless (Radix, Headless UI) and class-based (daisyUI) libraries depending on how much styling you want to own.

MODULE 16 · ADVANCED

Performance, accessibility and best practices

Minimum duration **1 day** | Level **Advanced**

This module covers shipping fast, accessible Tailwind projects and keeping class-heavy markup maintainable. It explains why Tailwind's CSS stays small and measures a real production build; reviews accessibility topics across the course (contrast in the default palette, focus-visible styles, screen-reader-only text, reduced motion, ARIA variants); sets out best practices for class order, the design scale, @apply and readable markup; summarises the changes from Tailwind 3 to 4, the upgrade tool and browser support; and gives a checklist for debugging classes that do not apply. Sizes and contrast values in this module were measured, not estimated.

IN THIS MODULE

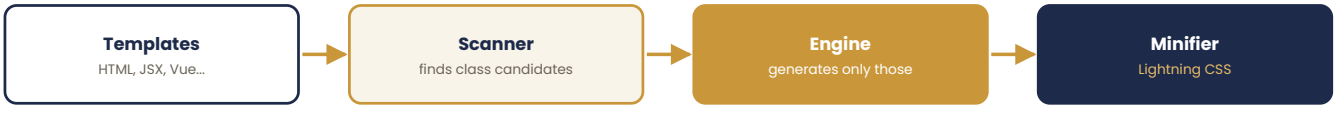
16.1 Performance
16.2 Accessibility
16.3 Best practices
16.4 Upgrading projects
16.5 Debugging

16.1 Performance

16.1.1 How Tailwind keeps CSS small

A traditional stylesheet grows with every new page and feature, and old rules are rarely deleted because nobody is sure they are unused. Tailwind's CSS grows with the number of **distinct** classes used, not with the number of pages: `p-4` is generated once whether it appears on one element or ten thousand. Three mechanisms keep it small:

- **On-demand generation**: only classes found in the scanned sources are generated; the thousands of possible utilities are never written out.
- **Reuse**: a project quickly reaches a plateau where new pages use classes that already exist, so the CSS barely grows.
- **Theme pruning**: theme variables are emitted only when something uses them (Module 13).



THE BUILD PIPELINE: NOTHING UNUSED REACHES THE OUTPUT

16.1.2 Production builds and minification

In development, the CSS is unminified and rebuilt incrementally in milliseconds. A production build minifies it (whitespace, shortest colour and number forms, merged rules) through the built-in Lightning

CSS. With Vite, `npm run build` does this automatically; with the CLI, add `--minify`. Servers then compress it with gzip or Brotli on the way to the browser.

PRODUCTION BUILDS shell

```
# Vite (React, Vue, Svelte...): minified automatically
npm run build

# Tailwind CLI
npx @tailwindcss/cli -i src/app.css -o dist/app.css --minify
```

16.1.3 Checking final CSS size

The following real build compiles the responsive landing page from Module 9 (with the Module 10 dark-mode toggle) and reports the sizes:

APP.CSS text

```
@import "tailwindcss";
@custom-variant dark (&:where(.dark, .dark *));
```

MEASURING A PRODUCTION BUILD terminal

```
tailwindcss -i app.css -o dist/dev.css 2>/dev/null
tailwindcss -i app.css -o dist/app.css --minify 2>/dev/null
echo "classes used:      $(grep -o 'class="[^"]*"' index.html | tr ' "' '\n\n' | grep -v '^class=' |
sort -u | grep -c .)"
echo "development CSS:   $(wc -c < dist/dev.css) bytes"
echo "minified CSS:      $(wc -c < dist/app.css) bytes"
echo "minified + gzip:   $(gzip -9 -c dist/app.css | wc -c) bytes"
```

TERMINAL

```
$ tailwindcss -i app.css -o dist/dev.css 2>/dev/null
$ tailwindcss -i app.css -o dist/app.css --minify 2>/dev/null
$ echo "classes used:      $(grep -o 'class="[^"]*"' index.html | tr ' "' '\n\n' | grep -v '^class=' |
sort -u | grep -c .)"
classes used:      74
$ echo "development CSS:   $(wc -c < dist/dev.css) bytes"
development CSS:   15743 bytes
$ echo "minified CSS:      $(wc -c < dist/app.css) bytes"
minified CSS:      12639 bytes
$ echo "minified + gzip:   $(gzip -9 -c dist/app.css | wc -c) bytes"
minified + gzip:   3224 bytes
```

The whole page, including Preflight and every utility it uses, ships a stylesheet that is a few kilobytes over the network, well under the 20 KB target in this module's checkpoint. In the browser, check the **Network** panel: the transferred size (compressed) and the resource size (uncompressed) are shown for `app.css`.

TIP

Tailwind is rarely the reason a page is slow. When Lighthouse reports a poor score, look first at images (size, format, `loading="lazy"`, explicit width and height), fonts (too many weights, no `font-display: swap`) and JavaScript.

16.2 Accessibility

16.2.1 Colour contrast with the default palette

WCAG requires a contrast ratio of at least **4.5:1** for normal text and **3:1** for large text (24px, or 18.5px bold) and for interface parts such as input borders and focus indicators. The table below was computed in Chromium from the default palette's real colours: each grey shade as text on white, and white text on each `600` shade.

CONTRAST RATIOS COMPUTED IN CHROMIUM

text-slate-300 on white	1.49:1	fails
text-slate-400 on white	2.63:1	fails
text-slate-500 on white	4.76:1	AA
text-slate-600 on white	7.58:1	AA
text-slate-700 on white	10.36:1	AA
text-gray-300 on white	1.47:1	fails
text-gray-400 on white	2.60:1	fails
text-gray-500 on white	4.84:1	AA
text-gray-600 on white	7.56:1	AA
text-gray-700 on white	10.30:1	AA
text-zinc-300 on white	1.48:1	fails
text-zinc-400 on white	2.62:1	fails
text-zinc-500 on white	4.83:1	AA
text-zinc-600 on white	7.72:1	AA
text-zinc-700 on white	10.43:1	AA
text-white on bg-indigo-600	6.46:1	AA
text-white on bg-indigo-500	4.58:1	AA
text-white on bg-blue-600	5.25:1	AA
text-white on bg-blue-500	3.76:1	large text / UI only
text-white on bg-emerald-600	3.65:1	large text / UI only
text-white on bg-emerald-500	2.47:1	fails
text-white on bg-amber-600	3.20:1	large text / UI only
text-white on bg-amber-500	2.13:1	fails
text-white on bg-rose-600	4.53:1	AA
text-white on bg-rose-500	3.75:1	large text / UI only
text-white on bg-sky-600	4.02:1	large text / UI only
text-white on bg-sky-500	2.71:1	fails

Rules of thumb that follow from these numbers: grey text on white needs shade **500 or darker** for normal text (600 is safer for small text); **400** is only for placeholders, disabled states or large text; white text works on most **600** backgrounds but not on yellow, amber or light green, which need dark text instead (`bg-amber-400 text-amber-950`).

16.2.2 Focus-visible styles on every interactive element

Every link, button, input and custom control must show where keyboard focus is. Preflight keeps the browser's default outline, so focus is visible unless you remove it. If you restyle it, use `focus-visible:` so the indicator appears for keyboard users, keep at least a 2px outline or ring with 3:1 contrast, and test by pressing Tab through the whole page. A consistent approach is a base style for all elements:

ONE FOCUS STYLE FOR THE WHOLE SITE

CSS

```css
@layer base {
  :focus-visible {
    outline: 2px solid var(--color-indigo-600);
    outline-offset: 2px;
  }
}
```

16.2.3 sr-only and not-sr-only

`sr-only` hides content visually but keeps it in the accessibility tree: labels for icon-only buttons, extra context for links (`Read more <span class="sr-only">about Dasara</span>`), headings for sections that are visually obvious. `focus:not-sr-only` reveals a skip link when it receives focus (Module 7). Do not use `hidden` or `invisible` for such text: they hide it from screen readers too.

16.2.4 Reduced motion

Guard decorative motion with `motion-safe:` or remove it with `motion-reduce:` (Module 11). Large movements, parallax, auto-playing carousels and infinite animations are the priority; small colour transitions are fine to keep.

16.2.5 ARIA variants for state styling

Style interactive state from the attribute that also tells assistive technology about it: `aria-expanded:`, `aria-selected:`, `aria-current:`, `aria-pressed:`, `aria-invalid:`, `aria-disabled:`. When the visual state is driven by the ARIA attribute, the two can never get out of sync, which is a common accessibility bug with class-toggling code (Modules 6 and 12).

16.3 Best practices

16.3.1 Consistent class order with the Prettier plugin

The official Prettier plugin sorts classes in the same order Tailwind outputs them: layout and positioning first, then box model, typography, visuals, and finally variants grouped by type, with responsive variants last. Everyone on the team sees the same order, diffs stay small, and duplicated or conflicting classes become easy to spot.

SETUP (TAILWINDSTYLESHEET LETS IT KNOW YOUR CUSTOM UTILITIES) javascript

```
npm install -D prettier prettier-plugin-tailwindcss

// .prettierrc
{
  "plugins": ["prettier-plugin-tailwindcss"],
  "tailwindStylesheet": "./src/app.css"
}
```

Before formatting	After prettier-plugin-tailwindcss
`class="text-white md:px-6 px-4 hover:bg-indigo-500 bg-indigo-600 rounded-lg py-2 font-semibold"`	`class="rounded-lg bg-indigo-600 px-4 py-2 font-semibold text-white hover:bg-indigo-500 md:px-6"`

16.3.2 Using the design scale instead of arbitrary values

Arbitrary values are an escape hatch, not a habit. `mt-[13px]`, `text-[#1b2a49]` and `w-[327px]` scattered through a code base recreate the inconsistency Tailwind is meant to prevent. Guidelines:

- Prefer the nearest scale value: designs rarely need 13px when 12px (`mt-3`) is available.
- If a value is a real part of the design, add it to the theme once (`--color-navy`, `--spacing-header`) and use the generated class.
- Keep arbitrary values for genuine one-offs: a precise `grid-cols-[...]` template, a `clamp()` size, an image position.

- Search the code base periodically for `-[` to find candidates for theme tokens.

16.3.3 When to use @apply and when not to

Use @apply	Avoid @apply
Styling HTML you cannot add classes to (Markdown, CMS, third-party widgets)	Making markup "look cleaner" by moving classes into CSS
Overriding a library's own class names	Recreating Bootstrap-style component classes for every element
A few global primitives in a project with no component system (`.btn`)	Anything a loop or framework component could reuse instead
Component stylesheets with `@reference` where a scoped style is required	Classes that need variants: write a `@utility` instead

16.3.4 Keeping markup readable

- **Extract components** when markup repeats (Module 12): readability problems are usually duplication problems.
- **Break long class lists across lines** in a logical grouping in templates that allow it, or rely on the Prettier order.
- **Use semantic HTML**: `<nav>`, `<article>`, `<button>` make the structure readable even with long class lists.
- **Name with data attributes or component names** when you need hooks for tests and analytics, not with presentational class names.
- **Use cva or variant maps** (Module 15) for components with many states instead of nested ternaries.
- **Do not fight utilities with custom CSS**: if many elements need the same exception, change the theme.

16.4 Upgrading projects

16.4.1 Tailwind 3 to 4 changes and the upgrade tool

Tailwind 4 (January 2025) is a rewrite with a new engine and CSS-first configuration. Most class names are unchanged, but some defaults and names moved. The official upgrade tool handles the bulk of the migration:

THE UPGRADE TOOL

shell

```
# requires Node.js 20+; run it in a clean git branch
npx @tailwindcss/upgrade
```

It updates dependencies, converts `tailwind.config.js` to `@theme` where possible, replaces the `@tailwind` directives with `@import "tailwindcss"`, and renames changed classes in your templates. Review the diff afterwards. The changes you are most likely to notice:

Area	v3	v4
Setup	`@tailwind base; @tailwind components; @tailwind utilities;`	`@import "tailwindcss";`

Area	v3	v4
Configuration	`tailwind.config.js`	`@theme` in CSS (JS config via `@config`)
Content paths	`content: [...]` required	automatic detection, `@source` for extras
PostCSS / CLI packages	`tailwindcss`, `autoprefixer`	`@tailwindcss/postcss`, `@tailwindcss/cli`, `@tailwindcss/vite`; no autoprefixer needed
Renamed scales	`shadow-sm`, `rounded-sm`, `blur-sm`	now `shadow-xs`, `rounded-xs`, `blur-xs` (and the bare names moved up one step)
Default border / divide colour	gray-200	`currentColor`: add a colour class
Default ring	3px, blue-500	1px, `currentColor`: use `ring-3 ring-blue-500` for the old look
`outline-none`	transparent outline	now `outline-hidden`; `outline-none` really removes it
Gradients	`bg-gradient-to-r`	`bg-linear-to-r` (old name still works)
Opacity utilities	`bg-opacity-50`	removed: use `bg-black/50`
Arbitrary variables	`bg-[--brand]`	`bg-(--brand)`
Important modifier	`!flex`	`flex!` (old form still accepted)
Stacked variant order	right to left	left to right
Button cursor	`cursor: pointer`	`cursor: default` (browser default)
Placeholder colour	gray-400	current text colour at 50% opacity

16.4.2 Browser support requirements

Tailwind 4 relies on modern CSS: cascade layers, `@property` registered custom properties, `color-mix()` and oklch colours. It therefore targets **Safari 16.4+, Chrome 111+ and Firefox 128+**. In those browsers everything works without prefixes or fallbacks. If a project must support older browsers (some enterprise or government environments), stay on Tailwind 3.4 until the requirement changes; there is no compatibility mode.

> **WATCH OUT**
> Check your audience before upgrading: an analytics report of browser versions settles the question. Older iPhones that cannot update past iOS 16.3 are the most common reason to wait.

16.5 Debugging

16.5.1 Classes not applying: common causes

Symptom	Likely cause	Fix
A class does nothing at all	typo or non-existent class (`text-bold`, `bg-red-550`)	check in IntelliSense; hover shows the CSS if it exists

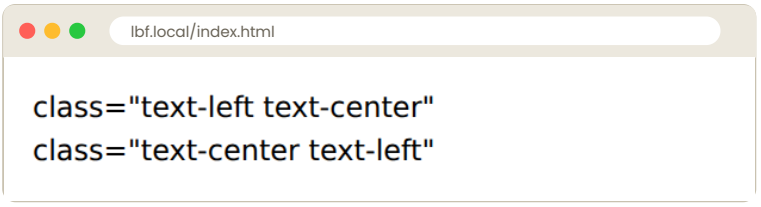
Symptom	Likely cause	Fix
Works in one file, not another	file not scanned (ignored by `.gitignore`, in `node_modules`, outside the project)	`@source "../path"`
Dynamic colour does nothing	class built from pieces: `` `bg-${c}-600` ``	map to full class names (Module 15)
Two classes conflict	both set the same property (`p-2` `p-4`); stylesheet order decides	remove one; use `tailwind-merge` in components
Utility loses to custom CSS	your CSS is unlayered, so it beats all layers	move it into `@layer base` / `components`
Utility loses to a library's CSS	library CSS is unlayered or more specific	import it into a layer, or use the `!` suffix (`flex!`) sparingly
`dark:` ignores the toggle	dark is still media-based	`@custom-variant dark (...)` (Module 10)
`hover:` does nothing on phones	v4 hover only applies with a hover-capable pointer	expected; use `active:` for touch feedback
`@apply` error in a component file	the stylesheet does not know the theme	`@reference "../app.css";`
No styles at all	CSS file not imported, or build plugin missing	check the import in the entry file and the Vite/PostCSS config

The "two classes conflict" case surprises many developers, so here it is in real output: the order in the attribute does not matter, only the order in the generated stylesheet.

INDEX.HTML

HTML

```
<p class="text-left text-center">class="text-left text-center"</p>
<p class="text-center text-left">class="text-center text-left"</p>
```



BOTH PARAGRAPHS RENDER THE SAME WAY: THE ATTRIBUTE ORDER IS IRRELEVANT

16.5.2 Inspecting generated CSS in DevTools

1. Right-click the element and choose **Inspect**. The **Styles** pane lists every rule that matches, with the escaped Tailwind selectors (`.md\:flex`) and the cascade layer each rule belongs to.
2. Crossed-out declarations were overridden; the winning rule is the one not struck through. Hover the layer name to see the layer order.
3. If the class is in the HTML but no rule appears, the CSS was not generated: search the loaded stylesheet (Sources panel, Ctrl+F) for the class name to confirm, then check scanning (16.5.1).
4. Use the **Computed** tab to see the final value and click the arrow to jump to the rule that set it.
5. Theme variables appear on `:root`; selecting `<html>` shows their current values, which is useful when debugging themes and dark mode.

6. Toggle states (`:hover`, `:focus-visible`) with the **:hov** button, and emulate dark mode or reduced motion from the **Rendering** panel.

PRACTICE CHECKPOINT

Audit the landing page with **Lighthouse**, fix accessibility issues, and confirm the **production CSS file is under 20 KB**.

A worked approach to the audit:

1. Build for production and serve the result (`npm run build` then `npm run preview`); never audit the development server.
2. In Chrome DevTools open **Lighthouse**, choose Mobile, tick Performance, Accessibility, Best practices and SEO, and run.
3. Fix each accessibility finding. Typical ones and their Tailwind fixes: low-contrast text (`text-slate-400` to `text-slate-600`); buttons without a name (`aria-label` or an `sr-only` span); images without `alt`; links that are not distinguishable (underline or `font-medium` plus colour); tap targets too small (`min-h-11` or more padding).
4. Re-run until Accessibility is 100, then test manually: tab through the page (focus visible everywhere?), zoom to 200% (does anything overlap?), enable reduced motion and dark mode.
5. Check the CSS size: the Network panel shows the transferred size of `app.css`; the build in Section 16.1.3 measured it with the CLI.

Tailwind's output stays small because only used classes are generated; verify it with a minified build. Check contrast (grey 500+ on white for text), keep focus-visible styles everywhere, drive state styles from ARIA attributes, sort classes with Prettier, prefer the theme over arbitrary values, upgrade with `npx @tailwindcss/upgrade`, and debug with the Styles pane and the common-causes table.

MODULE 17 · ADVANCED

Projects and interview preparation

Minimum duration **3 days** | Level **Advanced**

The final module brings everything together in portfolio projects and prepares learners for front-end interviews. It specifies six capstone projects with their requirements, the Tailwind techniques each one exercises and key parts of a reference solution; explains how to practise by recreating real designs; gives an interview checklist with model answers, a comparison of Tailwind, Bootstrap and plain CSS, and a walk-through of building a component live from a design; and suggests the next steps after the course.

IN THIS MODULE

17.1 Capstone projects
17.2 Practice
17.3 Interview preparation
17.4 Next steps

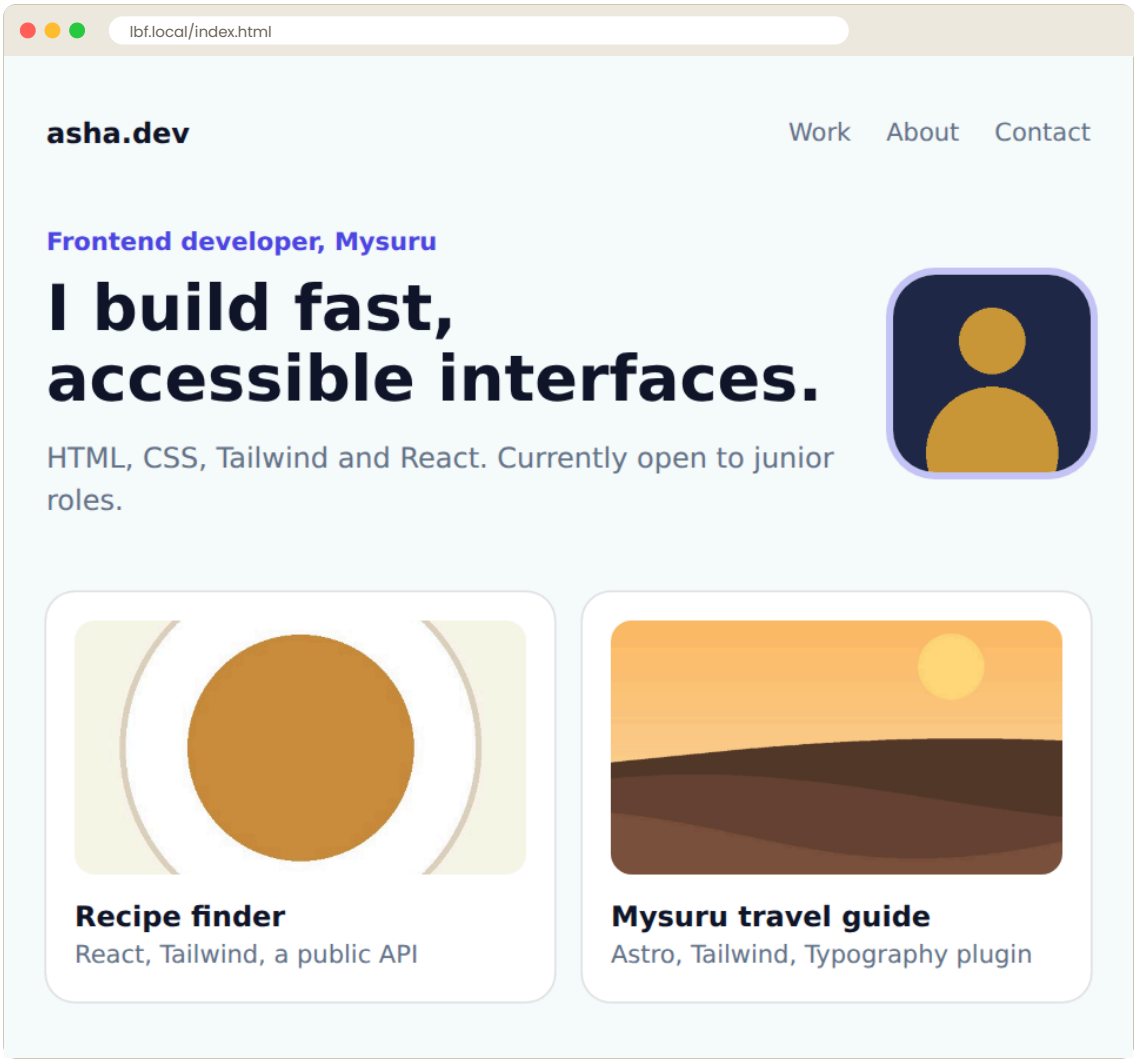
17.1 Capstone projects

Each project below lists what to build, the techniques it exercises (with the module where they were taught) and a definition of done. Build at least three, including one inside a framework, and deploy them (Netlify, Vercel, GitHub Pages and Cloudflare Pages all host static builds for free).

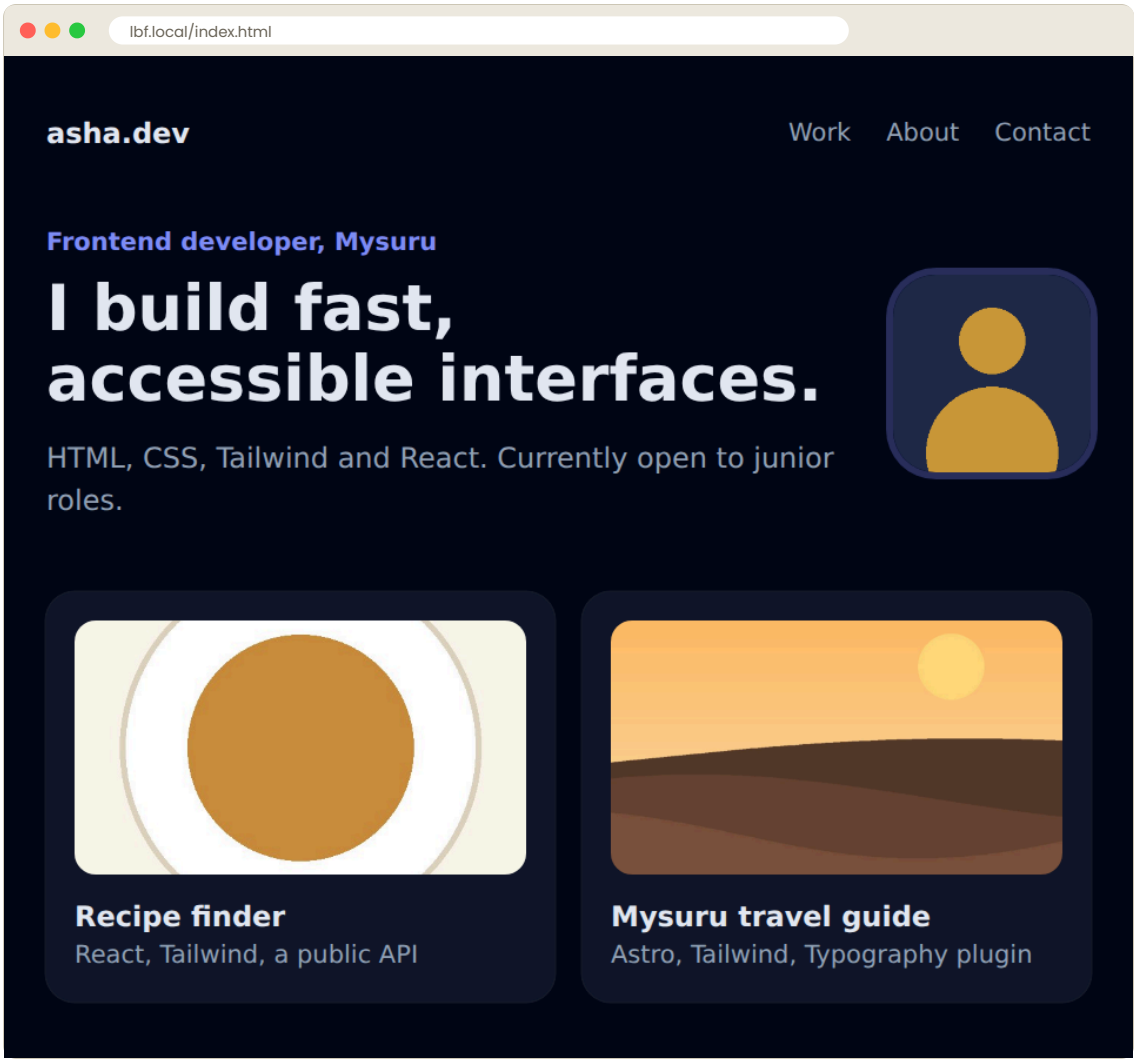
Project	Main techniques	Modules
Portfolio with dark mode	responsive layout, dark mode toggle, semantic colours, typography	4, 7, 9, 10
SaaS landing page	hero, feature grid, pricing, testimonials, FAQ accordion, motion	5, 8, 11, 12
Admin dashboard	grid shell, sidebar, stat cards, tables, charts area	7, 8, 12
E-commerce listing, product page and cart	card grids, galleries, forms, state variants, container queries	6, 8, 9, 12
Homepage clone	faithful reproduction of a real design, theme tokens	13
Blog in a framework	Typography plugin, components, routing, dark mode	4, 15

17.1.1 Responsive personal portfolio with dark mode

- **Sections**: header with navigation and theme toggle; hero with name, role and photo; projects grid with tags; about and skills; contact form; footer.
- **Requirements**: mobile-first from 320px; dark mode that follows the system and remembers the user's choice without a flash; semantic colour tokens; visible focus everywhere; Lighthouse accessibility 100.
- **Done when**: deployed, linked from your CV and GitHub profile, and every project card links to a live demo and the source code.



REFERENCE SOLUTION EXCERPT: LIGHT THEME



THE SAME MARKUP WITH THE DARK CLASS

17.1.2 SaaS landing page (hero, features, pricing, testimonials, FAQ)

- **Sections**: sticky translucent header; hero with headline, sub-text, two calls to action and a product screenshot; logo strip; three-to-six feature cards with icons; pricing with a highlighted plan and a monthly/yearly toggle (peer-checked or a small script); testimonials; FAQ with `<details>`; final call to action; footer with link columns.
- **Requirements**: responsive at every width; entry animations guarded with `motion-safe:`; a brand theme in `@theme` (no default palette classes in the markup); production CSS under 20 KB.
- **Reuse**: the pricing table and accordion from Module 12, the hero from Module 5.

17.1.3 Admin dashboard with charts area, tables and sidebar

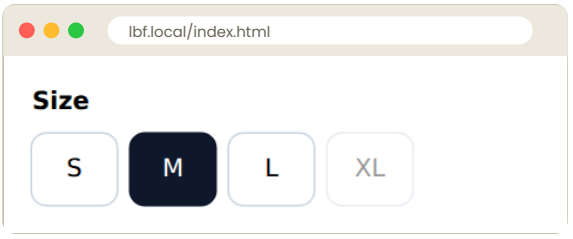
- **Layout**: `grid-cols-[240px_1fr]` shell that collapses to a top bar with a slide-in sidebar below `lg`; header with search and profile menu.
- **Content**: four stat cards; a chart area (a library such as Chart.js or Recharts reading theme colours through `getComputedStyle`, Module 13); a sortable data table with status badges and pagination; a recent-activity list.
- **Requirements**: dark mode; tables scroll horizontally on small screens (`overflow-x-auto`); sort buttons expose `aria-sort` and use `aria-[sort=ascending]:` variants.

17.1.4 E-commerce product listing, product page and cart

- **Listing**: filter sidebar (checkboxes styled with `accent-*` or the Forms plugin), auto-fit product grid, product cards with image zoom on hover (`group-hover:scale-105`) and badges.
- **Product page**: image gallery with a featured image and thumbnails, size picker built from radio buttons with `has-checked:`, quantity input, add-to-cart button with a loading state.
- **Cart**: line items with `divide-y`, a summary card that becomes sticky on desktop (`lg:sticky lg:top-24`), and a product card that uses container queries so it works in the grid and in the narrow cart drawer.

SIZE-PICKER.HTML: PART OF THE PRODUCT PAGE HTML

```
<fieldset class="flex gap-2 text-sm">
  <legend class="mb-2 font-semibold">Size</legend>
  <label class="grid h-10 w-12 cursor-pointer place-items-center rounded-lg ring-1 ring-slate-300 has-
checked:bg-slate-900 has-checked:text-white has-checked:ring-slate-900 has-disabled:cursor-not-allowed
has-disabled:opacity-40 has-focus-visible:outline-2 has-focus-visible:outline-offset-2">
    <input type="radio" name="size" value="S" class="sr-only">S
  </label>
  <label class="grid h-10 w-12 cursor-pointer place-items-center rounded-lg ring-1 ring-slate-300 has-
checked:bg-slate-900 has-checked:text-white has-checked:ring-slate-900 has-disabled:cursor-not-allowed
has-disabled:opacity-40 has-focus-visible:outline-2 has-focus-visible:outline-offset-2">
    <input type="radio" name="size" value="M" class="sr-only" checked>M
  </label>
  <label class="grid h-10 w-12 cursor-pointer place-items-center rounded-lg ring-1 ring-slate-300 has-
checked:bg-slate-900 has-checked:text-white has-checked:ring-slate-900 has-disabled:cursor-not-allowed
has-disabled:opacity-40 has-focus-visible:outline-2 has-focus-visible:outline-offset-2">
    <input type="radio" name="size" value="L" class="sr-only">L
  </label>
  <label class="grid h-10 w-12 cursor-pointer place-items-center rounded-lg ring-1 ring-slate-300 has-
checked:bg-slate-900 has-checked:text-white has-checked:ring-slate-900 has-disabled:cursor-not-allowed
has-disabled:opacity-40 has-focus-visible:outline-2 has-focus-visible:outline-offset-2">
    <input type="radio" name="size" value="XL" class="sr-only" disabled>XL
  </label>
</fieldset>
```



A SIZE PICKER FROM NATIVE RADIO BUTTONS: HAS-CHECKED: FOR THE SELECTION, HAS-DISABLED: FOR SOLD-OUT SIZES

17.1.5 Clone of a well-known website's homepage

Recreating a real, polished homepage (a product site, a news site, a streaming service) is excellent practice because the design decisions are already made: your job is translation into utilities. Measure spacing and font sizes with DevTools, map them to the nearest scale values or theme tokens, and match the responsive behaviour at three widths. Use your own placeholder text and images, do not copy logos or brand assets, and label the project clearly as a learning clone in your portfolio.

17.1.6 Blog built with a framework and the Typography plugin

- **Stack**: a framework with Markdown or MDX support (Astro, Next.js, Nuxt Content or SvelteKit).
- **Pages**: home with a list of post cards (title balanced, excerpt clamped to three lines, date and reading time); post page rendered inside `prose dark:prose-invert` with custom element modifiers;

tag pages; an about page.

- **Requirements**: custom fonts via `@theme`; code blocks styled; images with `prose-img:rounded-xl`; dark mode; an RSS feed and good meta tags for sharing.

17.2 Practice

17.2.1 Recreating designs from Dribbble or Frontend Mentor

Frontend Mentor provides real design files with challenges graded from newbie to guru, plus a community that reviews your solution; **Dribbble** and similar galleries offer endless (unspecified) interface designs to recreate. A productive routine:

1. Pick a design slightly above your level and estimate the time before starting.
2. Identify the design tokens first: colours, fonts, the spacing rhythm, radius and shadows. Put them in `@theme`.
3. Build mobile first, section by section, using only theme classes. Note every place you reach for an arbitrary value and ask whether it should be a token.
4. Compare with the design at the specified widths (overlay the screenshot at 50% opacity with a browser extension, or place them side by side).
5. Check accessibility (keyboard, contrast, alt text) and publish. Record the time it took; it should fall with each project.

17.3 Interview preparation

17.3.1 Core topics checklist

Interviewers for front-end roles that use Tailwind usually check the following. Each answer below is a model short answer; be ready to expand with an example.

Q. What does "utility-first" mean?
Styling by composing small single-purpose classes (`p-4`, `flex`, `text-sm`) directly in the markup instead of writing custom CSS classes per component. Each class does one thing and maps to one or a few declarations from a constrained design scale (Module 1).

Q. Why is Tailwind called mobile-first? What does md:flex mean?
Unprefixed utilities apply at all widths; a breakpoint prefix applies from that width upward. `md:flex` means display flex from 48rem (768px) up. Phone styles are written without a prefix (Module 9).

Q. How do variants work? Give three kinds.
A variant prefix wraps a utility in a condition: pseudo-classes (`hover:`, `focus-visible:`), media and feature queries (`md:`, `dark:`, `motion-reduce:`), relationships (`group-hover:`, `peer-invalid:`, `has-checked:`) and attributes (`aria-expanded:`, `data-[state=open]:`). They can be stacked (Module 6).

Q. Explain group and peer.
`group` marks a parent so children can react to its state (`group-hover:`); `peer` marks a previous sibling so later siblings can react (`peer-checked:`). Both can be named (`group/card`) when nested. Peers must come before the styled element because CSS only selects following siblings (Module 6).

Q. How do you implement dark mode?
By default `dark:` follows `prefers-color-scheme`. For a toggle, redefine it with `@custom-variant dark`

`(&:where(.dark, .dark *))`, set the class from localStorage in a blocking head script to avoid a flash, and preferably use semantic colour variables (Module 10).

Q. What is @theme and how does it differ from :root?
Variables in `@theme` are emitted as CSS variables and also generate utilities according to their namespace (`--color-brand` creates `bg-brand`, `text-brand`…). Variables in `:root` are only variables (Module 13).

Q. When would you use @apply?
For HTML you cannot add classes to (Markdown, CMS, third-party widgets), library overrides and a few global primitives. Not to tidy markup; repeated markup should become a component (Modules 12 and 16).

Q. What are arbitrary values and when should you avoid them?
Bracket syntax for values outside the scale (`w-[327px]`, `grid-cols-[200px_1fr]`, `[mask-type:alpha]`). Use them for genuine one-offs; repeated values belong in the theme (Module 3).

Q. Container queries versus breakpoints?
Breakpoints respond to the viewport and suit page layout; container queries (`@container` plus `@md:`) respond to the parent's width and suit reusable components that appear in containers of different sizes (Module 9).

Q. How does Tailwind avoid shipping unused CSS?
It scans source files as text for class candidates and generates CSS only for the classes it finds. Unused utilities are never generated, so no purge step is needed. This is also why class names must be complete strings (Modules 2 and 16).

Q. Why does `bg-${color}-500` not work?
Because the scanner never runs code: the complete string `bg-red-500` does not appear in the source, so no CSS is generated. Map values to full class names, use a CSS variable, or safelist with `@source inline()` (Module 15).

Q. How do you resolve conflicting classes passed to a component?
Which of two conflicting utilities wins depends on stylesheet order, not attribute order. Use `tailwind-merge` (often through a `cn()` helper with clsx) so the last class of a group wins (Module 15).

17.3.2 Tailwind vs Bootstrap vs plain CSS

	Tailwind CSS	Bootstrap	Plain CSS (with BEM, modules…)
Approach	utility classes composed in markup	pre-built component classes plus some utilities	custom class names and stylesheets
Look out of the box	none: you design it	recognisable Bootstrap look	none
Customisation	`@theme` tokens; every value adjustable	Sass variables; deeper changes fight the defaults	unlimited
CSS size	only used classes, usually 5 to 30 KB	large unless carefully trimmed	grows with the project
Naming	none needed	component names given	you name everything
Interactive JS	none (use headless libraries)	included (modals, dropdowns)	your own

	Tailwind CSS	Bootstrap	Plain CSS (with BEM, modules...)
Learning curve	class vocabulary; CSS knowledge still required	fastest for standard layouts	requires discipline to scale
Best for	custom designs, design systems, component frameworks	quick standard admin and internal tools	small sites, special effects, learning CSS

A balanced interview answer: Tailwind is not "better" in every case. It shines when a team builds a custom design in components and wants consistency and small CSS; Bootstrap is faster for a standard look with little design input; plain CSS remains essential knowledge, because Tailwind is CSS and every utility maps to properties you must understand.

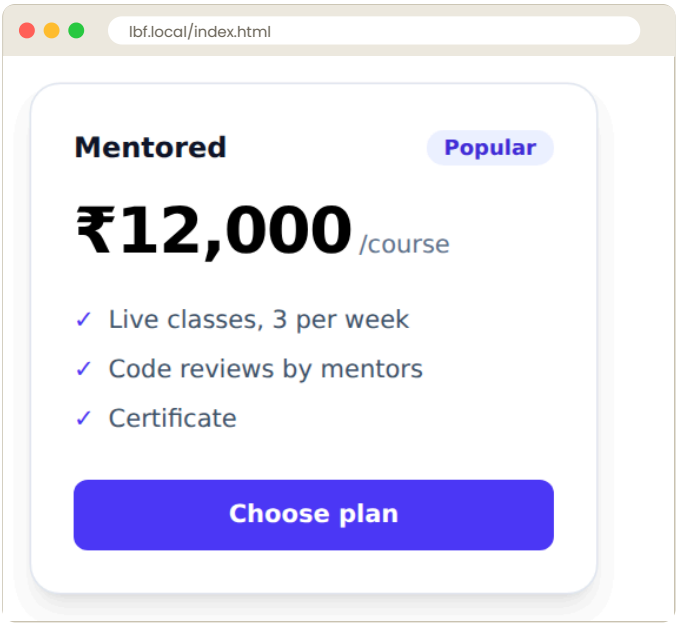
17.3.3 Building a component live from a design

Live-coding rounds often show a small design (a pricing card, a notification, a profile header) and ask you to build it while talking. A method that keeps you calm and visibly structured:

1. **Read the design aloud**: structure first ("a card with a header row, a price, a list and a button"), then tokens (colours, radius, shadow).
2. **Write semantic HTML first** with no classes: `article`, `h3`, `ul`, `button`.
3. **Layout**: add `flex` / `grid`, gaps and padding from the scale.
4. **Typography and colour**: sizes, weights, text colours, backgrounds.
5. **States**: hover, focus-visible, disabled; mention accessibility as you add them.
6. **Responsiveness**: shrink the window and add breakpoint or container variants where needed.
7. **Review**: point out what you would extract into a component or token in a real code base.

THE FINISHED LIVE-CODING COMPONENT HTML

```
<article class="w-80 rounded-2xl bg-white p-6 shadow-lg ring-1 ring-slate-200">
  <div class="flex items-center justify-between">
    <h3 class="font-semibold text-slate-900">Mentored</h3>
    <span class="rounded-full bg-indigo-50 px-2.5 py-0.5 text-xs font-semibold text-indigo-
700">Popular</span>
  </div>
  <p class="mt-4 flex items-baseline gap-1"><span class="text-4xl font-black tracking-
tight">&amp;#8377;12,000</span><span class="text-sm text-slate-500">/course</span></p>
  <ul class="mt-5 space-y-2 text-sm text-slate-600">
    <li class="flex gap-2"><span class="text-indigo-600" aria-hidden="true">&amp;#10003;</span>Live
classes, 3 per week</li>
    <li class="flex gap-2"><span class="text-indigo-600" aria-hidden="true">&amp;#10003;</span>Code
reviews by mentors</li>
    <li class="flex gap-2"><span class="text-indigo-600" aria-hidden="true">&amp;#10003;
</span>Certificate</li>
  </ul>
  <button class="mt-6 w-full rounded-lg bg-indigo-600 py-2.5 text-sm font-semibold text-white
hover:bg-indigo-500 focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-
indigo-600">Choose plan</button>
</article>
```



A PRICING CARD BUILT IN THE ORDER ABOVE

17.4 Next steps

17.4.1 React, Angular or Vue with Tailwind

Most Tailwind jobs combine it with a JavaScript framework. The natural continuation of this course is one of the LBF framework courses, applying everything here in components: React (with Next.js, shadcn/ui and cva), Angular (with standalone components and signals) or Vue (with Nuxt and the Composition API). In each, practise the patterns from Module 15: component variants, `cn()`, headless libraries, and themes shared across an application.

17.4.2 Design systems and UI engineering

Tailwind is a common foundation for **design systems**: shared tokens in `@theme`, a component library with documented variants, and accessibility built in. Skills to develop next: reading and maintaining design files in Figma, design tokens and their export, component documentation with Storybook, visual regression testing, and accessibility testing with screen readers. UI engineers who combine solid CSS, a framework and design-system thinking are in steady demand.

PRACTICE CHECKPOINT

Build and deploy **three portfolio-quality projects** with Tailwind, including **one built inside a JavaScript framework**.

A submission checklist for each project:

Item	Check
Live URL	deployed, loads over HTTPS, no console errors
Source code	public repository with a README: screenshot, features, stack, how to run
Responsive	tested at 320, 768, 1024 and 1440px; no horizontal scrolling
Accessible	Lighthouse accessibility 100; keyboard navigation works; focus always visible

Item	Check
Theme	colours, fonts and radius come from `@theme`; few arbitrary values
Dark mode	at least one project supports it without a flash
Performance	production CSS under 20 KB; images optimised
Framework	one project in React, Angular, Vue, Svelte or a meta-framework

Finish the course by building and shipping real projects: a portfolio, a landing page, a dashboard or shop, and one framework app. Be ready to explain utility-first, mobile-first breakpoints, variants, group and peer, dark mode, `@theme`, `@apply`, arbitrary values, container queries and how Tailwind avoids unused CSS, and to build a component live from a design.